```
<!DOCTYPE html>
<html lang="en-US">
<head>
<meta charset="UTF-8" />
<meta name="generator" content="LaTeX Lwarp package" />
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
<!--[if lt IE 9]>
<script src="http://html5shiv.googlecode.com/svn/trunk/html5.js"></script>
<![endif]-->
<title>Amaroo’s Consensus Strategy: Ultra Terminum</title>
<link rel="stylesheet" type="text/css" href="lwarp.css" />

<script>
// Lwarp MathJax emulation code
//
// Based on code by Davide P. Cervone.
// Equation numbering: https://github.com/mathjax/MathJax/issues/2427
// Starred and ifnextchar macros: https://github.com/mathjax/MathJax/issues/2428
// \left, \right delimiters: https://github.com/mathjax/MathJax/issues/2535
//
// Modified by Brian Dunn to adjust equation numbering and add subequations.
//
// LaTeX can use \seteqnumber{subequations?}{section}{number} before each equation.
// subequations? is 0 usually, 1 if inside subequations.
// section is a string printed as-is, or empty.
// number is auto-incremented by MathJax between equations.
//
MathJax = {
  subequations: "0",
  section: "",
  loader: {
    load: ['[tex]/tagformat', '[tex]/textmacros'],
  },
  startup: {
    ready() {
      //  These would be replaced by import commands if you wanted to make
      //  a proper extension.
      const Configuration = MathJax._.input.tex.Configuration.Configuration;
      const CommandMap = MathJax._.input.tex.SymbolMap.CommandMap;
      const Macro = MathJax._.input.tex.Symbol.Macro;
      const TexError = MathJax._.input.tex.TexError.default;
      const ParseUtil = MathJax._.input.tex.ParseUtil.default;
      const expandable = MathJax._.util.Options.expandable;

      //  Insert the replacement string into the TeX string, and check
      //  that there haven't been too many maxro substitutions (prevents
      //  infinite loops).
      const useArgument = (parser, text) => {
        parser.string = ParseUtil.addArgs(parser, text, parser.string.slice(parser.i));
        parser.i = 0;
        if (++parser.macroCount > parser.configuration.options.maxMacros) {
          throw new TexError('MaxMacroSub1',
          'MathJax maximum macro substitution count exceeded; ' +
          'is there a recursive macro call?');
        }
      }

      //  Create the command map for:
      //     \ifstar, \ifnextchar, \ifblank, \ifstrequal, \seteqnumber
      new CommandMap('Lwarp-macros', {
        ifstar: 'IfstarFunction',
        ifnextchar: 'IfnextcharFunction',
        ifblank: 'IfblankFunction',
        ifstrequal: 'IfstrequalFunction',
        seteqnumber: 'SeteqnumberFunction'
      }, {
        //  This function implements an ifstar macro.
        IfstarFunction(parser, name) {
          const resultstar = parser.GetArgument(name);
          const resultnostar = parser.GetArgument(name);
          const star = parser.GetStar();                  // true if there is a *
          useArgument(parser, star ? resultstar : resultnostar);
        },

        //  This function implements an ifnextchar macro.
        IfnextcharFunction(parser, name) {
          let whichchar = parser.GetArgument(name);
          if (whichchar.match(/^(?:0x[0-9A-F]+|[0-9]+)$/i)) {
            // $ syntax highlighting
            whichchar = String.fromCodePoint(parseInt(whichchar));
          }
          const resultnextchar = parser.GetArgument(name);
          const resultnotnextchar = parser.GetArgument(name);
          const gotchar = (parser.GetNext() === whichchar);
          useArgument(parser, gotchar ? resultnextchar : resultnotnextchar);
        },

        // This function implements an ifblank macro.
        IfblankFunction(parser, name) {
          const blankarg = parser.GetArgument(name);
          const resultblank = parser.GetArgument(name);
          const resultnotblank = parser.GetArgument(name);
          const isblank = (blankarg.trim() == "");
          useArgument(parser, isblank ? resultblank : resultnotblank);
        },

        // This function implements an ifstrequal macro.
        IfstrequalFunction(parser, name) {
          const strequalfirst = parser.GetArgument(name);
          const strequalsecond = parser.GetArgument(name);
          const resultequal = parser.GetArgument(name);
          const resultnotequal = parser.GetArgument(name);
          const isequal = (strequalfirst == strequalsecond);
          useArgument(parser, isequal ? resultequal : resultnotequal);
        },

        //  This function modifies the equation numbers.
        SeteqnumberFunction(parser, name) {
            //  Get the macro parameters
            const star = parser.GetStar();                  // true if there is a *
            const optBrackets = parser.GetBrackets(name);   // contents of optional brackets
            const newsubequations = parser.GetArgument(name);  // the subequations argument
            const neweqsection = parser.GetArgument(name);  // the eq section argument
            const neweqnumber = parser.GetArgument(name);   // the eq number argument
            MathJax.config.subequations=newsubequations ;   // a string with boolean meaning
            MathJax.config.section=neweqsection ;           // a string with numeric meaning
            parser.tags.counter = parser.tags.allCounter = neweqnumber ;
        }

      });

      //  Create the Lwarp-macros package
      Configuration.create('Lwarp-macros', {
        handler: {macro: ['Lwarp-macros']}
      });

      MathJax.startup.defaultReady();

      // For forward references:
      MathJax.startup.input[0].preFilters.add(({math}) => {
        if (math.inputData.recompile){
            MathJax.config.subequations = math.inputData.recompile.subequations;
            MathJax.config.section = math.inputData.recompile.section;
        }
      });
      MathJax.startup.input[0].postFilters.add(({math}) => {
        if (math.inputData.recompile){
            math.inputData.recompile.subequations = MathJax.config.subequations;
            math.inputData.recompile.section = MathJax.config.section;
        }
      });

        // For \left, \right with unicode-math:
        const {DelimiterMap} = MathJax._.input.tex.SymbolMap;
        const {Symbol} = MathJax._.input.tex.Symbol;
        const {MapHandler} = MathJax._.input.tex.MapHandler;
        const delimiter = MapHandler.getMap('delimiter');
        delimiter.add('\\lBrack', new Symbol('\\lBrack', '\u27E6'));
        delimiter.add('\\rBrack', new Symbol('\\rBrack', '\u27E7'));
        delimiter.add('\\lAngle', new Symbol('\\lAngle', '\u27EA'));
        delimiter.add('\\rAngle', new Symbol('\\rAngle', '\u27EB'));
        delimiter.add('\\lbrbrak', new Symbol('\\lbrbrak', '\u2772'));
        delimiter.add('\\rbrbrak', new Symbol('\\rbrbrak', '\u2773'));
        delimiter.add('\\lbag', new Symbol('\\lbag', '\u27C5'));
        delimiter.add('\\rbag', new Symbol('\\rbag', '\u27C6'));
        delimiter.add('\\llparenthesis', new Symbol('\\llparenthesis', '\u2987'));
        delimiter.add('\\rrparenthesis', new Symbol('\\rrparenthesis', '\u2988'));
        delimiter.add('\\llangle', new Symbol('\\llangle', '\u2989'));
        delimiter.add('\\rrangle', new Symbol('\\rrangle', '\u298A'));
        delimiter.add('\\Lbrbrak', new Symbol('\\Lbrbrak', '\u27EC'));
        delimiter.add('\\Rbrbrak', new Symbol('\\Rbrbrak', '\u27ED'));
        delimiter.add('\\lBrace', new Symbol('\\lBrace', '\u2983'));
        delimiter.add('\\rBrace', new Symbol('\\rBrace', '\u2984'));
        delimiter.add('\\lParen', new Symbol('\\lParen', '\u2985'));
        delimiter.add('\\rParen', new Symbol('\\rParen', '\u2986'));
        delimiter.add('\\lbrackubar', new Symbol('\\lbrackubar', '\u298B'));
        delimiter.add('\\rbrackubar', new Symbol('\\rbrackubar', '\u298C'));
        delimiter.add('\\lbrackultick', new Symbol('\\lbrackultick', '\u298D'));
        delimiter.add('\\rbracklrtick', new Symbol('\\rbracklrtick', '\u298E'));
        delimiter.add('\\lbracklltick', new Symbol('\\lbracklltick', '\u298F'));
        delimiter.add('\\rbrackurtick', new Symbol('\\rbrackurtick', '\u2990'));
        delimiter.add('\\langledot', new Symbol('\\langledot', '\u2991'));
        delimiter.add('\\rangledot', new Symbol('\\rangledot', '\u2992'));
        delimiter.add('\\lparenless', new Symbol('\\lparenless', '\u2993'));
        delimiter.add('\\rparengtr', new Symbol('\\rparengtr', '\u2994'));
        delimiter.add('\\Lparengtr', new Symbol('\\Lparengtr', '\u2995'));
        delimiter.add('\\Rparenless', new Symbol('\\Rparenless', '\u2996'));
        delimiter.add('\\lblkbrbrak', new Symbol('\\lblkbrbrak', '\u2997'));
        delimiter.add('\\rblkbrbrak', new Symbol('\\rblkbrbrak', '\u2998'));
        delimiter.add('\\lvzigzag', new Symbol('\\lvzigzag', '\u29D8'));
        delimiter.add('\\rvzigzag', new Symbol('\\rvzigzag', '\u29D9'));
        delimiter.add('\\Lvzigzag', new Symbol('\\Lvzigzag', '\u29DA'));
        delimiter.add('\\Rvzigzag', new Symbol('\\Rvzigzag', '\u29DB'));
        delimiter.add('\\lcurvyangle', new Symbol('\\lcurvyangle', '\u29FC'));
        delimiter.add('\\rcurvyangle', new Symbol('\\rcurvyangle', '\u29FD'));
        delimiter.add('\\Vvert', new Symbol('\\Vvert', '\u2980'));
    }   // ready
  },    // startup

  tex: {
    packages: {'[+]': ['tagformat', 'Lwarp-macros', 'textmacros']},
    tags: "ams",
        tagformat: {
            number: function (n) {
                if(MathJax.config.subequations==0)
                    return(MathJax.config.section + n);
                else
                    return(MathJax.config.section + String.fromCharCode(96+n));
            },
        },
  }
}
</script>

<script
    id="MathJax-script"
    src="https://cdn.jsdelivr.net/npm/mathjax@3/es5/tex-chtml.js"
></script>

</head>
<body>
<!--|Using lwarp|whitepaper.html|-->


<div class="bodywithoutsidetoc">


<main class="bodycontainer">


<section class="textbody">

<a id="whitepaper-autofile-0"></a>

<!--MathJax customizations:-->


<div class="hidden">

\(\newcommand{\footnotename}{footnote}\)

\(\def \LWRfootnote {1}\)

\(\newcommand {\footnote }[2][\LWRfootnote ]{{}^{\mathrm {#1}}}\)

\(\newcommand {\footnotemark }[1][\LWRfootnote ]{{}^{\mathrm {#1}}}\)

\(\let \LWRorighspace \hspace \)

\(\renewcommand {\hspace }{\ifstar \LWRorighspace \LWRorighspace }\)

\(\newcommand {\mathnormal }[1]{{#1}}\)

\(\newcommand \ensuremath [1]{#1}\)

\(\newcommand {\LWRframebox }[2][]{\fbox {#2}} \newcommand {\framebox }[1][]{\LWRframebox } \)

\(\newcommand {\setlength }[2]{}\)

\(\newcommand {\addtolength }[2]{}\)

\(\newcommand {\setcounter }[2]{}\)

\(\newcommand {\addtocounter }[2]{}\)

\(\newcommand {\arabic }[1]{}\)

\(\newcommand {\number }[1]{}\)

\(\newcommand {\noalign }[1]{\text {#1}\notag \\}\)

\(\newcommand {\cline }[1]{}\)

\(\newcommand {\directlua }[1]{\text {(directlua)}}\)

\(\newcommand {\luatexdirectlua }[1]{\text {(directlua)}}\)

\(\newcommand {\protect }{}\)

\(\def \LWRabsorbnumber #1 {}\)

\(\def \LWRabsorbquotenumber &quot;#1 {}\)

\(\newcommand {\LWRabsorboption }[1][]{}\)

\(\newcommand {\LWRabsorbtwooptions }[1][]{\LWRabsorboption }\)

\(\def \mathchar {\ifnextchar &quot;\LWRabsorbquotenumber \LWRabsorbnumber }\)

\(\def \mathcode #1={\mathchar }\)

\(\let \delcode \mathcode \)

\(\let \delimiter \mathchar \)

\(\def \oe {\unicode {x0153}}\)

\(\def \OE {\unicode {x0152}}\)

\(\def \ae {\unicode {x00E6}}\)

\(\def \AE {\unicode {x00C6}}\)

\(\def \aa {\unicode {x00E5}}\)

\(\def \AA {\unicode {x00C5}}\)

\(\def \o {\unicode {x00F8}}\)

\(\def \O {\unicode {x00D8}}\)

\(\def \l {\unicode {x0142}}\)

\(\def \L {\unicode {x0141}}\)

\(\def \ss {\unicode {x00DF}}\)

\(\def \SS {\unicode {x1E9E}}\)

\(\def \dag {\unicode {x2020}}\)

\(\def \ddag {\unicode {x2021}}\)

\(\def \P {\unicode {x00B6}}\)

\(\def \copyright {\unicode {x00A9}}\)

\(\def \pounds {\unicode {x00A3}}\)

\(\let \LWRref \ref \)

\(\renewcommand {\ref }{\ifstar \LWRref \LWRref }\)

\( \newcommand {\multicolumn }[3]{#3}\)

\(\require {textcomp}\)

\(\newcommand {\intertext }[1]{\text {#1}\notag \\}\)

\(\let \Hat \hat \)

\(\let \Check \check \)

\(\let \Tilde \tilde \)

\(\let \Acute \acute \)

\(\let \Grave \grave \)

\(\let \Dot \dot \)

\(\let \Ddot \ddot \)

\(\let \Breve \breve \)

\(\let \Bar \bar \)

\(\let \Vec \vec \)

\(\newcommand {\toprule }[1][]{\hline }\)

\(\let \midrule \toprule \)

\(\let \bottomrule \toprule \)

\(\def \LWRbooktabscmidruleparen (#1)#2{}\)

\(\newcommand {\LWRbooktabscmidrulenoparen }[1]{}\)

\(\newcommand {\cmidrule }[1][]{\ifnextchar (\LWRbooktabscmidruleparen \LWRbooktabscmidrulenoparen }\)

\(\newcommand {\morecmidrules }{}\)

\(\newcommand {\specialrule }[3]{\hline }\)

\(\newcommand {\addlinespace }[1][]{}\)

\(\require {mathtools}\)

\(\newenvironment {crampedsubarray}[1]{}{}\)

\(\newcommand {\smashoperator }[2][]{#2\limits }\)

\(\newcommand {\SwapAboveDisplaySkip }{}\)

\(\newcommand {\LaTeXunderbrace }[1]{\underbrace {#1}}\)

\(\newcommand {\LaTeXoverbrace }[1]{\overbrace {#1}}\)

\(\newcommand {\LWRmultlined }[1][]{\begin {multline*}}\)

\(\newenvironment {multlined}[1][]{\LWRmultlined }{\end {multline*}}\)

\(\let \LWRorigshoveleft \shoveleft \)

\(\renewcommand {\shoveleft }[1][]{\LWRorigshoveleft }\)

\(\let \LWRorigshoveright \shoveright \)

\(\renewcommand {\shoveright }[1][]{\LWRorigshoveright }\)

\(\newcommand {\shortintertext }[1]{\text {#1}\notag \\}\)

\(\newcommand {\vcentcolon }{\mathrel {\unicode {x2236}}}\)

\(\newcommand {\tcbset }[1]{}\)

\(\newcommand {\tcbsetforeverylayer }[1]{}\)

\(\newcommand {\tcbox }[2][]{\boxed {\text {#2}}}\)

\(\newcommand {\tcboxfit }[2][]{\boxed {#2}}\)

\(\newcommand {\tcblower }{}\)

\(\newcommand {\tcbline }{}\)

\(\newcommand {\tcbtitle }{}\)

\(\newcommand {\tcbsubtitle [2][]{\mathrm {#2}}}\)

\(\newcommand {\tcboxmath }[2][]{\boxed {#2}}\)

\(\newcommand {\tcbhighmath }[2][]{\boxed {#2}}\)

\(\newcommand {\nicefrac }[3][]{\mathinner {{}^{#2}\!/\!_{#3}}}\)

\(\require {cancel}\)

\(\newcommand {\iddots }{\mathinner {\unicode {x22F0}}}\)

\(\let \fixedddots \ddots \)

\(\let \fixedvdots \vdots \)

\(\let \fixediddots \iddots \)

\(\let \originalddots \ddots \)

\(\let \originalvdots \vdots \)

\(\let \originaliddots \iddots \)

\(\let \originaldddot \dddot \)

\(\let \originalddddot \ddddot \)

\(\newcommand {\LWRmarginnote }[1][]{}\)

\(\newcommand {\marginnote }[2][]{\qquad {\small \textrm {#2}}\LWRmarginnote }\)

\(\newcommand {\BeginAccSupp }[1]{}\)

\(\newcommand {\EndAccSupp }[1]{}\)

\(\newcommand{\ceilLWRsubstar}[2][]{{#1\left \lceil #2#1\right \rceil }}\)

\(\newcommand{\ceilLWRsubnostar}[2][]{{#1\lceil #2#1\rceil }}\)

\(\newcommand{\ceil}{\ifstar\ceilLWRsubstar\ceilLWRsubnostar}\)

\(\newcommand{\floorLWRsubstar}[2][]{{#1\left \lfloor #2#1\right \rfloor }}\)

\(\newcommand{\floorLWRsubnostar}[2][]{{#1\lfloor #2#1\rfloor }}\)

\(\newcommand{\floor}{\ifstar\floorLWRsubstar\floorLWRsubnostar}\)

</div>

<a id="whitepaper-autopage-1"></a>
```

<div class="titlepage">

<h1>Amaroo's Consensus Strategy: Ultra Terminum</h1>

<div class="author">

<div class="oneauthor">

<p>
by Max Kaye
</p>
</div>

</div>

<div class="titledate">

<div class="center">

<p>
<i>*Nos Ultra Terminum Ultimum Imus, Nunc*</i><br />
v1.1 — July 22<sup>nd</sup> 2025 / v1.1.1 — Updated November&nbsp;14<sup>th</sup>&nbsp;2025
</p>
</div>

</div>

</div>

```
<div class="abstract">


<div class="abstracttitle">Abstract</div>

<p>
This paper explains and constructs <em>Amaroo's</em> consensus mechanism: <em>Ultra Terminum</em>.
</p>

<p>
<em>Ultra Terminum</em> (UT) is a proof-agnostic cross-chain consensus strategy with multiple scaling configurations, and provides a practical solution to the Blockchain Trilemma (a.k.a. Scalability Trilemma). Although UT is proof-agnostic, using <em>Proof of
Work</em> as the foundational method of proof provides decisive advantages to security, scalability, and user experience.
</p>

<p>
The Trilemma takes \(O(c)\) as the measure of a traditional blockchain's scalability – \(c\) being a normal computer's capacity. UT's simplest configuration, \(\UT {1}\), scales quadratically — \(O(c^2)\). UT's other scaling configurations are: \(\UT {2}\)
\((O(c^3))\), \(\UT {3}\) \((O(c^4))\), and \(\UTinf {}\) (theoretically \(O(n)\), where \(n\) \(\sim \) size of the network).
</p>

<p>
<em>Proof of Reflection</em>, a new consensus primitive, is the key to UT's performance and security. Proof of Work configurations of UT meet <em>or exceed</em> the security of traditional PoW blockchains.
</p>

<p>
With reasonable parameters (i.e., a per-chain load comparable to Bitcoin, and one level of nested dapp-chains), a conservative estimate of capacity — using Amaroo's intended launch configuration (\(\UT {2}\)) — exceeds 350K TPS without <em>layer 2</em> scaling
methods.
</p>

<p>
Capacity of \(\UTinf {}\) is <em>independent</em> of those parameters and is theoretically <em>unbounded</em> by the architecture. It is combined with the previous variants to produce \(\UTinf {1}\), \(\UTinf {2}\), and \(\UTinf {3}\).
</p>

<p>
Confirmation times within UT are \(O(c^{-1})\), i.e., confirmation rate is \(O(c)\). The initial configuration will have a <em>maximum</em> confirmation frequency exceeding \(\sim 10\) Hz. Typically, sharded blockchains have a confirmation frequency \(&lt; 0.2\) Hz.
</p>

<p>
Some existing sharded (or otherwise \(O(c^2)\)) blockchain designs claim to be capable of 1M TPS. By comparison, \(\UT {2}\) configurations — with <em>equivalent</em> parameters — have a maximum capacity exceeding 10B TPS. Such a configuration would have a
system-wide maximum throughput exceeding 2000 GB/s of transaction data, and a confirmation frequency exceeding 400 Hz. UT out-scales the architectures of Eth2, Polkadot, Cardano, and Solana by orders of magnitude.
</p>

<p>
.7ex]0.5pt<br />
This paper's source code available at <a href="https://github.com/amaroohq/whitepaper" target="_blank" >https://github.com/amaroohq/whitepaper</a>  65000words.
</p>

</div>






</p>
```

```
<div class="center">

</div>

<p>
.7ex]0.5pt<span style="width:3.4445pt; display:inline-block"><!----></span>Begin DRAFT<span style="width:3.4445pt; display:inline-block"><!----></span>.7ex]0.5pt
</p>
```

```
<!--................................-->
<h4 id="autosec-6">HIGH Todos</h4>
<a id="whitepaper-autopage-6"></a>



<nav class="htodo">

</nav>
```

```
<!--................................-->
<h4 id="autosec-7">MED Todos</h4>
<a id="whitepaper-autopage-7"></a>


<nav class="mtodo">

</nav>
```

```
<!--................................-->
<h4 id="autosec-8">LOW Todos</h4>
<a id="whitepaper-autopage-8"></a>



<nav class="ltodo">

</nav>
```

```
<!--...............................-->
<h4 id="autosec-9">Todos</h4>
<a id="whitepaper-autopage-9"></a>


<nav class="todo">

</nav>
```

```html
<!--................................-->
<h4 id="autosec-10">Comments</h4>
<a id="whitepaper-autopage-10"></a>



<nav class="com">

</nav>

<p>
.7ex]0.5pt<span style="width:3.4445pt; display:inline-block"><!----></span>End DRAFT<span style="width:3.4445pt; display:inline-block"><!----></span>.7ex]0.5pt
</p>
```

```html
<!--................................-->
<h4 id="autosec-11">Contents</h4>
<a id="whitepaper-autopage-11"></a>



<nav class="toc">

</nav>


<div
    class="tcolorbox"
    style=" border:  1px solid #404040; background:  #FCD1E9; "
>


<div
    class="tcolorboxtitle"
    style=" border-top:  1px solid #404040; border-bottom:  1px solid #404040; background:  #404040; color:  #FFFFFF; "
>

<p>
TODO [1]
</p>
</div>


<div
    class="tcolorboxupper"
    style=" color:  #000000; "
>

<a id="todoref:1"></a>

<p>
explain header size: 112 = 80 + 32 (btc efficiency + an extra parent to support DAG data struct)
</p>
</div>

</div>

<a id="autoid-1" ></a >


<div
    class="tcolorbox"
    style=" border:  1px solid #404040; background:  #FCD1E9; "
>


<div
    class="tcolorboxtitle"
    style=" border-top:  1px solid #404040; border-bottom:  1px solid #404040; background:  #404040; color:  #FFFFFF; "
>

<p>
TODO [2]
</p>
</div>


<div
    class="tcolorboxupper"
    style=" color:  #000000; "
>

<a id="todoref:2"></a>

<p>
discuss impact of PoW/PoS choice for simplex-/dapp-chains — PoS overhead (sorta discussed atm, but only for headers)
</p>
</div>

</div>

<a id="autoid-2" ></a >

<p>
<a id="LWR-ht-introduction"></a>%
</p>
```

```
<!--..............................-->
<h4 id="autosec-12"><span class="sectionnumber">1&#x2003;</span>Introduction</h4>
<a id="whitepaper-autopage-12"></a>


<a id="introduction"></a>

<p>
Blockchains are complex systems and each component affects the system in a number of different ways. When designing a blockchain network, we must therefore carefully consider these components and their interactions, and the context that the chain will operate in.
These choices are encoded in the protocol and the consequences of those choices can last for the entire lifetime of the network.
</p>

<p>
Naturally, when designing systems, some problems are left to be solved later – not everything can or should be done up-front. The most common problem that blockchain designers have left in the ‘solve later’ pile is the long-term <em>scalability</em> of the network. In
turn, this neglect gave rise to a generation of blockchain designs that tried (and failed) to provide a scalable solution without abandoning the other core properties of a blockchain. Existing chains began to adapt, too – Bitcoin introduced SegWit, and Ethereum developed
a plan to transition to a sharded design secured by Proof of Stake (an effort which is still ongoing, 10 years later). Despite all of this, a solution to this problem remains elusive.
</p>

<p>
Before long, this problem was codified as a trilemma by Vitalik Buterin. In this paper, we’ll refer to it as the <em>Blockchain Trilemma</em> or <em>the Trilemma</em>, though it is also known as the <em>Scalability Trilemma</em>. It states that, for blockchain designs,
there is a ‘choose two out of three’-type relationship between (i)&nbsp;security, (ii)&nbsp;scalability and (iii)&nbsp;decentralization (discussed below in section <a href="whitepaper.html#subsec:trilemma">1.1</a>).
</p>

<p>
The shortcomings of popular existing designs which have prioritized security and decentralization are readily apparent thanks to <em>congestion</em>: transactions take longer to be confirmed, fees rise, business is disrupted, and costs explode. When security and
decentralization are prioritized over scalability, there will, inevitably, be a shortage of capacity at some stage.
</p>

<p>
Other combinations of trilemma properties have been tried as well. Secure and scalable “blockchains” have been attempted, but are fragile and centralized. Scalable and decentralized systems exist, but lack the fundamental properties of a blockchain, such as presenting a
single, consistent record of events to all users.
</p>

<p>
Amaroo’s underlying consensus mechanism, <em>Ultra Terminum</em> (UT), is a cooperative cross-chain consensus strategy that addresses the <em>Trilemma</em> and creates cohesion between chains.
</p>

<p>
Compared to existing consensus methods, UT provides <em>equal or better</em> security properties (including Bitcoin’s PoW method and variants, and <em>all</em> PoS variants). This is because UT combines existing consensus methods and, by way of construction, UT
can only <em>add</em> security to these methods.
</p>

<p>
UT is capable of supporting millions of transactions per second with similar chain parameters (like block size) to those of traditional blockchains (like Bitcoin or Eth1<sup>1</sup>).
</p>
```

<div role="note" class="footnotes">

<p>
<sup>1</sup>&nbsp; The term *Eth1* refers to the original Proof of Work Ethereum (pre-merge), and *Eth2* refers to the Sharded Proof of Stake Beacon Chain. The terms were originally adopted to describe the different designs on the Ethereum Roadmap; but they've since been deprecated. They've not been updated in this paper because they are functional for our purposes: to distinguish between PoW Ethereum and the various PoS iterations.
</p>

</div>

```
<!--...............................-->
<h5 id="autosec-14"><span class="sectionnumber">1.1&#x2003;</span>The Blockchain Trilemma</h5>
<a id="whitepaper-autopage-14"></a>


<a id="subsec:trilemma"></a>

<p>
<a href="https://web.archive.org/web/20220606205028/https://eth.wiki/sharding/Sharding-FAQs" target="_blank" >The trilemma</a> is as follows:
</p>


<div
    class="tcolorbox"
    style=" border:  1px solid #404040; background:  #FCD1E9; "
>


<div
    class="tcolorboxtitle"
    style=" border-top:  1px solid #404040; border-bottom:  1px solid #404040; background:  #404040; color:  #FFFFFF; "
>

<p>
TODO [3]
</p>
</div>


<div
    class="tcolorboxupper"
    style=" color:  #000000; "
>

<a id="todoref:3"></a>

<p>
via ZH: confusing using same variable name (\(c\),\(n\)) for multiple things. \(O(c)\) and \(O(n)\) somewhat unclear.
</p>
</div>

</div>

<a id="autoid-3" ></a >

<p>
Given \(c\) (computational resources per node, e.g., computation, bandwidth, storage, etc.), and \(n\) (the size of the network, e.g., transaction throughput, state size, user population, market cap, etc.), blockchain systems have no more than two of these three properties:
</p>

<ul style="list-style-type:none">


<li>
<p>
• Decentralization — the system can operate with participants that have only \(O(c)\) resources (e.g., a laptop, a raspberry pi, a VPS; typical internet bandwidth, storage space, etc.).
</p>


</li>
<li>


<p>
• Security — the system is secure against attackers with up to \(O(n)\) resources.<sup>2</sup>
</p>


</li>
<li>


<p>
• Scalability — the system can process \(O(n)\) transactions, with \(O(n) &gt; O(c)\); this means that, as the network grows, the throughput of the system grows faster than the computational resources required per user.
</p>
</li>
</ul>

<p>

</p>

<p>
This definition of scalability is perhaps problematic. If the network, which has \(O(n)\) TPS demand, becomes bottlenecked by an \(O(c^2)\) scaling method, is the network really scalable? I prefer an alternative definition of scalability: the system can process \(O(n)\) transactions in \(O(1)\) time, i.e., getting a transaction confirmed is neither too expensive nor delayed as \(n\) and/or \(c\) change.<sup>3</sup>
</p>

<p>
Why mention \(O(c^2)\) scaling particularly? Why is that an important breakpoint for scaling configurations? In a word: sharding. The standard method of sharding (or hosting child-chains generally) is to replace transactions with shard-headers in the host-chain. Extra data might also be required. If the host-chain has \(O(c)\) capacity, then it should support<sup>4</sup> \(O(c)\) shards. Each shard has \(O(c)\) capacity also, thus the full system has \(O(c^2)\) capacity.
</p>
```

```html
<div role="note" class="footnotes">

<a id="whitepaper-autopage-15"></a>

<p>
<sup>2</sup>&nbsp; Here, by convention, “up to \(O(n)\) resources” means that the system remains secure against attackers with less than some critical proportion of the network’s block production capacity — e.g., less than \(\nicefrac {1}{2}\) for typical PoW
chains, and less than \(\nicefrac {1}{3}\) for typical PoS chains.
</p>


<p>
<sup>3</sup>&nbsp; This does not mean there is no fee market for transactions, nor that localized congestion will not happen. Rather, we are interested in an equilibrium that provides substantial value and capacity whilst avoiding transactions being as free as sending an
email.
</p>


<p>
<sup>4</sup>&nbsp; Presuming a secure method of sharding is known and in use, and \(O(1)\) load per shard-header.
</p>


</div>


<div
    class="tcolorbox"
    style=" border:  1px solid #404040; background:  #ECECEC; "
>


<div
    class="tcolorboxupper"
    style=" color:  #000000; "
>

<p>
<span role="note" class="marginpar"><b>Aside</b></span> Traditionally, the use of big \(O\) notation when discussing the trilemma has not conformed to the strict mathematical meaning; instead, it is used quite loosely as a way to analyze the scalability of a
blockchain architecture. Naturally, we’ll continue this tradition.
</p>
</div>

</div>
```

```
<!--................................-->
<h6 id="autosec-16"><span class="sectionnumber">1.1.1&#x2003;</span>Core Conflict</h6>
<a id="whitepaper-autopage-16"></a>


<a id="sec:core-conflict"></a>

<figure id="autoid-4" class="figure ">
```

<div class="center">

<p>

<img
    src="./includes/ut/diags/trilemma/core_conflict_sag.png"
    class="inlineimage"
    alt="(image)"
>
</p>

<div class="figurecaption">

Figure 1: A “cloud” diagram of the core conflict of the <i>*Blockchain Trilemma*</i>.<br />
\(A \to B\) can be read “\(A\) so that \(B\)”; and \(A \leftarrow B\): “\(A\) requires \(B\)”.

</div>

</div>

</figure>

<p>
fig. 1 shows a cloud<sup>5</sup> for the core conflict. It reads: *safely increasing capacity* requires that we *stay decentralized*. *Staying decentralized* requires that we *use many chains and small blocks*. *Safely increasing capacity* requires that the network *stays secure*. *Staying secure* requires that we *use big blocks*. Big blocks are the opposite of small blocks, so we have a *conflict*. Additionally: using multiple chains compromises security; and big blocks compromise decentralization.
</p>

<p>
To understand the core conflict, we need to look at the underlying assumptions.
</p>

<p>
Buterin writes<sup>6</sup> in the Ethereum sharding FAQ regarding the *many chains with small blocks* strategy:
</p>

```html
<div role="note" class="footnotes">

<a id="whitepaper-autopage-18"></a>

<p>
<sup>5</sup>&nbsp; Clouds are diagrams of explanatory implications which reveal a conflict between two (apparently) necessary things. For more on clouds, see <a href="https://www.google.com/search?q=ISBN+0-88427-115-3" target="_blank" >It's Not Luck</a> by
Eli Goldratt (1994).
</p>


<p>
<sup>6</sup>&nbsp;See <a href="https://web.archive.org/web/20220606205028/https://eth.wiki/sharding/Sharding-FAQs" target="_blank" ><em>On sharding blockchains FAQs</em></a>.
</p>


</div>


<div
    class="tcolorbox"
    style=" border:  1px solid #404040; background:  #ECECEC; "
>


<div
    class="tcolorboxupper"
    style=" color:  #000000; "
>

<p>
<span role="note" class="marginpar"><b>Quote</b></span>  [&#x2026;] This greatly increases throughput, but at a cost of security: an N-factor increase in throughput using this method necessarily comes with an N-factor decrease in security, as a level of resources
1/N the size of the whole ecosystem will be sufficient to attack any individual chain. [&#x2026;]
</p>
</div>

</div>

<p>
He writes, regarding the <em>big blocks</em> strategy:
</p>


<div
    class="tcolorbox"
    style=" border:  1px solid #404040; background:  #ECECEC; "
>


<div
    class="tcolorboxupper"
    style=" color:  #000000; "
>

<p>
<span role="note" class="marginpar"><b>Quote</b></span>  [&#x2026;] such an approach inevitably has its limits: if one goes too far, then nodes running on consumer hardware will drop out, the network will start to rely exclusively on a very small number of
supercomputers running the blockchain, which can lead to great centralization risk.
</p>
</div>

</div>

<p>
A mistaken way to break the conflict is <em>merged mining</em> (a.k.a. AuxPoW). This method attempts to share security between chains, so that one might be able to have decentralization and security via small blocks and merged mined chains/shards (sometimes called
side-chains). Buterin writes:
</p>


<div
    class="tcolorbox"
    style=" border:  1px solid #404040; background:  #ECECEC; "
>


<div
    class="tcolorboxupper"
    style=" color:  #000000; "
>

<p>
<span role="note" class="marginpar"><b>Quote</b></span>  [&#x2026;] If all miners participate, this theoretically can increase throughput by a factor of N without compromising security. However, this also has the problem that it increases the computational and
storage load on each miner by a factor of N, and so in fact this solution is simply a stealthy form of block size increase.
</p>
</div>

</div>

<p>
fig. <a href="whitepaper.html#fig:trilemma-mm-conflict">2</a> shows the cloud for the merged mining conflict.
</p>

<figure id="autoid-5" class="figure ">
```

<div class="center">

<p>

```
<a href="./includes/ut/diags/trilemma/mm_conflict_sag.png" target="_blank" ><img
    src="./includes/ut/diags/trilemma/mm_conflict_sag.png"
    class="inlineimage"
    alt="(image)"
></a>
</p>
```

<div class="figurecaption">
Figure 2: A cloud showing the scaling conflict of <i>*merged mining*</i>.

</div>

</div>

</figure>

<p>
An underlying assumption here is that maximally sharing security across the network requires miners to maintain a record of all chains and do validation on all those chains. The naive solution (using many chains, mentioned above) conflicts with secure methods of merged mining. It seems like progress is impossible.
</p>

<p>
We have some hints to conditions that might belong to a solution:
</p>

<ul style="list-style-type:none">

<p>
- We can share security between chains/shards and can use small blocks.
</p>

<p>
- We can share security without miners keeping and validating all chains/shards.
</p>

<p>
This is the crux of the problem: how do you construct a network of blockchains (scalable) such that attacking an individual chain is about as difficult as attacking the full network (secure), whilst ensuring that the security of the network does not require validating all chains (decentralized)?
</p>

<p>
Prior Assumptions Here are some prior underlying assumptions that are either common or which I expect to be:
</p>

<ul style="list-style-type:none">

<p>
- Sharing PoW security requires merged mining.
</p>

<p>
- Sharing PoW security requires that chains use the same hashing algorithm.
</p>

<p>
- Multiple (non-merged-mined) PoW chains using the same hashing algorithm means that at least some of those chains are vulnerable to a 51% attack.
</p>

<p>
- Simultaneously securing a network with PoW and PoS is not possible without compromises (like that PoW miners could DoS PoS validators or vice versa).
</p>

<p>
- It is unsafe for miners/validators to build on unvalidated histories (as is done with SPV mining, which *is* unsafe).
</p>


```
<div
    class="tcolorbox"
    style=" border:  1px solid #404040; background:  #ECECEC; "
>

<div
    class="tcolorboxupper"
    style=" color:  #000000; "
>
```

<p>
Aside I call them *assumptions*; some people will likely (and rightly) take issue with that and call them *conclusions* instead. For our purposes there isn't really a difference; I include them here so that I can later show you conditions under which they are all simultaneously false.
</p>
</div>

</div>

```html
<!--................................-->
<h6 id="autosec-20"><span class="sectionnumber">1.1.2&#x2003;</span>Conjecture: A Principle of Scaling</h6>
<a id="whitepaper-autopage-20"></a>


<a id="sec:a-principle-of-scaling"></a>

<p>
A scalable system <em>can</em> have components with complexities worse than \(O(c)\) <em>if and only if</em> those components are not <em>bottlenecks</em>, i.e., <em>constraints</em>. As long as there is <em>excess capacity</em> in those components, the system can
still scale.
</p>
```

```
<!--...............................-->
<h5 id="autosec-21"><span class="sectionnumber">1.2&#x2003;</span>Ultra Terminum</h5>
<a id="whitepaper-autopage-21"></a>



<div
    class="tcolorbox"
    style=" border:  1px solid #404040; background:  #ECECEC; "
>


<div
    class="tcolorboxupper"
    style=" color:  #000000; "
>

<p>
<span role="note" class="marginpar"><b>Quote</b></span> <span role="note" class="marginpar"><b>Quote</b></span> Decoupling the underlying consensus from the state-transition has been informally proposed in private for at least two years—Max Kaye was
a proponent of such a strategy during the very early days of Ethereum.
</p>

<p>
.7ex]0.5pt
</p>
```

<div class="flushright">

<p>
<i>— *Dr. Gavin Wood;* *Polkadot Whitepaper* *(2016), s2.2*</i>
</p>
</div>

</div>

</div>

<p>
In essence, *Ultra Terminum* (UT) is not a method to scale a single blockchain, rather a method to scale a *network* of blockchains.
</p>

<p>
UT differs from existing consensus methods<sup>7</sup> in that it is a *novel modification* to particular components of preexisting consensus algorithms. These modifications allow those consensus algorithms to cooperate. This improves the maximal security and decentralization of the network, whilst also providing a foundation for scalability.
</p>

<p>
At the core of *Ultra Terminum* is a new method for sharing security: *Proof of Reflection* (section 2). This technique (which works *in conjunction* with PoW, PoS, etc) allows the incremental construction of complex blockchain networks with powerful scaling properties. Those blockchains use the technique *Proof of Reflection* to *mutually secure each other*, forming something called *the simplex* (section 3.2). Blockchains at this level are called *simplex-chains* and are able to host *dapp-chains* — chains that are dedicated to a dapp, and they can be application specific (e.g., a DEX) or general (e.g., a chain hosting an instance of the EVM). The vital properties provided by UT allow heterogenous chains to form a cohesive network and cooperate in securing each other. *Proof of Reflection* is how *Ultra Terminum* (excluding nested chains) scales with order \(O(c^2)\), and is the basis for unbounded \(O(n)\) scaling. UT's higher-order scaling configurations, \(O(c^3)\) and \(O(c^4)\), require dapp-chains: chains that are application-specific and which inherit security properties from the foundational structure. As UT is primarily a method of *structuring* a blockchain network, the scalability configurations mentioned herein *do not include* ***layer 2*** *methods*. That means that *layer 2* techniques (e.g., state/payment channels, ZK/optimistic rollups) can be implemented *on top* of UT.
</p>

<p>
UT's novel structure means that confirmation times within UT are *sub-constant* and of order \(O(c^{-1})\) (i.e., the confirmation rate is \(O(c)\)). This means that, as the network grows and \(c\) increases, confirmation times in UT will approach 0. This is an improvement over existing architectures, which are, ideally and at best, \(O(1)\).
</p>

<p>
Practically, there is little difference between a single blockchain and the simplex to the user. *Proof of Reflection* allows each simplex-chain, and each hosted dapp-chain, to do efficient SPV proofs regarding state-entries on other simplex-chains or dapp-chains. This allows for the creation of rich cross-chain functionality, such as the ability to use a single network-level token across all simplex-chains and dapp-chains.
</p>

<p>
Although SPV proofs can be hundreds of bytes long, UT is structured so that their length does not significantly impact scalability. UT does this by prioritizing a clean and simple protocol for simplex-chains, and allowing dapp-chains the freedom to implement whatever state- and transaction-protocols are suitable for the application in question. This means that complex or inefficient data-structures for a dapp-chain's state do not impact the network globally, preventing bottlenecks in foundational components.
</p>

<p>
Finally, a UT simplex can be modified to enable a *tiling of simplexes* (section 6). This technique *removes the upper bound* on a UT network's growth, without sacrificing security of the network.
</p>

<div role="note" class="footnotes">

<p>
<sup>7</sup>&nbsp; Whether *Ultra Terminum* is a consensus *method* or not is not immediately clear. On the one hand: a new combination of methods is still a method. On the other hand: UT provides a way to modify other consensus methods via the fork rule, and UT doesn't provide a way to run a *single* blockchain. This is why I introduced it as a cross-chain consensus *strategy*.
</p>

</div>

```html
<!--...............................-->
<h5 id="autosec-24"><span class="sectionnumber">1.3&#x2003;</span>Paper Roadmap</h5>
<a id="whitepaper-autopage-24"></a>


<p>
The main goals of this paper are to document, explain, discuss, and evaluate UT and its components. Some secondary goals of this paper are to be <em>simple and clear</em> — this is why the paper is written in this voice. When I say “we”, I am referring to <em>you and
I</em>. This document was written to be read.
</p>

<p>
Our journey is structured thus:
</p>
<ul style="list-style-type:none">

<li>
<p>
• section <a href="whitepaper.html#sec:proof-of-reflection">2</a> constructs the main primitive, <em>Proof of Reflection</em>, and covers related topics such as <em>conversion of work</em>.
</p>

</li>
<li>

<p>
• section <a href="whitepaper.html#lateximage-whitepaper-63">3</a> uses <em>Proof of Reflection</em> to construct the <em>Simplex</em>, <em>Ultra Terminum</em> (UT), and its scaling configurations: \(\UT {1}\), \(\UT {2}\) and \(\UT {3}\).
</p>

</li>
<li>

<p>
• section <a href="whitepaper.html#lateximage-whitepaper-63">4</a> covers the practical considerations for UT: the availability of blocks, state transition, protocol variants, confirmation times, the PoR graph, block-DAGs, DoS attacks, NIPoPoWs, simplex security,
cross-chain transactions, expedited transactions, and a possible initial configuration.
</p>

</li>
<li>

<p>
• section <a href="whitepaper.html#lateximage-whitepaper-63">5</a> analyzes the scalability of UT.
</p>

</li>
<li>

<p>
• section <a href="whitepaper.html#lateximage-whitepaper-63">6</a> constructs a <em>Tiling of Simplexes</em> and  <span
    id="lateximage-whitepaper-1"
    class="lateximagesource"
><!--
```

$UT_\aleph$

```
--><img
    src="whitepaper-images/9E017A0D0FF33F393D9613C6722CF454.svg"
    alt="(math image)"
    role="math"
    style="width:2.4025em ; vertical-align:-0.17647em "
    class="lateximage"
></span>, scaling horizontally with \(O(n)\) capacity.
</p>

</li>
<li>

<p>
• section <a href="whitepaper.html#lateximage-whitepaper-63">7</a> briefly summarizes UT’s resistance to attacks.
</p>

</li>
<li>

<p>
• ?? concludes by evaluating all UT variants against the Trilemma.
</p>

</li>
<li>

<p>
• ?? lists the major criticisms of UT with commentary.
</p>
</li>
</ul>
<p>
The appendices contain comparisons to other networks, details on the iterations of the simulation (section <a href="whitepaper.html#lateximage-whitepaper-63">4.10.4</a>), and nonessential protocol analysis.
</p>
```

```html
<!--...............................-->
<h4 id="autosec-27"><span class="sectionnumber">2&#x2003;</span>Proof of Reflection</h4>
<a id="whitepaper-autopage-27"></a>


<a id="sec:proof-of-reflection"></a>

<p>
In this section we will build up to and discuss a new consensus primitive: <em>Proof of Reflection</em> (PoR). This is a proof that one blockchain’s headers have been confirmed by a second chain. Using these proofs, we can share security between these two chains, such
that attacking either chain is as difficult as attacking both.
</p>

<p>
Our starting point is the question: is there anything <em>in principle</em> that prohibits blockchains sharing security? We already have some examples of this (e.g., merged mining), but those existing methods require each miner to opt-in and aren’t general. Can we come
up with a general way to share security between chains?
</p>


<div
    class="tcolorbox"
    style=" border:  1px solid #404040; background:  #ECECEC; "
>


<div
    class="tcolorboxupper"
    style=" color:  #000000; "
>

<p>
<span role="note" class="marginpar"><b>Aside</b></span> A note on the term “miner”: consensus protocols sometimes choose a new name for the role of <em>block producer</em>. For example: “validator”, “baker”, “collator”, etc. In this document, the term “miner”
usually refers to the generic role of <em>block producer</em> in an inclusive sense, rather than specifically to block producers of PoW based-chains.
</p>
</div>

</div>

<p>
Our starting point is the idea of one chain ‘tracking’ another chain via its headers — it’s the sort of thing that would enable a smart contract to do on-chain SPV proof evaluation. This isn’t a new idea: in 2013, I proposed a system which used this method to support rich
cross-chain exchange.<sup>8</sup> I also wrote a simplified implementation of this method for Bitcoin SPV proofs in the very early days of Ethereum<sup>9</sup> — a precursor to the later-successful BTC Relay<sup>10</sup> (although BTC Relay seems to have been
abandoned in late 2017).
</p>

<p>
What should we call an on-chain version of a headers-only chain? Since there is no established term, let’s call it a “projection”, and let’s call the <em>act</em> of one chain creating a projection of another “imaging”.
</p>
```

```
<div role="note" class="footnotes">

<a id="whitepaper-autopage-28"></a>

<p>
<sup>8</sup>&nbsp; see <a href="https://bitcointalk.org/index.php?topic=198032.0" target="_blank" >https://bitcointalk.org/index.php?topic=198032.0</a>, and <a href="https://bitcointalk.org/index.php?topic=598784.0" target="_blank"
>https://bitcointalk.org/index.php?topic=598784.0</a>
</p>

<p>
<sup>9</sup>&nbsp; <a href="\citeCopprChainheadersLink " target="_blank" >\citeCopprChainheadersLink </a> <a
href="https://web.archive.org/web/20240927044057/https://raw.githubusercontent.com/XertroV/coppr/refs/heads/master/chainheaders.py" target="_blank" >[m1]</a>
</p>

<p>
<sup>10</sup>&nbsp; <a href="\citeBTCRelayLink " target="_blank" >\citeBTCRelayLink </a>
</p>


</div>


<div
    class="tcolorbox"
    style=" border:  1px solid #404040; background:  #F2F2F2; "
>


<div
    class="tcolorboxupper"
    style=" color:  #000000; "
>

<p>
<span role="note" class="marginpar"><b>Term</b></span> <b><a href="whitepaper.html#??">Projection</a></b>:   A <em>projection</em> of a chain is its <em>headers-only</em> version that has been recorded and evaluated <em>by a different chain</em>. For
example, <a href="https://github.com/ethereum/btcrelay" target="_blank" >BTC Relay</a> is a smart contract by which Ethereum previously hosted a <em>projection</em> of Bitcoin. The <em>act</em> of one chain creating and maintaining the projection of
another is called <em>imaging</em>.
</p>
</div>

</div>
```

```html
<!--................................-->
<h5 id="autosec-29"><span class="sectionnumber">2.1&#x2003;</span>A Projection of Bitcoin in Ethereum</h5>
<a id="whitepaper-autopage-29"></a>


<p>
It is well understood that an Ethereum smart contract (SC) can <em>image</em> Bitcoin. This results in a <em>projection</em> of Bitcoin that is available to other Ethereum SCs (e.g., to use as the basis for SPV proofs). Since Bitcoin's consensus and PoW algorithm is
simple and has low overhead, implementing the necessary logic as an Ethereum SC is viable. In principle, any chain with a headers-only mode can be imaged in this way.<sup>11</sup>
</p>

<p>
Let's assume that a smart contract like this exists on the Ethereum chain and it is operational.
</p>

<p>
We're going to use some tables and diagrams as we go to show the sequence of events that occur and what data is produced and/or recorded. table <a href="whitepaper.html#tab:eth-images-btc">1</a> shows data and events for both Bitcoin and Ethereum as Bitcoin
blocks are produced and the projection of Bitcoin in Ethereum is maintained. fig. <a href="whitepaper.html#fig:pr-btc-eth-step1">3</a> illustrates this. Note: fig. <a href="whitepaper.html#fig:pr-btc-eth-step1">3</a> includes some variance in Ethereum's block
production rate, similar to what might be observed in a real-world environment, but table <a href="whitepaper.html#tab:eth-images-btc">1</a> does not.
</p>

<p>
After a Bitcoin block (\(\text {BTC}_k\)) is produced, a user (it doesn't matter who) produces a transaction informing the SC of that block's header. Then, an Ethereum miner includes that transaction in a block, which updates the SC. This process repeats when the
next Bitcoin block is mined, and so on.
</p>

<p>
Why would a chain want<sup>12</sup> to include a projection of another chain? The typical answer is to enable proofs of some state or that some transactions occurred on the imaged chain. This enables more complex SCs, e.g., a trustless \(\text {BTC}\leftrightarrow
\text {ETH}\) market.
</p>

<figure id="autoid-6" class="table ">
```

<div class="center">

<div
    class="fminipage"
    style="border: 0pt solid #000000 ; padding:0pt ; "
>

<div class="figurecaption">

Table 1: (Hypothetical) Data and events for both Bitcoin and Ethereum as a projection of Bitcoin in Ethereum is maintained via Bitcoin headers being tracked by an Ethereum SC.

</div>

```
<table>

<tr class="tbrule">
                Time step (~15 s  Bitcoin      Eth
<td class="tdl">increments)       block mined  block mined  Eth block contents  Eth state</td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
</tr>


<tr class="hline">
<td class="tdl">\(\vdots \)  </td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
</tr>


<tr>
<td class="tdl">0  \(k\)  </td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
</tr>


<tr>
<td class="tdl">1   \(j\)  \(\text {BTC}_k\) header  Records \(\text {BTC}_{0 \cdots k}\)</td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
</tr>


<tr>
<td class="tdl">\(\vdots \)  </td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
</tr>


<tr>
<td class="tdl">40  \(k+1\)  </td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
</tr>


<tr>
<td class="tdl">41   \(j+40\)  \(\text {BTC}_{k+1}\) header  Records \(\text {BTC}_{0 \cdots k+1}\)</td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
</tr>


<tr>
<td class="tdl">\(\vdots \)  </td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
</tr>


<tr class="tbrule">
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
</tr>

</table>

</div>

</div>

</figure>

<figure id="autoid-7" class="figure ">
```

```
<div class="center">

<p>

<a href="./includes/ut/diags/pow_refl_btc_eth_step1_sag.png" target="_blank" ><img
    src="./includes/ut/diags/pow_refl_btc_eth_step1_sag.png"
    style="
    height:228pt ;
    "
    class="inlineimage"
    alt="(image)"
></a>
</p>


<div class="figurecaption">
```

Figure 3: (Hypothetical) Bitcoin headers, as they are produced, are included in Ethereum’s state via a smart contract and user made transactions. This is roughly how BTC Relay worked.

```
</div>

<a id="fig:pr-btc-eth-step1"></a>

</div>

</figure>
```

```
<div role="note" class="footnotes">

<a id="whitepaper-autopage-33"></a>

<p>
<sup>11</sup>&nbsp; Practically, there can be constraints that prevent this. For example: Ethereum typically doesn’t support memory hard hashes, so hosting projections of chains like Litecoin on Ethereum is potentially impossible.
</p>

<p>
<sup>12</sup>&nbsp; Obviously, blockchains don’t have desires. We’ll personify them as a short-cut, which here means approximately: the users / community / developers of a blockchain have some common goal or incentive to do a particular thing.
</p>


</div>
```

```
<!--...............................-->
<h5 id="autosec-34"><span class="sectionnumber">2.2&#x2003;</span>Incrementally Implementing <em>Proof of Reflection</em></h5>
<a id="whitepaper-autopage-34"></a>


<a id="sec:two-blockchains"></a>

<p>
Let’s consider two hypothetical and distinct blockchains — chain  <span
    id="lateximage-whitepaper-2"
    class="lateximagesource"
><!--
```

L

```
--><img
    src="whitepaper-images/F5366111D1ABB083D5E37BFEB27668DC.svg"
    alt="(math image)"
    role="math"
    style="width:0.829em"
    class="lateximage"
></span>and chain <span
    id="lateximage-whitepaper-3"
    class="lateximagesource"
><!--
```

R

--><img
    src="whitepaper-images/586A7FCE0B3B0FD0E1039D04F1D7F6ED.svg"
    alt="(math image)"
    role="math"
    style="width:0.93071em"
    class="lateximage"
>. To begin with, you can imagine these as Bitcoin and Ethereum (before migrating to PoS). However, the changes that are required by *Proof of Reflection* would require major changes to an existing chain's consensus mechanism, so it's unlikely that changes like these would ever be integrated.
</p>

<p>
Our starting case for <span
    id="lateximage-whitepaper-4"
    class="lateximagesource"
><!--

L

```
--><img
    src="whitepaper-images/F5366111D1ABB083D5E37BFEB27668DC.svg"
    alt="(math image)"
    role="math"
    style="width:0.829em"
    class="lateximage"
></span>and <span
    id="lateximage-whitepaper-5"
    class="lateximagesource"
><!--
```

R

```
--><img
    src="whitepaper-images/586A7FCE0B3B0FD0E1039D04F1D7F6ED.svg"
    alt="(math image)"
    role="math"
    style="width:0.93071em"
    class="lateximage"
></span>is that both use Nakamoto consensus with the same Proof of Work algorithm. For simplicity, both chains also have identical block times and we won’t consider block production variance.
</p>
```

```
<!--................................-->
<h6 id="autosec-43"><span class="sectionnumber">2.2.1&#x2003;</span>Step 1. Chain  <span
    id="lateximage-whitepaper-6"
    class="lateximagesource"
><!--
```

R

```
--><img
    src="whitepaper-images/586A7FCE0B3B0FD0E1039D04F1D7F6ED.svg"
    alt="(math image)"
    role="math"
    style="width:0.93071em"
    class="lateximage"
></span>Images Chain <span
    id="lateximage-whitepaper-7"
    class="lateximagesource"
><!--
```

L

```
--><img
    src="whitepaper-images/F5366111D1ABB083D5E37BFEB27668DC.svg"
    alt="(math image)"
    role="math"
    style="width:0.829em"
    class="lateximage"
></span></h6>
<a id="whitepaper-autopage-43"></a><a id="whitepaper-autopage-47"></a>


<p>
This is similar to Ethereum imaging Bitcoin — see fig. <a href="whitepaper.html#fig:pow_refl_step1">4</a>. (We'll omit the table this time since it's basically the same as table <a href="whitepaper.html#tab:eth-images-btc">1</a>.)
</p>

<p>
Like before,  <span
    id="lateximage-whitepaper-8"
    class="lateximagesource"
><!--
```

R

```
--><img
    src="whitepaper-images/586A7FCE0B3B0FD0E1039D04F1D7F6ED.svg"
    alt="(math image)"
    role="math"
    style="width:0.93071em"
    class="lateximage"
></span>will include  <span
    id="lateximage-whitepaper-9"
    class="lateximagesource"
><!--
```

L

```
--><img
    src="whitepaper-images/F5366111D1ABB083D5E37BFEB27668DC.svg"
    alt="(math image)"
    role="math"
    style="width:0.829em"
    class="lateximage"
></span>'s headers as they are produced. Unlike before, let's assume this is a <em>protocol-level</em> implementation; so <span
    id="lateximage-whitepaper-10"
    class="lateximagesource"
><!--
```

R

```
--><img
    src="whitepaper-images/586A7FCE0B3B0FD0E1039D04F1D7F6ED.svg"
    alt="(math image)"
    role="math"
    style="width:0.93071em"
    class="lateximage"
></span>miners can include  <span
    id="lateximage-whitepaper-11"
    class="lateximagesource"
><!--
```

L

```
--><img
    src="whitepaper-images/F5366111D1ABB083D5E37BFEB27668DC.svg"
    alt="(math image)"
    role="math"
    style="width:0.829em"
    class="lateximage"
></span>headers directly with no transaction fees.
</p>

<figure id="autoid-8" class="figure ">
```

```
<div class="center">
```

```
<p>
```

```
<a href="./includes/ut/diags/pow_refl_step1_sag.png" target="_blank" ><img
    src="./includes/ut/diags/pow_refl_step1_sag.png"
    style="
    height:150pt ;
    "
    class="inlineimage"
    alt="(image)"
></a>
</p>
```

```
<div class="figurecaption">
```

Figure 4: Step 1. Chain R images Chain L; thus Chain R hosts a `<em>`*projection*`</em>` of Chain L.

```
</div>
```

```
<a id="fig:pow_refl_step1"></a>
```

```
</div>
```

```
</figure>
```

```
<!--................................-->
<h6 id="autosec-57"><span class="sectionnumber">2.2.2&#x2003;</span>Step 2.  <span
    id="lateximage-whitepaper-12"
    class="lateximagesource"
><!--
```

L

```
--><img
    src="whitepaper-images/F5366111D1ABB083D5E37BFEB27668DC.svg"
    alt="(math image)"
    role="math"
    style="width:0.829em"
    class="lateximage"
></span>Images <span
    id="lateximage-whitepaper-13"
    class="lateximagesource"
><!--
```

R

```
--><img
    src="whitepaper-images/586A7FCE0B3B0FD0E1039D04F1D7F6ED.svg"
    alt="(math image)"
    role="math"
    style="width:0.93071em"
    class="lateximage"
></span></h6>
<a id="whitepaper-autopage-57"></a><a id="whitepaper-autopage-61"></a>


<p>
Similar to <span
    id="lateximage-whitepaper-14"
    class="lateximagesource"
><!--
```

R

```
--><img
    src="whitepaper-images/586A7FCE0B3B0FD0E1039D04F1D7F6ED.svg"
    alt="(math image)"
    role="math"
    style="width:0.93071em"
    class="lateximage"
></span>, <span
    id="lateximage-whitepaper-15"
    class="lateximagesource"
><!--
```

L

```
--><img
    src="whitepaper-images/F5366111D1ABB083D5E37BFEB27668DC.svg"
    alt="(math image)"
    role="math"
    style="width:0.829em"
    class="lateximage"
></span>will begin including <span
    id="lateximage-whitepaper-16"
    class="lateximagesource"
><!--
```

R

```
--><img
    src="whitepaper-images/586A7FCE0B3B0FD0E1039D04F1D7F6ED.svg"
    alt="(math image)"
    role="math"
    style="width:0.93071em"
    class="lateximage"
></span>headers in each block via appropriate protocol-level changes. Just as  <span
    id="lateximage-whitepaper-17"
    class="lateximagesource"
><!--
```

R

```
--><img
    src="whitepaper-images/586A7FCE0B3B0FD0E1039D04F1D7F6ED.svg"
    alt="(math image)"
    role="math"
    style="width:0.93071em"
    class="lateximage"
></span>hosts a projection of <span
    id="lateximage-whitepaper-18"
    class="lateximagesource"
><!--
```

L

```
--><img
    src="whitepaper-images/F5366111D1ABB083D5E37BFEB27668DC.svg"
    alt="(math image)"
    role="math"
    style="width:0.829em"
    class="lateximage"
></span>, <span
    id="lateximage-whitepaper-19"
    class="lateximagesource"
><!--
```

L

```
--><img
    src="whitepaper-images/F5366111D1ABB083D5E37BFEB27668DC.svg"
    alt="(math image)"
    role="math"
    style="width:0.829em"
    class="lateximage"
></span>now hosts a projection of <span
    id="lateximage-whitepaper-20"
    class="lateximagesource"
><!--
```

R

```
--><img
    src="whitepaper-images/586A7FCE0B3B0FD0E1039D04F1D7F6ED.svg"
    alt="(math image)"
    role="math"
    style="width:0.93071em"
    class="lateximage"
></span>. This is shown in fig. <a href="whitepaper.html#fig:pow_refl_step2">5</a> and table <a href="whitepaper.html#tab:por-step-2">2</a>.
</p>

<figure id="autoid-9" class="table ">
```

<div class="center">

<div class="figurecaption">
Table 2: Step 2. Both Chain L and Chain R host a projection of each other.

</div>


```
<table>

<tr class="tbrule">
                    L block  L block              R block  R block
<td class="tdl">Time  made     contents  L state  made     contents  R state</td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
</tr>


<tr class="hline">
<td class="tdl">\(\vdots \)        </td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
</tr>


<tr>
<td class="tdl">0  \(k\)  \(R_{j-1}\) header  Records \(R_{0 \cdots j-1}\)    </td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
</tr>


<tr>
<td class="tdl">1      \(j\)  \(L_{k}\) header  Records \(L_{0 \cdots k}\)</td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
</tr>


<tr>
<td class="tdl">2  \(k + 1\)  \(R_{j}\) header  Records \(R_{0 \cdots j}\)    </td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
</tr>


<tr>
<td class="tdl">3      \(j + 1\)  \(L_{k+1}\) header  Records \(L_{0 \cdots k+1}\)</td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
</tr>


<tr>
<td class="tdl">\(\vdots \)        </td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
</tr>


<tr class="tbrule">
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
<td class="tdl"></td>
</tr>

</table>

</div>

</figure>

<figure id="autoid-10" class="figure ">
```

```
<div class="center">

<p>

<a href="./includes/ut/diags/pow_refl_step2_sag.png" target="_blank" ><img
    src="./includes/ut/diags/pow_refl_step2_sag.png"
    class="inlineimage"
    alt="(image)"
></a>
</p>


<div class="figurecaption">
```

Figure 5: Step 2. Chain L and Chain R contain a projection of each other's headers-only chain.

```
</div>

<a id="fig:pow_refl_step2"></a>

</div>

</figure>
```

```
<!--................................-->
<h6 id="autosec-79"><span class="sectionnumber">2.2.3&#x2003;</span>Step 3. A Reflection of  <span
    id="lateximage-whitepaper-21"
    class="lateximagesource"
><!--
```

L

```
--><img
    src="whitepaper-images/F5366111D1ABB083D5E37BFEB27668DC.svg"
    alt="(math image)"
    role="math"
    style="width:0.829em"
    class="lateximage"
></span>in <span
    id="lateximage-whitepaper-22"
    class="lateximagesource"
><!--
```

R

```
--><img
    src="whitepaper-images/586A7FCE0B3B0FD0E1039D04F1D7F6ED.svg"
    alt="(math image)"
    role="math"
    style="width:0.93071em"
    class="lateximage"
></span></h6>
<a id="whitepaper-autopage-79"></a><a id="whitepaper-autopage-83"></a>


<p>
Can we use a projection of a chain for a different purpose? What happens if Chain  <span
    id="lateximage-whitepaper-23"
    class="lateximagesource"
><!--
```

L

```
--><img
    src="whitepaper-images/F5366111D1ABB083D5E37BFEB27668DC.svg"
    alt="(math image)"
    role="math"
    style="width:0.829em"
    class="lateximage"
></span>tracks whether Chain  <span
    id="lateximage-whitepaper-24"
    class="lateximagesource"
><!--
```

L

```
--><img
    src="whitepaper-images/F5366111D1ABB083D5E37BFEB27668DC.svg"
    alt="(math image)"
    role="math"
    style="width:0.829em"
    class="lateximage"
></span>'s history is confirmed within Chain <span
    id="lateximage-whitepaper-25"
    class="lateximagesource"
><!--
```

R

```
--><img
    src="whitepaper-images/586A7FCE0B3B0FD0E1039D04F1D7F6ED.svg"
    alt="(math image)"
    role="math"
    style="width:0.93071em"
    class="lateximage"
></span>? This can be done via merkle branches<sup>13</sup><a id="whitepaper-autopage-89"></a> that prove Chain  <span
    id="lateximage-whitepaper-26"
    class="lateximagesource"
><!--
```

R

```
--><img
    src="whitepaper-images/586A7FCE0B3B0FD0E1039D04F1D7F6ED.svg"
    alt="(math image)"
    role="math"
    style="width:0.93071em"
    class="lateximage"
></span>'s relevant state. In essence, Chain <span
    id="lateximage-whitepaper-27"
    class="lateximagesource"
><!--
```

L

```
--><img
    src="whitepaper-images/F5366111D1ABB083D5E37BFEB27668DC.svg"
    alt="(math image)"
    role="math"
    style="width:0.829em"
    class="lateximage"
></span>uses its projection of Chain <span
    id="lateximage-whitepaper-28"
    class="lateximagesource"
><!--
```

R

```
--><img
    src="whitepaper-images/586A7FCE0B3B0FD0E1039D04F1D7F6ED.svg"
    alt="(math image)"
    role="math"
    style="width:0.93071em"
    class="lateximage"
></span>to prove that its <em>own history</em> matches that of its <em>projection</em> in Chain  <span
    id="lateximage-whitepaper-29"
    class="lateximagesource"
><!--
```

R

```
--><img
    src="whitepaper-images/586A7FCE0B3B0FD0E1039D04F1D7F6ED.svg"
    alt="(math image)"
    role="math"
    style="width:0.93071em"
    class="lateximage"
></span>. Chain <span
    id="lateximage-whitepaper-30"
    class="lateximagesource"
><!--
```

L

```
--><img
    src="whitepaper-images/F5366111D1ABB083D5E37BFEB27668DC.svg"
    alt="(math image)"
    role="math"
    style="width:0.829em"
    class="lateximage"
></span>proves that it is <em>reflected</em> in Chain  <span
    id="lateximage-whitepaper-31"
    class="lateximagesource"
><!--
```

R

```
--><img
    src="whitepaper-images/586A7FCE0B3B0FD0E1039D04F1D7F6ED.svg"
    alt="(math image)"
    role="math"
    style="width:0.93071em"
    class="lateximage"
></span>.
</p>

<p>
What does this proof look like? The following progression is shown in fig. <a href="whitepaper.html#fig:por-step3-parts">6</a>. First, Chain  <span
    id="lateximage-whitepaper-32"
    class="lateximagesource"
><!--
```

L

```
--><img
    src="whitepaper-images/F5366111D1ABB083D5E37BFEB27668DC.svg"
    alt="(math image)"
    role="math"
    style="width:0.829em"
    class="lateximage"
></span>must prove that its history is reflected, so we first find the most recently reflected header, \(L_{i+1}\) (ideally, this is the previous <span
    id="lateximage-whitepaper-33"
    class="lateximagesource"
><!--
```

L

```
--><img
    src="whitepaper-images/F5366111D1ABB083D5E37BFEB27668DC.svg"
    alt="(math image)"
    role="math"
    style="width:0.829em"
    class="lateximage"
></span>block). Secondly, we want to prove that \(L_{i+1}\) is also the <em>best block</em> (for Chain  <span
    id="lateximage-whitepaper-34"
    class="lateximagesource"
><!--
```

L

```
--><img
    src="whitepaper-images/F5366111D1ABB083D5E37BFEB27668DC.svg"
    alt="(math image)"
    role="math"
    style="width:0.829em"
    class="lateximage"
></span>) according to <em>Chain <span
    id="lateximage-whitepaper-35"
    class="lateximagesource"
><!--
```

R

```
--><img
    src="whitepaper-images/9211226889D6D3F2C5CA2BF0518EB270.svg"
    alt="(math image)"
    role="math"
    style="width:0.93071em"
    class="lateximage"
></span>’s</em> projection of Chain  <span
    id="lateximage-whitepaper-36"
    class="lateximagesource"
><!--
```

L

```
--><img
    src="whitepaper-images/F5366111D1ABB083D5E37BFEB27668DC.svg"
    alt="(math image)"
    role="math"
    style="width:0.829em"
    class="lateximage"
></span>, using the best known <span
    id="lateximage-whitepaper-37"
    class="lateximagesource"
><!--
```

R

```
--><img
    src="whitepaper-images/586A7FCE0B3B0FD0E1039D04F1D7F6ED.svg"
    alt="(math image)"
    role="math"
    style="width:0.93071em"
    class="lateximage"
```

`></span>`block, `\(R_{j+1}\)`. For that, we need a merkle branch showing `\(L_{i+1}\)` is part of `\(R_{j+1}\)`'s state — this is sometimes referred to (in this paper) as the `<em>`*missing*`</em>` merkle branch. Thirdly, we want to prove that `\(R_{j+1}\)` is the `<em>`*best block*`</em>` according to Chain `<span`

```
    id="lateximage-whitepaper-38"
    class="lateximagesource"
><!--
```

L

```
--><img
    src="whitepaper-images/F5366111D1ABB083D5E37BFEB27668DC.svg"
    alt="(math image)"
    role="math"
    style="width:0.829em"
    class="lateximage"
></span>'s projection of Chain <span
    id="lateximage-whitepaper-39"
    class="lateximagesource"
><!--
```

R

```
--><img
    src="whitepaper-images/586A7FCE0B3B0FD0E1039D04F1D7F6ED.svg"
    alt="(math image)"
    role="math"
    style="width:0.93071em"
    class="lateximage"
></span>. We can do that via a merkle branch, too, but full nodes of Chain  <span
    id="lateximage-whitepaper-40"
    class="lateximagesource"
><!--
```

L

```
--><img
    src="whitepaper-images/F5366111D1ABB083D5E37BFEB27668DC.svg"
    alt="(math image)"
    role="math"
    style="width:0.829em"
    class="lateximage"
></span>already know whether \(R_{j+1}\) is the best block or not, so this branch doesn't need to be explicit. However, <span
    id="lateximage-whitepaper-41"
    class="lateximagesource"
><!--
```

L

--><img
    src="whitepaper-images/F5366111D1ABB083D5E37BFEB27668DC.svg"
    alt="(math image)"
    role="math"
    style="width:0.829em"
    class="lateximage"
>'s nodes must be able to generate it. The *full* collection of information required to prove reflection is called a *Proof of Reflection* (PoR).
</p>

<figure id="autoid-11" class="figure ">

```
<div class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; ">

<a id="whitepaper-autopage-122"></a>

<p>

<a href="pow_refl_step3_1_sag" target="_blank" ><img
    src="pow_refl_step3_1_sag"
    style="
    width:128pt;
    "
    class="inlineimage"
    alt="(image)"
></a>
</p>


<div class="figurecaption">
(a) Find the most recently reflected  <span
    id="lateximage-whitepaper-42"
    class="lateximagesource"
><!--
```

L

```
--><img
    src="whitepaper-images/F5366111D1ABB083D5E37BFEB27668DC.svg"
    alt="(math image)"
    role="math"
    style="width:0.829em"
    class="lateximage"
></span>block.

</div>

<a id="fig:por_step3-part1"></a>

<a id="sub@fig:por_step3-part1"></a>

</div>

&#x2003;

&#x2003;
```

```
<div class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; ">

<a id="whitepaper-autopage-125"></a>

<p>

<a href="pow_refl_step3_2_sag" target="_blank" ><img
    src="pow_refl_step3_2_sag"
    style="
    width:128pt;
    "
    class="inlineimage"
    alt="(image)"
></a>
</p>


<div class="figurecaption">
(b) Prove that block is known to the most recently reflected  <span
    id="lateximage-whitepaper-43"
    class="lateximagesource"
><!--
```

R

```
--><img
    src="whitepaper-images/586A7FCE0B3B0FD0E1039D04F1D7F6ED.svg"
    alt="(math image)"
    role="math"
    style="width:0.93071em"
    class="lateximage"
></span>block.

</div>

<a id="fig:por_step3-part2"></a>

<a id="sub@fig:por_step3-part2"></a>

</div>

&#x2003;

&#x2003;
```

```
<div class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; ">

<a id="whitepaper-autopage-128"></a>

<p>

<a href="pow_refl_step3_3_sag" target="_blank" ><img
    src="pow_refl_step3_3_sag"
    style="
    width:128pt;
    "
    class="inlineimage"
    alt="(image)"
></a>
</p>


<div class="figurecaption">
(c) Prove that  <span
    id="lateximage-whitepaper-44"
    class="lateximagesource"
><!--
```

R

```
--><img
    src="whitepaper-images/586A7FCE0B3B0FD0E1039D04F1D7F6ED.svg"
    alt="(math image)"
    role="math"
    style="width:0.93071em"
    class="lateximage"
></span>block is known to the current <span
    id="lateximage-whitepaper-45"
    class="lateximagesource"
><!--
```

L

```html
--><img
    src="whitepaper-images/F5366111D1ABB083D5E37BFEB27668DC.svg"
    alt="(math image)"
    role="math"
    style="width:0.829em"
    class="lateximage"
></span>block.

</div>

<a id="fig:por_step3-part3"></a>

<a id="sub@fig:por_step3-part3"></a>

</div>


<div class="figurecaption">
Figure 6: Incrementally constructing a <em>proof of reflection</em> (PoR).

</div>

<a id="fig:por-step3-parts"></a>

</figure>

<p>
Segments of Chain  <span
    id="lateximage-whitepaper-46"
    class="lateximagesource"
><!--
```

L

```
--><img
    src="whitepaper-images/F5366111D1ABB083D5E37BFEB27668DC.svg"
    alt="(math image)"
    role="math"
    style="width:0.829em"
    class="lateximage"
></span>and <span
    id="lateximage-whitepaper-47"
    class="lateximagesource"
><!--
```

R

```
--><img
    src="whitepaper-images/586A7FCE0B3B0FD0E1039D04F1D7F6ED.svg"
    alt="(math image)"
    role="math"
    style="width:0.93071em"
    class="lateximage"
></span>(events and data) are shown in table <a href="whitepaper.html#tab:por-step-3">3</a> and fig. <a href="whitepaper.html#lateximage-whitepaper-63">7</a>.
</p>

<p>
Chain  <span
    id="lateximage-whitepaper-48"
    class="lateximagesource"
><!--
```

L

```
--><img
    src="whitepaper-images/F5366111D1ABB083D5E37BFEB27668DC.svg"
    alt="(math image)"
    role="math"
    style="width:0.829em"
    class="lateximage"
></span>now knows <em>which  <span
    id="lateximage-whitepaper-49"
    class="lateximagesource"
><!--
```

L

```
--><img
    src="whitepaper-images/4520F3C81B83B796FCAD32865797159F.svg"
    alt="(math image)"
    role="math"
    style="width:0.829em"
    class="lateximage"
></span>blocks are recorded by Chain  <span
    id="lateximage-whitepaper-50"
    class="lateximagesource"
><!--
```

R

```
--><img
    src="whitepaper-images/9211226889D6D3F2C5CA2BF0518EB270.svg"
    alt="(math image)"
    role="math"
    style="width:0.93071em"
    class="lateximage"
></span></em>, i.e., which local blocks are known about by some external source. Put another way: Chain  <span
    id="lateximage-whitepaper-51"
    class="lateximagesource"
><!--
```

L

```
--><img
    src="whitepaper-images/F5366111D1ABB083D5E37BFEB27668DC.svg"
    alt="(math image)"
    role="math"
    style="width:0.829em"
    class="lateximage"
></span>'s history is confirmed <em>not only</em> by new Chain  <span
    id="lateximage-whitepaper-52"
    class="lateximagesource"
><!--
```

L

```
--><img
    src="whitepaper-images/F5366111D1ABB083D5E37BFEB27668DC.svg"
    alt="(math image)"
    role="math"
    style="width:0.829em"
    class="lateximage"
></span>blocks, <em>but also</em> by Chain  <span
    id="lateximage-whitepaper-53"
    class="lateximagesource"
><!--
```

R

```
--><img
    src="whitepaper-images/586A7FCE0B3B0FD0E1039D04F1D7F6ED.svg"
    alt="(math image)"
    role="math"
    style="width:0.93071em"
    class="lateximage"
></span>blocks.
</p>
```

```html
<div role="note" class="footnotes">

<a id="whitepaper-autopage-149"></a>

<p>
<sup>13</sup>&nbsp; Vector commitments (or verkle branches) can be used, too (this applies to most uses of merkle trees / branches in this paper). For the sake of convenience and simplicity, verkle trees won’t be explicitly mentioned as an alternative unless there is a
specific purpose.
</p>


</div>


<div
    class="tcolorbox"
    style=" border:  1px solid #404040; background:  #ECECEC; "
>


<div
    class="tcolorboxupper"
    style=" color:  #000000; "
>

<p>
<span role="note" class="marginpar"><b>Aside</b></span> <b>Important:</b> Soon, these confirmations will have real and useful meaning. Under the right conditions, an appropriate configuration of <em>Proof of Reflection</em> results in an increase in the
<em>rate</em> that confirmations are acquired. This is the first hint of \(O(c^{-1})\) confirmation time.
</p>
</div>

</div>

<p>
At this point, if an attacker was to publish an alternate, better Chain  <span
    id="lateximage-whitepaper-54"
    class="lateximagesource"
><!--
```

L

```
--><img
    src="whitepaper-images/F5366111D1ABB083D5E37BFEB27668DC.svg"
    alt="(math image)"
    role="math"
    style="width:0.829em"
    class="lateximage"
></span>history, then Chain  <span
    id="lateximage-whitepaper-55"
    class="lateximagesource"
><!--
```

L

```
--><img
    src="whitepaper-images/F5366111D1ABB083D5E37BFEB27668DC.svg"
    alt="(math image)"
    role="math"
    style="width:0.829em"
    class="lateximage"
></span>nodes would reorganize around the <em>new</em> history published by the attacker, and the attacker's block headers would end up being recorded in Chain  <span
    id="lateximage-whitepaper-56"
    class="lateximagesource"
><!--
```

R

```
--><img
    src="whitepaper-images/586A7FCE0B3B0FD0E1039D04F1D7F6ED.svg"
    alt="(math image)"
    role="math"
    style="width:0.93071em"
    class="lateximage"
></span>and causing a reorganization there, too. Currently, this configuration does not add any security to Chain <span
    id="lateximage-whitepaper-57"
    class="lateximagesource"
><!--
```

L

```
--><img
    src="whitepaper-images/F5366111D1ABB083D5E37BFEB27668DC.svg"
    alt="(math image)"
    role="math"
    style="width:0.829em"
    class="lateximage"
></span>.
</p>

<p>
Could we use Chain  <span
    id="lateximage-whitepaper-58"
    class="lateximagesource"
><!--
```

L

```
--><img
    src="whitepaper-images/F5366111D1ABB083D5E37BFEB27668DC.svg"
    alt="(math image)"
    role="math"
    style="width:0.829em"
    class="lateximage"
></span>'s knowledge <em>that its own history is reflected in Chain  <span
    id="lateximage-whitepaper-59"
    class="lateximagesource"
><!--
```

R

```
--><img
    src="whitepaper-images/9211226889D6D3F2C5CA2BF0518EB270.svg"
    alt="(math image)"
    role="math"
    style="width:0.93071em"
    class="lateximage"
></span></em> to <em>prevent</em> such an attack?
</p>

<figure id="autoid-12" class="table ">
```

```
<div class="center">


<div
    class="fminipage"
    style="border:  0pt solid #000000 ; padding:0pt ; "
>


<div class="figurecaption">
Table 3: <a id="tab:por-step-3"></a> Step 3. Chain  <span
    id="lateximage-whitepaper-60"
    class="lateximagesource"
><!--
```

L

```
--><img
    src="whitepaper-images/F5366111D1ABB083D5E37BFEB27668DC.svg"
    alt="(math image)"
    role="math"
    style="width:0.829em"
    class="lateximage"
></span>records which of its headers are known about by Chain  <span
    id="lateximage-whitepaper-61"
    class="lateximagesource"
><!--
```

R

```
--><img
    src="whitepaper-images/586A7FCE0B3B0FD0E1039D04F1D7F6ED.svg"
    alt="(math image)"
    role="math"
    style="width:0.93071em"
    class="lateximage"
></span>. That is: Chain  <span
    id="lateximage-whitepaper-62"
    class="lateximagesource"
><!--
```

L

```
--><img
    src="whitepaper-images/F5366111D1ABB083D5E37BFEB27668DC.svg"
    alt="(math image)"
    role="math"
    style="width:0.829em"
    class="lateximage"
></span>includes <em>proofs of reflection</em>. Note: “Headers” is abbreviated to “Hdrs”.

</div>
```

```
<table>

<tr class="tbrule">
<td class="tdl">Time
 <span    id="lateximage-whitepaper-63"    class="lateximagesource"><!--
                                                                                                                                block
          Lblock                Rblock   Rblock
mined   contents   Lstate  mined    contents   Rstate</td>
<td class="tdat "></td>
<td class="tdbang "></td>
</tr>


<tr class="hline">
<td class="tdat "></td>
<td class="tdbang "></td>
<td class="td  ">&#x22EE;    </td>
<td class="tdat "></td>
<td class="tdbang "></td>
</tr>


<tr>
<td class="tdat "></td>
<td class="tdbang "></td>
<td class="td  ">0  k  R$_{j-1}$header, < br/ >
L$_{k-1}$PoRHdrs : R$_{0\cdots j-1}$, < br/ >
PoRs : L$_{0\cdots k-1}$ < /td >
< tdclass = "tdat" >< /td >
< tdclass = "tdbang" >< /td >
< /tr >


1     j  L$_k$headerHdrs : L$_{0\cdots k}$ < /td >
< tdclass = "tdat" >< /td >
< tdclass = "tdbang" >< /td >
< /tr >


2  k+1  R$_j$header, < br/ >
L$_k$PoRHdrs : R$_{0\cdots j}$, < br/ >
PoRs : L$_{0\cdots k}$ < /td >
< tdclass = "tdat" >< /td >
< tdclass = "tdbang" >< /td >
< /tr >


3     j+1  L$_{k+1}$headerHdrs : L$_{0\cdots k+1}$ < /td >
< tdclass = "tdat" >< /td >
< tdclass = "tdbang" >< /td >
< /tr >


&#x22EE;    </td>
<td class="tdat "></td>
<td class="tdbang "></td>
</tr>


<tr class="tbrule">
</div> </figure>

<figure id="autoid-13" class="figure ">
```

```
<div class="center">
<a href="./includes/ut/diags/pow_refl_step3_sag.png" target="_blank" ><img
    src="./includes/ut/diags/pow_refl_step3_sag.png"
    class="inlineimage"
    alt="(image)"
></a>
```

`<div class="figurecaption">`Figure 7: Step 3. Chain Lincludes `<i>`*proofs of reflection*`</i>` (PoRs) along with headers. Proofs of Reflection allow Chain $L$to know which of its own blocks are known to Chain R.

`</div> </div></figure>`

`<!--...............................-->`
`<h6 id="autosec-171"><span class="sectionnumber">`2.2.4&#x2003;`</span>`Step 4. One Way Reflection`</h6>`

Before we discuss a change that Chain Lcould make, it is important to note that chain-work done with one hashing algorithm is `<em>`*not generally convertible*`</em>` to ‘equivalent’ work done via another hashing algorithm. For example, there is no meaningful `<em>`*generic*`</em>` answer to the question `<em>`*how many double SHA256*`<sup>`*14*`</sup>` *hashes is one Ethash hash worth?*`</em>`

For the purposes of our hypothetical construction, let’s say that $L$and Rdo `<em>`*equal work over equal time*`</em>`. In the current example, that means that the work required to produce either L$_i or R_j is the same. < em > For the sake of this construction, we'll also presume this relationship doesn't change over time < /em >. Our constant of conversion is thus: 1 < em > R Blocks per L Block < /em >.$

<div role="note" class="footnotes">
<p>

<sup>*14*</sup>*&nbsp;Bitcoin uses Hash(x) = SHA256(SHA256(x)) as its PoW hash.*

</p>

</div>
<div
    class="tcolorbox"
    style=" border: 1px solid #404040; background: #ECECEC; "
>
<div
    class="tcolorboxupper"
    style=" color: #000000; "
> **Aside** We're not that concerned with whether this is a reasonable assumption in the real world or not; right now, we just need a way to convert the work done on each chain into the same units. Methods for converting work are discussed in section 2.3.   </div> </div>

Currently, the Chain Lnetwork chooses the "heaviest" (most worked) chain as its common history. Chain Lcalculates the "weight" of blocks (i.e., how much work went in to them) via an estimation of how many hashes were required — e.g., some number of *double SHA256 hashes*. For the purposes of illustration, let's convert this number to be in terms of *L Blocks* instead of *double SHA256 hashes*. That's easy, since each block is worth 1 *L Block* by definition. We can also measure the work of an Lblock in terms of *R Blocks* (1 *L-Block* = 1 *R-Block* by the constant of conversion above).

How can the network choose the heaviest chain? Well, a traditional blockchain might use a simple recursive function like algorithm 1.

<figure id="autoid-14" class="algorithm ruled">

<div class="figurecaption">Algorithm 1: Vanilla chain-weight algorithm (typical of traditional blockchains).

</div>
<ul style="list-style-type:none">

procedure CHAINWEIGHT(blocks, state)&#x25B7; The weight of a chain

if length(blocks) == 0 **then**

return 0

end **if**

return BLOCKWEIGHT(head(blocks), state) + CHAINWEIGHT(tail(blocks), state)

end **procedure**

procedure BLOCKWEIGHT(block, state) &#x25B7; The weight of an individual block

return LOCALBLOCKWEIGHT(block, state) &#x25B7; NB: defined by the consensus method

end **procedure**

</figure>

Now that we can convert block weights between Lblocks and Rblocks, could L's CHAINWEIGHT algorithm incorporate the idea that Chain Rhad confirmed part of L's history? Could Chain Luse this to thwart some types of attack?

Yes, and we must modify the block-weight calculation so that it accounts for work contributed by Chain R. **??** is such an algorithm. Essentially, additional weight is added to a block when it is *the best block* known to Chain R, i.e., when, according to Chain R, it is at the tip of Chain L. Note that this weight is still added if Chain Rknows of multiple competing chain-tips.

What is the meaning and impact of this change?

The *meaning* of this change is that Chain Lnow incorporates work done on Chain R*into Chain L's own calculation of the heaviest worked chain*.

When a chain does this we say *Chain L(or Chain L's work) is* ***reflected*** *in Chain R*. This technique is what is meant by the term *Proof of Reflection*.

<div
    class="tcolorbox"
    style=" border: 1px solid #404040; background: #F2F2F2; "
>
<div
    class="tcolorboxupper"
    style=" color: #000000; "
> **Term** **Proof of Reflection (PoR)**: The consensus technique whereby a blockchain becomes more difficult to attack by including work done by reflecting blockchains in its *fork rule*. </div> </div>

One particular *impact* of this change is that a doublespend attack on L(e.g., withholding a privately mined chain-segment that reverts a transaction) must now be performed *not only* against Chain L, *but also and simultaneously* against Chain R.

Why? The attacker's privately mined Lblocks *are not known about* by Chain R. Rather, Chain Rknows about the *public* Chain Lhistory *against which the attack competes*. Thus, *either*:

<ul style="list-style-type:none">

- the private chain-segment must contribute more total work to the Chain Lblockchain than the public chain-segment does (*including* the relevant Chain Rchain-segment); *or*

- the attacker must *additionally* produce a private Chain Rchain-segment such that the *total* work of both private chain-segments is greater than the total work of both public chain-segments, and publish both chain-segments simultaneously.

Note that, at this point, there is no benefit to Chain R's security. That's because Chain Risn't 'reading' the reflected work back from Chain L. Thus a doublespend attack against Chain Rhas the expected, non-reflected profile — it isn't more difficult to attack Chain Ryet. However, Chain Rcan take advantage of the reflection. The main requirements are: the inclusion of appropriate proofs of reflection that show known Chain Rblocks according to Chain L, and an update to Chain R's block-weight calculations to account for the reflected work. *Proof of Reflection* doesn't automatically secure both chains; each chain can proactively and independently take advantage of *Proof of Reflection*.

Naturally, if there were a large difference in target block frequencies (e.g., 10 minutes vs 15 seconds) then there would also be a good deal of latency between the points where the higher-frequency chain gains the security benefit from reflected work. For this reason, *Proof of Reflection* is most useful between high frequency chains, or chains of similar frequencies. One downside of this is that shortening the block production frequency requires the inclusion of more block headers. In the scheme of things, this can be somewhat significant but it is not a deal-breaker.

Exactly how one chain can properly account for reflected work requires that we cover how to compare (and convert) that work, and is the topic of section 2.3.

Note that, as the Chain Ltip is gaining reflections from Chain R, miners on Chain $L$are incented to include as many novel Chain Rheaders and PoRs as possible. That's because each new Chain Rheader (with a PoR) will increase the weight of the *ancestors* of the Chain Ldraft block, which helps the draft block compete with other draft Lblocks. This increases the overall chain-weight that the miner is building on, and thus contributes to their block becoming part of the most-worked chain.

<div
    class="tcolorbox"
    style=" border: 1px solid #404040; background: #ECECEC; "
>
<div
    class="tcolorboxupper"
    style=" color: #000000; "
> **Aside** Where do Chain Lminers get PoRs from? There are multiple answers, but one is for miners of Chain $L$to request them from Chain Rnodes — light-client protocols often support this sort of thing. The problem is discussed in section 4. For now, it's okay to assume that PoRs are broadcast alongside headers.   </div> </div>

There are still potential attacks on Chain L. For example: what if an attacker mines a doublespend in private and produces a longer chain-segment than the honest chain? That is, the attacker's segment — *excluding reflections* — is heavier than the honest chain-segment. At this point the attacker can publish their blocks even though the honest chain-segment — *including reflections* — is heavier. Chain Lnodes would *not* reorganize around this new chain-segment, so why would an attacker do this? If the projection of Chain Lin Chain R*does not account for reflections*, then the attacker's chain-segment will appear (to Chain R) to have more work than the honest chain-segment. Thus the *projection* of Lin Rwill reorganize to favor the attacker's chain-segment. If the attacker has more hash power than the honest miners (i.e., q > p<sup>15</sup>) then they might<sup>16</sup> be able to use this reorganization as a foothold — either to launch a traditional 51% attack against L, or to attack SPV verification and light clients.

<div role="note" class="footnotes">
<p>
<sup>15</sup>&nbsp; In Satoshi's Bitcoin: A Peer-to-Peer Electronic Cash System the parameters p and q represent the probability that the next block will be found by an honest node or the attacker, respectively. This convention has been continued in subsequent analysis, e.g., Rosenfeld's <a href="https://web.archive.org/web/20220209100515/https://cloudflare-ipfs.com/ipfs/QmNUWmY94QUievK8ptoxsPyAQUsKvx1cjRyCgPcfmysAVv" target="_blank"
>Analysis of hashrate-based double-spending (Meni Rosenfeld; 2012), and is continued here, also.
</p>

<p>
<sup>16</sup>&nbsp; In a limited case like this, where there are only two chains, there are many options for preventing these sorts of attacks on full nodes. However, in a more general case, where there might be many reflecting chains, we need to deal with the *root cause* of the vulnerability.
</p>

</div>
<div
    class="tcolorbox"
    style=" border: 1px solid #404040; background: #ECECEC; "
>
<div
    class="tcolorboxupper"
    style=" color: #000000; "
> **Aside** Note: Chain $R$is not *required* to actually evaluate Chain L's tip. PoR can still work if Chain Rsimply records every Chain Lheader that it can, and nothing more. However, this increases the complexity of a PoR implementation, and Chain Rusers won't have protocol-level access to a projection of L. Since a correctly-evaluated projection of $L$is useful (for users of either chain), we should solve this problem if we can.   </div> </div>

How can we prevent this kind of attack? The attack is only possible because Chain $R$was *not* accounting for reflected weight — if Chain R's projection of Chain Laccounts for reflections, then this attack is not possible. If Chain Rusers were *required* to run full nodes for both $L$and R, then we've essentially just combined $L$and Rinto one big, overly-complex blockchain — this change would thwart the attack, but it isn't a solution. Instead, we need to ensure that Chain Rcan cheaply and reliably evaluate the weight of L's reflections.

Let's add a field to L's header: the total chain-weight,<sup>17</sup> *including reflections*, of that block. If this value is *always* reliable, then it's trivial to correctly construct L's headers-only chain. With traditional blockchains (like Bitcoin) it's easy to verify the weight of a header, and thus a headers-only chain, because the header's difficulty is *already available* as part of the PoW's payload. In *this* case, though, *additional* data is required — specifically, the proofs of reflection. Full nodes of Chain Lalready verify that the claimed chain-weight is accurate — all the required data is contained in Lblocks — but this doesn't help light clients. We need additional protocol changes to ensure that the *claimed* chain-weight of a header is *always* reliable.

```
<div role="note" class="footnotes">
<p>
<sup>17</sup>&nbsp; Instead of the total chain-weight, the sum of reflected weight works too (these are essentially equivalent).
</p>


</div>
<div
    class="tcolorbox"
    style=" border:  1px solid #404040; background:  #ECECEC; "
>
<div
    class="tcolorboxupper"
    style=" color:  #000000; "
```

> **Aside** One solution is to adopt a design that allows Rnodes to independently calculate or verify L's reflections without evaluating L's state. Provided that Rnodes can calculate any missing merkle branches on demand, this will work. This method has substantial advantages, however, some configurations have considerable overhead. It is discussed in section 4.3 and section 4.4, and analyzed in section 5.7. </div> </div>

We *can*, at least, guarantee that a fraud proof will *always* be possible when a malicious Lblock lies about its total chain-weight. Additionally, other Lminers can detect the lie and link back to the malicious block as an invalid parent alongside the fraud proof. These are useful features, but they're overkill at this point.

For now, let's assume that Rrecords Lheaders *and* the corresponding PoRs, and that Rnodes verify Lheaders' chain weights.

<!--...............................-->
<h6 id="autosec-175">2.2.5&#x2003;Step 5. Mutual Reflection</h6>

The final step in this progression is *mutual reflection* — where both chains image one-another and include the necessary PoRs and modifications to their chain-weight algorithms. This is shown in fig. 8.

<figure id="autoid-15" class="figure ">

```
<div class="center">
<a href="./includes/ut/diags/pow_refl_step5_sag.svg" target="_blank" ><img
    src="./includes/ut/diags/pow_refl_step5_sag.svg"
    style="
    height:228pt ;
    "
    class="inlineimage"
    alt="(image)"
></a>
```

<div class="figurecaption">Figure 8: Step 5. <i>*Proof of Reflection*</i> between two UT Chains, Chain $L$and Chain R

</div> </div></figure>

When two chains (Chain $L$and Chain R) mutually reflect each other, detecting attacks becomes easier. The security of both Chain $L$and $R$are partially dependent on each others' histories (along with their own, of course). If one chain is attacked, where some alternate chain-segment is published, then that chain's nodes will know that those blocks have not been reflected — potentially indicating that the recently-published chain-segment was constructed in private or constructed after the fact.

There are several details that still require discussion, though, such as: *how exactly is weight contributed by a reflecting chain converted to weight in the local chain?* (discussed in section 2.3); and *how can proofs of reflection be calculated without the requirement that miners are full nodes of both chains?* (discussed in section 4). This last question is particularly important for moving beyond mutual reflection between only two chains.

The *essence* of *Proof of Reflection* should now be apparent. *In principle*, we can make blockchains more difficult to attack based on the idea that *blockchains can include a projection of the history of other blockchains (and confirm a chain's history like they do transactions)*. *In principle*, it is possible to increase the security of a blockchain via *reflection* and to increase the security of multiple blockchains via *mutual reflection*.

Remarks Typically, it is insecure for a weaker PoW blockchain to use the same hashing algorithm as a more popular, more heavily mined PoW blockchain. In such a situation, a small proportion of miners on the more popular chain could temporarily divert efforts to perform a doublespend or empty block DoS on the other chain, and thus the weaker chain is plainly insecure. However, if those two chains were using *mutual PoR*, then this kind of attack becomes impossible, and we've found a way for two PoW chains to coexist using the same PoW algorithm.

.7ex]0.5pt<!---->Begin DRAFT<!---->.7ex]0.5pt

.7ex]0.5pt<!---->End DRAFT<!---->.7ex]0.5pt

<!--...............................-->
<h5 id="autosec-177">2.3&#x2003;Comparing Incomparable Proofs of Work</h5>

For *Proof of Reflection* to work, we must alter the *fork rule* (which compares the chain-weight of two candidate best-blocks). We sum block-weights to get the chain-weight. PoW chains use *the expected number of hashes to produce a block* as the measure of *work*.<sup>18</sup> So, for mutually reflecting PoW chains, we need a way to *compare and convert* hashes done on R to hashes done on L (and vice versa).

If we are to convert work between PoW chains, then we *must* be able to convert between L-hashes and R-hashes *in a consistent way*. Consistency, here, means the conversion shouldn't change the outcome of the fork rule; if the fork rule says L$_{i\cdots j,a} > L_{i\cdots j,b} intermsofL - hashes, thenitshouldgivethesameresultfor < em > all < /em > caseswhentheunitsarechangedtoR - hashes.$

Since both weight and hashes can be *summed*, an easy way for this to work is for us to use a *linear* conversion method (i.e., the units are *proportional*, we have some *constant of conversion* for that specific situation, and all units *share an origin*). In effect, for any two values, the *ratio* between linearly converted values is *constant* provided those values are in same units, but independent of *which* units those values are represented in. This doesn't imply that all possible *mid-states* are linearly convertible, though. However, if some mid-states *are* linearly convertible (to and from hashes), then *the fork rule should work for those units, too*.

When it comes to comparing work, we'll need to work with particular factors (or properties) that are intrinsic to all blockchains (though the particular values vary by chain). These are the factors which *naturally come as rates*, even though they're often measured in absolutes. For example: instead of measuring work in raw hashes, we'll use *hashes per block*; instead of measuring *block rewards*, we'll measure *coins per block* and the *inflation rate*. Even though the act of block production is *discrete*, blockchains have a *block target time*; a way of keeping things consistent *over time* (at least in the short term). Since a block represents a discrete amount of work, and has a known, measurable target time, we can sometimes unify the two via the idea of *rate of work*, e.g., *hashes per second*. Making these sorts of conversions up-front (where appropriate) will simplify the work to come, since we'll be working with ratios of units which naturally allow for useful conversion.

<div role="note" class="footnotes">
<p>
<sup>18</sup>&nbsp; Alternatively, something that's *linearly convertible* to number-of-hashes (like difficulty) can be used instead.
</p>

```
</div>
<div
    class="tcolorbox"
    style=" border:  1px solid #404040; background:  #ECECEC; "
>
<div
    class="tcolorboxupper"
    style=" color:  #000000; "
```

> **Aside** When we can measure the *rate of work* in some direct way (e.g., number of hashes per block), we can also *scale block rewards according to both the network difficulty and the demonstrated rate of work*. This enables dynamic block production (i.e., blocks with a flexible minimum difficulty). This would (probably) be disruptive to a traditional blockchain, due to many stale blocks. For UT, it becomes relevant with section 4.8, and particularly with section 4.8.3. UT does not require support for dynamic block production, but it's useful to have as a technique in reserve (perhaps particularly for section 3.4).   </div> </div>

Here's the problem: *universal and generic* conversion between qualitatively different units is impossible.<sup>19</sup> We need a specific *goal* and *context* to successfully convert — in fact, the *goal* is what *enables* us to judge whether it's a successful method of conversion or not.

We only know one specific thing at the moment: our goal is to *convert* chain-work so that *PoR works*. The *conversion* shouldn't introduce any vulnerabilities, and it should work within the constraints that we've discussed up to this point.

In order to discuss *conversion methods* we need to know the *context* within which the conversion happens. We already know some background context, like that we're using blockchains, that root tokens (which are exact and fungible) are involved via block rewards, that nodes have access to certain information via some proof or because it's already in their state, etc. This background context isn't enough, though. What specific contexts can *facilitate* conversion? This section has two examples — section 2.4.1 and section 2.4.2.

Before we discuss the examples themselves, let's discuss some intermediate conversions that we'll use in those two examples.

<div role="note" class="footnotes">
<p>
<sup>19</sup>&nbsp; Attribution: I learned this, its significance for epistemology, and the importance of *goals* and *context* when evaluating *ideas*, from the philosopher Elliot Temple — particularly via his <a
href="https://curi.gumroad.com/l/mhtbA" target="_blank" >Critical Fallibilism Course (Elliot Temple; 2020) (which is a paid course and thus difficult to cite directly). His article <a
href="https://criticalfallibilism.com/multi-factor-decision-making-math" target="_blank" >Multi-Factor Decision Making Math (Elliot Temple; 2021) details many related concepts and is a good starting point.
</p>

</div>

<!--................................-->
<h6 id="autosec-180">2.3.1&#x2003;Theoretical Conversion</h6>

Consider a traditional blockchain (like Bitcoin, or Eth1). We know that traditional blockchains have properties specific to their blocks, like: reward per block (coins/block); a block target time (seconds/block) — or block frequency (blocks/second); and a difficulty (hashes/block). There are also *network-wide* properties, too, like the *inflation rate* (coins/second). The *instantaneous* relationship between these properties is mediated by various protocols — these protocols (e.g., difficulty adjustment algorithms) are part of the *context* of those properties and relationships. How can we use these relationships to our advantage?

```
<div
    class="tcolorbox"
    style=" border:  1px solid #404040; background:  #ECECEC; "
>
<div
    class="tcolorboxupper"
    style=" color:  #000000; "
```

> **Aside** With regards to Proof of Reflection, consider that we *only* need to convert *simultaneous* work. That means: PoR does not need to be able to convert chain-work between chains *over time*, only *for some given moment*.<sup>20</sup>   </div>

`<div role="note" class="footnotes">`
`<p>`
`<sup>`20`</sup>`&nbsp;At this point, we are discussing how PoR works with Proof of Work, discussions of where other methods, such as Proof of Stake, fit into the ecosystem will be discussed in section `<a href="whitepaper.html#lateximage-whitepaper-63">`2.5`</a>`.
`</p>`

`</div> </div>`

The units that we have to work with are: blocks, seconds, hashes, and coins`<sup>`21`</sup>`. There are actually multiple types of blocks (L-blocks and R-blocks), coins (L-coins and R-coins), and hashes (L-hashes and R-hashes). We can't combine those unless we're able to convert those values to common units.

If we ignore some of the normal constraints on consensus algorithms — like where information comes from — what information could help us convert? If we had `<em>`*an exchange rate*`</em>` between L-coins and R-coins, then we can trivially convert between them. If we have that, then, for our current purpose, we can treat L-coins and R-coins as interchangeable units — because we can `<em>`*always*`</em>` convert between them. So now we have L-blocks, R-blocks, coins, and L-hashes and R-hashes.

Let's consider Chain L, and give some of these properties variables: $\mathrm{L}_f(L - blocks/s)forblockfrequency, L_r(L - coins/L - block)fortheblockreward, andL_d(L - hashes/L - block) - - - thedifficulty. Wecanmultiplycombinationsofthesetogetnewunits:$
$L_f \cdot L_d givesusL - hashes/s; L_f \cdot L_r givesL - coins/s, and < spanclass = "textrm\ textmd\ textup\ textulc" >< spanclass = "numerator" > L_d < /span > / < spanclass = "denominator" > L_r < /span >< /span > givesus < b > L - hashes/L - coin < /b >.$

Now, let's add that exchange rate: $\mathrm{X}_{R \to L}(L - coins/R -$
$coin). Let'salsoaddvariablesforChainR, R_f, R_r, andR_d, whichcorrespondtothosewealreadyhaveforChainL. There'sasymmetrybetweenchainsLandR, sowealreadyknowthat < spanclass = "textrm\ textmd\ textup\ textulc" >< spanclass = "numerator" > R_d < /span > / < spanclass = "denominator" > R_r < /span >< /span > givesusR-$
$hashes/R - coin.$

In theory, can an exchange rate help us convert between R-hashes/R-coin and L-hashes/L-coin?

<div role="note" class="footnotes">
<p>
<sup>21</sup>&nbsp; Note that the terms *coin* and *root token* are synonymous. The choice of *coin* over *root token*, for these sections, is for practicality — we'll see this term *a lot*.
</p>

</div>

```
<!--...............................-->
<h6 id="autosec-183"><span class="sectionnumber">2.3.2&#x2003;</span>Converting Block-Weights</h6>
```

Can we find some function, $\text{ConvWork}_{R\rightarrow L}(w), thatconvertsR-hashestoL-hashes? <spanclass="hidden"> \backslash(\backslash seteqnumber\{0\}\{\}\{0\}\backslash) $
<!--

$$\begin{align}
& \frac{L_d}{L_r} && \frac{\text{L-hashes}}{\text{L-coin}} && \nonumber \\
\implies & \frac{L_d}{L_r}\cdot X_{R\rightarrow L} && \frac{\text{L-hashes}}{\text{R-coin}} && \text{Multiply by } X_{R\rightarrow L} \nonumber \\
\implies & \frac{L_d}{L_r}\cdot X_{R\rightarrow L}\cdot\frac{R_r}{R_d} && \frac{\text{L-hashes}}{\text{R-hash}} && \text{Divide by } {}^{R_d}/_{R_r} \tag{1} \\
\therefore \text{ConvWork}_{R\rightarrow L}(w) = & \frac{L_d}{L_r}\cdot X_{R\rightarrow L}\cdot\frac{R_r}{R_d}\cdot w && \text{R-hashes} \rightarrow \text{L-hashes} && \tag{2}
\end{align}$$

-->

```
\begin{align}
&amp; \frac {L_d}{L_r} &amp; &amp;\frac {\text {L-hashes}}{\text {L-coin}} &amp; &amp; \nonumber \\[0.5em] \implies \; &amp; \frac {L_d}{L_r} \cdot X_{R\rightarrow L} &amp; &amp;\frac {\text {L-hashes}}{\text {R-coin}} &amp; &amp;\text
{Multiply by }X_{R\rightarrow L} \nonumber \\[0.5em] \implies \; &amp; \frac {L_d}{L_r} \cdot X_{R\rightarrow L} \cdot \frac {R_r}{R_d} &amp; &amp;\frac {\text {L-hashes}}{\text {R-hash}} &amp; &amp;\text {Divide by }\nicefrac {R_d}{R_r}
\label {eq:por-conversion-const-1} \\[0.5em] \therefore \text {ConvWork}_{R\rightarrow L}(w) = \; &amp; \frac {L_d}{L_r} \cdot X_{R\rightarrow L} \cdot \frac {R_r}{R_d} \cdot w &amp; &amp;\text {R-hashes }\rightarrow \text { L-hashes}
&amp; &amp; \label {eq:por-conv-work}
\end{align}
```

With eq. (1), **we have just found our first constant of conversion for** **block-weight**.

```
<div
    class="tcolorbox"
    style=" border:  1px solid #404040; background:  #ECECEC; "
>
<div
    class="tcolorboxupper"
    style=" color:  #000000; "
```
> **Aside** eq. (1) has a natural symmetry. It's worth noting for later. </div> </div>

What's going on here? We start out by observing L$_d$ $$/L$_r$ $$ gives us a value in units of L-hashes$\overline{}$
$\text{L-coin.}Thisisaconstantofconversionfrom L-coinsto L-hashesforagivenmoment---ifsomeminerearnedxL-coinstoday, thenx\cdot <spanclass="textrm\ textmd\ textup\ textulc"><spanclass="numerator">L_d/<spanclass="denominator">L_rwouldtellyouroughlyhowmanyhashesweredonetoearnthatreward.Next, wemultiplybytheexchangeratetofindth$

Let's sanity check this.

```
<div
    class="tcolorbox"
    style=" border:  1px solid #404040; background:  #F2F2F2; "
>
<div
    class="tcolorboxupper"
    style=" color:  #000000; "
```
> **Term** **Root Token (RT)**: *aka* ***Coin***. The typically sole network-level token required by blockchain protocols. e.g., Bitcoin has BTC, Ethereum has ETH, Polkadot has DOT, Cardano has ADA, Amaroo has ROO, etc. </div> </div>

Consider two blockchains ($L$and R) that are *very* similar to Bitcoin. Unless otherwise specified, the chains are identical. Here are the key assumptions:

<ul style="list-style-type:none">

- Both $L$and Rstarted on the same day, with the same block rewards (in their respective root tokens), block frequencies, and inflation schedules.

- $L$and Rhave equal money supplies, and the exchange rate has been stable at $\text{X}_{R\rightarrow L} = 3L-coins/R-coin$.

- $L$and Ruse different PoW algorithms, Luses something like Scrypt (similar to Litecoin) and Ruses something like SHA256 (similar to Bitcoin).

- ASIC/FPGA mining doesn't exist yet, but GPU mining does.

- (In this thought experiment) the best GPUs for mining Scrypt and SHA256 are of the same brand and model — i.e.&nbsp;the same supply is responsible for the hardware of *all* miners, regardless of which chain they mine.

- There's no comparative advantage between GPU makes/models — i.e., a miner can't increase their revenue by cleverly organizing which GPUs mine which networks.

- The cost of running both $L$and Rnodes is negligible.

- $L$and Rhave perfect difficulty adjustment algorithms.

- The miner(s) used in this thought experiment are small relative to the total population of miners — their choices don't meaningfully impact network hash-rates or difficulty adjustments.

What should we expect regarding the conversion of work? To start with, let's note that GPU miners could work on either chain — good hardware for one chain is good hardware for the other, too. We know that $L$and R's block rewards (in root tokens) and block frequencies are the same — so the exchange rate is going to play a dominant role in RoI (since the only other difference is difficulty and hash-rate). If a miner could break even by making 30 L-coins, then they could also break even by making 10 R-coins. They'd need to make 3$\times asmanyL-coinsasR-coins---that'stheexchangerate.IfLandRusedthesamehashingalgorithm, thenwecouldcomparedifficultiestoseeifthismakessense---doesthatminermake3\times asmanyL-blocksastheywouldR-blocks?$

In this case, though, the difficulties are set for *different hashing algorithms* — so how many hashes can GPUs do for each hash? Say a GPU can do 7 SHA256 hashes for each 1 Scrypt hash. A miner that can do h Scrypt hashes/day should be able to do 7h SHA256 hashes/day. That same miner should be able to make h $\cdot <spanclass="textrm\ textmd\ textup\ textulc"><spanclass="numerator">L_r/<spanclass="denominator">L_dcoinsperday---L'scoinsperblock, dividedbyL'sdifficulty(hashesperblock)givesusaconstantofconversionwithunitsL-coins/L-hash.Ofcourse, theminercould, instead, mineonR, thusmaking7h\cdot <spanclass="textrm\ textmd\ textup\ textulc"><spanclass="numerator">R_r/<spanclass="denominator">R_dcoinsperday.Howdoweknowwhichisbetter?Weusetheexchangerate, ofcourse!$

If miners could swap from their current chain to the other chain and *increase their revenue*, then we should expect some to do that. In turn, we expect each chain's difficulty to change, reflecting that change in participation. If some miners (on the whole) moved from $L$to R, then we'd expect L's difficulty to decrease and R's difficulty to increase, corresponding to how many miners moved. Since this is an *arbitrage opportunity* (for miners), we expect that any profitability gap will quickly be closed. Thus, we can say that a miner's revenue is *equal* regardless of which chain they're mining: $\text{Revenue}_L = \text{Revenue}_R whenmeasuredinthesameunits. <spanclass="hidden"> \backslash(\backslash seteqnumber\{0\}\{\}\{2\}\backslash) $
<!--

$$\begin{align}
\text{Revenue}_L = h\cdot\frac{L_r}{L_d} && \text{L-coins} && \text{Revenue on } L \tag{3} \\
\text{Revenue}_R = 7h\cdot\frac{R_r}{R_d}\cdot X_{R\rightarrow L} && \text{L-coins} && \text{Revenue on } R \text{in L-coins} \tag{4} \\
\text{Revenue}_L = \text{Revenue}_R && \text{L-coins} && \text{The equality we set above} \nonumber \\
\therefore h\cdot\frac{L_r}{L_d} = 7h\cdot\frac{R_r}{R_d}\cdot X_{R\rightarrow L} && \text{L-coins} && \text{eq. (3) and eq. (4)} \nonumber \\
\frac{L_r}{L_d} = 7\cdot\frac{R_r}{R_d}\cdot X_{R\rightarrow L} && \frac{\text{L-coins}}{\text{L-hash}} && \text{Divide by } h \nonumber \\
\frac{R_d}{L_d} = 7\cdot\frac{R_r}{L_r}\cdot X_{R\rightarrow L} && \frac{\text{R-hashes}\cdot\text{L-blocks}}{\text{L-hash}\cdot\text{R-block}} && \text{Multiply by } \frac{R_d}{L_r} \nonumber \\
\frac{R_d}{L_d} = (7\cdot 3) && \frac{\text{R-hashes}\cdot\text{L-blocks}}{\text{L-hash}\cdot\text{R-block}} && \text{Sub } X_{R\rightarrow L},\ R_r, \text{ and } L_r \tag{5} \\
R_d = 21L_d && \frac{\text{R-hashes}}{\text{R-block}} && \text{Multiply by } L_d \tag{6}
\end{align}$$

-->

```
\begin{align}
\text {Revenue}_L &amp; = h \cdot \frac {L_r}{L_d} &amp; &amp; \text {L-coins} &amp; &amp; \text {Revenue on \cL } \label {eq:rev-L} \\[0.5em] \text {Revenue}_R &amp; = 7h \cdot \frac {R_r}{R_d} \cdot X_{R\rightarrow L} &amp; &amp; \text
{L-coins} &amp; &amp; \text {Revenue on \cR in L-coins} \label {eq:rev-R} \\[0.5em] \text {Revenue}_L &amp; = \text {Revenue}_R &amp; &amp; \text {L-coins} &amp; &amp; \text {The equality we set above} \nonumber \\[0.5em] \therefore h
\cdot \frac {L_r}{L_d} &amp; = 7h \cdot \frac {R_r}{R_d} \cdot X_{R\rightarrow L} &amp; &amp; \text {L-coins} &amp; &amp; \text {\autoref {eq:rev-L} and \autoref {eq:rev-R}} \nonumber \\[0.5em] \frac {L_r}{L_d} &amp; = 7 \cdot \frac
{R_r}{R_d} \cdot X_{R\rightarrow L} &amp; &amp; \frac {\text {L-coins}}{\text {L-hash}} &amp; &amp; \text {Divide by }h \nonumber \\[0.5em] \frac {R_d}{L_d} &amp; = 7 \cdot \frac {R_r}{L_r} \cdot X_{R\rightarrow L} &amp; &amp; \frac
{\text {R-hashes}\cdot \text {L-blocks}}{\text {L-hash}\cdot \text {R-block}} &amp; &amp; \text {Multiply by }\frac {R_d}{L_r} \nonumber \\[0.5em] \frac {R_d}{L_d} &amp; = (7 \cdot 3) &amp; &amp; \frac {\text {R-hashes}\cdot \text
{L-blocks}}{\text {L-hash}\cdot \text {R-block}} &amp; &amp; \text {Sub }X_{R\rightarrow L}\text {, }R_r\text {, and }L_r \label {eq:rev-diff-ratio-raw} \\[0.5em] R_d &amp; = 21 L_d &amp; &amp; \frac {\text {R-hashes}}{\text {R-block}}
&amp; &amp; \text {Multiply by }L_d \label {eq:rev-diff-ratio}
\end{align}
```

So, the ratio of *difficulties* should be 21 R-hashes·L-blocks$\overline{}$ $\text{L-hash}\cdot\text{R-block}; or, R'sdifficultyvalueshouldbe21\times L'sdifficultyvalue.$

Why did the substitution of
$\text{X}_{R\rightarrow L}$, R_r, and $L_r(eq. <spanclass="textup">(<ahref="\text{whitepaper.html}\#lateximage-whitepaper-63">5))equal3, though?First, noticethattheunitsdidnotchangewiththatoperation.Next, weknowtheexchangerateX_{R\rightarrow L} =$
$3; wesaidsoearlier.Soitmustbethat<spanclass="textrm\ textmd\ textup\ textulc"><spanclass="numerator">R_r/<spanclass="denominator">L_r =$
$1.Thissimplifyingstepisonlypossiblebecausewebegancalculatingnumericalvalues.WesaidearlierthatLandRhave---numerically---$
$identicalblockrewards, soitmustbethat<spanclass="textrm\ textmd\ textup\ textulc"><spanclass="numerator">R_r/<spanclass="denominator">L_r = 1inthiscase.$

Let's consider eq. (2) in light of the above. \(\seteqnumber{0}{}{6}\)
<!--

$$\begin{align}
\text{ConvWork}_{R\rightarrow L}(w) &= \frac{L_d}{L_r}\cdot X_{R\rightarrow L}\cdot\frac{R_r}{R_d}\cdot w && \text{R-hashes} \rightarrow \text{L-hashes} && \text{eq. (2)} \nonumber \\
&= \frac{1}{21}\cdot\frac{R_r}{L_r}\cdot X_{R\rightarrow L}\cdot w && \text{L-hashes} && \text{Sub } {}^{L_d}/_{R_d} \text{ from eq. (5)} \nonumber \\
&= \frac{3}{21}\cdot w = \frac{w}{7} && \text{L-hashes} && \text{Sub } \frac{R_r}{L_r}\cdot X_{R\rightarrow L} \text{ as before} \nonumber
\end{align}$$

-->

```
\begin{align}
\text {ConvWork}_{R\rightarrow L}(w) = \; &amp; \frac {L_d}{L_r} \cdot X_{R\rightarrow L} \cdot \frac {R_r}{R_d} \cdot w &amp; &amp;\text {R-hashes }\rightarrow \text { L-hashes} &amp; &amp;\text {\autoref {eq:por-conv-work}} \nonumber
\\[0.5em] = \; &amp; \frac {1}{21} \cdot \frac {R_r}{L_r} \cdot X_{R\rightarrow L} \cdot w &amp; &amp;\text {L-hashes} &amp; &amp;\text {Sub }\nicefrac {L_d}{R_d}\text { from \autoref {eq:rev-diff-ratio-raw}} \nonumber \\[0.5em] = \;
&amp; \frac {3}{21} \cdot w = \frac {w}{7} &amp; &amp;\text {L-hashes} &amp; &amp;\text {Sub }\frac {R_r}{L_r} \cdot X_{R\rightarrow L}\text { as before} \nonumber
\end{align}
```

We said this was true earlier — 1 L-hash is worth 7 R-hashes. Thus far, we have not yet found an inconsistency (i.e., we don't yet have a reason to think this won't work).

There is, however, an inconsistency lurking. Consider: \(\seteqnumber{0}{}{6}\)
<!--

$$\begin{align}
&& \frac{R_r}{L_r}\cdot X_{R\rightarrow L} && \frac{\text{L-blocks}}{\text{R-block}} \nonumber \\
&& \frac{L_f}{R_f} && \frac{\text{L-blocks}}{\text{R-block}} \nonumber \\
\text{But!} && \frac{L_f}{R_f} \ne \frac{R_r}{L_r}\cdot X_{R\rightarrow L} && \frac{\text{L-blocks}}{\text{R-block}} \tag{7}
\end{align}$$

-->

```
\begin{align}
&amp; &amp; &amp; \frac {R_r}{L_r} \cdot X_{R\rightarrow L} &amp; &amp;\frac {\text {L-blocks}}{\text {R-block}} \nonumber \\[0.5em] &amp; &amp; &amp; \frac {L_f}{R_f} &amp; &amp;\frac {\text {L-blocks}}{\text {R-block}} \nonumber
\\[0.5em] \text {But!} &amp; &amp; \frac {L_f}{R_f} \ne \; &amp; \frac {R_r}{L_r} \cdot X_{R\rightarrow L} &amp; &amp;\frac {\text {L-blocks}}{\text {R-block}} \label {eq:lfrf-ne-rrlr}
\end{align}
```

These two values are **not** equal (or comparable), and nothing we've said implies that they should be! There are *qualitative differences* between the two that is not represented in the current units. On the one hand, we have something like *relative block frequencies,* and on the other we have something like *a ratio of the* *weight or value* *of block creation*. But they have the same units! What's going on? How do we know whether a constant of conversion *works* for our purposes?

<!--................................-->
<h6 id="autosec-184">2.3.3&#x2003;Hold Up! We Need to Talk About L$_f < /span >/$R$_f < /span >$ and R$_r < /span >/$L$_r < /span >$ $\cdot X_{R\rightarrow L} < /h6 >$

```
<div
    class="tcolorbox"
    style=" border:  1px solid #404040; background:  #ECECEC; "
>
<div
    class="tcolorboxupper"
    style=" color:  #000000; "
```

> **Aside** This section regards some subtle ideas about when conversions work (i.e., give meaningful results), and when conversions don't. It's worth spending some time on these ideas because *when and how* you can convert is not always obvious. But, we *must* understand this to construct a meaningful method of converting block-weight — which PoR *requires*. </div> </div>

Let's consider some units with *real-world* interpretations. What can L-blocks/R-block *mean?*

<ul style="list-style-type:none">

- *Relative block frequencies* or *relative confirmation rates* — This has real-world meaning: Eth1 produces approximately 40 Ethereum-blocks in the same period (measured in seconds) that Bitcoin produces 1 Bitcoin-block.

- *Relative block weights* — This has real-world meaning: how much harder is it to generate a block on one network vs another network?

- *Relative confirmations* — This has real-world meaning: how many confirmations does one network take, compared to another, to reach equivalent security?<sup>22</sup>

Intuitively, *relative block weights* and *relative confirmations* sound related. If blocks on *L* are
5×heavierthanblocksonR,thenwe′dhaveaconstantofconversionof< spanclass = ”textrm textmd textup textulc” >< spanclass = ”numerator” > 1 < /span > / < spanclass = ”denominator” > 5 < /span >< /span >L − blocks/R − block;andachainof5R −
blockswouldbe< em > roughly < /em >ashardtocreateasachainof1L −
block.So< spanclass = ”textrm textmd textup textulc” >< spanclass = ”numerator” > 1 < /span > / < spanclass = ”denominator” > 5 < /span >< /span >seemslikeareasonableestimatefor< em > relativeconfirmations < /em >, too. < sup > 23 < /sup >

Naively, *relative block frequencies* seems to be in the same units as the other two: L-blocks/R-blocks; but they *cannot* be in the same units as *the values mean different things*. Let's consider *relative confirmation* ***rates*** particularly. What happens if we assume that *seconds* on each chain aren't the same thing, i.e., the units of *confirmation rate* are L-blocks/L-second (or R-blocks/R-second)?<sup>24</sup> Crucially, we can *not* cancel the *seconds* anymore: \(\seteqnumber{0}{}{7}\)
<!--

$$? = \frac{L_f}{R_f} \qquad \frac{\text{L-blocks} \cdot \text{R-seconds}}{\text{L-second} \cdot \text{R-block}} \qquad \text{Relative confirmation rates} \tag{8}$$

$$? = \frac{R_r}{L_r} \cdot X_{R\rightarrow L} \qquad \frac{\text{L-blocks}}{\text{R-block}} \cdot \cancelto{1}{\frac{\text{R-coins} \cdot \text{L-coins}}{\text{L-coins} \cdot \text{R-coins}}} \qquad \text{Relative block weights via } X_{R\rightarrow L} \tag{9}$$

-->

```
\begin{align}
?  &amp; = \frac {L_f}{R_f} &amp; &amp; \frac {\text {L-blocks} \cdot \text {R-seconds}}{\text {L-second} \cdot \text {R-block}} &amp; &amp; \text {Relative confirmation rates} \label {eq:rel-conf-hz} \\[0.5em] ?  &amp; = \frac {R_r}{L_r}
\cdot X_{R\rightarrow L} &amp; &amp; \frac {\text {L-blocks}}{\text {R-block}} \cdot \cancelto {1}{\frac {\text {R-coins} \cdot \text {L-coins}}{\text {L-coins} \cdot \text {R-coins}}} &amp; &amp; \text {Relative block weights via }
X_{R\rightarrow L} \label {eq:rel-block-weight}
\end{align}
```

We can see that eq. (8) and eq. (9) are now obviously not comparable.

The reason that L$_f < /span >/$R$_f < /span >$ did not make sense before is that we *were not including all necessary* *context**!* There is *implicit context* in some properties of blockchains — *participation*. Values like L$_f < /span >/$R$_f < /span >$ — when used to measure the *target block frequency* — *do not factor in participation*; the target block time is usually a *constant*, so it can hold no *network-specific context*.

Where does this network-specific context come from? How is it separated from "world" context — like target block frequencies? How is the network-specific context maintained over time? The answer to all three questions is the same: the **Difficulty Adjustment Algorithm** (DAA).

<div role="note" class="footnotes">
<p>
<sup>22</sup>&nbsp; “Equivalent security” means that a doublespend attempt on one network is just as risky, costly, etc, as a doublespend attempt on the other network. To do this comparison, we start by picking some q for the attacker on L, a transaction value (in L-coins), L’s block reward, and then find the boundary of attack-viability (measured in L-confirmations). The boundary of attack-viability is where rules of thumb around confirmation times come from, e.g., *for Bitcoin, a transaction is safe after 6 confirmations.* Next, we consider an *equivalently valuable* transaction on R(converting via the exchange rate), and an equivalent attacker (using eq. (2) to convert). How many confirmations are needed on Rso that P_L(attack success) = P_R(attack success)?
< /p >

<p>
<sup>23</sup>&nbsp; Due to the dynamics of confirmations, we can’t directly compare chain-segments like this, *generally* speaking — this example is here to help give you an intuition. The reason we can’t directly compare in this way is that simply *having more confirmations* is worth something in and of itself. The relationship is not linear. See <a href="https://web.archive.org/web/20220209100515/https://cloudflare-ipfs.com/ipfs/QmNUWmY94QUievK8ptoxsPyAQUsKvx1cjRyCgPcfmysAVv" target="_blank"
>Analysis of hashrate-based double-spending (Meni Rosenfeld; 2012) for more.
</p>

<p>
<sup>24</sup>&nbsp; Alternatively, you could assume that confirmation rates are *always* in the same units (i.e., *generic* blocks/second). That will yield similar results; the logic basically works either way with some minor tweaks. The important point is that the units of $\mathrm{L}_f < /span >$/$\mathrm{R}_f < /span >$ are **not** L-blocks/R-block.
</p>

```
</div>
<div
    class="tcolorbox"
    style=" border:  1px solid #404040; background:  #F2F2F2; "
>
<div
    class="tcolorboxupper"
    style=" color:  #000000; "
```

> **Term** **Difficulty Adjustment Algorithm (DAA)**: An algorithm which updates its chain’s difficulty as valid blocks are produced. The *output* of a DAA is *context laden* — units take on *additional context*. </div> </div>

DAA’s typically work like this: calculate a *ratio* by which to adjust (multiply) the prior difficulty, based on a *target* block production rate and the *measured* block production rate.

Bitcoin, for example, adjusts its difficulty every 2016 blocks.<sup>25</sup> A ratio is found by multiplying the previous difficulty
$(\mathrm{D}_{\text{prev}}) bythetargetduration(\Delta t_{\text{target}})for2016blocksanddividingbytheactualduration(\Delta t_{\text{actual}})ofthelast2016blocks. < sup > 26 < /sup > Notethattheunitsof \Delta t_{\text{actual}} areB-seconds/(2016B-blocks), andtheunitsof \Delta t_{\text{target}} areseconds/(2016blocks).$

DAA’s are special: they are the means by which *context* is added. DAA’s don’t explicitly deal with this context though — it’s not mentioned in the algorithm itself. The key to a DAA’s success is that it operates *relative to a past state that is* *already* *context laden.* So DAA’s don’t need to have any special awareness of context, just that multiplying the past difficulty by a *particular ratio* will adjust the *confirmation rate* to align with the *target block frequency*. It’s an *incremental and ongoing process*. Since DAA’s don’t have initial conditions, there’s no bootstrapping concern. To function, a DAA only needs to say how the difficulty should *change*; it doesn’t need to know what it actually *is*. We will use the subscript W$\rightarrow Btodenotetheideaofconvertingbetweensome< em > world < /em > context, andthe< em > networkcontext < /em > (ofBitcoin). < spanclass = ”hidden” > \backslash(\backslash seteqnumber\{0\}\{\}\{9\}\backslash) < /span >$
<!--

$$\begin{aligned}
&\text{Note that:} && \text{ConversionConst}_{W\rightarrow B} = \frac{\Delta t_{\text{target}}}{\Delta t_{\text{actual}}} && \frac{\text{B-blocks} \cdot \text{seconds}}{\text{B-second} \cdot \text{block}} \\[0.5em]
&\text{Bitcoin's DAA:} && \text{NextWork}_{W\rightarrow B}(D_{\text{prev}}) = \frac{\Delta t_{\text{target}}}{\Delta t_{\text{actual}}} \cdot D_{\text{prev}} && \frac{\text{B-hashes} \cdot \text{seconds}}{\text{B-second} \cdot \text{block}} \cdot \cancelto{1}{\frac{\text{B-blocks}}{\text{B-block}}}
\end{aligned}$$

-->

```
\begin{align}
\text {Note that:} &amp; &amp; \text {ConversionConst}_{W\rightarrow B} &amp; = \frac {\Delta t_\text {target}}{\Delta t_\text {actual}} &amp; &amp; \frac {\text {B-blocks} \cdot \text {seconds}}{\text {B-second} \cdot \text {block}}
\nonumber \\[0.5em] \text {Bitcoin&apos;s DAA:} &amp; &amp; \text {NextWork}_{W\rightarrow B}(D_\text {prev}) &amp; = \frac {\Delta t_\text {target}}{\Delta t_\text {actual}} \cdot D_\text {prev} &amp; &amp; \frac {\text {B-hashes} \cdot
\text {seconds}}{\text {B-second} \cdot \text {block}} \cdot \cancelto {1}{\frac {\text {B-blocks}}{\text {B-block}}} \nonumber
\end{align}
```

When a DAA adds context, it converts blocks
$\leftrightarrow B-blocks, andseconds \leftrightarrow B-seconds. Alternatively, itcould< em > strip < /em > context. Eitherwayworksbecausethe DAA actsasaboundaryoftheconvertiblecontextinbothcases. This meansthat< em > whenconverting < /em > wecancancelB-
secondswithseconds, B-blockswithblocks, etc. < spanclass = ”hidden” > \backslash(\backslash seteqnumber\{0\}\{\}\{9\}\backslash) < /span >$
<!--

$$\begin{aligned}
&\text{Alt. with context:} && \text{NextWork}_{W\rightarrow B}(D_{\text{prev}}) = \frac{\Delta t_{\text{target}}}{\Delta t_{\text{actual}}} \cdot D_{\text{prev}} && \frac{\text{B-hashes}}{\text{B-block}} && (10) \\[0.5em]
&\text{Divide by } D_{\text{prev}} && \implies \text{ConversionConst}_{W\rightarrow B} = \frac{\Delta t_{\text{target}}}{\Delta t_{\text{actual}}} && \cancelto{1}{\frac{\text{B-hashes} \cdot \text{B-blocks}}{\text{B-hash} \cdot \text{B-block}}}
\end{aligned}$$

-->

```
\begin{align}
\text {Alt.  with context:} &amp; &amp; \text {NextWork}_{W\rightarrow B}(D_\text {prev}) &amp; = \frac {\Delta t_\text {target}}{\Delta t_\text {actual}} \cdot D_\text {prev} &amp; &amp; \frac {\text {B-hashes}}{\text {B-block}} \label
{eq:bitcoin-daa} \\[0.5em] \text {Divide by } D_\text {prev} &amp; &amp; \implies \text {ConversionConst}_{W\rightarrow B} &amp; = \frac {\Delta t_\text {target}}{\Delta t_\text {actual}} &amp; &amp; \cancelto {1}{\frac {\text {B-hashes}
\cdot \text {B-blocks}}{\text {B-hash} \cdot \text {B-block}}} \nonumber
\end{align}
```

```html
<div role="note" class="footnotes">
<p>
<sup>25</sup>&nbsp; Note: in Bitcoin, a difficulty of 1 corresponds to 2^{32}hashes.
< /p >

<p>
<sup>26</sup>&nbsp; Note: in practice the ratio is clamped between <span class="textrm textmd textup textulc"><span class="numerator">1</span>/<span class="denominator">4</span></span> and 4. See Bitcoin's <a
href="https://github.com/bitcoin/bitcoin/blob/7fcf53f7b4524572d1d0c9a5fdc388e87eb02416/src/pow.cpp\#L49-L72" target="_blank" ><kbd>src/pow.cpp</kbd></a> for the implementation.
</p>


</div>
<div
    class="tcolorbox"
    style=" border:  1px solid #404040; background:  #F2F2F2; "
>
<div
    class="tcolorboxupper"
    style=" color:  #000000; "
>  <span role="note" class="marginpar"><b>Term</b></span> <b><a href="whitepaper.html#??">Convertible Context</a></b>:  The boundary of a group of values that are mutually convertible. Within a convertible context, all values must be of the same
<em>scale</em> or have known exact scaling factors. </div> </div>

The general case of a DAA's relationships (flows of <em>information</em> and <em>context</em>) are diagrammed in fig. <a href="whitepaper.html#lateximage-whitepaper-63">9</a>.

<figure id="autoid-16" class="figure ">
```

```
<div class="center">
<a href="diff_adjustment_alg_sag" target="_blank" ><img
    src="diff_adjustment_alg_sag"
    class="inlineimage"
    alt="(image)"
></a>
```

`<div class="figurecaption">`Figure 9: The difficulty adjustment algorithm governs the relationship between the inputs: the previous difficulty, the target block frequency, and network participation (chain history); and the output: the network difficulty. The DAA is how `<em>`*confirmations*`</em>` and `<em>`*coins*`</em>` become `<b>`**laden**`</b>` with `<em>`*implicit context*`</em>`. If we don't account for this `<em>`*implicit context*`</em>` then our conversions will be nonsensical. The implicit context is `<em>`*network participation*`</em>` — thus, `<kbd>N-</kbd>` prefixes the units which are `<em>`*context laden*`</em>`. Thick arrows indicate `<em>`*network context*`</em>`, and thin arrows indicate `<em>`*world context*`</em>`. Solid arrows show the `<em>`*flow*`</em>` of `<em>`*information*`</em>`. Dashed arrows show the `<em>`*flow*`</em>` of `<em>`*context*`</em>`. Two-way arrows $(\longleftrightarrow)linktwovaluesthatare< em > convertible < /em >.Thecollectionofvaluesmutuallylinkedbytwo-wayarrowsdefinethe< em > convertiblecontext < /em >.Valuescanonlybeconvertedwhenthereisadirecttwo-waypathbetweenthem.$

`</div> </div></figure>`

`<figure id="autoid-17" class="figure ">`

```
<div class="center">
<a href="diff_adjustment_alg_times_2_sag" target="_blank" ><img
    src="diff_adjustment_alg_times_2_sag"
    class="inlineimage"
    alt="(image)"
></a>
```

<div class="figurecaption">Figure 10: How are the convertible contexts of two different networks related? Without the market context, there's no conversion path that allows for the conversion of work — the conversion path between difficulties is a *consequence* of
$\mathrm{X}_{R\to L}(theexchangerate).Thisisthesameconvertiblecontextthatminersusetodeterminewhichnetworkismostprofitableforthem.Double-linedarrowsindicate< em > marketcontext < /em >.Thinsingle-lineddashedarrowsindicate< em > worldcontext < /em >.Noticethattheconvertiblepropertieswhichweareinterestedin(suchasL_dandR_d)use< e$
$wayarrows,indicatingthatweareusingnetworkcontext< em > and < /em >marketcontexttoconvertblock-weight.$

</div> </div></figure>

How do we know that *both* blocks and seconds become context laden via a DAA, though? Let's consider what $\mathrm{L}_f < /span >$/$\mathrm{R}_f < /span >$ means for the possible combinations of context laden values and note whether the meaning works for conversion or not (i.e., whether using it *appropriately* as a constant of conversion, or scaling factor, will produce sensible results).
\(\seteqnumber{0}{}{10}\)
<!--

$$\text{No context:} \quad \frac{L_f}{R_f} \quad \text{(unitless)} \quad \text{works} \tag{11}$$

$$\text{Context laden blocks:} \quad \frac{L_f}{R_f} \quad \frac{\text{L-blocks}}{\text{R-block}} \quad \text{fails} \tag{12}$$

$$\text{Context laden seconds:} \quad \frac{L_f}{R_f} \quad \frac{\text{R-seconds}}{\text{L-second}} \quad \text{fails} \tag{13}$$

$$\text{Both context laden:} \quad \frac{L_f}{R_f} \quad \frac{\text{L-blocks} \cdot \text{R-seconds}}{\text{L-second} \cdot \text{R-block}} \quad \text{?} \tag{14}$$

-->

```
\begin{align}
\text {No context:} &amp; &amp; \frac {L_f}{R_f} &amp; &amp; \text {(unitless)} &amp; &amp; \text {works} \label {eq:conv-both-no-ctx} \\[0.5em] \text {Context laden blocks:} &amp; &amp; \frac {L_f}{R_f} &amp; &amp; \frac {\text
{L-blocks}}{\text {R-block}} &amp; &amp; \text {fails} \label {eq:conv-both-ctx-blocks} \\[0.5em] \text {Context laden seconds:} &amp; &amp; \frac {L_f}{R_f} &amp; &amp; \frac {\text {R-seconds}}{\text {L-second}} &amp; &amp; \text
{fails} \label {eq:conv-both-ctx-seconds} \\[0.5em] \text {Both context laden:} &amp; &amp; \frac {L_f}{R_f} &amp; &amp; \frac {\text {L-blocks} \cdot \text {R-seconds}}{\text {L-second} \cdot \text {R-block}} &amp; &amp; \text {?} \label
{eq:conv-both-ctx-laden}
\end{align}
```

We've seen eq. (11) and eq. (12) before. The first represents the ratio of block frequencies (unitless) — that's straightforward and works. The second has units L-blocks/R-block, which sounds like it should be the ratio of block *weights* — but it's clear that it *isn't* that. (So this conversion method fails.)

eq. (13) has weird units, though. R-seconds/L-second means something like: the relative participation of each network compared with a recent past state; i.e., the ratio of the ratios of each network's *actual* block production compared to its *target* block production. (This conversion method also fails.)

eq. (14) measures something like *relative weighted confirmation rates*. It's not clear if is useful or not, but we *do* know that *no other* value we have access to has context laden seconds as a unit. How can we use it to convert between anything meaningful if context laden seconds can't be canceled via some conversion? (Do we even *need* to ever use those units, anyway?)

In general, it seems like the safe option is *not to use* $L_forR_fwhenconvertingwork < /em >$ — unless we have some *specific, context-driven* explanation for why it's okay in that case.

How do these ideas of context laden values work when converting values *between* these network-contexts? This is diagrammed in fig. 10.

In essence, an exchange rate provides meaningful conversion between L-coins and R-coins. Converting in this way *does not drop context*. Since network context is respected, we can use an exchange rate to build a meaningful constant of conversion *across networks*.

```
<div
    class="tcolorbox"
    style=" border:  1px solid #404040; background:  #ECECEC; "
>
<div
    class="tcolorboxupper"
    style=" color:  #000000; "
```

> **Aside** With regard to DAAs, it should be noted that Bitcoin's was the first, and the method has some undesirable properties. I quite like the algorithm named DAA-2 (which is used by Bitcoin Cash) in An Economic Analysis of Difficulty Adjustment Algorithms in Proof-of-Work Blockchain Systems (Noda, Okumura, Hashimoto; 2020). Experimentally, it seems to work well with section 4.8. </div> </div>

```
<!--...............................-->
<h6 id="autosec-189"><span class="sectionnumber">2.3.4&#x2003;</span>Conversions and Sums</h6>
```

We know that, after conversion, we can sum work from two different chains. Are there any *other* values (in units other than L-hashes) that we can sum up, though? When we're *summing* weights as part of calculating chain-weight (e.g., that of **??**, or **??**), do we need to sum *L-hashes*? Well, no. We only need to *end up* with L-hashes.

Consider the case for a two-stage linear conversion method. That is: we convert the input into some common units (which could be anything), then we convert those common units into the final units. If both partial-conversions are *linear*, then we must have a situation like this: \(\seteqnumber{0}{}{14}\)
<!--

$$\begin{aligned} \text{Convert}_{L\rightarrow R}(\dots) &= \text{Conv}_1(\text{Conv}_2(\dots)) && \\ &= V_1 \cdot \text{Conv}_2(\dots) && \text{For some constant of conversion, } V_1 \end{aligned}$$

-->

```
\begin{align*}
\text {Convert}_{L\rightarrow R}(\dots ) =&amp;\; \text {Conv}_1(\text {Conv}_2(\dots )) &amp; &amp; \\[0.5em] =&amp;\; V_1 \cdot \text {Conv}_2(\dots ) &amp; &amp;\text {For some constant of conversion, }V_1
\end{align*}
```

Let's sum multiple conversions, e.g., as done in **??**: \(\seteqnumber{0}{}{14}\)
<!--

$$\begin{aligned} \sum_{i=0}^{n} \text{Convert}_{L\rightarrow R}(\dots) &= \sum_{i=0}^{n} V_1 \cdot \text{Conv}_2(\dots) && \\ &= V_1 \cdot \sum_{i=0}^{n} \text{Conv}_2(\dots) && \text{Factorize out } V_1 \end{aligned}$$

-->

```
\begin{align*}
\sum \limits _{i=0}^n \text {Convert}_{L\rightarrow R}(\dots ) =&amp;\; \sum \limits _{i=0}^n V_1 \cdot \text {Conv}_2(\dots ) &amp; &amp;\text {} \\[0.5em] =&amp;\; V_1 \cdot \sum \limits _{i=0}^n \text {Conv}_2(\dots ) &amp; &amp;\text
{Factorize out }V_1
\end{align*}
```

Thus, *any* common units, which are linearly convertible both from a reflecting chain's block and to local chain-work, can be used during summation.

```
<div
    class="tcolorbox"
    style=" border:  1px solid #404040; background:  #ECECEC; "
>
<div
    class="tcolorboxupper"
    style=" color:  #000000; "
```

> **Aside** Before we move on, let's consider: \(\seteqnumber{0}{}{14}\)
<!--

$$\begin{aligned} & \frac{L_d}{L_r} \cdot X_{R\rightarrow L} && \frac{\text{L-hashes}}{\text{R-coin}} && \\ & \frac{R_r}{R_d} \cdot X_{R\rightarrow L} && \frac{\text{L-coins}}{\text{R-hash}} && \text{Similarly} \\ \therefore \text{ConvReward}_{R\rightarrow L}(w) = & \frac{R_r}{R_d} \cdot X_{R\rightarrow L} \cdot w && \text{R-hashes} \rightarrow \text{L-coins} && (15) \end{aligned}$$

-->

```
\begin{align}
&amp; \frac {L_d}{L_r} \cdot X_{R\rightarrow L} &amp; &amp;\frac {\text {L-hashes}}{\text {R-coin}} &amp; &amp; \text {} \nonumber \\[0.5em] &amp; \frac {R_r}{R_d} \cdot X_{R\rightarrow L} &amp; &amp;\frac {\text {L-coins}}{\text
{R-hash}} &amp; &amp; \text {Similarly} \nonumber \\[0.5em] \therefore \text {ConvReward}_{R\rightarrow L}(w) = \; &amp; \frac {R_r}{R_d} \cdot X_{R\rightarrow L} \cdot w &amp; &amp; \text {R-hashes} \rightarrow \text {L-coins} &amp;
&amp; \label {eq:por-conv-reward}
\end{align}
```

Is it possible that we can convert chain-work *via summing block rewards?* </div> </div>

<!--...............................-->
<h5 id="autosec-190">2.4&#x2003;Conversion Contexts</h5>

What blockchain contexts can facilitate the conversion of block-weight?

Whatever contexts we find, we will need to figure out a way to get the exchange rate that is *at least as secure* as the consensus algorithms (otherwise we'd be introducing a new weakest-link). That can't be too hard, right?

Can we *avoid* that exchange rate, though? Well, there is a context where
$\mathrm{X}_{R\to L} = 1 : < b > whenL - coins \equiv R - coins < /b >, i.e., bothchainsusethesameroottoken. Inthatcase, < spanclass = "textrm\ textmd\ textup\ textulc" >< spanclass = "numerator" > L_d < /span > / < spanclass = "denominator" > L_r < /span >< /span > \cdot$
$< spanclass = "textrm\ textmd\ textup\ textulc" >< spanclass = "numerator" > R_r < /span > / < spanclass = "denominator" > R_d < /span >< /span > givesusL - hashes/R - hashdirectly.$

```
<!--................................-->
<h6 id="autosec-191"><span class="sectionnumber">2.4.1&#x2003;</span>A Single Root Token Across Multiple Chains</h6>
```

The first example context is *one network split over two chains*. Both chains use the same *root token*. There are, naturally, questions to answer, like *How are block rewards managed?* and *How can the root token be on two blockchains at once?*

Let's consider the following *restricted case* first: the chains ($L$and R) have something like a two-way peg between them, ensuring that no coins are ever incorrectly created or destroyed.

In this case, for a single R-block, eq. (2) collapses to: \(\seteqnumber{0}{}{15}\)
<!--

$$\begin{aligned}
\text{ConvWork}_{R\rightarrow L}(w) &= \frac{L_d}{L_r}\cdot\frac{R_r}{R_d}\cdot w && && \text{Since } X_{R\rightarrow L}=1\\
\implies \text{ConvWork}_{R\rightarrow L}(R_d) &= \frac{L_d}{L_r}\cdot R_r && \text{L-hashes} && w=(R_d\cdot 1)\text{ R-hashes} \qquad (16)\\
\implies \text{ConvWork}_{R\rightarrow L}(R_d)\cdot\frac{L_r}{L_d} &= R_r && \text{L-coins} && \times\text{ L-coins/L-hashes}\\
& && && \text{Weight of an R-block in L-coins}
\end{aligned}$$

-->

```
\begin{align}
\text {ConvWork}_{R\rightarrow L}(w) = \; &amp; \frac {L_d}{L_r} \cdot \frac {R_r}{R_d} \cdot w &amp; &amp; &amp; &amp; \text {Since }X_{R\rightarrow L}=1 \nonumber \\[0.5em] \implies \text {ConvWork}_{R\rightarrow L}(R_d) = \; &amp;
\frac {L_d}{L_r} \cdot R_r &amp; &amp; \text {L-hashes} &amp; &amp; w = (R_d \cdot 1) \text { R-hashes} \label {eq:por-conv-work-srt} \\[0.5em] \implies \text {ConvWork}_{R\rightarrow L}(R_d) \cdot \frac {L_r}{L_d} = \; &amp; R_r &amp;
&amp; \text {L-coins} &amp; &amp; \genfrac {}{}{0pt}{}{\times \text { L-coins/L-hashes}\qquad \qquad \quad \;\;}{\text {Weight of an R-block in L-coins}} \nonumber
\end{align}
```

So, the weight of an R-block measured in L-coins is exactly equal to the Rchain's block reward. This, of course, makes sense. L-coins are fungible with R-coins in this context, so, naturally, the weight-in-coins of an Rblock is its own block reward.

If we convert like this (treating 1 R-block as worth
$R_r \cdot 1L - coins)thenwe'llneedtoconverttheresulttoL - hashes< em > eventually < /em >, but, if wehavemultiplereflectionstoprocess, wecancalculateinterimvalues(withunitsofL - coins)andsumthembeforethatfinalconversion. It'sworthrememberingthatblock - weightismeasured< em > perblock < /em >. Sincewe'reonlyconvertingwithrespectto< em > asingleblock < /em > atanyonetime, the< em > perblock < /em > partoftheresultingunits< em > cancelsout < /em > - - - e.g., eq. < spanclass = "textup" > (< ahref = "$whitepaper.html$#lateximage - whitepaper - 63" > 16 < /a >) < /span >.$

Let's use this context to write our first WeightOf function: algorithm 2 (we'll still return work in L-hashes, though, not L-coins).

```
<div
    class="tcolorbox"
    style=" border:  1px solid #404040; background:  #ECECEC; "
>
<div
    class="tcolorboxupper"
    style=" color:  #000000; "
```

> **Aside** Note: the algorithms we define here will be general — they'll work for *both* the local and any reflecting chains. </div> </div>

<figure id="autoid-18" class="algorithm ruled">

<div class="figurecaption">Algorithm 2: A WeightOf for networks of a single root token.

</div>
<ul style="list-style-type:none">

procedure WeightOf(B_i, *state*) $< spanclass = "floatright" > \&\#x25B7; TheweightofareflectingblockinL - hashes < /span >$

procedure
(T) $\leftarrow < spanclass = "textsc" > BlockTimestamp < /span > (B_i)$

procedure <span class="textsc">R $\leftarrow /spanspanclass = "textsc"ChainOf/span/span(B_i, state)/spanspanclass = "floatright"\&\#x25B7; Eitherthelocaloraref lectingchain/span/span$
/span/li
/spanli
/spanspan style="width:39pt; display:inline-block;"/span/span**/spanbprocedure/span/b** /spanspan
class="textsc"$R_r \leftarrow /spanspanclass = "textsc"BlockRewardOfAt/span/span(R, t, state)/spanspanclass = "floatright"\&\#x25B7; L - coins/R - block - - - bydefinition/span/span$
/span/li
/spanli
/spanspan style="width:52pt; display:inline-block;"/span/span**/spanbprocedure/span/b** /spanspan class="textsc"$L_d \leftarrow /spanspanclass = "textsc"LocalDifficultyAt/span/span(t, state)$
/span/li
/spanli
/spanspan style="width:65pt; display:inline-block;"/span/span**/spanbprocedure/span/b** /spanspan class="textsc"$L_r \leftarrow$
But where do we get the reflecting chain's block reward? To start with, we can't let each chain /spanem*set*/span/em its own reward, that would break the two-way peg. It also means /spanem*the reflecting chain shouldn't be the source of that data*/span/em. We could retrieve it from state (if it's already calculated), but for the sake of this example, let's calculate it in the /spanspan class="textsc"WeightOf/span/span function.
Right now, we don't have enough /spanem*context*/span/em to know what to do; /spanem*what contexts would be useful?*/span/em
What /spanem*purpose*/span/em does a block reward fulfil? It must be something about /spanem*security*/span/em, because that's /spanem*why*/span/em miners are given an incentive to mine (the /spanem*how*/span/em is via the /spanem*block reward*/span/em). If we have multiple chains (with the same root token), why should one chain be more secure than another? One reason is because /spanem*more commerce happens there;*/span/em i.e., more of the network relies on /spanem*that*/span/em chain compared to the other one.
What would make sense, given the context of a commerce-imbalance, to base the block reward on? A straightforward answer (which can be globally known, too) is /spanem*the ratio of root tokens on each chain.*/span/em We can have a /spanem*network wide*/span/em rule that there is some /spanem*network wide inflation rate*/span/em, and each chain's block reward is proportional to the /spanem*ratio*/span/em of coins on that chain relative to the entire supply. This way, the global inflation rate is predictable, even though the number of coins produced /spanem*on each chain*/span/em (via block rewards) is variable. Moreover, with this method we should expect that the /spanem*value to the community*/span/em of each chain roughly matches the /spanem*distribution of the community's activities*/span/em, and the /spanem*distribution of security*/span/em, too. This method is also consistent with the idea of /spanem*equal work for equal reward*/span/em. We can rely on that idea because /spanem*equality of work*/span/em is a result of the market (for block rewards) formed by participating miners.
If the network uses this rule to determine block reward, then, for either chain C, what can we say? Let: I (coins/s) be the network-wide inflation rate;
$G_t(coins)bethenetwork - wideroottokensupply; andC_t(coins)bethenumberofroottokensonchainC. C_f(blocks/s)istheblockproductionfrequencyofchainC./spanspanclass = "hidden"\backslash(\backslash seteqnumber\{0\}\{\}\{16\}\backslash)/span/span$
<!--

$$\begin{aligned}
C_r\cdot C_f &= \frac{C_t}{G_t}\cdot I && \frac{\text{C-coins}}{\text{second}} && \text{As set above}\\
\implies C_r &= \frac{C_t\cdot I}{G_t\cdot C_f} && \frac{\text{C-coins}}{\text{C-block}} && \qquad (17)\\
\therefore \frac{R_r}{L_r} &= \frac{R_t\cdot I}{G_t\cdot R_f}\cdot\frac{G_t\cdot L_f}{L_t\cdot I} && \frac{\text{L-blocks}}{\text{R-block}} && \text{Via eq. (17)}\\
\frac{R_r}{L_r} &= \frac{R_t\cdot L_f}{L_t\cdot R_f}\cdot\cancelto{1}{\frac{G_t\cdot I}{G_t\cdot I}} && \frac{\text{L-blocks}}{\text{R-block}} && \text{by \emph{definition}} \qquad (18)
\end{aligned}$$

-->

```
\begin{align}
C_R \cdot C_F &amp; = \frac {C_T}{G_T} \cdot I &amp; &amp; \frac {\text {C-coins}}{\text {second}} &amp; &amp; \text {As set above} \nonumber \\[0.5em] \implies C_R &amp; = \frac {C_T \cdot I}{G_T \cdot C_F} &amp; &amp; \frac {\text
{C-coins}}{\text {C-block}} &amp; &amp; \label {eq:srt-reward} \\[0.5em] \therefore \frac {R_R}{L_R} =&amp; \; \frac {R_T \cdot I}{G_T \cdot R_F} \cdot \frac {G_T \cdot L_F}{L_T \cdot I} &amp; &amp; \frac {\text {L-blocks}}{\text {R-block}}
&amp; &amp; \text {Via \autoref {eq:srt-reward}} \nonumber \\[0.0em] \frac {R_R}{L_R} =&amp; \; \frac {R_T \cdot L_F}{L_T \cdot R_F} \cdot \cancelto {1}{\frac {G_T \cdot I}{G_T \cdot I}} &amp; &amp; \frac {\text {L-blocks}}{\text {R-block}}
&amp; &amp; \text {by \emph {definition}} \label {eq:srt-block-ratios}
\end{align}
```

Hold up! Didn't we just go over — in great detail — why we can't use block frequencies ($L_f andR_f)inconversions? Whyisitokaytousethemnow?$
To answer this, let's consider /spanem*how*/span/em block rewards could be set network-wide so that they're consistent across all chains. Unlike a chain's difficulty, the block reward needs to be consistent with some global inflation rate. eq. /spanspan class="textup"(/spana href="whitepaper.html#lateximage-whitepaper-63"18/span/a)/span/span implies that
$R_r \cdot R_f \cdot L_t = L_r \cdot L_f \cdot R_t, soinsomecases/spanemwedon'tneedtoknowtheglobalstate, justthattherelationshipisenforced./span/emThatsaid, ifIisknownacrossthenetwork(i.e., itisconstantorawellknownfunction)thencalculatingG_t istrivial : G_t = I \cdot l + G_{t_0}, wherelisthelifetimeofthenetwork(inseconds), andG_{t_0} isanyinitialroottokensupplythatwasnotcreatedviainflation(e.g., viasometokengenerationevent).$
Could we maintain consistency across chains via something like a **/spanbReward Adjustment Algorithm/span/b** (RAA)? If each chain commits to the input values (e.g., via the inclusion of
$C_t intheirblock - headers), thenanRAAcouldbeevaluatedusingonlytheheader - chain(similartotheDAA). Thisway, ifaminer(orachain)triestosubvertorattacktheRAA, it'striviallydetectablebyreflectingchains.$

/spandiv
    class="tcolorbox"
    style=" border: 1px solid #404040; background: #F2F2F2; "

/spandiv
    class="tcolorboxupper"
    style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbTerm/span/b**/span/span **/spanb/spana href="whitepaper.html#??"Reward Adjustment Algorithm (RAA)/span/a/span/b**: An algorithm which updates the block reward of each chain in a /spanem*network*/span/em of chains that share a root token. Similar to a DAA, the /spanem*output*/span/em of an RAA is /spanem*context laden*/span/em. /span/div /span/div

/spandiv
    class="tcolorbox"
    style=" border: 1px solid #777472; background: #FEE8F4; "

/spandiv
    class="tcolorboxtitle"
    style=" border-top: 1px solid #777472; border-bottom: 1px solid #777472; background: #777472; color: #FFFFFF; "
TODO: [1] **/spanb(LOW)/span/b**/span/div
/spandiv
    class="tcolorboxupper"
    style=" color: #000000; "
RAA is long term context, DAA is short term — they don't interfere w/ eachother /span/div /span/div
/spanfigure id="autoid-19" class="figure "

/spandiv class="center"
/spana href="diff_adjustment_alg_times_3_sag" target="_blank" /spanimg
    src="diff_adjustment_alg_times_3_sag"
    class="inlineimage"
    alt="(image)"
/span/a

/spandiv class="figurecaption"Figure 11: How does implicit context change when considering networks of a single root token? In this case, because we set a network-wide inflation rate (eq. /spanspan class="textup"(/spana href="whitepaper.html#lateximage-whitepaper-63"17/span/a)/span/span), by */spanemdefinition/span/em* L_f*and*R_f*havemeaningintheconvertiblecontext.Inessence : bothLandRhaveconvertiblefactorsthataredependentonthesamemeaningof"seconds".Thecollectionofvaluesmutuallylinkedbytwo − wayarrows*(⟷ *)definethe/spanemconvertiblecontext/span/em.Double − linedarrowsindicatethe/spanemnew/span/emcontextusedintheRAA(whichisrelatedtotheworldcontext).ThickDouble − linedarrowsindicatethenewhybridcontextwhichallowsforconversion(formulti − chainnetworksofasingleroottoken).Asbefore, dashedarrowsindicatetheflowofcontext.*
/span/div /span/div/span/figure
If we use an RAA, then the RAA */spanemoutput/span/em* could become */spanemcontext laden/span/em* (depending on the RAA's input parameters). In the case of a global inflation rate, */spanemthis means that chains would rely on a common meaning of "seconds"!/span/em* So it is by */spanemconstruction and definition/span/em* that the block frequency introduced in eq. /spanspan class="textup"(/spana href="whitepaper.html#lateximage-whitepaper-63"17/span/a)/span/span becomes part of the convertible context. Thus, we can */spanemnow/span/em* include L_f*and*R_f*, thoughwecouldn'tbefore.*
Note that we also */spanemprove/span/em* eq. /spanspan class="textup"(/spana href="whitepaper.html#lateximage-whitepaper-63"7/span/a)/span/span for this case. We know that — when
L_f*and*R_f*areintheconvertiblecontext − − − wecanusethemviaa/spanemscalingfactor/span/emthattypically* $\neq 1$*.Forexample :*
toconvert/spanspanclass = "textrm textmd textup textsc"/spanspanclass = "numerator"L_f*/span/span//spanspanclass = "denominator"*R_f*/span/span/span/spantounitsofL − blocks/R −*
block, itmustbescaledby/spanspanclass = "textrm textmd textup textsc"/spanspanclass = "numerator"R_t*/span/span//spanspanclass = "denominator"*L_t*/span/span/span/span(whichiscontextualandunitless).Wecanfindascalingfactor/spanemhere/span/em(butnotgenerally)becausewe/spanemincludeditinaprotocol − levelrelationship/span*
This new situation and context are diagrammed in fig. /spana href="whitepaper.html#lateximage-whitepaper-63"11/span/a.
Let's use eq. /spanspan class="textup"(/spana href="whitepaper.html#lateximage-whitepaper-63"18/span/a)/span/span to create our next /spanspan class="textsc"WeightOf/span/span algorithm for this context of a */spanemsingle root token/span/em* (SRT) — **??**. /spanspan class="hidden" \(\seteqnumber{0}{}{18}\)/span/span
<!--

$$\begin{aligned}
& \frac{R_t \cdot L_f}{L_t \cdot R_f} \cdot L_d && \frac{\text{L-hashes}}{\text{R-block}} && \text{Via eq. (18)} \\
\therefore \text{ConvSRT}_{R\rightarrow L}(b) = & \frac{R_t \cdot L_f}{L_t \cdot R_f} \cdot L_d \cdot b && \text{R-blocks} \rightarrow \text{L-hashes} && (19)
\end{aligned}$$

-->

\begin{align}
&amp; \; \frac {R_t \cdot L_f}{L_t \cdot R_f} \cdot L_d &amp; &amp; \frac {\text {L-hashes}}{\text {R-block}} &amp; &amp; \text {Via \autoref {eq:srt-block-ratios}} \nonumber \\[0.5em] \therefore \text {ConvSRT}_{R\rightarrow L}(b) = &amp;
\; \frac {R_t \cdot L_f}{L_t \cdot R_f} \cdot L_d \cdot b &amp; &amp; \text {R-blocks } \rightarrow \text { L-hashes} &amp; &amp; \label {eq:conv-srt}
\end{align}

/spandiv
    class="tcolorbox"
    style=" border: 1px solid #404040; background: #ECECEC; "

/spandiv
    class="tcolorboxupper"
    style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbAside/span/b**/span/span These and the following conversions all collapse nicely if we set R
≡ *L.Thisisduetothesymmetrymentionedatthestartofsection /spanahref = "*whitepaper.html#*lateximage − whitepaper − 63"*2.3.2*/span/a./span/div* /span/div
/span/li
/span/ul

```
<!--...............................-->
```

/spanspan class="paragraph" id="autosec-193"/spanspan class="sectionnumber"2.4.1.1&#x2003;/span/spanDegenerate Case/span/span

There is an even simpler case when the same hashing algorithm is used and when the rewards and difficulties are equal. In this case, the conversion to/from remote units is trivial, since these are now, */spanemby definition,/span/em* identical to local units. Equivalently: all constants of conversion between corresponding values are 1. Subsequently, eq. /spanspan class="textup"(/spana href="whitepaper.html#lateximage-whitepaper-63"16/span/a)/span/span collapses to: /spanspan class="hidden" \(\seteqnumber{0}{}{19}\)/span/span

```
<!--
```

$$\text{ConvWork}_{R\rightarrow L}(w) = w \qquad \text{L-hashes} \qquad \text{If } R_d = L_d, R_r = L_r \tag{20}$$

```
-->

\begin{align}
\text {ConvWork}_{R\rightarrow L}(w) = \; &amp; w &amp; &amp; \text {L-hashes} &amp; &amp; \text {If } R_d = L_d, R_r = L_r \label {eq:por-conv-work-degenerate}
\end{align}
```

While this works fine for PoR, it offers little value for this discussion on conversion theory.

`<!--................................-->`
/spanh6 id="autosec-194"/spanspan class="sectionnumber"2.4.2&#x2003;/span/spanDifferent Root Tokens with a DEX/span/h6

/spandiv
    class="tcolorbox"
    style=" border: 1px solid #404040; background: #ECECEC; "

/spandiv
    class="tcolorboxupper"
    style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbAside/span/b**/span/span Before we explore this context, I should mention that we are discussing it primarily so that we can generalize our understanding of the conversion of work. This context presents substantial challenges that the SRT context does not. /span/div /span/div
Our starting point will be eq. /spanspan class="textup"(/spana href="whitepaper.html#lateximage-whitepaper-63"2/span/a)/span/span: /spanspan class="hidden" \(\seteqnumber{0}{}{20}\)/span/span
`<!--`

$$\text{ConvWork}_{R\rightarrow L}(w) = \frac{L_d}{L_r} \cdot X_{R\rightarrow L} \cdot \frac{R_r}{R_d} \cdot w \qquad \text{R-hashes} \rightarrow \text{L-hashes}$$

`-->`

```
\begin{align}
\text {ConvWork}_{R\rightarrow L}(w) = \; &amp; \frac {L_d}{L_r} \cdot X_{R\rightarrow L} \cdot \frac {R_r}{R_d} \cdot w &amp; &amp;\text {R-hashes }\rightarrow \text { L-hashes} \nonumber
\end{align}
```

It's easy to see that we can work with this — of course, the /spanspan class="textsc"WeightOf/span/span function needs access to R's difficulty and block reward, and the exchange rate, too. Do we */spanemneed/span/em* to use
R$_d$, *though?Surely/spanemany/span/emconversiontoL − hasheswillwork, right?*
Let's see: /spanspan class="hidden" \(\seteqnumber{0}{}{20}\)/span/span
`<!--`

$$\begin{aligned} & \frac{L_d}{L_r} \cdot X_{R\rightarrow L} \cdot R_r & & \frac{\text{L-hashes}}{\text{R-block}} \\ \therefore \text{ConvDEX}_{R\rightarrow L}(b) = & \frac{L_d}{L_r} \cdot X_{R\rightarrow L} \cdot R_r \cdot b & & \text{R-blocks} \rightarrow \text{L-hashes} \end{aligned} \tag{21}$$

`-->`

```
\begin{align}
&amp; \frac {L_d}{L_r} \cdot X_{R\rightarrow L} \cdot R_r &amp; &amp; \frac {\text {L-hashes}}{\text {R-block}} \nonumber \\[0.5em] \therefore \text {ConvDEX}_{R\rightarrow L}(b) = \; &amp; \frac {L_d}{L_r} \cdot X_{R\rightarrow L} \cdot
R_r \cdot b &amp; &amp; \text {R-blocks }\rightarrow \text { L-hashes} \label {eq:por-conv-dex}
\end{align}
```

Okay, so we */spanemcan/span/em* convert to L's block-weight (L-hashes) */spanemwithout knowing R's difficulty/span/em*. It's good to know that we have */spanemoptions/span/em* there — there's */spanemexcess capacity/span/em* in the values we can convert between. If one value is not available, then we might not even need it. (This is relevant for section /spana href="whitepaper.html#lateximage-whitepaper-63"2.5/span/a.)
To move forward, we need to use X$_{R\rightarrow L}$*inanew/spanspanclass = "textsc"WeightOf/span/spanfunction − − − /spanemwheredoesthatvaluecomefrom?/span/em*
The value can't be hard-coded, or provided by a third party — these would introduce vulnerabilities. We need an accurate exchange rate for the moment of conversion, too — one that is reactive enough to remain up-to-date, and precise enough to be useful. It also needs to be */spanemprovable/span/em*; in practice, it needs to be */spanemon-chain/span/em* and available to both chains (and they should agree on the exchange rate). If that wasn't the case, how could one chain validate the weight of another chain's PoRs? Ideally, the exchange rate should */spanemalready/span/em* be recorded in */spanemboth/span/em* chains. (This way, full nodes of either chain do not need extra data to calculate/verify chain-weight.)
/spanbHowever,/span/b there is a major loop-hole in some of the above requirements: they only matter for */spanemmutual PoR/span/em*. What's different if we consider */spanemone-way PoR/span/em* instead?
In the case of one-way PoR, the reflecting chain doesn't need to know much about the reflected chain, and the heavy lifting (like the burden of knowing exchange rates) can be offloaded to */spanemthe reflected chain only/span/em*.
Provided that the source of the exchange rate is a */spanemwell-built/span/em*, protocol-level DEX, are there any real issues?
Let's consider this limited case first: one-way PoR where the */spanemreflected/span/em* chain hosts a DEX between its root token, and the RT of the */spanemreflecting/span/em* chain. Using eq. /spanspan class="textup"(/spana href="whitepaper.html#lateximage-whitepaper-63"21/span/a)/span/span, we can write **??**.
This */spanemseems/span/em* okay; of course, it */spanemstill depends on the DEX/span/em*. That means that an attack on the DEX might be a way to attack the consensus algorithm. Did we just introduce a vulnerability?
If an attacker */spanemreduces/span/em* the exchange rate (L-coins/R-coin) somehow, then that will */spanemdecrease/span/em* the weight of reflected blocks. For example, an attacker might sell */spanema lot/span/em* of L-coins all at once (or cause that to happen through traditional market manipulation). If the exchange rate goes down, then the */spanemrate of work/span/em* (i.e., how fast chain-weight is accumulated) of Chain L will */spanemseem/span/em* to decrease.
However, this isn't */spanemalways/span/em* relevant: if Chain R is */spanemmuch/span/em* more secure than Chain L, then this doesn't change much in practice. Say that Chain R contributes
∼100×*asmuchchain − weightasLdoes.ForanattackertargetingChainL, reducingtheexchangerate/spanembyhalf/span/emmeansthattheweightofreflectingblocksishalved.Sotheattackerwouldneedtoadd*∼50 × *the/spanemcurrent/span/emhash − rateofLtocompeteagainstthereflectionsfromR.Iftheattackerwasminingin/spanempublic/span/em, thatwouldincreasethelocaldifficultyby*∼50×, *which/spanemincreases/span/emtheweightofreflectingblocksby*∼50×, *too(soanattackermininginpublicisself − defeating).Ofcourse, theattacker/spanemcould/span/emstillmineinprivate, /spansup27/span/supprovidedtheexchangeratestayslow.Eventhoughtheattackergetsa50%discountviamarketmanipulation, thisisstillaverydifficultattack(andverynoticeable).Ineffect, attackingLisatleastasdifficultasattackingR.*
If an attacker */spanemincreases/span/em* the exchange rate (L-coins/R-coin) somehow, then that will */spanemincrease/span/em* the weight of reflecting blocks. This does not help the attacker.
It seems that */spanemin principle/span/em* there are some cases where it's safe to use ConvDEX$_{R\rightarrow L}$*forone − wayPoR.We′llrevisitthislaterinsection /spanahref = "*whitepaper.html*#lateximage − whitepaper − 63"3.4.1/span/a.*

/spandiv role="note" class="footnotes"
/spanp
/spansup27/span/sup&nbsp; For the purpose of UT and Amaroo, this turns out to be a non-issue. See section /spana href="whitepaper.html#lateximage-whitepaper-63"3.2/span/a, section /spana href="whitepaper.html#lateximage-whitepaper-63"3.4.1/span/a, section /spana href="whitepaper.html#lateximage-whitepaper-63"4.8/span/a, and section /spana href="whitepaper.html#lateximage-whitepaper-63"4.9/span/a.
/span/p

/span/div
/spandiv
class="tcolorbox"
style=" border: 1px solid #404040; background: #ECECEC; "

/spandiv
class="tcolorboxupper"
style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbAside/span/b**/span/span What about mutual PoR, though? Notice that **??** is trivially generalized for mutual reflection by replacing /spanspan class="textsc"GetLocalDexRateFromAt/span/span with a suitable /spanspan class="textsc"ExchangeRateToLocalOfAt/span/span, i.e., a DEX that satisfies the requirements we specified earlier. So it seems like our */spanemconversion method/span/em* should work; the weak-link is the DEX, not the conversion.

In the context of */spanemUltra Terminum/span/em* and */spanemAmaroo/span/em*, this isn't an important problem to solve. After all, this is a paper about */spanemconsensus/span/em*, not trustless, decentralized, cross-chain, protocol-level markets. If */spanemmutual PoR/span/em* is ever used to secure multiple chains with heterogenous tokens, this problem will need to be answered, though. /span/div /span/div

<!--...............................-->
/spanh6 id="autosec-196"/spanspan class="sectionnumber"2.4.3&#x2003;/span/spanWhat About SPV?/span/h6

Both contexts (SRT and DEX) require that participating chains can do on-chain SPV against one another. Chains need some ability to */spanemintrospect/span/em* reflecting chains — e.g., SRT requires that users can move root tokens between chains, and the DEX context requires two chains to agree on the exchange rate between their root tokens. Even without this requirement, some method of cross-chain communication is clearly desirable.

Eventually, we'll need to construct a method for SPV between mutually reflecting chains that works and is safe. However, there are still other problems that we have not yet solved, and the solutions may motivate certain blockchain designs over others. The difference between these designs will likely impact whether (and how) SPV can be done safely. So, attempting to solve the SPV problem at this point is premature.

We will proceed on the */spanemassumption/span/em* that SPV is possible and easy to do in a reasonable time period, and we'll investigate the problem of SPV between reflecting chains in detail in section /spana href="whitepaper.html#lateximage-whitepaper-63"4.11/span/a.

```
<!--...............................-->
```

/spanh5 id="autosec-197"/spanspan class="sectionnumber"2.5&#x2003;/span/spanConverting Confirmations/span/h5

So far, we've considered PoW chains only. Conversion of chain-weight between PoW chains can work */spanemif and only if/span/em* we can convert between */spanemwork/span/em* (i.e., hashes) done on each chain — given an appropriate context. For a given PoW block, the network knows exactly how much work is implied by that block — the expected number of hashes to produce it. Thus, for PoW chains, there is an exact conversion between */spanemwork and confirmations/span/em* (for some context at some point in time). Over short time-scales, this conversion ratio is approximately constant (in general it's a function that takes a timestamp as an input parameter). Thus,
/spanemchain-weight/span/em (as represented in figures via
$\Sigma_w, e.g.\&nbsp; fig.\ /spanahref = "\text{whitepaper.html}\#lateximage-whitepaper-63"23/span/a)canberepresentedeitherinsomethinglike/spanemhashes/span/emor/spanemdifficulty/span/em/spanbor/span/bchain-weightcansimplybeintermsof/spanemconfirmations/span/em.$

/spandiv
class="tcolorbox"
style=" border: 1px solid #404040; background: #ECECEC; "

/spandiv
class="tcolorboxupper"
style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbAside/span/b**/span/span If we convert */spanemwork to confirmations/span/em*, will we end up with something */spanemincompatible and contradictory/span/em* to the traditional notion of "a confirmation"? There are definitely differences. For example: if we convert confirmations, then */spanemwe'll have non-integer confirmations/span/em*, and what does 0.88 confirmations mean? Is that less good than a normal confirmation? This problem arises because */spanemwe're not actually converting work to confirmations/span/em*, per se: we're converting */spanemanother chain's work/span/em* into */spanemequivalent-confirmations/span/em* relative to something. Equivalent-confirmations are another chains confirmations that have been */spanemconverted to be in terms of the local chain's confirmations/span/em*. Most likely, those equivalent-confirmations will be relative either to some known historical confirmation, or to that of the */spanemcurrent/span/em* block. /span/div /span/div
Why think about chain-weight in terms of */spanemequivalent-confirmations/span/em* instead of */spanemwork/span/em*? There are a few reasons. First, */spanemconfirmations are general!/span/em* If we reason in terms of */spanemconfirmations/span/em* instead of */spanemwork/span/em*, then */spanemmaybe/span/em* we can apply these ideas to */spanemother chains/span/em* that don't use PoW. Second, it */spanemsimplifies thinking/span/em*. The purpose of converting chain-weight is clearer and easier to reason about. Finally, it makes explicit the requirement that */spanemwe can only compare to a grounded context/span/em*.
There is no way to say */spanemX work on Lis worth Y work on R/span/em* without adding necessary context like */spanemwhen/span/em* that conversion is happening. Confirmations (like work) require that grounding, since they need to be scaled when converting between different chains. What about confirmations from the same chain? Unlike work (which can be summed directly), confirmations always require conversion to a */spanemknown standard/span/em* — even when they're */spanemfrom the same chain/span/em*. For example, we can say that the single confirmation provided by Bitcoin block 704610 is */spanemequivalent/span/em* to approximately 19,893,045,000,000 genesis-confirmations./spansup28/span/sup
The conversion-ratio is equal to the difficulty of block 704610. That is, it would take a chain of ∼ 20 trillion blocks, each with 1 genesis-confirmation worth of work, to match the weight of block 704610. Curiously, while this works for the */spanemfork rule/span/em* (since it compares chain weights), it does not work when waiting for transaction confirmations — in that case, 20 trillion confirmations at 1 difficulty is */spanemmuch/span/em* more secure than 1 confirmation at 20 trillion difficulty./spansup29/span/sup
Now, **/spanbconverting confirmations,/span/b** how do we actually do it? If we want to convert confirmations, then we'll need to abstract away from the idea of */spanemdifficulty/span/em* in our conversion method. /spanspan class="hidden" \(\seteqnumber{0}{}{21}\)/span/span

```
<!--
```

$$\begin{align} \text{Consider:}\quad & \frac{R_r}{L_r}\cdot X_{R\to L} && \frac{\text{L-blocks}}{\text{R-block}} \nonumber \\ \therefore \text{ConvBlocks}_{R\to L}(b) = & \frac{R_r}{L_r}\cdot X_{R\to L}\cdot b && \text{R-blocks} \to \text{L-blocks} \tag{22} \end{align}$$

```
-->

\begin{align}
\text {Consider:  } \;\;\; &amp; \frac {R_R}{L_R} \cdot X_{R\rightarrow L} &amp; &amp;\frac {\text {L-blocks}}{\text {R-block}} &amp; &amp; \text {} \nonumber \\[0.5em] \therefore \text {ConvBlocks}_{R\rightarrow L}(b) = \; &amp; \frac
{R_R}{L_R} \cdot X_{R\rightarrow L} \cdot b &amp; &amp;\text {R-blocks }\rightarrow \text { L-blocks} &amp; &amp; \label {eq:por-conv-blocks}
\end{align}
```

So $1\times Rconfirmationsisworth\left(\frac{R_r}{L_r}\cdot X_{R\to L}\right)Lconfirmations.Niceandsimple.$

/spandiv role="note" class="footnotes"
/spanp
/spansup28/span/sup&nbsp; A genesis-confirmation is relative to the Bitcoin genesis block — which had a difficulty of exactly 1.
/span/p

/spanp
/spansup29/span/sup&nbsp; For an explanation of */spanemwhy/span/em*, see /spana href="https://web.archive.org/web/20220209100515/https://cloudflare-ipfs.com/ipfs/QmNUWmY94QUievK8ptoxsPyAQUsKvx1cjRyCgPcfmysAVv"
target="_blank" Analysis of hashrate-based double-spending (Meni Rosenfeld; 2012)/span/a and Section /spana href="whitepaper.html#lateximage-whitepaper-63"4.7/span/a.
/span/p

/span/div

`<!--...............................-->`
/spanh6 id="autosec-199"/spanspan class="sectionnumber"2.5.1&#x2003;/span/spanCoins per Confirmation/span/h6

Given a multi-chain network, could we measure block-weight in coins? It seems promising and elegant if it works, but does it have any real-world meaning?
One example where measuring chain-weight in coins does have some meaning is section /spana href="whitepaper.html#lateximage-whitepaper-63"2.4.1/span/a (the SRT context). Let's consider this, starting with the conversion used in eq. /spanspan class="textup"(/spana href="whitepaper.html#lateximage-whitepaper-63"18/span/a)/span/span. /spanspan class="hidden" \(\seteqnumber{0}{}{22}\)/span/span
`<!--`

$$\begin{aligned}
C_r &= L_r \cdot \frac{C_t}{L_t} \cdot \frac{L_f}{C_f} && \frac{\text{L-coins}}{\text{C-block}} && \text{Via eq. (18)} \\
C_r \cdot \frac{C_f}{L_f} &= L_r \cdot \frac{C_t}{L_t} && \frac{\text{L-coins}}{\text{L-block}} && \\
\therefore \sum_{C \in \{L,R\}} L_r \cdot \frac{C_t}{L_t} &= L_r \cdot \frac{L_t + R_t}{L_t} && \frac{\text{L-coins}}{\text{L-block}} && \text{Sum coins (as a proxy for weight)} && \text{(23)} \\
L_r \cdot \frac{C_t}{L_t} &= \frac{L_t \cdot I}{G_t \cdot L_f} \cdot \frac{C_t}{L_t} && \frac{\text{L-coins}}{\text{L-block}} && \text{Via eq. (17)} \\
&= \frac{C_t \cdot I}{G_t \cdot L_f} && \frac{\text{L-coins}}{\text{L-block}} && \\
\therefore \sum_{C \in \{L,R\}} \frac{C_t \cdot I}{G_t \cdot L_f} &= \frac{L_t + R_t}{G_t} \cdot \frac{I}{L_f} && \frac{\text{L-coins}}{\text{L-block}} && \text{Sum coins (as a proxy for weight)} && \text{(24)}
\end{aligned}$$

`-->`

```
\begin{align}
C_r = \; &amp; L_r \cdot \frac {C_t}{L_t} \cdot \frac {L_f}{C_f} &amp; &amp; \frac {\text {L-coins}}{\text {C-block}} &amp; &amp; \text {Via \autoref {eq:srt-block-ratios}} \nonumber \\[0.5em] C_r \cdot \frac {C_f}{L_f} = \; &amp; L_r \cdot
\frac {C_t}{L_t} &amp; &amp; \frac {\text {L-coins}}{\text {L-block}} &amp; &amp; \text {} \nonumber \\[0.5em] \therefore \sum \limits _{C \in \{L, R\}} L_r \cdot \frac {C_t}{L_t} = \; &amp; L_r \cdot \frac {L_t + R_t}{L_t} &amp; &amp;\frac
{\text {L-coins}}{\text {L-block}} &amp; &amp;\text {Sum coins (as a proxy for weight)} \label {eq:chain-coin-weight} \\[0.5em] L_r \cdot \frac {C_t}{L_t} = \; &amp; \frac {L_t \cdot I}{G_t \cdot L_f} \cdot \frac {C_t}{L_t} &amp; &amp;
\frac {\text {L-coins}}{\text {L-block}} &amp; &amp; \text {Via \autoref {eq:srt-reward}} \nonumber \\[0.5em] = \; &amp; \frac {C_t \cdot I}{G_t \cdot L_f} &amp; &amp; \frac {\text {L-coins}}{\text {L-block}} &amp; &amp; \text {} \nonumber
\\[0.5em] \therefore \sum \limits _{C \in \{L, R\}} \frac {C_t \cdot I}{G_t \cdot L_f} = \; &amp; \frac {L_t + R_t}{G_t} \cdot \frac {I}{L_f} &amp; &amp;\frac {\text {L-coins}}{\text {L-block}} &amp; &amp;\text {Sum coins (as a proxy for
weight)} \label {eq:chain-coin-weight2}
\end{align}
```

What does eq. /spanspan class="textup"(/spana href="whitepaper.html#lateximage-whitepaper-63"23/span/a)/span/span imply if $L$and $R$are the only two chains in a context like section /spana href="whitepaper.html#lateximage-whitepaper-63"2.4.1/span/a? Notice that, in this case,
$L_t + R_t = G_t, thenetwork - widecurrencysupply.Oneimplicationisthatweight(measuredincoins)effectivelycounts/spanemhowmuchofthefullnetwork/span/emiscontributingtoChainL'ssecurity - - -$
$representedviathecoinsthatwereminted in those contributing blocks. It'seasiertoseeineq. /spanspanclass = "textup"(/spanahref = $"whitepaper.html#$lateximage - whitepaper - 63"24/span/a)/span/spanasthesumcollapsesto/spanspanclass = "textrm textmd textup textsc"/spanspanclass = "numerator"I/span/span//spanspanclass = "denom$
If the all chains in the network are functioning well, we should expect that summing a chain's weight in coins */spanemover the full history of the chain/span/em* should be close to the sum of all coins minted through block rewards. Of course, this is only useful over */spanemmultiple/span/em* chains. **/spanbIf a single, traditional blockchain tried to do this, then all chain-weights would be basically identical!/span/b**/spansup30/span/sup This happens because these conversion methods */spanemdon't try to convert work done at different times./span/em* PoR only ever converts */spanemnear-simultaneous work/span/em*, i.e., if the coin-weights of reflecting blocks are summed, that is always converted to local work */spanemwith respect to some specific moment in time./span/em*
While measuring weight in coins (in this case, at least) seems to have some meaning, we probably shouldn't */spanemleave/span/em* chain-weight in those units. The difficulty of a PoW network converts network size (participation) into hashes, and it is adjusted regularly. If a chain-weight measurement doesn't account for this, then */spanemhow does it include participation at all?/span/em* Without including participation in chain-weight, how can two local alternate histories be meaningfully compared? When measuring and converting chain-work, we */spanemalways/span/em* want to convert confirmations or coins back to meaningful units which factor in */spanemparticipation/span/em* in some way.

/spandiv role="note" class="footnotes"
/spanp

/spansup30/span/sup&nbsp; This may be a new criticism of PoS. In essence: a blockchain needs something like a DAA to factor-in participation. PoS chains use */spanemcoins/span/em* instead of */spanemhashes/span/em*, but */spanemcoins/span/em* will never provide a way to determine which chain has higher participation. Moreover, */spanemcoins/span/em* are actually a very */spanembad/span/em* way to measure participation (for a standalone PoS chain), because the */spanemmost valuable future network/span/em* is one where coins are being used for */spanemactual trade/span/em*, and this must happen at the expense of the number of coins used for staking. Thus, PoS chains */spanemcan only ever have objectively secure fork-rules/span/em* when other factors are included in their conversion contexts (like using PoR with a PoW chain, or some system of automatic checkpoints). One thing PoS chains could try is: measuring weight */spanemin another chain's hashes/span/em* via PoR.
/span/p

/span/div

```
<!--................................-->
/spanh5 id="autosec-201"/spanspan class="sectionnumber"2.6&#x2003;/span/spanReflection Between PoW and PoS Chains/span/h5



/spandiv
    class="tcolorbox"
    style=" border: 1px solid #404040; background: #ECECEC; "

/spandiv
    class="tcolorboxupper"
    style=" color: #000000; "
/spanspan role="note" class="marginpar"/spanbAside/span/b/span/span Whether PoS systems /spanemcan/span/em be secure is not a focus of this paper. There are still /spana
href="https://github.com/zack-bitcoin/amoveo-docs/blob/master/other_blockchains/proof_of_stake.md" target="_blank" criticisms of PoS/span/a without /spana
href="https://github.com/zack-bitcoin/amoveo-docs/blob/master/other_blockchains/the_defence_of_pos.md" target="_blank" adequate answers/span/a./spansup31/span/sup The intention of sections like this is not to endorse PoS, but
rather to explore what is possible /spanemif/span/em PoS can be done securely.    /span/div
```

/spandiv role="note" class="footnotes"
/spanp

/spansup31/span/sup&nbsp; Links are to /spana href="https://web.archive.org/web/20220309063128/https://github.com/zack-bitcoin/amoveo-docs/blob/master/other_blockchains/proof_of_stake.md" target="_blank" Proof of Stake/span/a (criticisms) by Zack Hess and /spana href="https://web.archive.org/web/20220309063254/https://github.com/zack-bitcoin/amoveo-docs/blob/master/other_blockchains/the_defence_of_pos.md" target="_blank" The Defense of PoS/span/a (replies).

/span/p

/span/div /span/div

Perhaps one of the most interesting features of /spanem*Proof of Reflection*/span/em is that PoW chains and PoS chains can reflect one another. Up till now, we've contextualized the weight of a reflection via the /spanem*work*/span/em required to produce a block. But the concept of /spanem*work*/span/em does not neatly apply to foundational consensus mechanisms that do not require the utilization of some physical resource — such as PoS.

/spandiv
    class="tcolorbox"
    style=" border: 1px solid #404040; background: #F2F2F2; "

/spandiv
    class="tcolorboxupper"
    style=" color: #000000; "

/spanspan role="note" class="marginpar"**/spanbTerm/span/b**/span/span **/spanb/spana href="whitepaper.html#??"Foundational Consensus Mechanisms/span/a/span/b**: Those mechanisms, like PoW and PoS, which can work in some /spanem*standalone*/span/em fashion; PoR is a cross-chain /spanem*extension*/span/em to such mechanisms. /span/div /span/div

Putting the issue of /spanem*conversion*/span/em aside for a moment, is it possible /spanem*in principle*/span/em for PoW and PoS chains to reflect one another? Yes. Additionally, PoR provides decisive advantages /spanem*both*/span/em for PoW chains /spanem*and*/span/em PoS chains, though there are some additional problems that must be solved, too.

If a PoW chain is reflected in a PoS chain, then an attacker will likely need more than just computational resources to attack the PoW chain. Consider a PoW chain and a PoS chain that share a root token, and each chain hosts approximately 50% of the total supply. If the two chains have equal block production frequencies, then (using **??**) 50% of the network's security comes from each chain.

Consider an attack on the PoW chain and presume that the difficulty on the PoW chain is constant over the attack, i.e., the PoW chain's difficulty doesn't adjust quickly enough to react to the attack. Additionally, assume the attacker has /spanem*not*/span/em been contributing to the network before the attack, i.e., their hash-rate is not accounted for in the PoW chain's difficulty. Given the two chains are mutually reflecting, half of the network's security is provided by the PoS chain (and thus immune to the attacker in this case). Therefore, a successful attacker — /spanem*using the traditional method of mining a competing chain-segment in private*/span/em — must generate more blocks than both chains combined. That means the attacker needs /spanem*twice*/span/em the honest hash-rate for a guaranteed successful attack.

However, consider the case that /spanem*the security contribution of the PoW chain is /spanspan class="uline"capped/span/span at 50%*/span/em — i.e., capped at the proportion of root tokens hosted on that chain. For our purposes, this situation is approximately equivalent to that where the PoW chain has a /spanem*perfect*/span/em difficulty adjustment algorithm, i.e., the network instantly adapts to keep the block production frequency constant. For the sake of this demonstration, assume that these chains /spanem*retroactively*/span/em adjust block weightings to ensure this cap holds. Let p > 0 be the honest miners' contribution to /spanem*overall*/span/em network security, and q > 0 be the attacker's contribution. As the PoW contribution to overall security is capped at 50%, the equality p + q = 0.5 is enforced. In this case, the attacker will have a maximum chain-weight contribution rate of

$\frac{1}{2\cdot\frac{q}{q+p} andthehonestchain-segmentswillhaveamaximumcontributionrateof \frac{1}{2}\cdot\frac{p}{q+p}+\frac{1}{2}.Theconditionforasuccessfulattackisshownineq. /spanspanclass="textup"(/spanahref="}$whitepaper.html$\#lateximage-whitepaper-63"25/span/a)/span/span,andtheinequalityhasnosolutions./spanspanclass="hidden"\backslash(\backslash seteqnumber\{0\}\{\}\{24\}\backslash)/span/span$ <!--

$$\begin{aligned} \frac{1}{2}\cdot\frac{q}{q+p} &> \frac{1}{2}\cdot\frac{p}{q+p}+\frac{1}{2} \\ q &> p+(q+p) \\ 0 &> 2p && \text{which is a contradiction since } p>0 \qquad (25) \end{aligned}$$

-->

```
\begin{align}
&amp;&amp; \frac {1}{2} \cdot \frac {q}{q + p} &amp; &gt; \frac {1}{2} \cdot \frac {p}{q + p} + \frac {1}{2} \notag \\ &amp;&amp; q &amp; &gt; p + (q + p) \notag \\ &amp;&amp; 0 &amp; &gt; 2p &amp;&amp; \text {which is a contradiction
since\;} p &gt; 0 \label {eq:refl-pow-pos-1}
\end{align}
```

Given the right set-up, a PoW chain gains an /spanem*incredible*/span/em security advantage from mutual reflection with a PoS chain.

/spandiv
    class="tcolorbox"
    style=" border: 1px solid #404040; background: #ECECEC; "

/spandiv
    class="tcolorboxupper"
    style=" color: #000000; "

/spanspan role="note" class="marginpar"**/spanbAside/span/b**/span/span **/spanbNote:/span/b** The attack scenario above assumes that the attacker is not attacking the PoS chain that is reflecting the PoW chain. That is not a safe assumption. Additionally, with traditional blockchains (which are trees), an empty-block DoS is possible — this is addressed in section /spana href="whitepaper.html#lateximage-whitepaper-63"4.8/span/a. /span/div /span/div

What about the PoS chain, though; what benefits does it gain from this relationship? The answer here is simple: by using mutual PoR with a PoW chain, the PoS chain gains /spanem*thermodynamic security*/span/em; the PoS chain's history is /spanem*thermodynamically secured*/span/em by the PoW chain. **/spanbThis solves the /spanem*Nothing at Stake*/span/em problem for any well constructed PoS scheme./span/b**/spansup32/span/sup Furthermore, it is possible for error-correction methods like /spanem*slashing*/span/em to be implemented /spanem*on the PoW chain*/span/em, not the PoS chain. Moving the staking and error correction methods to a different chain will require subtle and precise protocol design, but such changes are /spanem*in principle*/span/em possible with tolerable overhead.

There are some (as yet) unsolved problems that arise through this design, such as the /spanem*economic*/span/em details of managing block rewards across the PoW and PoS chains. Given that solutions to this problem likely depend on the specific details of the relevant PoS systems, this problem is not addressed here. Note: conversion methods for reflected weight, like **??**, will work provided a well defined /spanspan class="textsc"WeightOf/span/span function exists.

There are some other conjectured solutions to the /spanem*Nothing at Stake*/span/em problem.

/spandiv role="note" class="footnotes"
/spanp
/spansup32/span/sup&nbsp; I consider the */spanemNothing at Stake/span/em* problem and */spanemlong range/span/em* attacks to be two sides of the same coin. Maybe it's worth explicitly mentioning that mutual PoR solves long-range attacks, too.
/span/p

/span/div
/spandiv
class="tcolorbox"
style=" border: 1px solid #404040; background: #ECECEC; "

/spandiv
class="tcolorboxupper"
style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbQuote/span/b**/span/span /spanspan role="note" class="marginpar"**/spanbQuote/span/b**/span/span Long-range “nothing-at-stake” attacks are circumvented through a simple “checkpoint” latch which prevents a dangerous chain-reorganisation of more than a particular chain-depth. To ensure newly-syncing clients are not able to be fooled onto the wrong chain, regular “hard forks” will occur (of at most the same period of the validators' bond liquidation) that hard-code recent checkpoint block hashes into clients.
.7ex]0.5pt

/spandiv class="flushright" /spani— *Dr. Gavin Wood; /spana href="https://web.archive.org/web/20240927041732/https://gateway.pinata.cloud/ipfs/QmbH4TzUB7izvuwidG598DNnk3Nmd1aWEyf8KLxeAkrvkK" target="_blank" Polkadot Whitepaper/span/a, s5.2/span/i* /span/div /span/div /span/div

/spandiv
 class="tcolorbox"
 style=" border: 1px solid #404040; background: #ECECEC; "

/spandiv
 class="tcolorboxupper"
 style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbQuote/span/b**/span/span /spanspan role="note" class="marginpar"**/spanbQuote/span/b**/span/span Provided that stakeholders are frequently online, nothing at stake is taken care of by our analysis of forkable strings (even if the adversary brute-forces all possible strategies to fork the evolving blockchain in the near future, there is none that is viable), and our chain selection rule that instructs players to ignore very deep forks that deviate from the block they received the last time they were online.
.7ex]0.5pt

/spandiv class="flushright" */spani— /spana href="https://web.archive.org/web/20240927042139/https://gateway.pinata.cloud/ipfs/QmWCAHyi35SeXH2E4e8jRVk7yNse2x6D14uPfABnhagbvN" target="_blank" Ouroboros: A Provably Secure Proof-of-Stake Blockchain Protocol/span/a, s10/span/i* /span/div /span/div /span/div

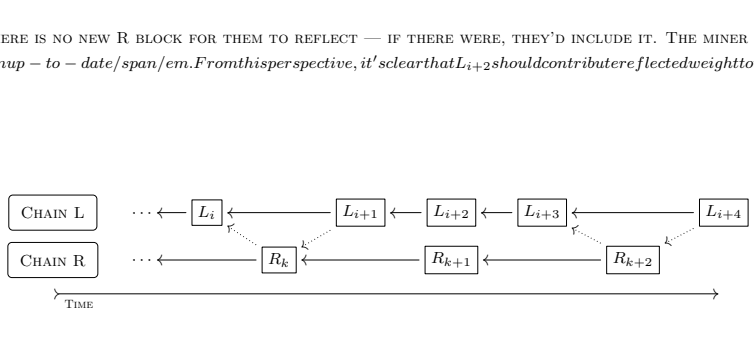
These two examples solve the */spanemNothing at Stake/span/em* problem via mechanisms that are */spanemexternal/span/em* to the protocol itself, i.e., hard-coded checkpoints and the requirement that nodes are online "frequently".

The solution provided by mutual reflection with a PoW blockchain — i.e., thermodynamic security — is provided */spanemby the protocol itself/span/em* and can only */spanemincrease/span/em* the security of PoS mechanisms. Thus, UT's solution to */spanemNothing at Stake/span/em* is qualitatively superior.

`<!--................................-->`

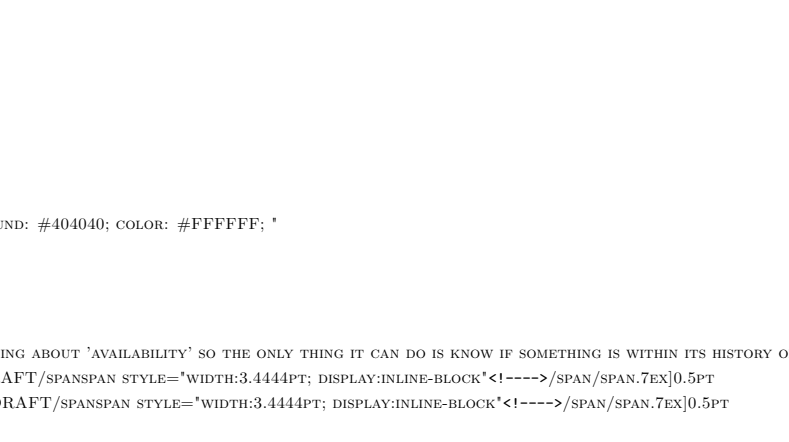
/spanh5 id="autosec-206"/spanspan class="sectionnumber"2.7&#x2003;/span/spanCounting Work/span/h5

When chains L and R have the same block frequency and produce blocks in an orderly fashion, it's trivial to count how much work is contributed by each PoR. However, real-world blockchains do not produce blocks like this — even if chains L and R have the same block frequencies, sometimes L will produce 2 or 3 (or more) blocks before R produces its next one (or vice versa).
/spanspan class="hidden" \(\setEqNumber{0}{}{25}\)/span/span `<!--`



```
-->
\begin{equation*}
\xyMChains { \xyL {\text {Chain L}} &amp; \cdots &amp; \xyB {L_i} \ar [l] &amp; &amp; \xyB {L_{i+1}} \ar [ll] \ar @{.&gt;}[ld] &amp; \xyB {L_{i+2}} \ar [l] &amp; \xyB {L_{i+3}} \ar [l] &amp; \\ \xyL {\text {Chain R}} &amp; \cdots &amp;
&amp; \xyB {R_j} \ar [ll] \ar @{.&gt;}[lu] &amp; &amp; &amp; &amp; \xyB {R_{j+1}} \ar [llll] \ar @{.&gt;}[lu] \\ \xyTimeHoriz {rrrrrrr} &amp; &amp; &amp; &amp; &amp; &amp; &amp; \\ }
\end{equation*}
```

Should R_{j+1} include 3 PoRs, or just 1? How should R_{j+1} calculate the weight of those PoRs and the total weight contributed to chain R?
If R_{j+1} had reflected only L_{i+1}, then it's clear that we would count work from both L_{i+1} and R_{j+1} — that much is easy. However, R_{j+1} reflected */spanemthree/span/em* L headers. Should R_{j+1} count work from */spanemall three/span/em*, or something else? Why?
Consider the situation from the miner's perspective (the miner of L_{i+2}). There is no new R block for them to reflect — if there were, they'd include it. The miner is effectively claiming — via the absence of a new reflection — that L_{i+2}'s parent (L_{i+1}) reflects the most recent R header (R$_j$)$; that L_{i+1} is still /spanemup-to-date/span/em. From this perspective, it's clear that L_{i+2} should contribute reflected weight to the best R headers that were most recently reflected: in this case, just R_j$.
We can evaluate L_{i+3} the same way.
So, yes R_{j+1} should include and count work from all 3 PoRs in this case.
What about a more complicated example?
/spanspan class="hidden" \(\setEqNumber{0}{}{25}\)/span/span `<!--`



```
-->
\begin{equation*}
\xyMChains { \xyL {\text {Chain L}} &amp; \cdots &amp; \xyB {L_i} \ar [l] &amp; &amp; \xyB {L_{i+1}} \ar [ll] \ar @{.&gt;}[ld] &amp; \xyB {L_{i+2}} \ar [l] &amp; \xyB {L_{i+3}} \ar [l] &amp; &amp; \xyB {L_{i+4}} \ar [ll] \ar @{.&gt;}[ld] \\
\xyL {\text {Chain R}} &amp; \cdots &amp; &amp; \xyB {R_k} \ar [ll] \ar @{.&gt;}[lu] &amp; &amp; \xyB {R_{k+1}} \ar [ll] &amp; &amp; \xyB {R_{k+2}} \ar [ll] \ar @{.&gt;}[lu] &amp; \\ \xyTimeHoriz {rrrrrrrr} &amp; &amp; &amp; &amp; &amp;
&amp; &amp; &amp; \\ }
\end{equation*}
```

R_{k+2} can be evaluated by similar logic to R_{j+1} above, but what about L_{i+4}?
Does L_{i+4} count the weight of R_{k+1}? If so, does it count towards L_{i+3}'s chain-weight?
The difficulty in answering these questions — with what we've covered so far — is that */spanemthe fork-rule doesn't care/span/em* where the weight is added, just that it */spanemis/span/em* added. When the fork rule compares L_{i+4} to an alternative block, only the */spanemtotal/span/em* chain-weight of each is relevant. However, to understand exactly how PoRs */spanemshould/span/em* be counted, we need to know more.
The essence of PoR is */spanemthe confirmation of another chain's /spanspan class="uline"history/span/span/span/em* — so a reflection cannot contribute to a block which exists */spanemin the reflecting block's /spanspan class="uline"future/span/span/span/em*. It must be that reflections contribute to the */spanemmost recent common ancestor/span/em* of all */spanempossible/span/em* L blocks which */spanemcould/span/em* include that specific PoR.
This leads us to a simple and elegant way to count reflected work: follow the arrows back from the reflecting block to the most recently reflected local block.
L_{i+4}'s reflection of R_{k+2} contributes chain-weight to L_{i+3} because */spanemany/span/em* block that builds on L_{i+3} could reflect R_{k+2}. Following the arrows: $R_{k+2} \dashrightarrow L_{i+3}$.
Similarly, L_{i+4}'s reflection of R_{k+1} (which could be either explicit or implicit via R_{k+2}) must contribute chain-weight to L$_i because /spanemany/span/em block building on L_i could reflect R_{k+1} and include that PoR. Following the arrows: R_{k+1} \longrightarrow R_k \dashrightarrow L_i$.
We can also say more about R_{k+2}'s reflection of L_{i+3}, now, too. Particularly, that the reflected work of $L_{i+1} \cdots L_{i+3} should contribute to R_k, /spanemnot/span/em R_{k+1}$.
.7ex]0.5pt/spanspan style="width:3.4444pt; display:inline-block"`<!---->`/span/spanBegin DRAFT/spanspan style="width:3.4444pt; display:inline-block"`<!---->`/span/span.7ex]0.5pt

/spandiv
class="tcolorbox"
style=" border: 1px solid #404040; background: #FCD1E9; "

/spandiv
class="tcolorboxtitle"
style=" border-top: 1px solid #404040; border-bottom: 1px solid #404040; background: #404040; color: #FFFFFF; "
TODO [4]/span/div
/spandiv
class="tcolorboxupper"
style=" color: #000000; "
consider a situation where an attacker reflects 1 honest block and 1 competing attacker block, and the attacker block is not released. for the sake of the argument, assume the reflected blocks weight much less than the reflecting block. /span/div /span/div

/spandiv
class="tcolorbox"
style=" border: 1px solid #404040; background: #FCD1E9; "

/spandiv
class="tcolorboxtitle"
style=" border-top: 1px solid #404040; border-bottom: 1px solid #404040; background: #404040; color: #FFFFFF; "
TODO [5]/span/div
/spandiv
class="tcolorboxupper"
style=" color: #000000; "
consensus enforces availability of refl blocks but counting work doesn't know anything about 'availability' so the only thing it can do is know if something is within its history or not. asymmetry. /span/div /span/div
.7ex]0.5pt/spanspan style="width:3.4444pt; display:inline-block"`<!---->`/span/spanEnd DRAFT/spanspan style="width:3.4444pt; display:inline-block"`<!---->`/span/span.7ex]0.5pt
.7ex]0.5pt/spanspan style="width:3.4444pt; display:inline-block"`<!---->`/span/spanBegin DRAFT/spanspan style="width:3.4444pt; display:inline-block"`<!---->`/span/span.7ex]0.5pt

<!--................................-->
/spanh5 id="autosec-207"/spanspan class="sectionnumber"2.8&#x2003;/span/spanIncompatibility between Merged Mining and PoR/span/h5

/spanul style="list-style-type:none"

/spanli
- Merged Mining allows attacking merged chains at 0 cost.

/span/li
/spanli
- that means that if a parent chain and a merged mined child chain where to reflect one another, then the weight contributed via merged mining must be 0 — no additional work was actually done beyond that of the parent-chain.

/span/li
/spanli
- Also, if some other chain reflects both a parent chain */spanemand/span/em* a merged mined child chain, then the net benefit is equal to */spanemonly/span/em* the work contributed by the reflecting parent chain.

/span/li
/span/ul

/spandiv
class="tcolorbox"
style=" border: 1px solid #404040; background: #FCD1E9; "

/spandiv
class="tcolorboxtitle"
style=" border-top: 1px solid #404040; border-bottom: 1px solid #404040; background: #404040; color: #FFFFFF; "
TODO [6]/span/div
/spandiv
class="tcolorboxupper"
style=" color: #000000; "
PoR in general: (nb: check if this is sufficiently answered) reflect only chains that reflect your history; if they favor a different history, then you should be building on that history instead, so don't reflect those blocks — i.e. ppl should calculate weight to be 0. /span/div /span/div
.7ex]0.5pt/spanspan style="width:3.4444pt; display:inline-block"<!---->/span/spanEnd DRAFT/spanspan style="width:3.4444pt; display:inline-block"<!---->/span/span.7ex]0.5pt

<!--...............................-->
/spanh4 id="autosec-208"/spanspan class="sectionnumber"3&#x2003;/span/spanUT$_i$: Constructing /spanemUltra Terminum/span/em/span/h4

/spanemProof of Reflection/span/em can be used to build UT$_1$ — an O(c^2)foundationforablockchainnetwork(called/spanemthesimplex/span/em).Thissectiondetailstheconstructionofsuchafoundation,andhowitcanbeextendeduptoUT$_3$ − − − whichhasO(c^4)complexity.TheO(n)scalingconfiguration(UT$_\aleph$)isdetailedinsection /spanahref = ”whitepaper.html#lateximage − whitepaper − 63”6/span/a.

Such a foundation (/spanemthe simplex/span/em) is /spanemnot/span/em a sharded blockchain — there's no requirement that participating chains are interchangeable or using the same primitives. This was demonstrated via the example in section /spana href="whitepaper.html#sec:two-blockchains"2.2/span/a. Rather, /spanemthe simplex/span/em is an emergent construct that is created via the /spanemrelationships/span/em between blockchains. Instead of one blockchain being split into many (as occurs with sharding), /spanemthe simplex/span/em is many blockchains becoming one coherent network.

<!--...............................-->
/spanh5 id="autosec-209"/spanspan class="sectionnumber"3.1&#x2003;/span/spanGeneralizing Reflection/span/h5

*/spanem*Proof of Reflection*/span/em* is, in essence, the idea that a chain can acknowledge that its history has been confirmed by a different chain, and that this fact can be used to share security between chains. That is a simplification, but
it */spanemis/span/em* the essence of it.
In principle, the necessary capabilities (and actions) that some chains, C$_A andC_B, musthave(anddo)inorderforC_A tobereflectedbyC_B are:$
/spanul style="list-style-type:none"

/spanli
1. The headers of C$_A canbe(andare)freelyrecorded --- promptlyandunambiguously --- inC_B;$
/span/li
/spanli
2. The headers of C$_B canbe(andare)freelyrecorded --- promptlyandunambiguously --- inC_A; and$
/span/li
/spanli
3. C$_A isableto(anddoes)promptlyprovethatitspastheadershavebeenrecordedinC_B, andhasfullknowledgeofwhichheadershavebeenrecorded.$
/span/li
/span/ul
The benefits from */spanem*Proof of Reflection*/span/em* begin as soon as C$_A integratesthisknowledgeintoitschain - weightingalgorithm, byamethodsuitablysimilarto$ **??** $andalgorithm$ /spanahref = "whitepaper.html#$lateximage - whitepaper - 63$"2/span/a.
If C$_A andC_B aredoing/spanemmutual/span/emProofofReflection, thenbothchainsmustsatisfyallrequirements.$
Is
C$_A ableto/spanemsimultaneously/span/emdoreflectionwithmorethanoneotherchain, e.g., C_C...C_Z? Yes. Thereisnothingthatwehavecoveredsofarthatwouldpreventthis. If/spanemProofofReflection/span/emisviablewithasingleotherchain, thenitisviablewith/spanemmany/span/emotherchains. However, thedynamicsdobecomeincreasinglycomp$
In order to support arbitrarily many reflections, we need to modify /spanspan class="textsc"ReflectedBlockWeight/span/span from **??** as shown in **??**.
Note that **??** integrates the cap on weight contributed by each reflecting chain, as suggested in section /spana href="whitepaper.html#lateximage-whitepaper-63"2.6/span/a.

<!--................................-->
/spanh5 id="autosec-210"/spanspan class="sectionnumber"3.2&#x2003;/span/spanUT$_1$: The Simplex/span/h5

/spanfigure id="autoid-20" class="figure "

/spandiv class="minipage" style="vertical-align:middle ; justify-content:flex-start ; "

/spana href="./includes/ut/diags/simplex__2__sag.png" target="__blank" /spanimg
src="./includes/ut/diags/simplex__2__sag.png"
style="
height:128pt ;
"
class="inlineimage"
alt="(image)"
/span/a

/spandiv class="figurecaption"(a) Proof of Reflection between 2 blockchains. A 1-simplex. The most basic non-trivial simplex.
/span/div
/span/div

/spandiv class="minipage" style="vertical-align:middle ; justify-content:flex-start ; "

/spana href="./includes/ut/diags/simplex_7_sag.png" target="_blank" /spanimg
src="./includes/ut/diags/simplex_7_sag.png"
style="
height:128pt ;
"
class="inlineimage"
alt="(image)"
/span/a

/spandiv class="figurecaption"(b) Proof of Reflection between 7 blockchains; a 7-chain simplex; a 6-simplex.
/span/div
/span/div

/spandiv class="minipage" style="vertical-align:middle ; justify-content:flex-start ; "

/spana href="./includes/ut/diags/simplex__17__sag.png" target="__blank" /spanimg
src="./includes/ut/diags/simplex__17__sag.png"
style="
height:128pt ;
"
class="inlineimage"
alt="(image)"
/span/a

/spandiv class="figurecaption"(c) A 17-chain simplex; a 16-simplex. It has 136 unique mutual reflections in total.
/span/div
/span/div

/spandiv class="figurecaption"Figure 12: Simplexes of increasing capacity. Vertices are simplex-chains. Edges are the reflections between simplex-chains.
/span/div /span/figure

/spandiv
class="tcolorbox"
style=" border: 1px solid #404040; background: #F2F2F2; "

/spandiv
class="tcolorboxupper"
style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbTerm/span/b**/span/span **/spanb/spana href="whitepaper.html#??"Simplex/span/a/span/b**: The single coherent structure that emerges from a collection of blockchains that mutually reflect each other. /span/div /span/div
When two or more blockchains */spanemmutually reflect/span/em* each other, they form a */spanemsimplex/span/em*/spansup33/span/sup. For the sake of brevity: all */spanemreflections/span/em* within a simplex are */spanemmutual reflections/span/em*, and I will omit */spanemmutual/span/em* from now on when discussing them. Examples of simplexes are shown in fig. /spana href="whitepaper.html#lateximage-whitepaper-63"12/span/a.
When a blockchain is part of a simplex, it is called a */spanemsimplex-chain/span/em* (as distinct from */spanemdapp-chains/span/em*).

/spandiv role="note" class="footnotes"
/spanp
/spansup33/span/sup&nbsp;The name is taken from geometry (particularly: the higher-dimensional kind). A simplex, for a given dimensionality, is the uniquely simplest polytope; e.g., a line in 1D space, a triangle in 2D space, a tetrahedron in 3D space, etc. A k-dimensional simplex is known as a k-simplex. As shown in fig. /spana href="whitepaper.html#lateximage-whitepaper-63"12/span/a, a particular 2D projection of a k-simplex (which produces a regular (k+1)-gon with additional edges between all pairs of vertices), is identical to a diagram of all possible mutual reflections between k+1 blockchains, where each chain is represented by a vertex and each mutual reflection is represented by an edge.
/span/p

/span/div
/spandiv
    class="tcolorbox"
    style=" border: 1px solid #404040; background: #F2F2F2; "

/spandiv
    class="tcolorboxupper"
    style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbTerm/span/b**/span/span **/spanb/spana href="whitepaper.html#??"Simplex-chain/span/a/span/b**: A blockchain that is part of a */spanemsimplex/span/em*; it mutually reflects all other simplex-chains in that simplex. /span/div /span/div
To maintain consistency with the geometric usage of the term */spanemsimplex/span/em*: a simplex with k+1 chains is called a k-simplex or a (k+1)-chain simplex/spansup34/span/sup. In a k-simplex, each simplex-chain has k reflections (one reflection for each of the other simplex-chains). A k-simplex has, in total, k+1 $(_{2reflections.})$
The security of simplexes is discussed in section /spana href="whitepaper.html#lateximage-whitepaper-63"4.10.1/span/a.

/spandiv role="note" class="footnotes"
/spanp

/spansup34/span/sup&nbsp;NB: I will ignore this distinction for $k \gg 1$.

/span/p

/span/div

<!--................................-->
/spanh5 id="autosec-216"/spanspan class="sectionnumber"3.3&#x2003;/span/spanUT$_1$: Scaling Complexity Intuition/span/h5

/spandiv
    class="tcolorbox"
    style=" border: 1px solid #404040; background: #ECECEC; "

/spandiv
    class="tcolorboxupper"
    style=" color: #000000; "

/spanspan role="note" class="marginpar"**/spanbAside/span/b**/span/span Although we will rigorously investigate UT's scaling complexity in section /spana href="whitepaper.html#lateximage-whitepaper-63"5/span/a, it will prove useful to introduce $N_1 hereandtogainanintuitionforUT'sscalabilitybeforesection\ /spanahref=$ "whitepaper.html$\#lateximage-whitepaper-63"4/span/a./span/div$ /span/div

How scalable is UT$_1$?

Let us start by assuming that a single blockchain has O(c) capacity (measured in transactions per second) and is thus O(c) scalable. We'll also assume that the */spanemheaders/span/em* are O(1) bytes (i.e., a constant size).

Note that a single blockchain is a 0-simplex.

A 1-simplex has two blockchains, and each chain must record the headers of the other and the respective PoRs. For the sake of simplicity, let's assume that the PoRs are also O(1). Thus, each chain loses O(1) of its O(c) capacity, and the network's total capacity is 2O(c)-2O(1) = O(c).

Since each chain has finite capacity, there is an absolute maximum size to a simplex. We can keep adding more chains until all blocks are 100% full of headers and PoRs. However, this system has 0 capacity for transactions. So, there must be a capacity maxima with more than 2 simplex-chains and fewer than the absolute maximum.

What we're observing is that the overall capacity is the */spanemproduct/span/em* of the size of the simplex (in chains) and each chain's remaining capacity — a quadratic relationship.

Let's call the proportion of capacity for transactions t, and the proportion for reflections r. We know that t + r = 1, since t+r is 100% of a chain's capacity. However, the */spanemnetwork-wide/span/em* capacity is proportional to the */spanemproduct/span/em* of t and r. Maximizing t r suggests that a chain's capacity should be split approximately 50/50 between t and r, so

t=r=$/spanspanclass="textrm\ textmd\ textup\ textsc"/spanspanclass="numerator"1/span/span//spanspanclass="denominator"2/span/span/span/span$=O(1). Given that t and r are proportions of the capacity, we can say O(ct) + O(cr) = O(c).

Thus, the overall capacity is given by O(ct) $\cdot O(cr) = O(c^2)$.

This relationship is analogous to the relationship between a rectangle's maximum area for a given perimeter length: the area scales with the square of the perimeter (and such a rectangle is a square).

Let's say $N_1 isthenumberofchainswhichmaximizesasimplex'scapacity(fortransactions).TogetafeelforN_1, let'sestimateitusingexamplechainsbasedonmodifiedBitcoinparameters.$

First, we'll add a merkle root of headers and PoRs to the header, increasing header size from 80 to 112 bytes. Second, we'll reduce the block frequency down from 10 minutes to 1 minute, and scale the block limit down from 1 MB to 100 kB (this will give us more realistic numbers). Let's assume the merkle branch leading to a header is 256 bytes, so the total PoR overhead is 368 B / block / chain. We want to use 50% of the overall capacity (100 kB / block) for PoRs, so each block should contain ∼50 kB worth of PoRs. Dividing the PoR capacity by the per-chain overhead gives us $N_1 \approx 139 chainsandanetwork-widecapacityequivalentto{\sim}70 Bitcoinnetworks.$

<!--................................-->
/spanh5 id="autosec-217"/spanspan class="sectionnumber"3.4&#x2003;/span/spanUT$_2$: Dapp-chains/span/h5

/spanemDapp-chains/span/em are a method by which /spanemUltra Terminum/span/em exceeds O(c^2)*scaling/spanemwithout/span/emusingthemethoddescribedinsection* /spanahref = "whitepaper.html#*lateximage − whitepaper* − 63"6/span/a.*Tobeclear* : *theO*(c^2)*configurationofUTiscompatiblewiththatothermethod*; *dapp* − *chainsarea*/*spanemseparateandindependent*/*span*/*emmethodofscaling*.*However*, *thereare*/*spanemdecisive*/*span*/*emreasonstointroduceanduse*/*spanemdapp* − *chains*/*span*/*em*.*Dapp* − *chainsprovidefeaturesthattheO*(n)*scalingconfigurationalonecannot*/*spanemeasily*/*span*/*emprovide*.*Additionally*, *dapp* − *chainsincreasethesimplex*′*sscalabilitytoO*(c^3)*orO*(c^4).

/spandiv
class="tcolorbox"
style=" border: 1px solid #404040; background: #F2F2F2; "

/spandiv
class="tcolorboxupper"
style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbTerm/span/b**/span/span **/spanb/spana href="whitepaper.html#??"Dapp-chain/span/a/span/b**: An /*spanemapplication-specific/span/em* child-chain that is secured via the parent-chain. Dapp-chains may have architecturally distinct state- and transaction-schemes (distinct from those schemes used in the simplex, and other dapp-chains). /span/div /span/div

Intrinsically, dapp-chains are not restricted to any particular foundational consensus method. They might use PoW, or PoS, or PoA, or something else. However, dapp-chains also use /*spanemProof of Reflection/span/em* with their host simplex-chain. With a suitable foundational consensus method, PoR enables dapp-chains to be as secure as their host simplex-chain with little overhead. Note that, since dapp-chains can use whichever foundational consensus method, they can optionally have /*spanemtheir own/span/em* root token (and use that for mining rewards, transaction fees, etc).

It's preferable that a simplex-chain validate the headers of its dapp-chains (similar to a light client), though this is not required. For some dapp-chain consensus methods (such as PoS), there might be special primitives that a host simplex-chain must support. However, only that host simplex-chain requires those primitives; other simplex-chains do not./spansup35/span/sup This means that simplex-chains can /*spanemspecialize/span/em* in hosting /*spanemparticular types/span/em* of dapp-chains, providing rich and efficient environments (for nodes of both simplex-chains /*spanemand/span/em* dapp-chains).

Validating dapp-chain headers, on-chain, can be done via the following simple, clean, and extensible method: /*spanemencode dapp-chain headers as simplex-level transactions/span/em*. This means that supporting new dapp-chain consensus methods is about as difficult as introducing new transaction types (or opcodes), and different simplex-chains have a great deal of freedom in choosing which dapp-chain consensus methods to support.

/spandiv role="note" class="footnotes"
/spanp
/spansup35/span/sup&nbsp; The caveat here is that other simplex-chains may need to be capable of validating fraud proofs for the simplex-chain in question. So they don't need these primitives */spanemavailable to local transactions/span/em*, but do need to be capable of executing those primitives if a fraud proof involves one.
/span/p

/span/div
/spandiv
    class="tcolorbox"
    style=" border: 1px solid #404040; background: #F2F2F2; "

/spandiv
    class="tcolorboxupper"
    style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbTerm/span/b**/span/span **/spanb/spana href="whitepaper.html#??"Header-transactions/span/a/span/b**: Dapp-chain headers that are encoded as simplex-level transactions; i.e., they are processed by a simplex-chain as a transaction, but they also function as the header for a dapp-chain block. /span/div /span/div

Practically speaking, a simple input-output transaction system with light scripting capabilities (like that of Bitcoin) can be created to facilitate the necessary primitives. Additionally, different simplex-chains can implement different scripting systems, effectively facilitating */spanemany/span/em* compatible consensus mechanism. There is not much (if any) overhead to using an input-output system like this: a header's parent hash is like a transaction input, the */spanemoutput/span/em* is the header or its hash/spansup36/span/sup, and any other particulars of the header can be treated as an input script to the transaction/spansup37/span/sup.

/spandiv role="note" class="footnotes"
/spanp

/spansup36/span/sup&nbsp;A header-transaction's output script can be generic (or templated) as it is the same for all header-transactions for that dapp-chain. In practice this can be as simple as a single opcode that validates that header. In Bitcoin, an output script is known as the /spankbdscriptPubKey/span/kbd.

/span/p

/spanp

/spansup37/span/sup&nbsp;In Bitcoin, the input script to a transaction is called the /spankbdscriptSig/span/kbd; see /spana href="https://en.bitcoin.it/wiki/Transaction" target="_blank" https://en.bitcoin.it/wiki/Transaction/span/a.

/span/p

/span/div

<!--...............................-->
/spanh6 id="autosec-220"/spanspan class="sectionnumber"3.4.1&#x2003;/span/spanDapp-chain Security/span/h6

If the headers of dapp-chains are simplex-level transactions, what can we say about the security of dapp-chains?

First, notice that there is no substantive difference between standalone headers and header-transactions. That means that */spanemzero-confirmation/span/em* header-transactions are */spanemexactly/span/em* as secure as a standalone counterparts (and at least as secure as zero-confirmation transactions). This is not very secure in the case of a PoW dapp-chain, but it means that a PoS dapp-chain's zero-confirmation header-transactions could be just as secure as blocks from an equivalent standalone PoS blockchain. (It also means that the PoS dapp-chain is more secure after header-transactions are confirmed, compared to the standalone equivalent.)

When a header-transaction is confirmed by the simplex, the corresponding dapp-chain can efficiently use one-way PoR to partially/spansup38/span/sup inherit the security (and security properties) of the host simplex-chain./spansup39/span/sup Similar to mutual PoR, this can provide a */spanemsecurity context/span/em* where otherwise-insecure methods of consensus can be done securely.

With regards to doublespends, one-way PoR means that the reflected chain is */spanemat least/span/em* as difficult to attack as the reflecting chain (as we covered in section /spana href="whitepaper.html#lateximage-whitepaper-63"2.2.4/span/a). Since the parent simplex-chain is as difficult to attack as the complete simplex, each dapp-chain must therefore */spanemalso/span/em* be that difficult to attack. Provided that a dapp-chain's blocks are available, attacking a dapp-chain is as difficult as attacking the entire network.

Note that parent-chains (generally) need to record their child-chains' headers */spanemanyway/span/em*, so this use of one-way PoR — where a simplex-chain reflects child dapp-chains — has near-zero overhead for both the simplex-chain and the dapp-chain.

One major, generic concern for dapp-chains is */spanempreventing DoS attacks/span/em.*/spansup40/span/sup This is one reason to favor PoS (or PoA) dapp-chains over PoW dapp-chains. Another concern is the */spanemavailability/span/em* of dapp-chain blocks.

<!--...............................-->
/spanspan class="paragraph" id="autosec-221"/spanspan class="sectionnumber"3.4.1.1&#x2003;/span/spanSpam, Availability, and Dapp-chains: A PoW/PoS Asymmetry/span/span

PoW Dapp-chains Proof of Work systems operate best when all practicable hashing resources are contributing to the same system. That is: all economically available miners are mining on the same network.
We can explore this with a thought experiment. Say I forked the Bitcoin codebase and created a new genesis block with today's date, which in turn creates a new blockchain network. Crucially, I do not change the hashing algorithm.
How secure would this network be? Given that the actual Bitcoin network has an established hash-base, and there are many old, uneconomical mining units out there, the overwhelming majority of hash power would not be working on my new network (either it would be off, or working on Bitcoin). In fact, even if all the units that were powered off were used to mine my network, it would still have but a small fraction of Bitcoin's hash-rate. The effect of this is that *\/spanemAt any point/span/em* a small proportion of Bitcoin miners could divert their miners and perform a 51% attack against my network. Therefore, *\/spanemGiven Bitcoin exists,/span/em* my network cannot be considered secure in practice.
If two traditional PoW chains share a hashing algorithm, then the one with more hashing power is the only secure candidate.
How does this apply to PoW simplex dapp-chains? First, notice that if a PoW dapp-chain has a comparable difficulty to a simplex-chain, then the simplex as a whole (including other dapp-chains) would benefit if that hashing power were used on the simplex instead of the dapp-chain. Additionally, moving hashing power the other way (from simplex to dapp-chain) would weaken the overall security of the simplex. From this, we can intuit that the most stable, secure system is one that heavily prioritizes mining PoW simplex-chains over PoW dapp-chains. This brings us to our second observation: that PoW dapp-chains will naturally have a much lower difficulty than their host simplex-chains.
In a functioning PoW network, the work serves not only as the method of securing the chain, but also a method to prevent block spam. This is due to the DAA regulating block production rates. DAAs are unreliable, though, when there are large reserves of mining power that might come online at any moment for an unknown period of time, and then disappear. One typical consequence of a sharp drop in mining power is a period where the difficulty is much higher than the remaining miners are capable of meeting, resulting in reduced block production. In extreme cases, this may require a hard-fork to resolve. The usual solution to this is a rule whereby blocks at lower difficulty can be mined, provided that a suitably long duration has passed since the prior block. If PoW dapp-chain headers are verified by the simplex, a hard-fork to change the DAA of a PoW dapp-chain would require the host simplex-chain to be aware of this change. This is not the only issue with block spam, though. If a miner can trivially create blocks (because the difficulty is too low), then they could flood the host simplex-chain with valid (but useless) header-transactions. This is a problem, not just for that dapp-chain, but for all other dapp-chains hosted by that simplex-chain.

/spandiv role="note" class="footnotes"
/spanp
/spansup38/span/sup&nbsp;Some security properties of the host simplex-chain are not inherited. For example: the simplex ensures data availability of simplex-chain blocks, but not dapp-chain blocks. Other security properties, like thermodynamic security, are inherited.
/span/p

/spanp
/spansup39/span/sup&nbsp;Note: PoW dapp-chains will have a much lower difficulty than the host simplex-chain. Although a simplex-chain could do mutual PoR with dapp-chains, this is unnecessary and inefficient — provided that this difficulty asymmetry exists. Although there is no fundamental reason that PoW dapp-chains must have a much lower difficulty, we should take care to avoid any implementation that would compromise or reduce the security of the simplex. Practically, this probably means avoiding PoW dapp-chains (see **??**&nbsp;3.4.1.1).
/span/p

/spanp
/spansup40/span/sup&nbsp; Unfortunately, the strategy we use in section /spana href="whitepaper.html#lateximage-whitepaper-63"4.8.3/span/a to protect simplex-chains does not work here.
/span/p

/span/div
/spandiv
    class="tcolorbox"
    style=" border: 1px solid #404040; background: #ECECEC; "

/spandiv
    class="tcolorboxupper"
    style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbAside/span/b**/span/span These are substantial problems, but might be soluble. The next problem, however, is not.   /span/div /span/div
Spam is not the nail in the coffin for PoW dapp-chains. Rather, that honor falls to */spanemavailability/span/em*.
The problem itself is simple to describe: a malicious miner creates valid blocks, and publishes the header-transactions to the host chain, but does not publish the block bodies to the dapp-chain network.
A key requirement of dapp-chains is that they cannot increase the load of the miner above O(c). This means that processing a single header-transaction cannot exceed O(/spanspan class="textrm textmd textup textsc"/spanspan class="numerator"c/span/span//spanspan class="denominator"N$_D/span/span$/span/span), where
N$_Disthenumberofdapp - chainsthatasimplex - chainiscapableofhosting - -or, moreaccurately, thenumberofheadertransactionsitcanprocessperblock; we'llassumethesearequivalent.IdeallyO(N_D) =$
$O(c), assumingitisnotpossibletodobetterthanO(1)whenprocessingaheader - transaction.$
If we are to have reliable inter-chain operation between simplex-chains and dapp-chains, then we must have some assurance that the recorded blocks are valid, which requires them to be available. Invalid blocks would allow a miner to steal funds via cross-chain transactions, which is obviously bad. Provided that we only allow (public/spansup41/span/sup) dapp-chains which support fraud proofs, then we have the foundation for that assurance. But, fraud proofs only work if honest nodes have access to the block to calculate said fraud proof. It is generally not possible to create a fraud proof if the block is unavailable.
Since availability is a requirement, how can we ensure that dapp-chain blocks are available and not withheld?

/spandiv role="note" class="footnotes"
/spanp
/spansup41/span/sup&nbsp; If a dapp-chain is designed to be private, opaque, or non-financial, then we might not care (or even be able to care) about the validity of its state transition.
/span/p

/span/div
/spandiv
    class="tcolorbox"
    style=" border: 1px solid #404040; background: #ECECEC; "

/spandiv
    class="tcolorboxupper"
    style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbAside/span/b**/span/span Note that *spanemProofs of Availability/span/em* are of no use here, because verifying them requires at least as many bytes as the block/spansup42/span/sup itself; it is just as bandwidth intensive as downloading all blocks. So, at best, we might be able to do some kind of challenge-response, but, practically speaking, this isn't the solution we're looking for. /span/div

/spandiv role="note" class="footnotes"
/spanp

/spansup42/span/sup&nbsp; This is somewhat intuitive since proofs of availability allow for the reconstruction of the original block.

/span/p

/span/div /span/div

I suspect that, for PoW dapp-chains, it is */spanemin principle/span/em* not possible to ensure the availability of blocks. This is due to the principles by which PoW blockchains operate. Particularly, */spanemproofs of work/span/em* are created */spanemin isolation/span/em.* Only */spanemafter/span/em* a PoW is found is a block shared with the network (see fig. /spana href="whitepaper.html#lateximage-whitepaper-63"13a/span/a). Therefore, it is always possible to partially share a block in (pure) PoW networks.

/spanfigure id="autoid-21" class="figure "

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "

/spana href="25-constructing-ut-10-pow-chain-fc_sag" target="_blank" /spanimg
src="25-constructing-ut-10-pow-chain-fc_sag"
class="inlineimage"
alt="(image)"
/span/a

/spandiv class="figurecaption"(a) The PoW block creation process (simplified).
/span/div
/span/div

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "

/spana href="25-constructing-ut-11-pos-chain-fc_sag" target="_blank" /spanimg
    src="25-constructing-ut-11-pos-chain-fc_sag"
    class="inlineimage"
    alt="(image)"
/span/a

/spandiv class="figurecaption"(b) An example of the kind of PoS block creation process we're after.
/span/div
/span/div

/spandiv class="figurecaption"Figure 13: The block creation process (simplified) for PoW chains and some variants of PoS chains.
/span/div /span/figure

PoS Dapp-chains First, it's important to note that there are many variants within the "Proof of Stake" family of consensus methods. For our purposes right now, we're interested in those where multiple validators are involved before a block is finalized / validated. fig. /spana href="whitepaper.html#lateximage-whitepaper-63"13b/span/a demonstrates a simplification of such a block creation processes.
Provided that the PoS method is well-constructed, this kind of PoS solves both the spam and availability problems.
Typically, such systems prevent validators from creating competing blocks by imposing an economic penalty (/spanem*slashing*/span/em) on those validators who help create two or more blocks at the same height (or slot, or whatever). Since slashing will destroy a validator's capital, and thus destroy their ability to be a validator, we can infer that this process will be rare. That's good, because we don't have to care much about the overhead of resolving these cases. This solves the spam problem.
The availability problem is solved provided there is at least one honest validator with knowledge of the block contents (because honest validators want to broadcast the block).

/spandiv
    class="tcolorbox"
    style=" border: 1px solid #404040; background: #ECECEC; "

/spandiv
    class="tcolorboxupper"
    style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbAside/span/b**/span/span The term "well-constructed" is doing most of the heavy lifting here. I don't think we need to worry, though, because these sorts of problems are what PoS designers spend a lot of time on. So, if any contemporary PoS chains are secure, then we /spanem*should*/span/em have a viable option for PoS dapp-chains. /span/div /span/div

PoA Dapp-chains &amp; Others Regarding spam and availability — what about PoA? Fortunately this case is trivial: we offload the responsibility /spanem*completely*/span/em to the PoA operators. The reason is simple: if PoA operators are withholding blocks or committing fraud, then that dapp-chain is already compromised.
And to cover all our bases: if interoperability with the simplex is not required, then we don't care what consensus method is used; the simplex-chain is just time-stamping data.
/spanfigure id="autoid-22" class="figure "

/spandiv class="center"
/spana href="25-constructing-ut-20-dappchain-proof-choice__sag" target="__blank" /spanimg
    src="25-constructing-ut-20-dappchain-proof-choice__sag"
    class="inlineimage"
    alt="(image)"
/span/a

/spandiv class="figurecaption"Figure 14: A flow chart to decide which proof system to use for a dapp-chain, assuming a secure PoS method is available.
/span/div /span/div/span/figure

It is worth noting that there are other directions that dapp-chains could take, too. Notably: we could use some kind of rollup; we could use some kind of zero-knowledge proof (ZKP) system; or we could go in a more traditional layer-2 direction, such as state channels.

Design Position With regards to how we should actually design simplex-chains to best support dapp-chains, there are a few important takeaways from the above. First, different dapp-chain use-cases have different technical requirements; we don't need a one-size-fits-all design. Second, no PoS system is yet adapted for dapp-chain use, and if PoS is generally insecure then we cannot assume secure PoS dapp-chains are possible. Third, scaling research (in a broad sense) is often focused on methods that should be compatible with our dapp-chain design, so new methods may become available at any point. Therefore, we should aim to support many different methods of running dapp-chains and providing the necessary primitives for users to do so.

<!--................................-->
/spanh6 id="autosec-228"/spanspan class="sectionnumber"3.4.2&#x2003;/span/spanThree General Incentive Models for Dapp-chain Reflection/span/h6

If dapp-chain headers are included alongside transactions in simplex-blocks, is it not the case that both must pay some kind of */spanemtransaction fee/span/em*? If not, how are simplex-chain miners to prioritize what to include in their blocks? Even if such a fee is */spanemnot always necessary/span/em*, the */spanemability/span/em* to provide a fee has decisive advantages — like creating asymmetry between an attacker and honest simplex-chain miners.
If it is possible to implement dapp-chains (or any system of child-chains) such that those chains have */spanemfreedom of protocol/span/em* and */spanemfreedom of incentivization/span/em* whilst inheriting the parent-chain's security, then we should strive to achieve that.

/spandiv
 class="tcolorbox"
 style=" border: 1px solid #404040; background: #F2F2F2; "

/spandiv
 class="tcolorboxupper"
 style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbTerm/span/b**/span/span **/spanb/spana href="whitepaper.html#??"Freedom of Incentivization/span/a/span/b**: The property whereby child-chains have free choice of incentive-system (i.e., the nature and dynamics of their root token, or lack thereof). /span/div /span/div

/spandiv
 class="tcolorbox"
 style=" border: 1px solid #404040; background: #F2F2F2; "

/spandiv
 class="tcolorboxupper"
 style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbTerm/span/b**/span/span **/spanb/spana href="whitepaper.html#??"Freedom of Protocol/span/a/span/b**: The property whereby child-chains have free choice of protocol (including consensus mechanism, scripting, accounting methods, block structures, etc). /span/div /span/div
section /spana href="whitepaper.html#lateximage-whitepaper-63"2.4.2/span/a details a conversion method whereby PoR is possible between chains using different root tokens via a DEX. Could dapp-chains use a */spanemprotocol-level/span/em* DEX to abstract their protocol and incentive method away from those of its parent-chain? Yes.
Is this required for this kind of abstraction? No.
Here are three methods of abstraction which maintain the above freedoms.

<!--................................-->

/spanspan class="paragraph" id="autosec-229"/spanspan class="sectionnumber"3.4.2.1&#x2003;/span/spanMethod 1: Pay the simplex miner on the dapp-chain/span/span

In this method, the dapp-chain uses its root token to pay both the dapp-chain miner and the simplex-chain miner (who includes the relevant dapp-chain header in their block).

Since all dapp-chain miners are required to run a full node of the parent-chain, this is trivial. In essence, the host simplex-chain is a subset of the dapp-chain. Simplex-miners can run light clients/spansup43/span/sup of the dapp-chain to regularly collect block-rewards.

A dapp-chain could, perhaps, have a rule like */spanemX root tokens are created as part of the coinbase transaction and the miner of that dapp-block has free choice of the proportion of those which are provided as a transaction fee to the host-miner/span/em.*

Example use-case: an existing blockchain migrates to become an */spanemAmaroo/span/em* dapp-chain.

<!--...............................-->

/spanspan class="paragraph" id="autosec-230"/spanspan class="sectionnumber"3.4.2.2&#x2003;/span/spanMethod 2: Pay the simplex miner via a native DEX/span/span

When a dapp-chain hosts a native DEX, it can use that DEX for PoR. The general case (where a reflecting chain contributes far more chain-work than the reflected chain) was discussed in section /spana href="whitepaper.html#lateximage-whitepaper-63"2.4.2/span/a.

Consider the limited context of a DEX with only one required trading pair (between the dapp-chain's root token and the ROO), combined with the security-contribution differential between a simplex-chain and a dapp-chain. Note that a conservative implementation of a DEX between this pair */spanemonly relies on local state/span/em* — that of the host simplex-chain and the dapp-chain, all of which is accessible to dapp-chain full nodes. The simplest method of preventing market manipulation (that might allow for some attack on the dapp-chain) is to calculate PoR weight via an */spanemold/span/em* exchange rate (e.g., from 24 hours ago), or to use an */spanemaverage/span/em* over some period of time. Both of these ensure that */spanemcompetition between blocks/span/em* (at any given time) is not dependent on the */spanemcurrent/span/em* DEX execution. With regards to dapp-chains using Proof of Reflection, this is sufficient.

Given a DEX, the dapp-chain can use this to automatically convert some of the mining reward to the root token of the host simplex-chain. These rewards could accrue over time and be bundled into far fewer transactions than would otherwise be necessary and automatically managed by the DEX. Unlike the previous method, this method doesn't require the simplex-chain miner to ever interact with dapp-chains (besides including their header-transactions); however, the protocol is more complex.

Example use-case: a greenfield dapp-chain uses an Amaroo-compatible DEX (which requires no development effort) so that simplex-miners have lower operating costs; thus incenting simplex-miners to include their headers over those of others.

<!--...............................-->
/spanspan class="paragraph" id="autosec-231"/spanspan class="sectionnumber"3.4.2.3&#x2003;/span/spanMethod 3: Pay the simplex miner directly/span/span

If the dapp-chain is willing to forego more efficient SPV transactions (or otherwise doesn't require them), and it is willing to bear the full burden of PoR in this context, then simply recording the hash of a dapp-chain header might be sufficient. In such a case, transactions (in the style of Bitcoin's /spanspan class="textsc"op_return/span/span transaction format) provide everything required. This makes sense if the dapp-chain has exceptionally large headers, or if the dapp-chain does not wish to */spanemdisclose/span/em* the headers themselves (perhaps it is a private/permissioned network). In any case, since it is impossible to stop users including hashes in transactions, this is always a method by which dapp-chains can enable PoR with the host simplex-chain.
Example use-cases:
/spanul style="list-style-type:none"

/spanli
- An existing */spanemanchored/span/em*/spansup44/span/sup blockchain migrates to become an Amaroo dapp-chain.

/span/li
/spanli
- A new (and ephemeral) dapp-chain is created to facilitate a national election that will result in a 200 GB audit log (facilitating unprivileged verification of the election result) and a peak votes-per-second over $10^5. Thisdemonstratesboth/spanemfreedomofincentivization/span/em(asthereisnone)and/spanemfreedomofprotocol/span/emasnopaymentsaremadeandnorestrictionisplacedonthenatureofthisdapp-chain'spayload.$

/span/li
/span/ul

/spandiv role="note" class="footnotes"
/spanp
/spansup43/span/sup&nbsp;A simplex-miner could use other methods too, like maintaining full nodes of each dapp-chain and continuously cycling through them (alternating which are running and which are not) to avoid massive computation requirements. Light clients seem obviously preferable where possible.
/span/p

/spanp
/spansup44/span/sup&nbsp;**/spanbAnchoring/span/b**: The process by which the hash of some data (perhaps a secondary chain's blocks) is included in transactions of a primary blockchain (e.g., /spana href="https://www.reddit.com/r/Bitcoin/comments/5xkvc1/psa_were_running_a_stress_test_of_our_blockchain/" target="_blank" Bitcoin/span/a). Anchoring */spanemwould/span/em* be a progenitor to PoR, except that I believe the idea of an /spana href="https://github.com/XertroV/coppr/blob/master/chainheaders.py" target="_blank" on-chain light client predates/span/a the term */spanemanchoring/span/em*. Though I think that the idea of time-stamping a hash (e.g., via an /spanspan class="textsc"op_return/span/span transaction on Bitcoin) predates the idea of an on-chain light client.
/span/p

/span/div

<!--...............................-->
/spanh6 id="autosec-233"/spanspan class="sectionnumber"3.4.3&#x2003;/span/spanPoS Dapp-chains/span/h6

If the headers of dapp-chains are encoded as simplex-transactions, then techniques like */spanemslashing/span/em* can be first-class operations within the */spanemhybrid/span/em* PoW context provided by the simplex. This solves the */spanemNothing at Stake/span/em* and */spanemlong-range/span/em* problems for PoS dapp-chains, so long as the necessary PoS primitives can be encoded in a simplex-transaction.

The abstraction layer between simplex-chains and dapp-chains brings practical benefits, too. For example: existing (open-source) PoS blockchain schemes can be easily integrated as dapp-chains. Given that dapp-chains inherit security properties of their parent-chain (via one-way PoR), if such a dapp-chain's consensus method supports */spanemother/span/em* features — e.g., /spana href="https://github.com/w3f/consensus/blob/master/pdf/grandpa.pdf" target="_blank" finality guarantees/span/a — we should get those features for */spanemfree/span/em*. (Sharding, too, for that matter&#x2026;)

The most likely method of integration has three core components: modification of the headers (and integration of PoR), modification of existing slashing protocols, and support for intra-simplex SPV proofs. For example: /spana href="https://github.com/openethereum/openethereum" target="_blank" OpenEthereum (Parity; 2022)/span/a /spansup45/span/sup could be integrated as a dapp-chain with the creation of a new /spana href="https://github.com/openethereum/openethereum/blob/582bca385fedb1af682e989e5bcc6b3b2cf53028/crates/ethcore/types/src/header.rs" target="_blank" header format/span/a, the creation or modification of a suitable /spana href="https://github.com/openethereum/openethereum/blob/582bca385fedb1af682e989e5bcc6b3b2cf53028/crates/ethcore/src/engines/basic_authority.rs" target="_blank" engine/span/a, and the implementation of suitable /spana href="https://github.com/openethereum/openethereum/blob/582bca385fedb1af682e989e5bcc6b3b2cf53028/crates/vm/builtin/src/lib.rs" target="_blank" builtins/span/a that facilitate intra-simplex SPV proofs/spansup46/span/sup and any other useful features. Naturally, there are some other components that are necessary (like a component for broadcasting header-transactions), but those components are common over many dapp-chain integrations and only need to be written once for each */spanemkind/span/em* of dapp-chain.

/spandiv role="note" class="footnotes"
/spanp

/spansup45/span/sup&nbsp;OpenEthereum itself has, since this section was written, been deprecated. This section is left unchanged since the specific client we might modify is immaterial to the main point: that relatively few modifications can be applied to existing clients to create a dapp-chain version of that kind of blockchain.

/span/p

/spanp

/spansup46/span/sup&nbsp;Note: instead of builtins, these requirements could be met via EVM/WASM smart contracts.

/span/p

/span/div

<!--...............................-->
/spanh6 id="autosec-235"/spanspan class="sectionnumber"3.4.4&#x2003;/span/spanGoing Further/span/h6

There are two more ideas we'll explore a little in this section: dapp-chain simplexes and dapp-dapp-chains. Both of these are somewhat speculative, in the sense that we're adding more and more assumptions and should thus be somewhat conservative in our conclusions. However, if these techniques can be done practically and securely, then they will give us some nice features.

<!--.................................-->

/spanspan class="paragraph" id="autosec-236"/spanspan class="sectionnumber"3.4.4.1&#x2003;/span/spanDapp-chain Simplexes/span/span

Say we have a simplex with a reasonable number of dapp-chains — enough that we're interested in increasing the efficiency of communication between them. Currently, the worst case situation for moving something (tokens, information, permissions, whatever) from one dapp-chain (D_{α}hosted on L) to another (D_{β}on R) involves at least three transactions: $D_{\alpha} \rightarrow L \rightarrow R \rightarrow D_{\beta}.Ifwearejustprovingsomethingto D_{\beta}, wecan(atleast)dothisinonetransaction, butwestillneedtoproveboththatsomeblockon D_{\alpha} isvalidandthatsomestateentrieshavesomeparticularvalues.(TheproofsforLandRblocks, andtherelevantPoRs, areleftasimplicitsinceallfullnodeshavethosealready.)$

If a path between two dapp-chains is common enough, is there any way to make this process more efficient? Yes: dapp-chain simplexes.

The simpler case is when all the dapp-chains are on the same simplex-chain. In this case, the dapp-level simplex's PoRs can be verified by the host simplex-chain, and we should gain some of the properties that the main simplex enjoys./spansup47/span/sup Participating dapp-chains would know the chain-tips for all the other participating dapp-chains, so SPV-esq proofs between those chains are roughly halved in size.

The more complex case is a dapp-chain simplex where the dapp-chains are hosted on different simplex-chains. Enforcing this at the simplex-level would require some additional protocol support — the host chain would need to verify something that depends on another simplex-chain. As we'll see in section /spana href="whitepaper.html#lateximage-whitepaper-63"4/span/a (particularly section /spana href="whitepaper.html#lateximage-whitepaper-63"4.11/span/a), this kind of thing can be done without breaking O(c) constraints. This does not, however, mean that we can have an unlimited number of dapp-chain simplexes that span multiple simplex chains. If we tried that, we'd find that a fully validating simplex-chain node would need to follow O(c) many dapp-chain simplexes, each of which has O(c) many chains, resulting in an overall load of O(c^2).

So, in conclusion, dapp-chain simplexes are possibly useful but should be considered on a case-by-case basis until we have a better understanding of them.

<!--................................-->

/spanspan class="paragraph" id="autosec-237"/spanspan class="sectionnumber"3.4.4.2&#x2003;/span/spanUT$_3$: Dapp-dapp-chains/span/span

Let's assume for a moment that there exists a feature-complete, production-ready, open source, sharded PoS blockchain. It's out there, in the wild, chugging away processing O(c^2)*manytransactions.*

Given that we just discussed how existing blockchain clients can be modified to run as dapp-chains on the simplex, what, */spanemin principle/span/em*, is stopping us from using the same techniques with this sharded PoS chain? If the answer is */spanemnothing/span/em*, then it's self-evident that this is a path to take UT from

O(c^3)*capacitytoO*(c^4)*capacity* − − − *thatis*, $\mathrm{UT}_2 \to \mathrm{UT}_3$.*Theremay*, *ofcourse*, *bemoreadjustmentsthatneedtobemadedependingonthePoSprotocol.But*, *iftheprotocolissecureinisolation*, *thenit′shardtoseehowaddingthermodynamicsecurityviaone* −
wayPoRwithasimplex − *chainwould/spanemnecessarily/span/embesomehowincompatibleorproblematic.Ingeneral*, *ifsomeO*(c^x)*scalingmethodworksforatraditionalblockchain*, *thencombiningitwithUTresultsinanO*(c^{2+x})*scalablenetwork.*

It's worth noting that this technique only works in one direction: UT can accommodate other chains (as dapp-chains), but other chains cannot accommodate UT in the same way. If a sharded chain were adapted such that each shard were part of a simplex, then either overall capacity decreases due to PoR overhead, or the chain hosting the shards is redundant and we just end up with an implementation of UT. This is yet another asymmetry between UT and traditional blockchains.

.7ex]0.5pt/spanspan style="width:3.4444pt; display:inline-block"<!---->/span/spanBegin DRAFT/spanspan style="width:3.4444pt; display:inline-block"<!---->/span/span.7ex]0.5pt

.7ex]0.5pt/spanspan style="width:3.4444pt; display:inline-block"<!---->/span/spanEnd DRAFT/spanspan style="width:3.4444pt; display:inline-block"<!---->/span/span.7ex]0.5pt

/spandiv role="note" class="footnotes"
/spanp

/spansup47/span/sup&nbsp; These properties (of the main simplex) are explored in-depth in section /spana href="whitepaper.html#lateximage-whitepaper-63"4/span/a. Exactly which properties are gained in which contexts will be left for future research, but I'd expect there to be benefits around censorship resistance, reduced proof sizes, DoS mitigation, and more.

/span/p

/span/div

<!--................................-->
/spanh4 id="autosec-239"/spanspan class="sectionnumber"4&#x2003;/span/spanPractical Considerations for UT's Design/span/h4

<!--...............................-->

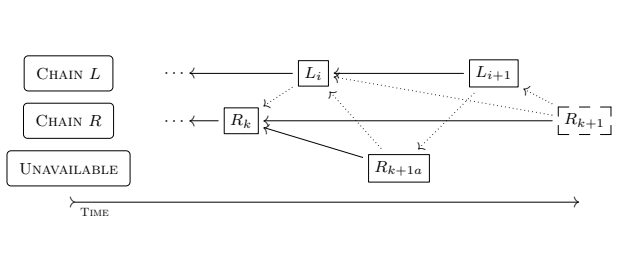
/spanh5 id="autosec-240"/spanspan class="sectionnumber"4.1&#x2003;/span/spanThe Availability of Reflected Blocks/span/h5

What happens if a chain reflects a valid block that is not available? Could an attacker use this to their advantage?

For example, consider the following situation (fig. /spana href="whitepaper.html#lateximage-whitepaper-63"15/span/a). L and R are two mutually reflecting chains, and the most recently produced block is L_i, which reflected R_k. Next, a malicious R miner produces a valid block (R_{k+1a}) which reflects L_i. That miner broadcasts the */spanemheader/span/em* of R_{k+1a} along with a PoR branch proving that L_i was reflected, but does not broadcast the rest of the block. The L miners receive the header and PoR branch, and shortly thereafter an L miner produces L_{i+1}, which reflects R_{k+1a}. Since other (honest) R miners cannot build on R_{k+1a}, they must work on the draft block R_{k+1} instead. Note that, although R_{k+1} will almost certainly reflect L_i, whether it reflects L_{i+1} is at the miner's discretion.

/spanfigure id="autoid-23" class="figure "

/spanspan class="hidden" \(\seteqnumber{0}{}{25}\)/span/span <!--



-->

```
\begin{equation*}
\xyMChains { &amp; \xyL {\text {Chain \cL }} &amp; \cdots &amp; &amp; \xBL {} \arf [dl] \ar [ll] &amp; &amp; \xBL {1} \ar [ll] \arf [ldd] &amp; &amp; \\ &amp; \xyL {\text {Chain \cR }} &amp; \cdots &amp; \xBR {} \ar [l] &amp; &amp; &amp; &amp; \xyBDraft {\BR {1}} \ar [llll] \arf [lllu] \arf [lu] \\ &amp; \xyL {\text {Unavailable}} &amp; &amp; &amp; &amp; \xBR {1a} \ar [llu] \arf [luu] &amp; &amp; &amp; \\ &amp; \xyTimeHoriz {rrrrrr} &amp;&amp;&amp;&amp;&amp;&amp; }
\end{equation*}
```

/spandiv class="figurecaption"Figure 15: Chain-segments showing an unavailable Rblock that was reflected anyway. The dashed frame of R_{k+1} indicates that it is a draft block (i.e., not yet mined, but being actively worked on by miners). Note: R_{k+1} reflects L_{i+1} at the miner's discretion — it is optional.

/span/div /span/figure

The attacker can use this situation to their advantage if R_{k+1a} has precedence over R_{k+1} (i.e., it's favored by the fork-rule). In that case, the attacker can publish R_{k+1a} */spanemafter/span/em* R_{k+1} is mined for an advantage./spansup48/span/sup When does this case occur?

If R_{k+1} does */spanemnot/span/em* reflect L_{i+1}, then R_{k+1} will claim the same chain-weight as R_{k+1a}. However, R_{k+1a} is favored over R_{k+1} since the PoR via L_{i+1} contributes weight to R_{k+1a} (as discussed in section /spana href="whitepaper.html#lateximage-whitepaper-63"2.7/span/a). So the attacker */spanemalways/span/em* has the advantage if R_{k+1} */spanemdoes not/span/em* reflect L_{i+1}.

If R_{k+1} does reflect L_{i+1}, then R_{k+1a} is implied as an earlier block, so should have precedence.

Both these situations make sense: sometimes two Rblocks will be created near-simultaneously, and occasionally only one of those blocks will be reflected by an Lblock. In those naturally arising cases, we'd */spanemexpect/span/em* the reflected block (R_{k+1a} in this case) to take precedence. Either because it was reflected when R_{k+1} wasn't, or because, for R_{k+1} to reflect L_{i+1}, R_{k+1a} must have existed first.

So, we seem to have a problem: on the one hand, an attacker could withhold a block (but not the header or PoRs) to gain some advantage; on the other hand, prioritizing the earlier block seems to be necessary and correct behavior when this situation arises naturally.

The only difference between these two situations is whether R_{k+1a} is available or not; so this will be our discriminating factor.

Can Rminers do anything to prevent Lminers reflecting R_{k+1a} before the block body is released? The Rminers can't prevent a withheld Rblock from being produced or reflected, but, Rminers can still influence Lminers' choice to reflect R_{k+1a} (and withheld blocks in general). Lminers want their blocks to be reflected by Rminers so that future Lblocks will build on top of theirs. So Rminers could refuse to reflect some Lblocks — particularly, Lblocks that reflect Rblock headers without a known body (such as R_{k+1a}).

Since Lminers face the same problem (withheld but reflected Lblocks), both L and Rminers are incentivized to cooperate. This forms a stable equilibrium provided that the honest miners have excess bandwidth and that the attacker has less than half the combined (post-conversion) hash-rate.

Another way to look at this is that PoR only works when all blocks are available. This isn't surprising because blockchains generally only work when all (canonical) blocks are available. Since miners only want to participate in systems that work (how else would they earn income?), they will adopt the practices required to make the system work.

/spandiv role="note" class="footnotes"
/spanp

/spansup48/span/sup&nbsp; This is similar to the /*spanemselfish mining*/span/*em* attack published in /spana href="https://web.archive.org/web/20240927042356/https://gateway.pinata.cloud/ipfs/QmNukb1L8BhEsiCbrmnkEJWAvUjhBHidinKMZKfCaLG6ep" target="_blank" Majority is not Enough: Bitcoin Mining is Vulnerable (Eyal, Sirer; 2013)/span/a. It could also be chained to effect a doublespend.
/span/p

/span/div

<!--................................-->
/spanh6 id="autosec-242"/spanspan class="sectionnumber"4.1.1&#x2003;/span/spanThe Axiom of Availability/span/h6

To codify this as a foundation of UT, we will introduce the first network */spanemaxiom/span/em*: the /spana href="whitepaper.html#lateximage-whitepaper-63"Axiom of Availability/span/a.

/spandiv
class="tcolorbox"
style=" border: 1px solid #333333; background: #F2F2F2; "

/spandiv
class="tcolorboxtitle"
style=" border-top: 1px solid #333333; border-bottom: 1px solid #333333; background: #333333; color: #FFFFFF; "
Axiom of Availability/span/div
/spandiv
class="tcolorboxupper"
style=" color: #000000; "
/spanemPrinciple:/span/em All reflected blocks must be verifiably complete and readily available./spanspan role="note" class="marginpar"**/spanbAxiom/span/b**/span/span
/span/div
/spandiv
class="tcolorboxlower"
style="border-top: 1px dashed #333333; color: #000000; "

/spanemPredicate:/span/em A block, L_i, cannot yet be valid unless, for each reflected block R_k: /spanul style="list-style-type:none"

/spanli
i. R_k has been downloaded (i.e., the block is available); and
/span/li
/spanli
ii. the cryptographic integrity of R_k has been verified; and
/span/li
/spanli
iii. R_k is valid under this axiom.
/span/li
/span/ul
/span/div /span/div

/spandiv
class="tcolorbox"
style=" border: 1px solid #404040; background: #F2F2F2; "

/spandiv
class="tcolorboxupper"
style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbTerm/span/b**/span/span **/spanb/spana href="whitepaper.html#??"Axiom/span/a/span/b**: */spanemNetwork axioms/span/em* are foundational rules expressed as a principle and predicate. Consensus-forming nodes must adhere to them. See definitions: /spana href="whitepaper.html#lateximage-whitepaper-63"Availability/span/a, /spana href="whitepaper.html#lateximage-whitepaper-63"Maximal Reflection/span/a, /spana href="whitepaper.html#lateximage-whitepaper-63"Unified Ancestry/span/a. /span/div /span/div

<!--...............................-->
/spanh6 id="autosec-243"/spanspan class="sectionnumber"4.1.2&#x2003;/span/spanBandwidth Requirements/span/h6

In the case of just two chains, the bandwidth requirements for *L*miners are approximately doubled, but the storage and computational requirements are not. While there is some storage and computational overhead on the part of reflected *R*blocks, these are close-to-negligible when compared to the mined chains: miners need only to store */spanemrecent/span/em* *R*blocks and verify that they are */spanemstructurally/span/em* valid. While this is technically an O(c) computational load per block (since blocks are O(c) bytes large), practically this load is far smaller than the O(c) load of */spanemfully processing/span/em* a block, so we can treat it as a constant overhead (O(1)). Since only recent blocks are stored, the storage requirements are also O(1).

If there are O(c) many chains, then requirements are:

O(c^2)*forbandwidth*, *O*(*c*)*forstorage*, *andO*(*c*)*forcomputation*.*Aswewillseeinsection* /*spanahref* = "whitepaper.html#*lateximage* − *whitepaper* − 63"5.8/*span*/*a*, *thepracticalbandwidthrequirementsareacceptable*(*ontheorderof*1*MB*/*sorso*/*spansup*49/*span*/*sup*).*Assumingthatholdstrue*, *weconcludethatthe*/*spanahref* = "whitepaper.html#

See section /spana href="whitepaper.html#lateximage-whitepaper-63"5.7/span/a for a deeper discussion of bandwidth complexity.

.7ex]0.5pt/spanspan style="width:3.4444pt; display:inline-block"<!---->/span/spanBegin DRAFT/spanspan style="width:3.4444pt; display:inline-block"<!---->/span/span.7ex]0.5pt

```
/spandiv role="note" class="footnotes"
/spanp
/spansup49/span/sup&nbsp; A rule of thumb: 1 MB/s here corresponds to 1,000-4,000 TPS. Also note that this is only a requirement for miners and full nodes /spanemafter/span/em a desired chain has been fully synchronized.
/span/p

/span/div
/spandiv
      class="tcolorbox"
      style=" border: 1px solid #404040; background: #FCD1E9; "

/spandiv
      class="tcolorboxtitle"
      style=" border-top: 1px solid #404040; border-bottom: 1px solid #404040; background: #404040; color: #FFFFFF; "
TODO [7]/span/div
/spandiv
      class="tcolorboxupper"
      style=" color: #000000; "
  argue that hiding blocks can cause massive reorgs and invalidate future blocks after those future blocks have been mined (note: might be covered in part by 10-refl-availability.tex)  /span/div /span/div

/spandiv
      class="tcolorbox"
      style=" border: 1px solid #404040; background: #FCD1E9; "

/spandiv
      class="tcolorboxtitle"
      style=" border-top: 1px solid #404040; border-bottom: 1px solid #404040; background: #404040; color: #FFFFFF; "
TODO [8]/span/div
/spandiv
      class="tcolorboxupper"
      style=" color: #000000; "
  note that it is vital that new stuff doesn't invalidate old stuff / cause a re-org; or check that it's noted (ties into SPV)  /span/div /span/div
.7ex]0.5pt/spanspan style="width:3.4444pt; display:inline-block"<!---->/span/spanEnd DRAFT/spanspan style="width:3.4444pt; display:inline-block"<!---->/span/span.7ex]0.5pt
```

<!--...............................-->
/spanh5 id="autosec-245"/spanspan class="sectionnumber"4.2&#x2003;/span/spanProving Reflection/span/h5

If simplex-chains' consensus protocols require accounting for reflected work, then nodes must have some method whereby they know which work (in a particular chain's history) has been reflected. That is: a node for chain L must be able to answer the question /spanem*For each other simplex-chain, which blocks in chain L's history have been reflected?*/span/em This means that each node must have $N_1 - 1answers, perblock, forasimplexofN_1chains$.
There is a trivial method: with each header, include the corresponding merkle branch which proves reflection. Specifically: when a miner on chain L mines a block that includes a header from chain R, they should also include — alongside the header — a merkle branch that shows the most recent chain L ancestor that has been reflected by chain R. For example, block
$B_{L,i+1} mightincludeaproofthatH_{L,i} wasreflectedbyB_{R,j}.Thatbranchistheonlyrequiredbranch(i.e., the/spanemmissing/span/embranch), aschainLnodesare/spanemalready/span/emawarewhetherH_{R,j} wasreflectedbyB_{L,i+1}$.
Miners would need to do this for /spanem*all*/span/em simplex-chains that they reflect. Predictably, this has overhead with order
$O(N_1 \cdot \log_2 N_1), whereN_1 isthenumberofchainsinthesimplex.ThismethodhascomplexityO(c \cdot \log_2 c) whichisdiscussedinsection\ /spanahref = $"whitepaper.html$\#lateximage - whitepaper - 63$"$5.6.2/span/a$.

/spandiv
    class="tcolorbox"
    style=" border: 1px solid #404040; background: #F2F2F2; "

/spandiv
    class="tcolorboxupper"
    style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbTerm/span/b**/span/span **/spanb/spana href="whitepaper.html#??"Explicit Proofs (+PoRs)/span/a/span/b**: The UT protocol variant wherein miners/validators explicitly record /spanem*both*/span/em reflected headers /spanem*and*/span/em the single missing merkle branch required to prove reflection. /span/div /span/div
Do we /spanem*need*/span/em to include proofs of reflection, though? Is it possible to avoid the explicit inclusion of those proofs, potentially allowing for O(c) complexity instead?
If miners of any simplex-chain download the blocks of /spanem*all*/span/em simplex-chains — as mentioned in section /spana href="whitepaper.html#lateximage-whitepaper-63"4.1/span/a — then including all necessary proofs of reflection can be made redundant. Since miners, theoretically, have all the necessary data to construct the proofs, do those miners need to actually include those proofs? Could we treat those proofs as witnesses and prune them — similar to Segregated Witness (SegWit)/spansup50/span/sup?

/spandiv role="note" class="footnotes"
/spanp
/spansup50/span/sup&nbsp;/spana href="https://web.archive.org/web/20240926154239/https://en.bitcoin.it/wiki/Segregated_Witness" target="_blank" Segregated Witness/span/a was a new “witness” structure introduced to bitcoin blocks, separate from the transaction merkle tree. The structure contains data required to check transaction validity but not required to determine the transaction effects. Introduced with /spana href="https://web.archive.org/web/20240423183945/https://en.bitcoin.it/wiki/BIP_0141" target="_blank" BIP-141/span/a.
/span/p

/span/div
/spandiv
    class="tcolorbox"
    style=" border: 1px solid #404040; background: #F2F2F2; "

/spandiv
    class="tcolorboxupper"
    style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbTerm/span/b**/span/span **/spanb/spana href="whitepaper.html#??"Omitted Proofs (+OP)/span/a/span/b**: The UT protocol variant wherein miners/validators explicitly transmit /span*emonly/span/em* the reflected header component of PoRs, such that necessary proofs of reflection themselves are deterministically recalculable. /span/div /span/div

There would be some downsides to omitting the proofs of reflection. For one, it would mean that simplex-chain nodes of a single chain, during an initial sync, would not be able to verify the PoRs without auxiliary data — potentially a lot. Secondly, it would mean that miners /span*emmust/span/em* track the state of /span*emall/span/em* reflections in the simplex for some period of time so that they ensure the integrity of the reflection protocol. Although, given the /spana href="whitepaper.html#lateximage-whitepaper-63"Axiom of Availability/span/a, this is possible without significant overhead.

A practical method for treating proofs of reflection as witnesses that may be excluded/pruned is discussed in section /spana href="whitepaper.html#lateximage-whitepaper-63"4.3/span/a.

.7ex]0.5pt/spanspan style="width:3.4444pt; display:inline-block"<!---->/span/spanBegin DRAFT/spanspan style="width:3.4444pt; display:inline-block"<!---->/span/span.7ex]0.5pt

<!--...............................-->
/spanh6 id="autosec-247"/spanspan class="sectionnumber"4.2.1&#x2003;/span/spanVerkle Trees and Shorter PoRs/span/h6

/spanemVerkle trees/span/em are a new alternative to merkle trees. Similar to merkle trees, they allow efficient proofs of membership against a cryptographically secure root.

/spandiv
class="tcolorbox"
style=" border: 1px solid #BC842C; background: #FEE8F4; "

/spandiv
class="tcolorboxtitle"
style=" border-top: 1px solid #BC842C; border-bottom: 1px solid #BC842C; background: #BC842C; color: #FFFFFF; "
[TODO: [1] **/spanb(MED)/span/b**/span/div
/spandiv
class="tcolorboxupper"
style=" color: #000000; "
probs remove — not worth explaining here. /span/div /span/div
.7ex]0.5pt/spanspan style="width:3.4444pt; display:inline-block"<!---->/span/spanEnd DRAFT/spanspan style="width:3.4444pt; display:inline-block"<!---->/span/span.7ex]0.5pt

`<!--...............................-->`
/spanh5 id="autosec-248"/spanspan class="sectionnumber"4.3&#x2003;/span/spanSegmented State/span/h5

Traditionally, blockchain protocols have some */spanemglobal/span/em* state and a state-transition function. For example, the Ethereum Yellow Paper says:

/spandiv
    class="tcolorbox"
    style=" border: 1px solid #404040; background: #ECECEC; "

/spandiv
    class="tcolorboxupper"
    style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbQuote/span/b**/span/span /spanspan role="note" class="marginpar"**/spanbQuote/span/b**/span/span Ethereum, taken as a whole, can be viewed as a transaction-based state machine: we begin with a genesis state and incrementally execute transactions to morph it into some current state. It is this current state which we accept as the canonical “version” of the world of Ethereum. /spanli
[&#x2026;]
/span/li
/spanli
A valid state transition is one which comes about through a transaction. Formally:
/spanspan class="hidden" \(\seteqnumber{0}{}{25}\)/span/span `<!--`

$$\sigma_{t+1} \equiv \Upsilon(\sigma_t, T)$$

```
-->
\begin{equation*}
\sigma _{t+1} \equiv \Upsilon (\sigma _t, T)
\end{equation*}
```

where $\Upsilon is the Ethereum state transition function.$
.7ex]0.5pt

/spandiv class="flushright" /spani— *Dr. Gavin Wood; /spana href="https://web.archive.org/web/20240927042236/https://gateway.pinata.cloud/ipfs/QmcdwaEqKjsASs1sZqxBNPw5vmypE5YL61zSvWdGoX7wtC" target="_blank" Ethereum Yellow Paper / Petersburg Version 41c1837/span/a, s2/span/i* /span/div /span/div /span/div

One of the reasons for this tradition is that transactions are (typically) permitted to depend on any part of the global state. For example: a Bitcoin transaction is permitted to spend any UTXO, and an Ethereum smart contract may interact with any other smart contract on the Ethereum blockchain.

However, it is not necessary for a protocol to allow */spanemany and all/span/em* transactions to depend on global state. A protocol could specify that certain transactions may depend only on a strictly defined subset of global state, i.e., a well defined */spanemsegment/span/em* of global state that is independently calculable.

Simplex-chains can use this technique to their advantage by segmenting both transactions and state which are specific to Proof of Reflection. That way, the state of a simplex-chain's reflections can be calculated without needing to calculate the remaining state for that simplex-chain.

We could specify the state-transition of simplex-chains (using Ethereum's nomenclature) like this:

/spanspan class="hidden" \(\seteqnumber{0}{}{25}\)/span/span <!--

$$\begin{split} \sigma_{R,t+1} &\equiv \Upsilon_R(\sigma_{R,t}, T) \\ \sigma_{\star,t+1} &\equiv \Upsilon_{\star}(\sigma_{R,t} + \sigma_{\star,t}, T) \end{split} \tag{26}$$

-->

\begin{equation}

\begin{split} \label {eq:segregated-state} \sigma _{R,t+1} &amp; \equiv \Upsilon _R(\sigma _{R,t}, T) \\ \sigma _{\star ,t+1} &amp; \equiv \Upsilon _{\star }(\sigma _{R,t} + \sigma _{\star ,t}, T) \end {split}

\end{equation}

Where, at some time t: $\sigma_{R,t} isthesegmentofstatethatistrackingreflectionsandheaders; \sigma_{\star,t} istheglobalstateexcluding \sigma_{R,t} : \Upsilon_R isthestate-transitionfunctionforthereflectionssegment; and \Upsilon_{\star} isthestatetransitionfunctionforallremainingsegments. NotethatifTisareflection-transaction(i.e., itcontainsheaderstobereflected)then \Upsilon_{\star} doesnothing, andifTisanyothertypeoftransactionthen \Upsilon_R doesnothing.$

In essence eq. /spanspan class="textup"(/spana href="whitepaper.html#lateximage-whitepaper-63"26/span/a)/span/span shows that $\sigma_{R,t} depends/spanemonly/span/emonthe \sigma_{R,t-1} state - segmentandthecurrenttransaction, whereas \sigma_{\star,t} dependsonglobalstate.$

If simplex-chains are segmented in this manner, then miners will be able to calculate the reflection-state of other simplex-chains without calculating their complete state. This would allow them to deterministically calculate proofs of reflection for all other simplex-chains.

/span/li

<!--...............................-->
/spanh5 id="autosec-250"/spanspan class="sectionnumber"4.4&#x2003;/span/spanExploiting Segmented State/span/h5

Given that the reflection-segments of simplex-chains will contain mostly redundant data (i.e., headers), numerous optimizations are possible.

For example, it's not necessary for a miner's node to re-download reflected headers (which are part of other chains' blocks), since it can download them in advance and as they become available. We can reconstruct the PoRs root, provided that we know /spanem*which*/span/em headers are reflected and in what order. Transmitting the /spanem*hashes*/span/em of headers, only, reduces the effective size of simplex-blocks/spansup51/span/sup from b to ~b $\cdot(\frac{g+B_h}{2B_h}), wheregisthesizeoftherelevantdigestinbytes.Forg = 32; B_h = 112, thisreduceseffectiveblocksizeto{\sim}0.643b --- animprovementof{\sim}35\%.$

However, /spanem*instead*/span/em of using that technique to /spanem*minimize bandwidth*/span/em we could instead use it to /spanem*maximize the number of simplex-chains*/span/em. If simplex-blocks dedicate /spanspan class="textrm textmd textup textsc"/spanspan class="numerator"1/span/span//spanspan class="denominator"2/span/span/span/span of their capacity to reflections, and assuming +OP, then we can reduce that burden by 1 - /spanspan class="textrm textmd textup textsc"/spanspan class="numerator"32/span/span//spanspan class="denominator"B$_h$/span/span/span/span $\approx 70\%, /spanemor/span/emwecouldincreasethecapacityforreflectionsby/spanspanclass = "textrm\ textmd\ textup\ textsc"/spanspanclass = "numerator"B_h/span/span//spanspanclass = "denominator"32/span/span/span/span \approx 300\%!$

/spandiv role="note" class="footnotes"
/spanp
/spansup51/span/sup&nbsp;Assuming those blocks dedicate 50% capacity to transactions, and 50% to reflected headers (without PoRs).
/span/p

/span/div
/spandiv
    class="tcolorbox"
    style=" border: 1px solid #404040; background: #F2F2F2; "

/spandiv
    class="tcolorboxupper"
    style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbTerm/span/b**/span/span **/spanb/spana href="whitepaper.html#??"Header Omission (+HO)/span/a/span/b**: The UT protocol variant wherein miners/validators explicitly record */spanemonly/span/em* the hashes of reflected headers. A requirement is that block producers must eagerly download the headers of all simplex-chains and deterministically recalculate the relevant Proofs of Reflection. /span/div
/span/div

We have just reached the +HO variant via */spanemomitting proofs/span/em* (+OP). This time, however, it is not the */spanemproofs/span/em* that are redundant, but the */spanemheaders/span/em*.

Does */spanemheader omission/span/em* with */spanemexplicit proofs/span/em* provide any advantages? Yes, in some cases.

Particularly, if miners include only the single missing merkle branch associated with each necessary PoR, then */spanemno additional information/span/em* is required besides the */spanemheader/span/em* itself. Headers are trivial to acquire from the network, and each only needs to be acquired once, regardless of the number of PoRs it is a part of. Since the */spanemhash of each header/span/em* is */spanempart/span/em* of the missing PoR merkle branch, miners only need to provide */spaneman ordered list of merkle branches/span/em* for full PoR verifiability. Additionally, these merkle branches */spanemwill be part of specific SPV proofs/span/em*, so that when a cross-chain SPV transaction (which uses those branches) is made, it can omit those parts of the proof (replacing them with a pointer).

This UT protocol variant is +HOPoRs: the combination of */spanemheader omission/span/em* (+HO) and */spanemexplicit proofs/span/em* (+PoRs). It may present decisive advantages for implementations of */spanemsimplex tilings/span/em* (which are introduced in section /spana href="whitepaper.html#lateximage-whitepaper-63"6/span/a).

<!--................................-->
/spanh6 id="autosec-252"/spanspan class="sectionnumber"4.4.1&#x2003;/span/spanHash Compression &amp; Truncation/span/h6

/spandiv
class="tcolorbox"
style=" border: 1px solid #404040; background: #F2F2F2; "

/spandiv
class="tcolorboxupper"
style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbWarning/span/b**/span/spanThe following applies to PoW chains with compatible header PoW-hashing algorithms./span/div /span/div

Consider a fairly normal PoW chain, in that the PoW algorithm compares the hashes of headers as a number with a target. That is, the output hash has a bunch of zero digits at one end (or can be losslessly converted into such a form).

For simplicity, we'll assume that the zero digits are the most significant and are leading (and the least significant bits are trailing).

When we serialize this hash as a binary string, there is a trivial compression method. Since we */spanemknow/span/em* that one end of the hash has multiple zero bytes, we can replace this substring with the number of zero bytes replaced./spansup52/span/sup

This reduces the length of the hash (in bytes) from g to g - z + 1 (z being the number of zero bytes), */spanemhowever/span/em*, the */spanemsecurity/span/em* of the hash /spanspan class="uline"is still 8g bits/span/span. This is a limited form of hash compression. In a mature network using +HO, reducing hash length by ~/spanspan class="textrm textmd textup textsc"/spanspan class="numerator"1/span/span//spanspan class="denominator"4/span/span/span/span is possible, corresponding to an increase in maximum capacity of ~/spanspan class="textrm textmd textup textsc"/spanspan class="numerator"1/span/span//spanspan class="denominator"3/span/span/span/span or so.

This kind of compression is possible (and valuable) because the hashes are */spanemladen/span/em* with special properties by the PoW algorithm. There are two main properties we are concerned with. The first is that better header hashes, interpreted as numbers, are smaller. The second is subtle.

Consider a mature, healthy PoW chain, like Bitcoin. The proofs of work produced by such a chain are, through market feedback mechanisms, the */spanembest/span/em* proofs of work that the market as a whole (i.e., the global economy) is capable of producing profitably. Therefore, the proofs of work are an approximate */spanemmeasure/span/em* of the global capacity for hashing. More specifically, the number of leading zero-bits loosely encodes humanity's collective ability to produce arbitrary partial collisions via brute force. For example, Bitcoin block 879,273 has a difficulty around 1.1
$\times 10^{14}, correspondingtoatleast78leadingzero - bits./spansup53/span/supGivenBitcoinproducesaround52600blocksperyear(andallelsebeingequal), wecanexpectthebestblockhashproducedinthelastyeartohavearound78 + \log_2(52600) \approx 94leadingzero -$
$bits.Practicallyspeaking, allthesiliconintheworldworkingforayear, singularly, onfindingapartialSHA256collisionwouldnotdomuchbetterthan94bits.Wecanbeconfidentinthis/spanemparticular/span/empredictionbecausethemarketforBitcoinASICsismature -$
$-thoseASICsusethelatestfabricationprocesses, arenumerousenough, andareordersofmagnitudemoreefficientthangeneralprocessors(CPUs, GPUs, etc).Forlessmaturenetworks, ornetworksusingalgorithmsthatbenefitlessfromASICs, thedifferencewillbegreaterandmoredependentonexternalcomputeresourceswhicharenotreflectedinthePoWdifficulty.But,$
$2z, thisdoesnotpresentanissue - -thelowvalueofzmeanswehavemorebufferuntilwepasstheinsecuritybreakpoint, sothesetwoforcesroughlycanceloutinallbutextremecases.$

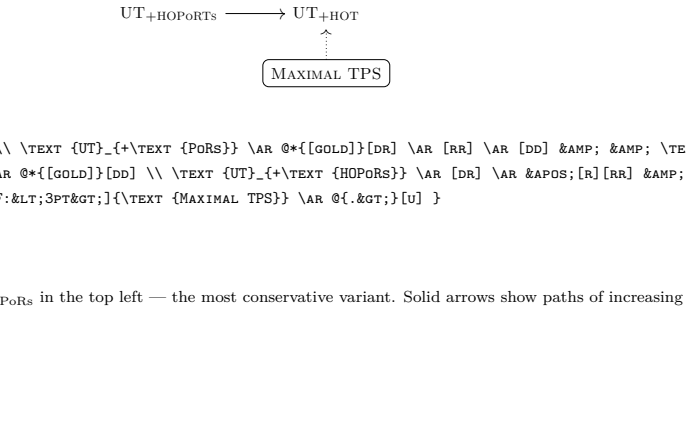
We will make use of this second property, that the leading zero-bits are related to the global hashing capacity, to justify the safety of */spanemhash truncation/span/em* as an optimization.

/spandiv role="note" class="footnotes"
/spanp
/spansup52/span/sup&nbsp; Since we're now dealing with a variable length byte-string, a typical encoding scheme would prefix this with the length of the bytes. From this we can recover the number of zero bytes, so we don't need to explicitly encode it.
/span/p

/spanp
/spansup53/span/sup&nbsp; The block itself had 81 leading zero-bits.
/span/p

/span/div
/spandiv
    class="tcolorbox"
    style=" border: 1px solid #404040; background: #F2F2F2; "

/spandiv
    class="tcolorboxupper"
    style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbTerm/span/b**/span/span **/spanb/spana href="whitepaper.html#??"Hash Truncation (+T)/span/a/span/b**: The UT protocol variant wherein miners/validators refer to reflected headers using */spanemonly/span/em* the least significant half of the hash. This effectively halves the hash size in throughput calculations for +OP and +HO variants. /span/div /span/div
The idea behind +T is that the security of a truncated hash in bits, assuming $g > 2z$, is given by 8(/spanspan class="textrm textmd textup textsc"/spanspan class="numerator"g/span/span//spanspan class="denominator"2/span/span/span/span + z)/spansup54/span/sup, and that this is */spanemalways/span/em* sufficient, given that z is intimately related to the largest reasonable attack that we could expect at that moment. The $g > 2z$ condition is there for two reasons. First, if $2z \geq$
gthenthesecurityofthetruncatedhashisjust8gbits, likenormal. Second, usingsignificantlymorethanhalfthehashforPoWisproblematicbecausethechanceofcollisionsbetweenvalidheadersincreasesdramatically. So, intuitively, thereissomebreakpointthatwecrossasthebitsusedforPoWincreases. Crossingthatbreakpointindicatesthatthehashisnolonger
−we've"maxeditout"andneedahashwithmorebits, orahashthatishardertogenerate(orslower, etc). Therefore, thereisalsosomesimilarandrelatedbreakpointwherethesafetyofhashtruncationdegrades. Perhapsitdoesnotdegradecompletely, butatleastenoughthatpartialcollisions(withouttherequiredPoW)oftheleastsignificantbitsbecomepracticaltogen
2zshouldprovideusareasonablesafetymargintoavoidsuchissues − − − ifwegetclosetoit, it'stimetochangethehash.
Combining +HO and +T gives us +HOT, the highest capacity variant of UT_1. In this configuration: headers are smaller, the information required for PoR regeneration is minimal, and the number of chains per simplex within given constraints is maximal. Since +T halves the effective hash length (which is all the data in the PoRs half of the block), the overall simplex capacity increases 2 ×.
/spanfigure id="autoid-24" class="figure "
/spanspan class="hidden" \(\seteqnumber{0}{}{26}\)/span/span <!--



```
-->
\begin{equation*}
\xymatrix @R=14pt@!R@C=-24pt@!C@M=4pt{ *+[F:&lt;3pt&gt;]{\text {Conservative}} \ar @{.&gt;}[d] \\ \text {UT}_{+\text {PoRs}} \ar @*{[gold]}[dr] \ar [rr] \ar [dd] &amp; &amp; \text {UT}_{+\text {OP}} \ar &apos;[d][dd] \ar [dr] &amp; \\ &amp;
\text {UT}_{+\text {PoRTs}} \ar [dd] \ar @*{[gold]}[rr] &amp; &amp; \text {UT}_{+\text {OPT}} \ar @*{[gold]}[dd] \\ \text {UT}_{+\text {HOPoRs}} \ar [dr] \ar &apos;[r][rr] &amp; &amp; \text {UT}_{+\text {HO}} \ar [dr] &amp; \\ &amp; \text
{UT}_{+\text {HOPoRTs}} \ar [rr] &amp; &amp; \text {UT}_{+\text {HOT}} \\ &amp; &amp; &amp; *+[F:&lt;3pt&gt;]{\text {Maximal TPS}} \ar @{.&gt;}[u] }
\end{equation*}
```

/spandiv class="figurecaption"Figure 16: Possible upgrade paths between UT variants, starting at UT_{+PoRs} in the top left — the most conservative variant. Solid arrows show paths of increasing capacity.
/span/div /span/figure

/spandiv role="note" class="footnotes"
/spanp
/spansup54/span/sup&nbsp; I use g here, even though it's measured in bytes, for consistency with the rest of the whitepaper. A more rigorous method would measure everything in bits instead, and avoid the somewhat awkward multiplication by 8, but it's not important for this discussion.
/span/p

/span/div

<!--.................................-->
/spanh5 id="autosec-255"/spanspan class="sectionnumber"4.5&#x2003;/span/spanStateless Full Nodes and Fraud Proofs/span/h5

A stateless full node (a.k.a. a stateless client) is a node that verifies all blocks, but does not actively track the current state of the chain. Instead, a stateless full node will use some minimal /spanem*witness data*/span/em (specific to each block) to verify that the state transition was valid. This is possible because /spanem*most*/span/em of the state tree remains untouched and thus the state root can be recalculated easily. The mutations to the state tree made by a block's transactions are all calculable from the witness data and the transactions, alone. In other words: the prior state root, the witness data, and the full block are all that is required to calculate the output state root of that block.

/spandiv
    class="tcolorbox"
    style=" border: 1px solid #404040; background: #ECECEC; "

/spandiv
    class="tcolorboxupper"
    style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbAside/span/b**/span/span We won't focus too much on the specifics of stateless nodes here, since there are many existing resources/spansup55/span/sup on the topic. Note that some stateless node designs require that each individual /spanem*transaction*/span/em include appropriate witnesses, rather than each block having a combined witness.
Although we will design a method that supports transaction witnesses, we won't require that transactions include them. /span/div /span/div
Supporting stateless nodes introduces some constraints on the cryptographic design of state and blocks. Updates to the state tree must be computationally efficient — something like
O(log *m*)*complexityformtotalstateelements.Practically,itappearsthatthisrequiresusinganappendonlystructure*(*e.g.,a/spanahref* = "*https* : //*github.com/opentimestamps/opentimestamps* − *server/blob/master/doc/merkle* − *mountain* − *range.md*" *target* = "_*blank*" *MerkleMountainRange*(*PeterTodd*; 2018)/*span/a*),*orsomekindofmerkliz*
This is a good start, and we can now produce witnesses for each transaction.
However, the witness associated with a transaction /spanem*before*/span/em it is confirmed /spanem*will rarely*/span/em be the same as the witness /spanem*after*/span/em it is confirmed. The before-witness will prove some state against the prior block's state root. However, the state is almost always mutated before a transaction is executed — the state root is different. The after-witness should use, instead of the state root of the prior block, the root of the /spanem*intermediate*/span/em state immediately prior to the execution of that transaction. This way, the headers-only chain, the after-witness, and the transaction are the only data required to verify the validity of the transaction (assuming that the merkle proof of the transaction in the block also includes the intermediate state roots directly before and after the a transaction).
Why does this after-witness matter so much? Because it /spanem*is*/span/em the /spanem*fraud proof*/span/em for any fraudulent transaction.

/spandiv role="note" class="footnotes"
/spanp
/spansup55/span/sup&nbsp; See: /spana href="https://web.archive.org/web/20210523201110/https://github.com/zack-bitcoin/amoveo-docs/blob/master/design/stateless_full_node.md" target="_blank" */spanemStateless Full Node/span/em*/span/a (Zack Hess), /spana href="https://web.archive.org/web/20210602053116/https://ethresear.ch/t/the-stateless-client-concept/172" target="_blank" */spanemThe Stateless Client Concept/span/em*/span/a (V. Buterin), /spana href="https://web.archive.org/web/20210224050038/https://petertodd.org/2016/delayed-txo-commitments" target="_blank" */spanemMaking UTXO Set Growth Irrelevant With Low-Latency Delayed TXO Commitments/span/em*/span/a (Peter Todd).
/span/p

/spanp
/spansup56/span/sup&nbsp; This isn't */spanemstrictly/span/em* necessary — it could be recorded elsewhere in a block, e.g., the coinbase transaction. However, in that case, the effective header size becomes the actual header, plus the branch to the coinbase tx, plus the coinbase tx. Committing elsewhere in the block doesn't have any overhead for stateless full nodes (since they have the full block data, anyway), but it does increase witness sizes for individual transactions.
/span/p

/span/div
/spandiv
class="tcolorbox"
style=" border: 1px solid #404040; background: #F2F2F2; "

/spandiv
class="tcolorboxupper"
style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbTerm/span/b**/span/span **/spanb/spana href="whitepaper.html#??"Fraud Proof/span/a/span/b**: Cryptographic evidence that a transaction, block, or state transition was invalid, incorrectly executed, or incorrectly not executed. /span/div /span/div
We do, however, need to ensure these mid-state roots are also recorded in the block. Typically, a block will commit to its transactions via something like the root of a merkle tree of those transactions. We can trivially modify this design by replacing each leaf with the corresponding {mid-state root, transaction} pair. Thus, if a block is invalid due to containing a fraudulent or otherwise invalid transaction, the fraud proof for that block consists of: the merkle branch proving the {mid-state root, transaction} pair, the pair itself, and the witness data for that transaction matching the mid-state root.
Additionally, such fraud proofs are all but */spanemautomatically/span/em* generated by full nodes during block verification — the witness data is exactly those state entries that the node requires to validate the transaction. In some cases, the witness data may not even be required if, for example, the transaction has an invalid signature. We could even optimize certain cases to reduce witness sizes by showing only the minimal data to prove that a particular condition was violated.

<!--...............................-->
/spanh5 id="autosec-257"/spanspan class="sectionnumber"4.6&#x2003;/span/spanThe PoR Graph/span/h5

As a simplex ages, a complex graph of PoRs is produced (and it is a DAG). Each vertex represents a block in some simplex-chain, and the outgoing edges point to reflected blocks. An example section of a simulated PoR graph is shown in fig. /spana href="whitepaper.html#lateximage-whitepaper-63"17/span/a.
/spanfigure id="autoid-25" class="figure "

/spandiv class="center"
/spana href="27-practical-30-fig1-por-graph-sim_sag" target="_blank" /spanimg
    src="27-practical-30-fig1-por-graph-sim_sag"
    class="inlineimage"
    alt="(image)"
/span/a

/spandiv class="figurecaption"Figure 17: A simulated PoR graph of a 9-simplex over ∼10 block periods. Approximately 900 reflections are shown. Opaque horizontal arrows point from child to parent, and the semitransparent arrows point to reflected blocks.
/span/div /span/div/span/figure

Since there are N$_1 chains, andeachblockhas\sim N_1 reflectionsonaverage, thetotalnumberofreflections, perblockperiod, is\sim {N_1}^2 = O(c^2). DoesthispresentaproblemforfullyvalidatingnodesthatwishtoverifytheentirePoRgraph?$

Recall that in section /spana href="whitepaper.html#lateximage-whitepaper-63"4.4/span/a, we took advantage of redundant data within the PoR graph. Particularly that, provided we knew /*spanemwhich/span/em* blocks were reflected and the /*spanemordering/span/em* used to create the PoRs root, we could deterministically reconstruct those parts of any simplex-chain block. However, this relied on either tracking /*spanemthe entire/span/em* PoR graph, or additionally downloading all missing merkle branches corresponding to PoRs used by the reflected block.

Why is so much information required? The reason is that each miner can choose /*spanemwhether/span/em* to reflect any remote block (assuming that the block is available), and has freedom to choose the /*spanemposition/span/em* of each reflected block in the merkle tree of PoRs.

Regarding the /*spanemposition/span/em* of reflected blocks — this is easy to solve. All we need to do is ensure there is only one position that a block can be in. For example, we could require that reflected blocks are ordered based on the ordering of their genesis blocks (which is fixed). If two or more blocks from a single simplex-chain are reflected, we can use a sub-ordering method (of which there are many options). This gives us a total ordering, meaning that we only need to know /*spanemwhich/span/em* blocks were reflected to reconstruct this segment of a block. Alternatively, we could use a merklized prefix tree which achieves the same goal. Either way, we have reduced the required information from a /*spanemlist/span/em* of block hashes to a /*spanemset/span/em* of them.

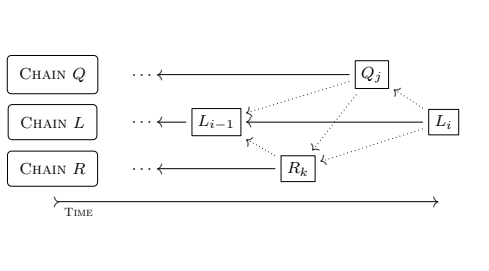
It is clear that miners need some freedom to choose /*spanemwhether/span/em* to reflect a block or not. For one, miners must be able to choose to reflect /*spanemnew/span/em* blocks as they appear. Additionally, due to the /spana href="whitepaper.html#lateximage-whitepaper-63"Axiom of Availability/span/a, miners must not reflect unavailable blocks, even if a valid PoR can be generated. However, because all honest miners obey the /spana href="whitepaper.html#lateximage-whitepaper-63"Axiom of Availability/span/a, there is never a reason for an honest miner /*spanemnot/span/em* to reflect a block that is available — doing so will always increase the overall chain-weight of that miner's draft block.

So, if an honest miner's block, L_i, reflects some block, R_k, then any block reflected by R_k must also be available (since the /spana href="whitepaper.html#lateximage-whitepaper-63"Axiom of Availability/span/ais recursive). If a block is available then a PoR is possible. Therefore, an honest miner should reflect all non-local blocks that were reflected by R_k but have not already been reflected by an ancestor of L_i.

Let's call this a state of /*spanemmaximal reflection/span/em*, and say that such a PoR graph is /*spanemmaximally reflective/span/em*.

<!--...............................-->
/spanh6 id="autosec-259"/spanspan class="sectionnumber"4.6.1&#x2003;/span/spanMaximally Reflective PoR Graphs/span/h6

Specifically, we can say this about any PoR graph in a state of maximal reflection: for every block L_i, and for every block R_k reflected by L_i, then every block b that is reflected by R_k satisfies one of: b is reflected by L_i; or b was reflected by an ancestor of L_i; or b is an ancestor of L_i.
Notice that, if $L_i$reflected any blocks, there will always be at least one block that */spanemwas/span/em* reflected by $L_i$that was */spanemnot/span/em* reflected by any other block that was reflected by L_i. Let us call these blocks the */spanemtips/span/em* of the PoR graph from L_i's perspective. PoR graph tips might reflect other blocks reflected by L_i, too.
For example, consider this PoR graph segment between 3 chains L, Q, and R:
/spanspan class="hidden" \(\seteqnumber{0}{}{26}\)/span/span <!--



-->
\begin{equation*}
\xyMChains { \xyL {\text {Chain \cQ }} &amp; \cdots &amp; &amp; &amp; \xBQ {} \arm [lll] \arf [lld] \arf [ldd] &amp; \\ \xyL {\text {Chain \cL }} &amp; \cdots &amp; \xBL {-1} \arm [l] &amp; &amp; &amp; \xBL {} \arm [lll] \arf [lld] \arf [lu] \\ \xyL {\text {Chain \cR }} &amp; \cdots &amp; &amp; \xBR {} \arf [lu] \arm [ll] &amp; &amp; \\ \xyTimeHoriz {rrrrr} &amp;&amp;&amp;&amp;&amp; }
\end{equation*}

Here, L_i reflects both Q_j and R_k, and Q_j also reflects R_k. Thus, from L_i's perspective, Q_j is a */spanemgraph tip/span/em*; and, from Q_j's perspective, R_k is a */spanemgraph tip/span/em*.
Starting from L_{i-1}, we could completely describe this section like so: /spanul style="list-style-type:none"

/spanli
- R_k reflects L_{i-1}; and

/span/li
/spanli
- Q_j reflects $\{R_k, L_{i-1}\}$; and

/span/li
/spanli
- L_i reflects $\{Q_j, R_k\}$.

/span/li
/span/ul
If the PoR graph is not maximally reflective, then we must specify the complete set of reflections for each block. However, */spanemwhen maximally reflective/span/em*, most of that information is redundant (about half in this example).
We */spanemdo/span/em* still need some information for each block, however, we can recreate the full PoR segment with less information — some of the reflections are */spanemimplied/span/em* and can be omitted: /spanul style="list-style-type:none"

/spanli
- the PoR graph segment is maximally reflective; and

/span/li
/spanli
- R_k reflects the tip L_{i-1}; and

/span/li
/spanli
- Q_j reflects the tip R_k; and

/span/li
/spanli
- L_i reflects the tip Q_j.

/span/li
/span/ul
We can see that the maximally reflective quality has at least two important properties. Firstly: we have */spanemdrastically/span/em*/spansup57/span/sup reduced the amount of information needed to completely describe the graph segment: $O(c^2) \to \sim O(c) - - - anexampleisshowninfig.\ /spanahref = "$whitepaper.html$\#lateximage - whitepaper - 63"18/span/a.Secondly:$
$thePoRgraph, accordingtoeachchain, /spanemconverges/span/emintoasingle, consistentPoRgraph.Inessence, anyblockinamaximallyreflectivePoRgraphsharesa/spanemcommonhistory/span/emwithallpriorblocksinthegraph.$
/spanfigure id="autoid-26" class="figure "

/spandiv class="center"
/spana href="27-practical-30-fig3-por-graph-tips_sag" target="_blank" /spanimg
src="27-practical-30-fig3-por-graph-tips_sag"
class="inlineimage"
alt="(image)"
/span/a

/spandiv class="figurecaption"Figure 18: This demonstrates how maximally reflective PoR graph segments with O(c) many chains can be described using ~O(c) information. Thick dotted arrows represent reflections of PoR graph */spanemtips/span/em*, and semitransparent arrows show other reflections — these reflections and implied via the reflection of the graph tips. This also shows that the DAG of tip-reflections is the */spanemlongest/span/em* path of reflections between two blocks.
/span/div /span/div/span/figure

Is there an intuitive explanation for the reduction in complexity of the PoR graph? Yes. To start with, let us assume that there is no */spanempropagation delay/span/em* — i.e., all miners instantly receive, validate, and incorporate all blocks into the PoR graphs of their draft blocks. In this case, no blocks are ever mined simultaneously, and so each block will reflect exactly one PoR graph tip. Since there are O(c) many chains, and we need to add O(1) extra information per header (the reflected tip), this is O(c) extra information all up.

In practice, blocks */spanemwill/span/em* sometimes be
mined simultaneously. So, even if */spanemmost/span/em* of the time a block only reflects one tip, */spanemsome/span/em* of the time that block will reflect multiple tips. The frequency of simultaneous blocks is dependent on the number of chains $(N_1), theblockfrequencyofeachchain(B_f), andsomeaveragepropagation/spansup58/span/supdelay(\phi seconds).Twoblocksaresimultaneousiftheyareminedwithin{\sim}\phi secondsofeachother.Inthe\phi secondsbeforethatmostrecenttipwasmined, weexpectthat{\sim}N_1B_f\phi othertipsweremined(fromanysimplex-chain).Unlessablockcontainsnoreflections(whichhappenswithprobability1-{N_1}^{-1}), itmustreflectatleastonegraphtip.So, weexpecteachblocktoreflect1+N_1B_f\phi tips.ForN_1=100, B_f=$
$/spanspanclass="textrm\ textmd\ textup\ textsc"/spanspanclass="numerator"1/span/span//spanspanclass="denominator"15/span/span/span/span, \phi=1, weexpecteachblocktobesimultaneouswith{\sim}7others, andeachblocktoreflect8graphtipsonaverage.$

Therefore, the overhead to reconstruct the PoR graph — and every PoR segment of every simplex-chain block — is, on average, g (1 +
N$_1B_f\phi)bytesperblock, wheregisthelengthofahash.Asg, 1, andB_fareconstants, andifwetreat\phi asconstant, theoverheadthushascomplexityO(c)/spanempersimplex-chain/span/em.Sincewe'retrackingO(c)chains, theoverallloadisO(c^2).$

The total bandwidth required to track the PoR graph (excluding verifying the availability of blocks) is: the overhead per block, plus the block header, all multiplied by the network-wide blocks per second rate. Thus, the total bandwidth is N$_1B_f(B_h+g(1+N_1B_f\phi))B/s---whichisO(c^2).$

However, we estimated before that we should expect to require 7 graph tips on average per block (which seems much less than O(c)). Practically speaking, the overhead is only significant when the header size is much smaller than the overhead: B$_h\ll g(1+N_1B_f\phi).Ifwepluginourroughnumbersfrombefore, wefindg(1+N_1B_f\phi)\approx 250\approx 2B_h=O(1).$

There */spanemis/span/em* a practical limit (depending on B$_f\phi)forhowlargeN_1cangrowbeforethisoverheadisnon-negligible./spanemHowever/span/em, ifwecappedtheexpectedoverheadto, say, 256bytes---g(1+N_1B_f\phi)\leq 256---thenwefindN_1\leq$
$7(B_f\phi)^{-1}\approx 105\cdot\phi^{-1}.Cappingtheoverheadto128bytesyieldsN_1\lesssim 45\cdot\phi^{-1}.NotethatmaximalvaluesofN_1areinverselyproportionaltoB_f, somodifyingB_fdoesn'tchangetheexpectedoverhead.$

/spandiv role="note" class="footnotes"
/spanp
/spansup57/span/sup&nbsp; The theoretical overhead is still O(c) per block, but practically we expect the overhead to be a similar size to the header which is O(1). It's explained shortly.
/span/p

/spanp
/spansup58/span/sup&nbsp; In particular, we are concerned with propagation */spanembetween miners/span/em*, not the network as a whole.
/span/p

/span/div

<!--...............................-->
/spanh6 id="autosec-262"/spanspan class="sectionnumber"4.6.2&#x2003;/span/spanOptimization or Contradiction?/span/h6

We calculated the overhead to be
g(1+N$_1 B_f \phi) = O(c) --- thisisbecauseweassumedthatO(B_f \phi) = O(1). However, ifthiswerereallythecase, weshouldbeabletooptimizethePoRsectionsofblocks(similartosection$ /spanahref = "whitepaper.html#$lateximage - whitepaper - 63$"4.4/span/a).
We could try to design the protocol like this: /spanem*instead of recording all reflections, each block records only the minimum necessary to reconstruct the PoR graph*/span/em. This would increase the overall bandwidth required to track the PoR graph, but our expression remains the same: N$_1 B_f (B_h + g(1 + N_1 B_f \phi))$.
Since the overhead is O(c), and recording all reflections is also O(c), can we swap in this new process to increase scalability?
The old PoR sub-block takes up N$_1 B_f \alpha B/s of capacity (where \alpha represents the bytes per reflected simplex-chain and \alpha \in [16, 512])$.
The new PoR sub-block instead takes up
B$_f \cdot g(1 + N_1 B_f \phi) B/s of capacity --- we exclude the block's header since that's already part of the block. If k_{1,B} B/s of a chain's capacity is dedicated to reflections, then we have: /spanspanclass = "hidden"\backslash(\backslash seteqnumber\{0\}\{\}\{26\}\backslash)/span/span$
<!--

$$
\begin{aligned}
k_{1,B} &= B_f g(1 + N_1 B_f \phi) \\
N_1 &= \frac{1}{B_f \phi}\left(\frac{k_{1,B}}{B_f g} - 1\right) \qquad (27)
\end{aligned}
$$

-->

\begin{align}
k_{1,B} &amp;= B_f g(1 + N_1 B_f \phi) \nonumber \\ N_1 &amp;= \frac {1}{B_f \phi } \Big (\frac {k_{1,B}}{B_f g} - 1\Big) \label {eq:bad-n1-with-phi}
\end{align}
Setting k$_{1,B} = 1,500 B/s and evaluating yields N_1 \approx 10{,}500 \cdot \phi^{-1}$.
Of course, ten thousand chains is a lot, and full tracking the PoR graph would require ∼15 MB/s of bandwidth and around 500 TB of storage per year. Each block would reflect, on average, 10k other blocks. Every block period, keeping up with the PoR graph would require calculating 10k PoRs roots — although there are significant optimizations to avoid recalculating these from scratch.

/spandiv
class="tcolorbox"
style=" border: 1px solid #404040; background: #ECECEC; "

/spandiv
class="tcolorboxupper"
style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbAside/span/b**/span/spanI don't know about you, but I'm getting the suspicion that this isn't really an O(c) load. /span/div /span/div
Since this conflicts with our notion of an O(c) load, let's examine one of our premises: that O(N$_1 B_f \phi) = O(c). Certainly O(N_1) = O(c), and O(B_f) = O(1), but does O(\phi) = O(1)$?
There is a good reason to think O$(\phi) = O(1). At some point, propagation delay is limited by the speed of light. So latency must have a definite minimum, which is a constant.$
However, the physical limits on latency do not dominate real-world latency yet. Near-future networking technologies, like LEO satellite constellations, will reduce latencies over significant distances. Moreover, even as the best-case minimum latency approaches physical limits, technology will continue to improve latency in /spanem*bandwidth intensive*/span/em contexts./spansup59/span/sup Therefore,
O$(\phi)/spanem should not/span/em be considered O(1), but rather should be considered O(c^{-1}): as technology improves, \phi decreases.$
If O$(\phi) = O(c^{-1}), this resolves the contradiction. It means that O(N_1 B_f \phi) = O(1). That matches our intuition from earlier, /spanem and/span/em means that maximally reflective PoR graphs really are only O(c) complex, rather than O(c^2)$.
We can also explain why we had such a large N$_1 estimate before --- we took an O(1) load and blew it up to an O(c) load. So we effectively changed O(N_1) from O(c) \rightarrow O(c^2). That meant that network-wide bandwidth and storage requirements went from O(c^2) \rightarrow O(c^3)$.
There is also an argument that
$\phi depends on the number of nodes in the P2P network --- that would take the form of a \log_b(n) term in O(\phi). While this is true for naive gossip protocols, in practice we see that miners will go to some effort to specifically reduce latency for block propagation. Under near-ideal conditions, miners can propagate new blocks and reflections to one another with near-zero local overhead (compared to the milliseconds of latency between them). Since this propagation is constrained by the latency between these special nodes, and not by the size of the network as a whole, we will ignore any \log_b(n) component of \phi.$

/spandiv role="note" class="footnotes"
/spanp
/spansup59/span/sup&nbsp; This is something we didn't consider when estimating N_1ineq. /spanspanclass = "textup"(/spanahref = "whitepaper.html#lateximage − whitepaper − 63"27/span/a)/span/span − − − thatthepropagationdelaywillincreaseastheexcesscapacityofnodes′availablebandwidthdecreases.Likeallsuchsystems, responsetimeisinverselyproportionaltoexcesscapacity.
/span/p

/span/div

<!--...............................-->
/spanh6 id="autosec-264"/spanspan class="sectionnumber"4.6.3&#x2003;/span/spanAxiom of Maximal Reflection/span/h6

It should be clear that there are decisive advantages for UT if the PoR graph is maximally reflective. Particularly, we can accurately describe the PoR graph entirely via */spanemnew reflections/span/em*, greatly reducing PoR overhead. Thus, we will require it via a protocol rule: the /spana href="whitepaper.html#latexImage-whitepaper-63"Axiom of Maximal Reflection/span/a.

/spandiv
class="tcolorbox"
style=" border: 1px solid #333333; background: #F2F2F2; "

/spandiv
class="tcolorboxtitle"
style=" border-top: 1px solid #333333; border-bottom: 1px solid #333333; background: #333333; color: #FFFFFF; "
Axiom of Maximal Reflection/span/div
/spandiv
class="tcolorboxupper"
style=" color: #000000; "
/spanemPrinciple:/span/em A miner must reflect all possible blocks./spanspan role="note" class="marginpar"**/spanbAxiom/span/b**/span/span

/span/div
/spandiv
class="tcolorboxlower"
style="border-top: 1px dashed #333333; color: #000000; "

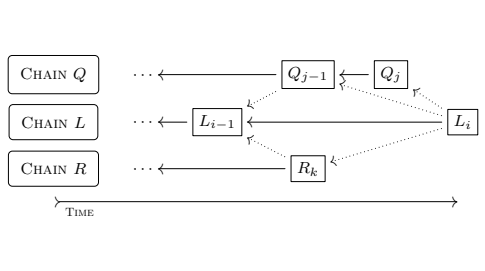
/spanemPredicate:/span/em A block, L_i, is invalid unless, for each reflected block R_k: /spanul style="list-style-type:none"

/spanli
i. R_k is valid under this Axiom; and
/span/li
/spanli
ii. each block that was reflected by R_k or is a parent of R_k: /spanul style="list-style-type:none"

/spanli
a. is reflected by L_i; or
/span/li
/spanli
b. is an ancestor of L_i; or
/span/li
/spanli
c. was reflected by an ancestor of L_i.
/span/li
/span/ul
/span/li
/span/ul
/span/div /span/div
It is worth noting that this axiom does not impact honest miners — it is in their interest to produce maximally reflective PoR graphs. However, this */spanemdoes/span/em* impact dishonest miners since reflecting other chains becomes all-or-nothing.
If a dishonest miner is determined to selectively reflect only */spanemsome/span/em* simplex-chains, their valid blocks cannot reflect any other blocks that are less selective. Honest miners, on the other hand, */spanemcan/span/em* reflect the dishonest miner's blocks. Thus, the /spana href="whitepaper.html#latexImage-whitepaper-63"Axiom of Maximal Reflection/span/acreates an asymmetry between honest and dishonest miners.

<!--................................-->
/spanh6 id="autosec-265"/spanspan class="sectionnumber"4.6.4&#x2003;/span/spanThe Longest PoR Chain/span/h6

fig. /spana href="whitepaper.html#lateximage-whitepaper-63"18/span/a shows us an interesting feature of maximally reflective PoR graphs: between two given blocks, the union of the paths via PoR graph tips both */spanemvisit every block/span/em* and are the */spanemlongest/span/em* paths between any two blocks. This is somewhat self-evident: since the PoR graph is a DAG, there is no way to return to a block once we have visited it. Thus, if a path exists that visits every block, then it must be a longest path because there are no unvisited blocks. A longer path would need to visit more blocks, but there are no more blocks, therefore no longer path exists.
Although it makes sense to refer to these paths as the */spanemlongest/span/em* — this isn't always strictly true. First, we should notice that multiple paths are possible, and are expected to occur when blocks are created near-simultaneously. In some cases, these paths can be different lengths. For example:
/spanspan class="hidden" \(\seteqnumber{0}{}{27}\)/span/span <!--



```
-->
\begin{equation*}
\xyMChains { \xyL {\text {Chain \cQ }} &amp; \cdots &amp; &amp; \xBQ {-1} \arm [ll] \arf [ld] &amp; \xBQ {} \arm [l] &amp; \\ \xyL {\text {Chain \cL }} &amp; \cdots &amp; \xBL {-1} \arm [l] &amp; &amp; &amp; \xBL {} \arm [lll] \arf [lld]
\arf [lu] \arf [llu] \\ \xyL {\text {Chain \cR }} &amp; \cdots &amp; &amp; \xBR {} \arf [lu] \arm [ll] &amp; &amp; \\ \xyTimeHoriz {rrrrr} &amp;&amp;&amp;&amp;&amp; }
\end{equation*}
```

Here, we can see that L_i reflects the tips $\{Q_j, R_k\}$. There are two paths from L_i to L_{i-1} in this case: one via
$Q_j \to Q_{j-1}, andtheothervia R_k. Botharealongestpathinthesensethattheyarelocallymaximal : nopathsincluding R_k (or Q_j) arepartofalongerpathto L_{i-1}$.
So, really, what we have identified is the DAG of the union of the longest paths between two blocks. This is exactly the information that is necessary to reconstruct a maximally reflective PoR graph.
Let's refer to this as the */spanemlongest PoR chain/span/em*, even though it is technically a DAG. Of all possible PoR DAGs that both include all edges leading to a graph tip and visit every block, the */spanemlongest PoR chain/span/em* has the fewest edges, and is thus the most chain-like of those DAGs. Additionally, in section /spana href="whitepaper.html#lateximage-whitepaper-63"4.8.7/span/a we will discuss some symmetries between blockchains and the longest PoR chain.

Intermission and Note

/spandiv
    class="tcolorbox"
    style=" border: 1px solid #404040; background: #ECECEC; "

/spandiv
    class="tcolorboxupper"
    style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbAside/span/b**/span/span Let us return to the /spana href="whitepaper.html#lateximage-whitepaper-63"Axiom of Maximal Reflection/span/aagain for a moment. I would have said something earlier, but I wanted to give you some time to chew, first. If you were left wondering */spanemWhat happens when R_k reflects an Lblock that is not an ancestor of L_i?/span/em* then I say */spanem'well spotted'/span/em* to you — finding criticisms on a first reading is worthy of praise. /spanli

/span/li
/spanli
Now, the answer to the question: on the one hand, if the *L*chain deliberately forks, and two distinct chains emerge, then they should reflect one another like any other pair of simplex-chains. On the other hand&#x2026; well the other hand is a problem that we can't solve, yet — we'll ignore it for now and return to it in section /spana href="whitepaper.html#lateximage-whitepaper-63"4.8.7/span/a. /span/div /span/div
/span/li

<!--................................-->
/spanh6 id="autosec-266"/spanspan class="sectionnumber"4.6.5&#x2003;/span/spanGeneralizing NIPoPoWs/span/h6

/spandiv
class="tcolorbox"
style=" border: 1px solid #404040; background: #F2F2F2; "

/spandiv
class="tcolorboxupper"
style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbWarning/span/b**/span/spanThis section is speculative; it's about how NIPoPoWs might be possible for the longest PoR chain. Subsequent sections regarding NIPoPoWs will, likewise, be speculative. UT won't depend on NIPoPoWs, but they are useful and it would be good to support them, if possible./span/div /span/div

<!--................................-->
/spanspan class="paragraph" id="autosec-267"/spanspan class="sectionnumber"4.6.5.1&#x2003;/span/spanNIPoPoWs Primer/span/span

Prior work: /spana href="https://web.archive.org/web/20190123153947/https://eprint.iacr.org/2017/963.pdf" target="_blank" Non-Interactive Proofs of Proof-of-Work (Kiayias, Miller, and Zindros; 2018)/span/a.

/spandiv
class="tcolorbox"
style=" border: 1px solid #404040; background: #ECECEC; "

/spandiv
class="tcolorboxupper"
style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbQuote/span/b**/span/span /spanspan role="note" class="marginpar"**/spanbQuote/span/b**/span/span Non-Interactive Proofs of Proof-of-Work (NIPoPoWs) are short stand-alone strings that a computer program can inspect to verify that an event happened on a proof-of-work-based blockchain without connecting to the blockchain network and without downloading all block headers. For example, these proofs can illustrate that a cryptocurrency payment was made.
.7ex]0.5pt

/spandiv class="flushright" /spani— /spana href="https://web.archive.org/web/20210520174729/https://nipopows.com/" target="_blank" *What are NIPoPoWs? (nipopows.com; 2019)/span/a/span/i* /span/div /span/div /span/div

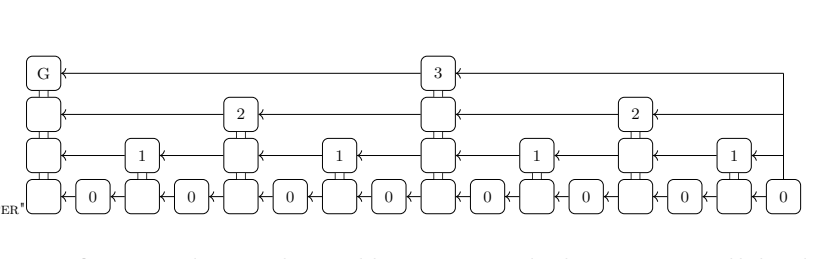
NIPoPoWs are possible due to the statistical properties of proofs of work. They require an additional interlink structure between blocks. The interlink structure is a kind of skip-list, where a block at height h links back to $\sim\log_2 hotherblocks.$

The insight that enables NIPoPoWs (and PoPoWs more generally) is that some PoWs will be */spanemmuch/span/em* lower than the difficulty target, and that the frequency of these special PoWs is dependent on the total number of PoWs that have been produced. That is: if the PoW target is $2^{184}, thenallvalidPoWswillbe \leq 2^{184}, 50\% willbe \leq 2^{183}, 25\% willbe \leq 2^{182}, andsoon. Wecangeneralizethisviathestatement : P(\mathrm{PoW} \leq 2^{184-\mu}) = 2^{-\mu} --- theprobabilitythattheproofofworkislessthanorequalto 2^{184-\mu} equals 1/2^{\mu}. Inessence, \mu isameasureofthe/spanemexcesscapacity/span/emofablock'sPoW : howmanyorders-of-magnitudelower/spanemcould/span/emthetargethavebeen, giventhe/spanemactual/span/emPoW.$

When a block has $\mu> 0$, it is a μ-superblock, and it is these superblocks that the interlink structure uses. Particularly, for each integer value of μ, each block links back to the most recent μ-superblock. The chain of μ level blocks is the μ-superchain. Note that the 0-superchain (or 0-chain) is identical to the blockchain itself. This interlink structure is partially shown in fig. /spana href="whitepaper.html#lateximage-whitepaper-63"19/span/a, which shows each μ-superchain according to the perspective of the right-most block. The full interlink structure is shown in fig. /spana href="whitepaper.html#lateximage-whitepaper-63"20/span/a.

/spanfigure id="autoid-27" class="figure "

/spandiv class="center"



/spandiv class="figurecaption"Figure 19: The μ-superchains used by NIPoPoWs. Each column represents one block. The y-axis indicates the μ level of each block. A μ-superblock is also a (μ-1)-superblock. */spanemCredit:/span/em* the structure and content of this figure is credited to the authors of the NIPoPoW paper (Kiayias, Miller, Zindros) — this figure is a recreation of one of their diagrams./spansup60/span/sup
/span/div /span/div/span/figure
/spanfigure id="autoid-28" class="figure "

/spandiv class="center"
/spana href="diags/27-practical-30-nipopows-all-interlinks_sag" target="_blank" /spanimg
    src="diags/27-practical-30-nipopows-all-interlinks_sag"
    class="inlineimage"
    alt="(image)"
/span/a

/spandiv class="figurecaption"Figure 20: The full interlink structure used by NIPoPoWs for the chain segment shown in fig. /spana href="whitepaper.html#lateximage-whitepaper-63"19/span/a.
/span/div /span/div/span/figure

Intuitively, a NIPoPoW works because it uses the (very short) top-most chain of μ-superblocks as a basis to hone in on a target block (denoted C[-k] in the NIPoPoW paper) a few confirmations behind the chain tip. This is done by progressively inspecting the most recent segments of lesser and lesser μ-superchains to establish the expected chain-weight that has been contributed to that target block./spansup61/span/sup This is the main /spanem*suffix proof*/span/em, as it proves the blocks at the recent end (suffix) of the blockchain. (Additionally, the most recent k blocks (C[-k:]) are included alongside the proof.) The work required to generate the main suffix proof is (approximately) as much work as has been contributed to the entire blockchain history. Thus, an adversarial NIPoPoW requires (approximately) as much work as the total chain-weight of the honest chain, so NIPoPoWs are secure./spansup62/span/sup

Now that the chain-head and relevant μ-superchains have been proven, arbitrary blocks in the chain's history can be proven. This is done by 'following down' from a top-level μ-superblock to the target block, following the interlink structure. Starting with the /spanem*earliest*/span/em μ-superblock between the target block and C[-k], up to μ levels are progressively descended until the target block is reached. 'Following down' means: for a μ-superblock, check if the previous block at level (μ-1) occurred /spanem*after*/span/em the target block — if so, that block becomes the next in the series, otherwise decrease μ by one and repeat with the current block. The proof terminates at the target block. The blocks that are traversed comprise the /spanem*infix proof*/span/em for that block.

An example of a NIPoPoW with a suffix and infix proof is shown in fig. /spana href="whitepaper.html#lateximage-whitepaper-63"21/span/a.

/spanfigure id="autoid-29" class="figure "

/spandiv class="center"
/spana href="diags/27-practical-30-nipopows-large_sag" target="_blank" /spanimg
    src="diags/27-practical-30-nipopows-large_sag"
    class="inlineimage"
    alt="(image)"
/span/a

/spandiv class="figurecaption"Figure 21: The blocks of a traditional chain that comprise a NIPoPoW. For a given $\mu>$ 0, each superblock at that level is */spanemthe same/span/em* block as the one directly below it; only half of the blocks at level μ-1 are also μ-superblocks. Yellow blocks are part of the main suffix proof. Patterned blocks comprise the infix proof of the target block. Patterned blue blocks are unique to the infix proof. The striped pink and gray block is an explanatory aid and is a 3-superblock.
/span/div /span/div/span/figure

Notice that there is no concept of */spanemstate/span/em* used when reasoning about NIPoPoWs — we're only considering a chain of interlinked blocks with PoWs. Put another way: there is no requirement that these blocks belong to the same blockchain; only the PoW and interlink structure matters.

So, at first glance, it appears that the longest PoR chain might work with NIPoPoWs, or some modified variant, at least.

There are two important caveats that are mentioned in the NIPoPoW paper; these are problems for us and we will have to address them.

First: the authors assume a single chain of blocks, not a graph. Thus, */spanemany/span/em* simultaneous blocks in the PoR graph are an issue.

Second: the authors assume a constant difficulty. For a traditional blockchain, this isn't too much of a problem. For UT, however, we ideally want uncoupled simplex-chain difficulties, even if they use the same hashing algorithm. Speaking of which, how does a NIPoPoW work if more than one hashing algorithm is used?

<!--..............................-->
/spanspan class="paragraph" id="autosec-272"/spanspan class="sectionnumber"4.6.5.2&#x2003;/span/spanNon-Interactive Proofs of Proof of Work Reflection (NIPoPoWRs)/span/span

Before we discuss the problems mentioned above, it is important to note that the method we will devise cannot make use of converted work from non-PoW blocks (e.g., if some simplex-chain used PoS, or PoA, etc). That's because there is no real sense to such a block being 'lucky' in the way that PoW superblocks are. We can still link back to non-PoW blocks as 0-blocks, though (i.e., they're at the lowest μ level)./spansup63/span/sup
Thus, these will not be NIPoPoRs — this method requires PoW, so, in terms of PoRs, we are only concerned with PoW reflection, rather than all reflection.

/spandiv role="note" class="footnotes"
/spanp
/spansup60/span/sup&nbsp; Source diagram: /spana href="https://nipopows.com/images/hierarchical-ledger.png" target="_blank" https://nipopows.com/images/hierarchical-ledger.png/span/a
/span/p

/spanp
/spansup61/span/sup&nbsp; The NIPoPoW algorithms don't do this exactly, but they embody similar and convergent knowledge.
/span/p

/spanp
/spansup62/span/sup&nbsp; This is a simplification — see /spana href="https://web.archive.org/web/20220329012237/https://cloudflare-ipfs.com/ipfs/QmYDpsHgs8wjX4qedMUDzanErehZuUcpV8fp2dBFbr9c5r" target="_blank" Non-Interactive Proofs of Proof-of-Work/span/a (Kiayias, Miller, and Zindros; 2018)for full details.
/span/p

/spanp
/spansup63/span/sup&nbsp; Note that, currently, this implies that non-PoW blocks are considered to contribute some work, which they do not in this context (it is not verifiable). We will resolve this conflict shortly — at the end of the section, it will make sense to consider them 0-blocks at the lowest μ level.
/span/p

/span/div
/spandiv
    class="tcolorbox"
    style=" border: 1px solid #404040; background: #ECECEC; "

/spandiv
    class="tcolorboxupper"
    style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbAside/span/b**/span/span You should know, I am only confident about the */spanemprinciples/span/em* of NIPoPoWs, not the specifics. So, I claim the following only as an */spanemin principle/span/em* argument. Further diligence is required. /span/div /span/div

Problem: DAG Compatibility If the longest PoR chain is a DAG, then the interlink will need to be a DAG, too. Sometimes, two (or more) blocks will be simultaneous, so we will have two (or more) graph tips in those cases. When that happens, we will always have at least two 0-superblocks (since a 3-superblock is also a 2-superblock, etc). In fact, at and below the μ level of the lightest graph tip, all interlink levels will point back to all graph tips.
We are therefore dealing with μ-superDAGs, not μ-superchains. The existing NIPoPoW /spankbdfollowDown/span/kbd/spansup64/span/sup algorithm should work fine for high level μ-superblocks. Since simultaneous blocks at such levels will be exceptionally rare, we can expect higher μ-superDAGs to be much more chain-like than the longest PoR chain.
At lower levels (or any level), it doesn't matter which path to the target block we take (if more than one is even available). The PoR graph has no conflicts and no concept of a "doublespend" — rather, it is a massive graph of interconnected merklized tree structures where vertexes are merkle roots and comprise the leaves of future vertex-roots; thus, edges are merkle branches. Put another way, there's no sense of */spanempriority/span/em* between siblings of the same μ level. This is not true for chains using GHOST (or block-DAGs)/spansup65/span/sup — in those cases it is possible for an uncle block to be of a higher μ level than its non-uncle sibling, so this construction doesn't cover those cases. But, in the case of the longest PoR chain, we are */spanemonly/span/em* concerned about */spanemwhether/span/em* a block exists — it doesn't matter */spanemhow/span/em* we get there, just that we */spanemcan/span/em* get there.
Curiously, this change is an alternate solution to the problem of an attacker "[breaking] chain superquality with non-negligible probability" addressed in the NIPoPoW paper (Section 7).
The attack is one that allows an attacker to manipulate the public chain's "superquality" (a measure of the frequency of μ-superblocks), such that fewer superblocks become canonical. In essence, the attacker selectively performs a selfish-mining-like attack whenever a significant μ-superblock is found. Their goal is to mine an alternate longest chain, such that the superblock becomes stale. This then allows the attacker to produce, in private, a superior NIPoPoW (compared to that of the public chain) for fewer resources than should be required.
The authors solve this problem via introducing a concept of "*/spanemgoodness/span/em*" — which is basically that proofs must be succinct and "*/spanemgood at every level/span/em*" (when this is not possible, provide the whole chain instead). The authors then show that, when goodness is not violated, the attack is impractical.
In our modified graph-based version, though, there is no such problem: blocks cannot become stale. Additionally, the /spana href="whitepaper.html#lateximage-whitepaper-63"Axiom of Maximal Reflection/span/aguarantees that the interlink history will fully span the entire PoR graph.

Problem: Variable Difficulty and Multiple Hashing Algorithms There are really two problems here for us to think about. If we can solve the problem of blocks using different hashing algorithms, then we will need to handle variable difficulty anyway, so let's work on the critical path first (variable difficulty).
Let's consider the superchains of two very similar traditional chains, $\mathcal{A}$and $\mathcal{B}$, using the same PoW algorithm. Since these chains use the same hash, we can scale their respective μ values depending on their relative difficulties.
If the difficulty of $\mathcal{B}$is half that of $\mathcal{A}$, then we can convert $\mu_{\mathcal{A}} levels into \mu_{\mathcal{B}} levels by adding 1. In this way, all blocks in \mathcal{A} are at level \mu_{\mathcal{A}} = 0 which is equivalent to level \mu_{\mathcal{B}} = 1$.
We can observe this equivalence via the following thought experiment: what does $\mathcal{B}$'s 1-superchain look like if $\mathcal{B}$'s block frequency is twice that of $\mathcal{A}$? First, we should expect that, for a given period, there are twice as many $\mathcal{B}$blocks as there are $\mathcal{A}$blocks. Since about half of $\mathcal{B}$blocks will have $\mu_{\mathcal{B}} \geq 1, there will be approximately as many \mathcal{B} blocks in the (\mu_{\mathcal{B}} = 1) - superchain as in \mathcal{A}'s (\mu_{\mathcal{A}} = 0) - superchain$.
This makes sense: there are twice as many blocks at half the difficulty, so we expect the same hash-rate to be mining on each chain. If both chains have the same hash-rate, then they should produce the same number of blocks at a given $\mu_{\mathcal{A}} level. Since \mathcal{B} has the lower difficulty, this is not true for a given \mu_{\mathcal{B}} level --- \mathcal{A} produces /span emno/span/em blocks at level \mu_{\mathcal{B}} = 0$.
Now, let us modify $\mathcal{B}$'s protocol in a strange way. We'll say that $\mathcal{B}$blocks */spanemmust/span/em* explicitly include all transactions (in sequence) from all ancestor blocks between the block itself and the prior $(\mu_{\mathcal{B}} \geq 1) - superblock. We'll also assume that there are suitable optimizations such that this isn't an issue for the \mathcal{B} network. This modification means that no \mathcal{B} N1PoPoW ever needs to descend to below \mu_{\mathcal{B}} = 1 again$.
For all intents and purposes, $\mathcal{A}$and $\mathcal{B}$produce almost indistinguishable NIPoPoWs (excepting, of course, that they prove blocks in different chains). Without analysis of the $\mathcal{B}$protocol, and ignoring that all $\mathcal{B}$proofs end at level $\mu_{\mathcal{B}} = 1, there is no sign that the \mathcal{B} chain has twice as many blocks at half the difficulty. The proofs are similar sizes, descend the same number of \mu levels, have similar numbers of blocks at each (adjusted) \mu level, etc. So we might ask the question : how do we know that \mathcal{A} doesn't produce blocks at level \mu_{\mathcal{A}} = -1, similar to \mathcal{B}$?
The point of this is to demonstrate that μ levels are relative and convertible between $\mathcal{A}$and $\mathcal{B}$, and there is no appreciable difference based on any μ-offset.
Therefore, we can express
$\mu_{\mathcal{A}} and \mu_{\mathcal{B}} in objective terms by measuring them relative to an arbitrarily low difficulty (e.g., 1). Let's call the \mu level in terms of a difficulty of 1 the /span emobjective \mu/span/em level. /span sup66/span/sup A consequence of this is that duplicate interlink entries (over sequential levels) will become common when there is a large difference between difficulties (i.e., \mu_{\mathcal{A}} \gg \mu_{\mathcal{B}}). Thus, we should also modify the interlink structure to prepend the \mu level to each group of previous blocks at that level. This allows us to omit the duplicates, assuming we use an appropriate cryptographic structure (e.g., a merklix tree seems suitable at first glance)$.
This works because no proof for either $\mathcal{A}$or $\mathcal{B}$will ever need to descend below the minimum objective μ of that chain's difficulty, and the statistical properties of all higher μ levels are maintained. It is no easier for an attacker to generate a malicious NIPoPoW than it would be otherwise.
Further, this is consistent with the observation that, if $\mathcal{A}$'s hash-rate doubled, then it would appear as though the minimum μ level increased by 1 (assuming that μ levels are not recalibrated according to the current difficulty).
However, this method is only partially consistent with the notion of */spanemgoodness/span/em* introduced in the NIPoPoW paper — the statistical distributions of μ-superblocks will only hold until the objective μ level of a chain's difficulty. We will ignore this problem as the graph-interlink structure has no need of a goodness measurement.
This resolves the first part of the problem: variable difficulties.
With regards to multiple hashing algorithms, PoR itself has already laid the groundwork for a solution: the conversion of work. Recall that in eq. /spanspan class="textup"(/spana href="whitepaper.html#lateximage-whitepaper-63"2/span/a)/span/span we derived $\text{ConvWork}_{R \to L}$: a function for converting R-hashes $\to L - hashes$.
We can use this to calculate a PoW's equivalent μ level in terms of another hash. We can think of converting a block's weight as starting with the chain Rtarget, converting that to the difficulty $(R_d hashes/block), applying \text{ConvWork}_{R \to L}(R_d), and converting this to the L target. Similarly, we can convert \mu levels using the same process, except that we replace the /span emactual/span/em target with the /span emlowest possible satisfiable target/span/em for the block in question$.
However, these μ levels are only comparable after linear scaling, so we need to differentiate them. From here out, we only need objective μ values, so let's use μ_R to refer to the objective μ of the Rchain, and μ_L to refer to that of the Lchain. We should note that, for a chain C, if we want the μ_C level relative to the difficulty, we just need to subtract $\log_2(C_d) (which is the minimum objective \mu_C level of any chain C block)$.
Conversion Example: let's start with an Rchain PoW target of
$2^{184} --- corresponding to a difficulty of 2^{72}. Next, a block is produced with a PoW of 2^{181}; it has an objective \mu_R of 75. The minimum target that the block would satisfy is 2^{181}, which corresponds to a maximum difficulty of 2^{75} (R - hashes). Note that \log_2($max. difficulty$) = \mu_R. We then convert this via \text{ConvWork}_{R \to L}(2^{75}) to get L - hashes. All that remains is to take and take \log_2 to get the converted objective \mu_L$.
What can we say about μ_R's relationship with μ_L? Consider that, in the SRT context, eq. /spanspan class="textup"(/spana href="whitepaper.html#lateximage-whitepaper-63"1/span/a)/span/span simplifies to
$L_d R_r (R_d L_r)^{-1}. Therefore : /span span class = "hidden"\(\backslash seteqnumber\{0\}\{\}\{27\}\backslash)/span/span$
<!--

$$
\begin{aligned}
2^{\mu_R} \cdot L_d R_r (R_d L_r)^{-1} &= 2^{\mu_L} \\
2^{\mu_R} \cdot R_r / R_d &= 2^{\mu_L} \cdot L_r / L_d \\
\log_2(2^{\mu_R} \cdot R_r / R_d) &= \log_2(2^{\mu_L} \cdot L_r / L_d) \\
\mu_R - \log_2(R_d) + \log_2(R_r) &= \mu_L - \log_2(L_d) + \log_2(L_r) \qquad (28) \\
\mu_R + \log_2(L_d R_r) - \log_2(L_r R_d) &= \mu_L \qquad (29)
\end{aligned}
$$

-->

\begin{align}
2^{\muType \cR } \cdot \cL _d \cR _r (\cR _d \cL _r)^{-1} &amp;= 2^{\muType \cL } \nonumber \\ 2^{\muType \cR } \cdot \cR _r / \cR _d &amp;= 2^{\muType \cL } \cdot \cL _r/\cL _d \nonumber \\ \log _2(2^{\muType \cR } \cdot \cR _r / \cR _d) &amp;= \log _2(2^{\muType \cL } \cdot \cL _r/\cL _d) \nonumber \\ \muType \cR - \log _2(\cR _d) + \log _2(\cR _r) &amp;= \muType \cL - \log _2(\cL _d) + \log _2(\cL _r) \label {eq:mu-relative-conv} \\ \muType \cR + \log _2(\cL _d \cR _r) - \log _2(\cL _r \cR _d) &amp;= \muType \cL \label {eq:mu-type-conv}
\end{align}
Note that μ_R- $\log_2(R_d) (eq. /span span class = "textup"(/span a href = "whitepaper.html#lateximage - whitepaper - 63"28/span/a)/span/span) is the block's relative \mu_R level (and similarly for \mu_L on the right - hand side)$.
All that remains is for each block in the longest PoR chain to calculate objective μ values of the same type using eq. /spanspan class="textup"(/spana href="whitepaper.html#lateximage-whitepaper-63"29/span/a)/span/span.
Earlier, we determined that objective μ values of the same type can accommodate variable difficulties. Thus, we have resolved both parts of the problem.

NIPoPoWRs: Summary We have sketched the explanatory framework and construction of a variant of NIPoPoWs that is compatible with the longest PoR chain: NIPoPoWRs.
NIPoPoWs were modified in the following ways: /spanul style="list-style-type:none"

/spanli
1. Convert (via PoR) each chain's native μ values into a common type for the interlink structure.
/span/li
/spanli
2. Use objective μ values to accommodate variable difficulties.
/span/li
/spanli
3. Replace the tree-interlink structure with a graph-interlink structure.
/span/li
/span/ul
The /spana href="whitepaper.html#lateximage-whitepaper-63"Axiom of Availability/span/ameans that the PoR graph does consensus on which blocks exist. The /spana href="whitepaper.html#lateximage-whitepaper-63"Axiom of Maximal Reflection/span/ameans that the interlink structure spans the entire PoR graph. Thus, any path to a block is a valid path and we don't need to introduce the notion of */spanemgoodness/span/em*.
We should also note that this construction does not (yet) work for block-DAGs (including GHOST). We will continue this construction in section /spana href="whitepaper.html#lateximage-whitepaper-63"4.8.8/span/a.

/spandiv role="note" class="footnotes"
/spanp
/spansup64/span/sup&nbsp;The /spankbdfollowDown/span/kbd algorithm produces the necessary blocks to connect a superblock to a proceeding regular block. It is defined in Section 5.1 and Algorithm 6 of /spana href="https://web.archive.org/web/20220329012237/https://cloudflare-ipfs.com/ipfs/QmYDpsHgs8wjX4qedMUDzanErehZuUcpV8fp2dBFbr9c5r" target="_blank" Non-Interactive Proofs of Proof-of-Work/span/a (Kiayias, Miller, and Zindros; 2018)
/span/p

/spanp
/spansup65/span/sup&nbsp; GHOST and block-DAGs are the specific focus of section /spana href="whitepaper.html#lateximage-whitepaper-63"4.8/span/a, and we'll discuss this issue properly, then.
/span/p

/spanp
/spansup66/span/sup&nbsp; The difficulty is the expected number of hashes to find a solution, so a difficulty of 0 (or below) is impossible.
/span/p

/span/div
/spandiv
class="tcolorbox"
style=" border: 1px solid #404040; background: #ECECEC; "

/spandiv
class="tcolorboxupper"
style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbAside/span/b**/span/span Although we are considering a graph-interlink structure, we should note an obvious optimization. First, let's notice that: when introducing multiple back-links to blocks of equal μ level, those new blocks */spanemmust/span/em* be a subset of the PoR tips recorded in the header (since there might be other tips that are of a lesser μ level). Thus, we don't actually need to include both/all in the interlink structure, since we always have access to both/all paths via the PoR tips in the header, which is already part of the NIPoPoWR. /span/div /span/div

<!--...............................-->
/spanh5 id="autosec-275"/spanspan class="sectionnumber"4.7&#x2003;/span/spanConfirmation Times/span/h5

A confirmation is a */spanemdiscrete/span/em* event that occurs when a block is produced. When an attacker is performing a hash-rate based doublespend attack, they are, effectively, racing the honest network; that race is measured in confirmations, not */spanemtime/span/em*.

/spandiv
    class="tcolorbox"
    style=" border: 1px solid #404040; background: #ECECEC; "

/spandiv
    class="tcolorboxupper"
    style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbQuote/span/b**/span/span /spanspan role="note" class="marginpar"**/spanbQuote/span/b**/span/span The probability of success [of a double-spend attempt] depends on the number of blocks [by which the honest network has an advantage], and not on the time constant T_0.
.7ex]0.5pt

/spandiv class="flushright" /spani— Meni Rosenfeld; /spana href="https://web.archive.org/web/20220209100515/https://cloudflare-ipfs.com/ipfs/QmNUWmY94QUievK8ptoxsPyAQUsKvx1cjRyCgPcfmysAVv" target="_blank" Analysis of hashrate-based double-spending (Meni Rosenfeld; 2012)/span/a/span/i /span/div /span/div /span/div

In a traditional blockchain (e.g., Bitcoin, Ethereum Classic) confirmations occur, on average, at a predictable rate (that of the target block production frequency). Thus, for any /spanemparticular/span/em traditional blockchain, a convenient time-based /spanemrule of thumb/span/em can be devised, e.g., a Bitcoin transaction is safe to accept after 1 hour. However, this approximation only works because blocks (and thus confirmations) are only produced locally (to that blockchain) and at a probabilistic (roughly constant) rate. Put another way, the frequency of confirmations is identical to the frequency of blocks, B$_f Hz.SinceO(B_f) = O(1), thetime - complexityofconfirmationinthesenetworksisalsoO(1).$

When using PoR, though, the assumptions behind that /spanemrule of thumb/span/em do not hold — while blocks on a single chain may be produced at a constant rate, that chain also gains a security benefit from other chains. For the case of a 2-chain simplex (where those chains have the same block production frequency), the rate of confirmations will be twice the rate of block production. This is easily generalized: for an N$_1 - simplexwithsimplex - chainsthatsharesomeblockfrequencyB_f, therateofconfirmationwillbe\mathbb{C}' = N_1 \cdot B_f Hz.Thus, therateofconfirmationshascomplexityO(\mathbb{C}') = O(N_1 \cdot B_f) = O(N_1) = O(c).$

Let /spanemconfirmation time/span/em be the duration breakpoint beyond which enough confirmations have occurred to consider a transaction /spanemsafe/span/em. This is equivalent to the /spanemrule of thumb/span/em mentioned earlier. For a traditional blockchain, as mentioned, this is the product of some constant and the expected duration between blocks: B$_f^{-1}.Forasimplex, though, theexpected/spanemduration/span/emis\mathbb{C}'^{-1} \propto \frac{1}{N_1 \cdot B_f}.Thus, asthesimplexgrows --- asN_1/spanemincreases/span/em --- theentirenetwork'srateofconfirmationsalsoincreases, andthus/spanemconfirmationtime/span/emapproaches0/spansup67/span/sup.$

A 200-simplex with B$_f = /spanspanclass = "textrm\ textmd\ textup\ textsc"/spanspanclass = "numerator"1/span/span//spanspanclass = "denominator"15/span/span/span/spanhasaconfirmationrateof\mathbb{C}' = /spanspanclass = "textrm\ textmd\ textup\ textsc"/spanspanclass = "numerator"40/span/span//spanspanclass = "denominator"3/span/span/span/span \approx 13.3Hz.An800 - simplexwithB_f = /spanspanclass = "textrm\ textmd\ textup\ textsc"/spanspanclass = "numerator"1/span/span//spanspanclass = "denominator"60/span/span/span/spanhasthesameconfirmationrate.A1400 - simplex(themostoptimizedmaximalsimplexgiven/spanemAmaroo's/span/eminitialconfiguration)withB_f = /spanspanclass = "textrm\ textmd\ textup\ textsc"/spanspanclass = "numerator"1/span/span//spanspanclass = "denominator"15/span/span/span/spanhas\mathbb{C}' \approx 93Hz --- \sim 46.5 \times fasterthanEOS/Solana, \sim 1116 \times fasterthanEth2, \sim 1400 \times fasterthanEth1, and \sim 55,800 \times fasterthanBitcoin.$

Note that PoR incents miners to publish blocks as soon as possible so that those blocks begin gaining reflections. If a miner does not publish a block immediately, then the reflections in that block become out-of-date very quickly as there are new, additional headers to reflect arriving constantly. Additionally, any competing block (published immediately by an honest miner) will begin acquiring reflections earlier, and contains more valuable reflected headers (incenting other miners to subsequently reflect it). So the published block has two distinct advantages over the withheld block. This mitigates the selfish mining/spansup68/span/sup attack.

/spandiv role="note" class="footnotes"
/spanp

/spansup67/span/sup&nbsp;To say that confirmation time approaches 0 only tells the latter half of the process by which a transaction becomes confirmed. The first half of that process is */spanemgetting an initial confirmation/span/em*, which is effectively a small, but constant, overhead.
/span/p

/spanp
/spansup68/span/sup&nbsp;See /spana href="https://web.archive.org/web/20240927042356/https://gateway.pinata.cloud/ipfs/QmNukb1L8BhEsiCbrmnkEJWAvUjhBHidinKMZKfCaLG6ep" target="_blank" Majority is not Enough: Bitcoin Mining is Vulnerable/span/a by Ittay Eyal and Emin Gu&#x0308;n Sirer.
/span/p

/span/div

<!--...............................-->
/spanh5 id="autosec-278"/spanspan class="sectionnumber"4.8&#x2003;/span/spanDoS and DAGs/span/h5

Up to this point, simplex-chains have been treated like traditional blockchains, where each block has only one parent. Since the vast majority of a simplex-chain's security is provided by other simplex-chains (and only a small proportion comes from that chain's foundational consensus method), are attacks like an empty-block Denial of Service/spansup69/span/sup (DoS) possible? If a simplex-chain were to use PoW, then it might be (relatively) trivial for an attacker to perform such an attack. This is because — in traditional blockchains — controlling more than 50% of the blocks produced provides */spanemexclusive/span/em* control over */spanemwhich candidate child blocks win/span/em* (i.e., are accepted into the canonical chain)./spansup70/span/sup Is there a way that we can mitigate this risk? If blocks were permitted */spanemmore/span/em* than a single parent, can this */spanemexclusivity/span/em* be denied?
%

/spandiv role="note" class="footnotes"
/spanp
/spansup69/span/sup&nbsp; For an example of this attack, see /spana href="https://bitcointalk.org/index.php?topic=56675.msg678006\#msg678006" target="_blank" Luke Jr's attack on Coiledcoin/span/a.
/span/p

/spanp
/spansup70/span/sup&nbsp; Exclusive control of this nature also allows for protocol changes via soft-forks. Additionally, because these soft-forks can be undetectable (when done well) and don't necessarily affect the income of a miner substantially, bribery (of miners) is theoretically inexpensive. WRT UT, this problem is resolved in section /spana href="whitepaper.html#lateximage-whitepaper-63"4.8.3/span/a.
/span/p

/span/div

<!--...............................-->
/spanh6 id="autosec-280"/spanspan class="sectionnumber"4.8.1&#x2003;/span/spanBlock-DAG Lineage/span/h6

The idea that blocks in a chain can have multiple parents — i.e., the chain forms a Directed Acyclic Graph (DAG) that is not also a tree — dates back to (at least) late 2013 with the publication of GHOST/spansup71/span/sup by Sompolinsky and Zohar. However, GHOST disallows multiple */spanemcanonical/span/em* parents, and a chain using GHOST defines its */spanemcanonical history/span/em* — the */spanemmain chain/span/em* — via the chain formed exclusively from the first parent of each block. A block's other, non-canonical parents are linked to */spanemonly/span/em* for the purpose of contributing to the total chain-weight.

In the two years after GHOST was published, a number of DAG-based blockchain designs were developed that facilitated merging histories from multiple parent blocks.

In mid-2014 I created a prototype DAG-based blockchain called Quanta/spansup72/span/sup with a novel method of linearization that converged to a complete and stable ordering of blocks. This method was independently rediscovered/spansup73/span/sup in mid-2015 by Lewenberg, Sompolinsky, and Zohar/spansup74/span/sup — who also further developed and analyzed the method, which they named */spanemthe inclusive protocol/span/em*. Additionally, in 2016, Paul Firth further developed Quanta in his */spanemTrustless Eventual Total Order/span/em* draft./spansup75/span/sup

In late-2015 several new alternative methods were also published, however, these are not generalizations of Nakamoto consensus. Namely: Lerner's DagCoin/spansup76/span/sup, and IOTA's Tangle/spansup77/span/sup. Since then, multiple other models have been proposed, and some built.

For the purposes of this paper, we are concerned with Quanta / the Inclusive Protocol (which is detailed in /spana href="https://web.archive.org/web/20210426004857/https://cloudflare-ipfs.com/ipfs/QmPb3oZBwyg1EJCR2CivnjTKWkf9UxhVbU8JByv6SW1pXy" target="_blank" Inclusive Block Chain Protocols (Lewenberg, Sompolinsky, Zohar; 2015)/span/a).

%

/spandiv role="note" class="footnotes"
/spanp
/spansup71/span/sup&nbsp; /spana href="https://web.archive.org/web/20220308084727/https://cloudflare-ipfs.com/ipfs/QmTDz4WuAXt2rV7Et3pHHKTFCYGPeDbDoAkmypkHdJnnKe" target="_blank" Secure High-Rate Transaction Processing in Bitcoin (Sompolinsky, Zohar; 2013)/span/a
/span/p

/spanp
/spansup72/span/sup&nbsp; /spana href="https://github.com/XertroV/quanta-test/blob/ba598d5fe89d3b16db07533957a2080edb19a9cd/quanta.py\#L157" target="_blank" Quanta source code/span/a, /spana href="https://bitcointalk.org/index.php?topic=1057342" target="_blank" Quanta BitcoinTalk thread/span/a.
/span/p

/spanp
/spansup73/span/sup&nbsp; As far as I can tell, the linearization methods produce identical results.
/span/p

/spanp
/spansup74/span/sup&nbsp;/spana href="https://web.archive.org/web/20210426004857/https://cloudflare-ipfs.com/ipfs/QmPb3oZBwyg1EJCR2CivnjTKWkf9UxhVbU8JByv6SW1pXy" target="_blank" Inclusive Block Chain Protocols (Lewenberg, Sompolinsky, Zohar; 2015)/span/a
/span/p

/spanp
/spansup75/span/sup&nbsp;/spana href="https://github.com/wildbunny/docs/blob/master/T.E.T.O-draft.pdf" target="_blank" T.E.T.O Draft (Paul Firth; 2016)/span/a
/span/p

/spanp
/spansup76/span/sup&nbsp; /spana href="https://web.archive.org/web/20240825150407/https://bitslog.com/wp-content/uploads/2015/09/dagcoin-v41.pdf" target="_blank" DagCoin Draft (Demian Lerner; 2015)/span/a
/span/p

/spanp
/spansup77/span/sup&nbsp; /spana href="https://web.archive.org/web/20250126094356/https://bitcointalk.org/index.php?topic=1216479.0" target="_blank" IOTA BitcoinTalk Thread (Come-From-Beyond; 2015)/span/a
/span/p

/span/div

<!--...............................-->
/spanh6 id="autosec-282"/spanspan class="sectionnumber"4.8.2&#x2003;/span/spanBasic Structure/span/h6

There are decisive advantages to using DAGs (instead of trees) as the fundamental structure of a chain. Namely, multiple histories (both compatible and incompatible) can be merged into a single, consistent history — a feature which eliminates stale blocks and thwarts attacks like an empty-block DoS. UT's simplex-chains must be block-DAGs to remain functional and avoid such DoS attacks.
Often, the motivation for using a block-DAG — instead of a block-tree — is to increase the block frequency. Since block-DAGs can reference multiple parent blocks, the stale-rate can theoretically approach (or reach) 0. In UT, increasing the block frequency eventually becomes counter-productive, though, since UT is sensitive to the size and number of headers that are produced (see section /spana href="whitepaper.html#lateximage-whitepaper-63"5.9/span/a). In UT, the purpose of using block-DAGs is to thwart certain attacks, not to increase the block frequency. The intention is for UT simplex-chains to use fairly typical block frequencies (e.g., 1 block per 15 s) — possibly decreasing those frequencies over time to increase capacity. Slower block frequencies also decrease the incidence of multiple parents — this matters because each parent typically increases the header size by 32 bytes.
Some basic block-dag segments are shown in fig. /spana href="whitepaper.html#lateximage-whitepaper-63"22/span/a.
/spanfigure id="autoid-30" class="figure "

```
/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "

/spana href="./includes/ut/diags/dag_simple_sag.png" target="_blank" /spanimg
    src="./includes/ut/diags/dag_simple_sag.png"
    style="
    height:128pt ;
    "
    class="inlineimage"
    alt="(image)"
/span/a

/spandiv class="figurecaption"(a) A simple 2-parent example of a block-dag segment.
/span/div
/span/div
```

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "

/spana href="./includes/ut/diags/dag__simple2__sag.png" target="__blank" /spanimg
src="./includes/ut/diags/dag__simple2__sag.png"
style="
height:128pt ;
"
class="inlineimage"
alt="(image)"
/span/a

/spandiv class="figurecaption"(b) A slightly more complex example of a block-dag segment.
/span/div
/span/div

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "

/spana href="./includes/ut/diags/dag__simple3__sag.png" target="__blank" /spanimg
src="./includes/ut/diags/dag__simple3__sag.png"
style="
height:128pt ;
"
class="inlineimage"
alt="(image)"
/span/a

/spandiv class="figurecaption"(c) A 3-parent example of a block-dag segment.
/span/div
/span/div

/spandiv class="figurecaption"Figure 22: Some examples of simple block-dag segments.
/span/div /span/figure

The essence of DAG-based consensus (at least the kind we're concerned with) is to */spanemprioritize execution/span/em* of blocks and transactions based on the */spanemsecurity contribution/span/em* (i.e., weight) of each parent block (and that parent's ancestry). Based on a */spanemmost recent common primary/spansup*78*/span/supancestor/span/em*, we can decide which parent's */spanemhistory/span/em* has priority execution. Prioritized blocks are positioned */spanemearlier/span/em* in the final ordering.

When a block has more than one parent, the prioritized parent is the */spanemprimary/span/em* one. The chain of primary parents forms the */spanemmain chain/span/em*.

/spandiv role="note" class="footnotes"
/spanp
/spansup78/span/sup&nbsp; In DAG (or DAG-like) chains, primary ancestors form the */spanemmain chain/span/em*, a.k.a. the */spanempivot chain/span/em*. Provided that parent blocks */spanemare prioritized by cumulative work/span/em*, the chain of */spanemprimary ancestors/span/em* between a given block and the genesis block must be the */spanemheaviest path/span/em* between the two.
/span/p

/span/div
/spandiv
    class="tcolorbox"
    style=" border: 1px solid #404040; background: #F2F2F2; "

/spandiv
    class="tcolorboxupper"
    style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbTerm/span/b**/span/span **/spanb/spana href="whitepaper.html#??"Main Chain/span/a/span/b**: In a block-DAG, the */spanemmain chain/span/em* is the continuous chain of primary parents from the best block to the genesis block. Blocks that are part of the main chain are */spanemon-main/span/em*, and blocks that are not are */spanemoff-main/span/em*. /span/div /span/div
Let's limit block-DAG blocks to two parents. If the best block has two parents, then each parent will have a */spanemsubgraph of blocks/span/em* between itself and the */spanemmost recent common primary ancestor/span/em* of the two parents. The subgraph which takes priority is that of the */spanemprioritized parent's ancestry/span/em*. If that subgraph is a chain, then the ordering and execution of blocks is trivial. If it is not, then there must be another subgraph within that subgraph, and this algorithm is applied recursively to the prioritized parent. After the prioritized subgraph is processed, the remaining blocks (those that are only ancestors of the secondary parent block, if it exists) are ordered and applied — invoking recursion where necessary. Finally, the best block is applied. In this way, all blocks are executed after their ancestors, and there is a clear and total ordering that trivially converges.
/spanfigure id="autoid-31" class="figure "

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "

/spana href="dag_example1_sag" target="_blank" /spanimg
src="dag_example1_sag"
style="
width:130pt;
"
class="inlineimage"
alt="(image)"
/span/a

/spandiv class="figurecaption"(a) How should we order this block-DAG? Note: children should */spanemalways/span/em* be after their parents, and */spanemprioritization/span/em* means */spanemearlier execution/span/em*.
/span/div
/span/div

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "

/spana href="dag_example1_expanded_order_00_sag" target="_blank" /spanimg
    src="dag_example1_expanded_order_00_sag"
    style="
    width:130pt;
    "
    class="inlineimage"
    alt="(image)"
/span/a

/spandiv class="figurecaption"(b) The first thing we should do is create any virtual nodes that are required ($V_{i+2,1}$).
/span/div
/span/div

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "

/spana href="dag__example1__expanded__order__10__sag" target="__blank" /spanimg
    src="dag__example1__expanded__order__10__sag"
    style="
    width:130pt;
    "
    class="inlineimage"
    alt="(image)"
/span/a

/spandiv class="figurecaption"(c) Since there are two parents, we look at the */spanemprioritized subgraph/span/em* (i.e., most worked).
/span/div
/span/div

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "

/spana href="dag__example1__expanded__order__20__sag" target="__blank" /spanimg
    src="dag__example1__expanded__order__20__sag"
    style="
    width:130pt;
    "
    class="inlineimage"
    alt="(image)"
/span/a

/spandiv class="figurecaption"(d) Again, there are two parents, so we look at the */spanemnext/span/em* prioritized subgraph.
/span/div
/span/div

```
/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "

/spana href="dag__example1__expanded__order__30__sag" target="__blank" /spanimg
     src="dag__example1__expanded__order__30__sag"
     style="
     width:130pt;
     "
     class="inlineimage"
     alt="(image)"
/span/a

/spandiv class="figurecaption"(e) We've found a chain. These blocks have the highest priority, so are executed first.
/span/div
/span/div
```

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "

/spana href="dag__example1__expanded__order__40__sag" target="__blank" /spanimg
src="dag__example1__expanded__order__40__sag"
style="
width:130pt;
"
class="inlineimage"
alt="(image)"
/span/a

/spandiv class="figurecaption"(f) We're now */spanemordering/span/em* the blocks — the solid arrows represent the final ordering. In this step we order the highest priority blocks.
/span/div
/span/div

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "

/spana href="dag_example1_expanded_order_50_sag" target="_blank" /spanimg
    src="dag_example1_expanded_order_50_sag"
    style="
    width:130pt;
    "
    class="inlineimage"
    alt="(image)"
/span/a

/spandiv class="figurecaption"(g) Now that the highest priority blocks are ordered, we can order the */spanemprevious/span/em* subgraph.
/span/div
/span/div

```
/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "

/spana href="dag_example1_expanded_order_60_sag" target="_blank" /spanimg
    src="dag_example1_expanded_order_60_sag"
    style="
    width:130pt;
    "
    class="inlineimage"
    alt="(image)"
/span/a

/spandiv class="figurecaption"(h) Once more, we order the next-in-line subgraph.
/span/div
/span/div
```

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "

/spana href="dag_example1_expanded_order_70_sag" target="_blank" /spanimg
    src="dag_example1_expanded_order_70_sag"
    style="
    width:130pt;
    "
    class="inlineimage"
    alt="(image)"
/span/a

/spandiv class="figurecaption"(i) And finally we order the last remaining blocks. (We could remove virtual blocks too).
/span/div
/span/div

/spandiv class="figurecaption"Figure 23: Example: sorting a moderately complex block-DAG; note that the left parent is always the best parent, so will have priority. Each block is annotated with its */spanemchain-weight/span/em* (Σ_w).
/span/div /span/figure

There is a simple generalization of the above to allow more than two parent blocks, too. That is: replace */spanemall/span/em* but the last (worst) parent with a */spanemvirtual parent block/span/em* that links back to all remaining */spanemactual/span/em* parent blocks (but contributes zero block-weight itself). Replacing the best parents (rather than the worst parents) with a virtual block means that the fork rule works when selecting the virtual block over the least-prioritized parent. This can be repeated to allow for arbitrarily many parents.

fig. /spana href="whitepaper.html#lateximage-whitepaper-63"23/span/a is an example of this algorithm for a moderately complex chain-segment ($B_i \cdots B_{i+3} whichis7blockstotal), andeachstepisenumeratedandexplained.$

We should expect conflicting transactions (which might otherwise be attempted doublespends) to arise during this process. Ancestors of one parent may not be ancestors of another parent. The exact protocol for handling conflicts is up to the implementation, but there is no reason that secondary parents should be treated as invalid by default. We will discuss merging histories in section /spana href="whitepaper.html#lateximage-whitepaper-63"4.8.5/span/a.

<!--................................-->
/spanh6 id="autosec-296"/spanspan class="sectionnumber"4.8.3&#x2003;/span/spanBlock-DAGs Prevent DoS Attacks/span/h6

Consider the situation where an attacker is attempting to deny service via the production of empty blocks, and that the attacker can create blocks faster than the honest network. Such a situation is illustrated in fig. /spana href="whitepaper.html#lateximage-whitepaper-63"24/span/a. Since the goal of the attack is to prevent transactions from occurring, the attacker must/spansup79/span/sup produce empty/spansup80/span/sup blocks. Furthermore, if the attacker links back to any honest blocks then the honest blocks' histories will be merged with the canonical history; thus the attack would fail. The attacker's only available strategy is to produce a single chain of empty blocks.
/spanfigure id="autoid-32" class="figure "

/spandiv class="center"
/spana href="dag_dos_v2_sag" target="_blank" /spanimg
  src="dag_dos_v2_sag"
  style="
  width:434pt;
  "
  class="inlineimage"
  alt="(image)"
/span/a

/spandiv class="figurecaption"Figure 24: An attempted empty-block DoS attack on a block-DAG. The attacker's blocks, A$_{i+x}, areminedinpublicandcontainnotransactions.Eachblockisannotatedwithits/spanemchain-weight/span/em(\Sigma_w); adoubleoutlineindicatesthatablockispartofthemainchain.Eventhoughtheattackerproduces2\times asmanyblocksasthehonestnetwork, theattackinevitablyfailsafterashortwhile.Note: H_i isdefinedtohave\Sigma_w = 0 forillustrativeconvenience.$
/span/div /span/div/span/figure

The challenge of such a DoS attack is to prevent honest miners from extending the attacker's chain-segment. For traditional (non-DAG) chains — where each non-genesis block has exactly one parent — this is accomplished as soon as the attacker is able to reliably produce a heavier chain-segment than the honest network for a given period./spansup81/span/sup

However, if blocks are allowed to have */spanemmore/span/em* than one parent then there */spanemis no point/span/em* where an attacker can */spanemmaintain/span/em* a DoS attack indefinitely. Instead, they can only */spanemdelay the execution/span/em* of some transactions for a short period of time. Particularly, if an attacker can produce A$_{\text{blocks}} > 1 forevery1blockproducedbythehonestnetwork, thentheattackcandelayhonesttransactionsforuptoapproximately(A_{\text{blocks}}{}^2 - 1) \cdot B_f{}^{-1} seconds, whereB_f isthefrequencyofblockproduction(inHz).Afterthis(approximate)point, theweightofthehonestchain-segment, whichincludestheattacker'schain-segment, isalwaysgreatest.$

If an attacker performs a */spanemrepeating cycle/span/em* of these attacks, then it may be able to decrease the effective capacity of the chain by a factor of $\sim$A$^2_{\text{blocks}}.Theopportunitycostofthisattack, fortheattacker, isatleastasmuchasthelosttransactionfees.$

We'll discuss DoS attacks again, shortly, in section /spana href="whitepaper.html#lateximage-whitepaper-63"4.8.7/span/a.

%

/spandiv role="note" class="footnotes"
/spanp
/spansup79/span/sup&nbsp; The attacker could also fill the blocks with spam transactions. That's more work for the attacker, but also more work for the honest network (calculating and storing that state, maybe indefinitely). It's preferable that the attacker has minimal transactions in their blocks. It's tempting to think of ideas like: */spanemsince the attacker's blocks are empty, we can let honest nodes make larger blocks via some kind of weighted average block size calculation plus some flexibility in the size of blocks produced/span/em*. The problem with this is that it incents the attacker to fill their blocks with spam transactions, which is counterproductive.
/span/p

/spanp
/spansup80/span/sup&nbsp; Note that the attacker should still be reflecting other simplex-chains so as to maximize the total weight of the attacker's chain-segment. Given this, the attacker's blocks will contain reflected simplex-chain headers but no transactions.
/span/p

/spanp
/spansup81/span/sup&nbsp; For a traditional blockchain, this would also imply a 51% attack were possible, so this isn't a situation that those chains have to deal with. In a simplex, however, a miner can have >50% of a chain's local hash-rate, so we need to handle this case.
/span/p

/span/div

<!--...............................-->
/spanh6 id="autosec-299"/spanspan class="sectionnumber"4.8.4&#x2003;/span/spanA Criticism of GHOST/span/h6

GHOST allows for blocks to link to a single canonical parent and multiple */spanemuncle/span/em* blocks. In the full GHOST algorithm, uncle blocks contribute */spanemweight/span/em* to the canonical chain-segment, but do not contribute */spanemtransactions/span/em*. Thus, uncles have */spanemno ability/span/em* to substantially contribute to the canonical chain's */spanemstate/span/em*.

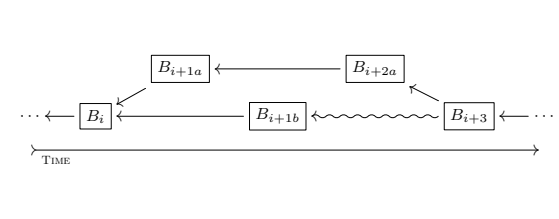
Consider an empty-block DoS against a chain using GHOST. If an attacker were to perform an empty-block DoS, the attacker could link back to honest miners' blocks as uncles, but never parents. Given this, there is no easy way for the honest miners to end or mitigate the DoS. The attacker can include honest miners' chain-work in a purely */spanembeneficial/span/em* way — there is a symmetry, thus honest miners (and the network) are at the mercy of the attacker.

Why does this symmetry exist? Because the */spanemcumulative weight/span/em* of each block (including uncles) is */spanemdivorced/span/em* from */spanemthe set of transactions/span/em* that is contributed by that block. However, with a full block-DAG, when an attacker links to uncles in this way, */spanemthey must allow for the execution of all non-conflicting transactions/span/em* (i.e., those which would not cause a doublespend to occur). Thus, GHOST */spanemdoes not mitigate/span/em* empty-block DoS attacks; */spanemonly/span/em* a full block-DAG can do that.

<!--................................-->
/spanh6 id="autosec-300"/spanspan class="sectionnumber"4.8.5&#x2003;/span/spanMerging Histories/span/h6

If a block B_{i+2} has two valid parents, how can we resolve this into one consistent history?
Consider the following subgraph, where squiggly arrows indicate secondary parents: fig. /spana href="whitepaper.html#lateximage-whitepaper-63"25/span/a.
/spanfigure id="autoid-33" class="figure "

/spandiv class="center"
/spanspan class="hidden" \(\seteqnumber{0}{}{29}\)/span/span <!--



-->
\begin{equation*}
\xymChains { &amp; &amp; &amp; \xBB {1a} \arm [ld] &amp; &amp; \xBB {2a} \arm [ll] \\ &amp; \cdots &amp; \xBBi \arm [l] &amp; &amp; \xBB {1b} \arm [ll] &amp; &amp; \xBB 3 \arm [lu] \ars [ll] &amp; \cdots \ar [l] \\ &amp; \xyTimeHoriz {rrrrrr} &amp;&amp;&amp;&amp;&amp;&amp; }
\end{equation*}

/spandiv class="figurecaption"Figure 25: A simple block subgraph. Standard arrows indicate primary parents, and squiggly arrows indicate secondary parents.
/span/div /span/div/span/figure

By the ordering algorithm we covered above, we know that the blocks will executed in this order: B_i, B_{i+1a}, B_{i+2a}, B_{i+1b}, B_{i+3}.

Typically, a blockchain's state transition function can (at least partially) be verified via the pattern in algorithm /spana href="whitepaper.html#lateximage-whitepaper-63"3/span/a. We first load the parent's state, apply each transaction in sequence, and then calculate the new state root. If this matches the state root recorded in the block header, then the state transition is valid (all else being equal). This works fine for some blocks that have only one parent (e.g., B_{i+1a}, B_{i+2a}).
/spanfigure id="autoid-34" class="algorithm ruled"

/spandiv class="figurecaption"Algorithm 3: A basic state transition verification function
/span/div
Omitted due to stack size. /span/figure

We /spanemcan/span/em use this algorithm /spanemfrom B_{i+1b}'s perspective/span/em to ensure B_{i+1b} is internally consistent; however, we /spanemcannot/span/em use it with the above subgraph.

Why? Because B_{i+1b}'s parent is B_i, but the block immediately executed prior to B_{i+1b} was B_{i+2a}. Thus, the algorithm doesn't work for B_{i+1b}: we need to use a different prior state, and therefore the /spanemresulting/span/em state root /spanemwill not match/span/em the state root in B_{i+1b}'s header.

Moreover, the algorithm doesn't exactly work for B_{i+3}, either. In B_{i+3}'s case, its primary parent is B_{i+2a}, but it's state root should be the result of applying B_{i+1b}'s transactions to B_{i+2a}'s output state, and then applying B_{i+3}'s transactions to that mid-state.

It is clear that the algorithm breaks for B_{i+3} because it has more than one parent.

The reason that it breaks for B_{i+1b} is more subtle: B_{i+1b} is not part of the /spanemmain chain/span/em.

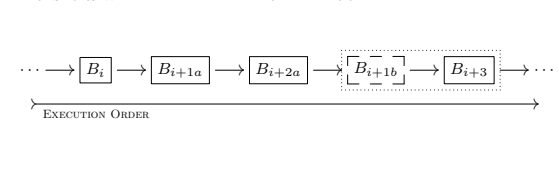
Observe that the pattern works for on-main blocks with a single parent, but not for off-main blocks, nor for on-main blocks with multiple parents.

The solution is to combine new off-main ancestors (B_{i+1b}) with the relevant on-main descendant (B_{i+3}) and process them as a single operation. In principle, we're collapsing the full block-DAG down to just the main chain, so that traditional blockchains' state-transition logic works for block-DAGs, too.

This has two important and related implications. First: transactions in off-main blocks may not be valid at execution time, because the previous state root changes. Second: only the state roots of on-main blocks are accurate and reliable.

We should be careful to verify proofs against /spanemonly/span/em on-main blocks — methods like SPV /spanemare not safe/span/em against off-main blocks.

We can now describe the execution in terms of the main chain by grouping novel off-main ancestors with the next main chain block:
/spanspan class="hidden" \(\seteqnumber{0}{}{29}\)/span/span <!--



-->
\begin{equation*}
\xymChains { &amp; \cdots \arm [r] &amp; \xBBi \arm [r] &amp; \xBB {1a} \arm [r] &amp; \xBB {2a} \arm [r] &amp; \xyBDraft {\BB {1b}} \arm [r] \blkGrp 1 &amp; \xBB 3 \ar [r] &amp; \cdots \\ &amp; \xyLabelHoriz {Execution Order}{0pt}{rrrrrr}{r30pt} &amp;&amp;&amp;&amp;&amp;&amp; }
\end{equation*}

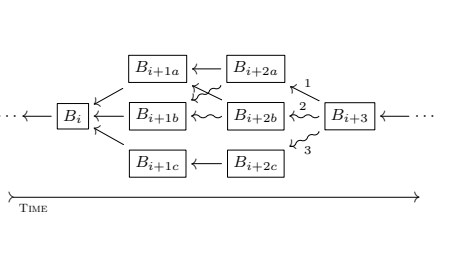
The dotted boundary around (B_{i+1b}
$\to B_{i+3})indicatesthattheblocksareprocessedasagroup.ThedashedboundaryaroundB_{i+1b}indicatesthatitis,inasense,/spanemvirtual/span/emwithinthestate-transitionfunction.B_{i+1b}ismorelikeapartofB_{i+3}thanablockinitsownright.$

See fig. /spana href="whitepaper.html#lateximage-whitepaper-63"26/span/a for an addition example.

If we want to support stateless full nodes (and fraud proofs), this grouping strategy adds a constraint to the block structure. Particularly, the complete witness structure of B_{i+3} is not simply the witnesses for B_{i+3}'s transactions combined with the witnesses for B_{i+1b}'s. Instead, it is a new witness structure initialized from B_{i+2a}'s state, and must cover /spanemboth/span/em blocks' transactions.

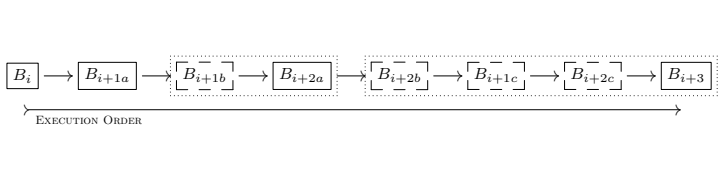
In regards to fraud proofs, the technique that we discussed in section /spana href="whitepaper.html#lateximage-whitepaper-63"4.5/span/a requires additional data to be included in B_{i+3}. Recall that the transaction root should correspond to a merkle tree with leaves alternating between state roots and transactions, with each root being that of the state immediately prior to the transaction's execution. Thus, B_{i+3} must provide intermediate state roots that uniquely correspond to each transaction in each novel off-main ancestor. Furthermore, the transaction root should commit to /spanemall/span/em of these {mid-state root, transaction} pairs, not only those of transactions that were included directly in B_{i+3}.
/spanfigure id="autoid-35" class="figure "

/spandiv class="center"
/spanspan class="hidden" \(\seteqnumber{0}{}{29}\)/span/span <!--



```
-->
\begin{equation*}
\xyMChains { &amp; &amp; &amp; \xBB {1a} \arm [ld] &amp; \xBB {2a} \arm [l] \ars [ld] \\ &amp; \cdots &amp; \xBBi \arm [l] &amp; \xBB {1b} \arm [l] &amp; \xBB {2b} \arm [lu] \ars [l] &amp; \xBB 3 \arm [lu]_1 \ars [l]_2 \ars [ld]^3 &amp;
\cdots \ar [l] \\ &amp; &amp; &amp; \xBB {1c} \arm [lu] &amp; \xBB {2c} \arm [l] \\ &amp; \xyTimeHoriz {rrrrr} &amp;&amp;&amp;&amp;&amp; }
\end{equation*}
```

/spanspan class="hidden" \(\seteqnumber{0}{}{29}\)/span/span <!--



```
-->
\begin{equation*}
\xyMChains { &amp; \xBBi \arm [r] &amp; \xBB {1a} \arm [r] &amp; \xyBDraft {\BB {1b}} \arm [r] \blkGrpM [r]1 &amp; \xBB {2a} \arm [r] &amp; \xyBDraft {\BB {2b}} \arm [r] \blkGrpM [rrr]2 &amp; \xyBDraft {\BB {1c}} \arm [r] &amp; \xyBDraft
{\BB {2c}} \arm [r] &amp; \xBB 3 \\ &amp; \xyLabelHoriz {Execution Order}{0pt}{rrrrrrr}{r30pt} &amp;&amp;&amp;&amp;&amp;&amp;&amp; }
\end{equation*}
```

/spandiv class="figurecaption"Figure 26: A complex block subgraph and corresponding execution order. In the subgraph, standard arrows indicate primary parents, and squiggly arrows indicate secondary parents. In the execution order, dotted boxes indicate groups of blocks that are executed together — when an on-main block has multiple parents, the novel off-chain ancestors (indicated by a dashed outline) are executed as part of the on-main block's execution.
/span/div /span/div/span/figure

<!--................................-->
/spanh6 id="autosec-303"/spanspan class="sectionnumber"4.8.6&#x2003;/span/spanAdapted NIPoPoWs for Block-DAGs: NIPoPPoWs/span/h6

/spandiv
    class="tcolorbox"
    style=" border: 1px solid #404040; background: #F2F2F2; "

/spandiv
    class="tcolorboxupper"
    style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbWarning/span/b**/span/spanReminder: section /spana href="whitepaper.html#lateximage-whitepaper-63"4.8.6/span/a and sections on NIPoPoWs are speculative./span/div /span/div
Recall that in **??**&nbsp;4.6.5.2 we noted that block-DAGs are currently incompatible with NIPoPoWs. This is because the traversal method of NIPoPoWs does not know anything about whether a block is on-main or not. Additionally, the fork rule does not (and should not) prioritize blocks based on μ values. So we should expect that some off-main blocks have a higher μ value than their on-main sibling and/or their first on-main descendant. Therefore, currently, a valid NIPoPoW can lead to an off-main block — i.e., a block that is not useful for SPV.
We do not have a problem traversing via off-main blocks, though. They share the same cryptographic properties (with regards to interlinks) as any other block.
So, our major goal is to find a way to always arrive at on-main blocks. A valid block-DAG NIPoPoW should never terminate at an off-main block.
One problem is that, when a block is created, the network does not know if it will be on-main or not. Future blocks determine whether it is on-main or not.
Furthermore, we cannot be certain that all future blocks will agree that any specific block is on-main or not. We should expect that all far-future blocks will agree, but how long should we wait and is it reliable?
Could we simply modify the interlink structure to add a flag for off-main blocks?
No — this is not sufficient. There is only one way (in principle) to always arrive at an on-main block: via ever more recent on-main blocks. It is only because of an on-main block's */spanemposterity/span/em* that it becomes (and remains) on-main. Simply flagging off-main blocks does not work because, once we reach an off-main μ-superblock, we know nothing about which prior blocks are on-main or not. An off-main block is (on its own) never a reliable source of prior blocks' on-main status.
Here is how we can reliably arrive at on-main blocks: first, starting from all chain-tips, we find the most recent common on-main ancestor. If this is very close to the chain-tips, we can, instead, use an on-main ancestor a few confirmations back — so we can meet any particular security parameter we might choose. We can be confident that this ancestor is on-main, regardless of which chain-tip eventually ends up as the on-main block. We must then follow the chain of primary parents.
Therefore, the interlink structure */spanemmust/span/em* include enough information to traverse the μ-superDAGs via on-main ancestors, only. Just because we */spanemcan/span/em* traverse via on-main blocks only does not mean that we will, though. In fact, we will traverse via two paths: one normally, and the other via on-main blocks only.
To support this, we'll modify the interlink structure such that each link has an optional second component: the most recent on-main ancestor at that μlevel. Most of the time, we should expect blocks to be on-main, so there will only be one interlinked entry at that μlevel. However, in the case that a μ-superblock is off-main, the interlink entry is both that off-main μ-superblock, and the most recent on-main μ-superblock (which is necessarily older than the off-main μ-superblock).
Thus, we effectively have two μ-superDAGs: one being the */spanembest-blocks/span/em* μ-superDAG, and the other being the */spanemon-main only/span/em* μ-superchain. (Each block in a block-DAG has only one primary parent, so a chain-tip's primary ancestry really is a chain, not a DAG.)
We then construct the main suffix proof for two cases: the */spanembest-blocks/span/em* suffix proof (regardless of on-main status), and the */spanemon-main only/span/em* suffix proof. These two proofs will always be consistent, though the best-blocks proof may not include all μ-superblocks that are included in the on-main proof. To account for this, we can require that, for each μlevel, the best-blocks μ-superDAG suffix subgraph is a superset of the corresponding on-main μ-superchain suffix. (That is: we make sure the necessary on-main blocks are included in the proof if they wouldn't be otherwise.) Both together comprise the suffix proof.
Regarding infix proofs, we can use the existing/spansup82/span/sup `/spankbdfollowDown/span/kbd` algorithm, is used to navigate between superblocks and regular blocks, with only one modification: an additional parameter determines whether `/spankbdfollowDown/span/kbd` should traverse the */spanembest-blocks/span/em* path to the block, or the */spanemon-main only/span/em* path. To generate proofs, we run `/spankbdfollowDown/span/kbd` over each path. We similarly modify the `/spankbdverify-infx/span/kbd` (*/spanemsic/span/em*) and `/spankbdancestors/span/kbd` functions used to verify infix proofs.
It is worth noting that the on-main proofs alone are not sufficient for secure NIPoPoWs: in some cases the on-main only proofs will imply substantially less total chain-weight than the best-blocks proofs. For example: if only 50% of blocks were on-main, then the resources an attacker needs to generate a competing NIPoPoW (for the on-main proofs) is only 50% of what they would need compared to a best-blocks proof. Thus, we prioritize one NIPoPoW over another based on the */spanembest-blocks/span/em* proofs (rather than the */spanemon-main/span/em* proofs). We do, however, validate that the target of the NIPoPoW is on-main via the accompanying on-main proof. The on-main path is only inconsistent with the best-blocks path in the case that the target block is off-main./spansup83/span/sup
A major difference between the best-blocks and on-main proofs should not be an issue for UT, though, as UT uses modest block frequencies for each chain. We should expect blocks to have a single parent most of the time. (Using the logic from section /spana href="whitepaper.html#lateximage-whitepaper-63"4.6.1/span/a, we'd expect simultaneous blocks ~2% of the time — give or take.)
As a final note, what happens when there are multiple paths in a μ-superDAG? Note that this situation only occurs in the best-blocks proof. Since the best-blocks proof is not concerned with the posterity of blocks (their on-main status), there is no issue here. We can choose between the paths by whichever means we like — e.g., picking the block with the lowest PoW, or picking on-main blocks over off-main blocks when they are siblings in the same μ-superDAG.
We've just sketched the construction of a new type of NIPoPoW that can prove */spanemboth/span/em* the full chain-weight and the posterity of blocks. The best-blocks half of the proof is almost a regular NIPoPoW on its own, but the other half of the proof (traversing on-main blocks only) is not independently secure. However, */spanemtogether/span/em* they are secure.
We should distinguish between this new kind of NIPoPoW and the regular kind. The main difference is that what we have just sketched proves */spanemwork and posterity/span/em*, rather than just work. Thus, it is a */spanemNon-Interactive Proof of Posteritorial/span/em*/spansup84/span/sup*/spanemProof of Work/span/em* (NIPoPPoW).

/spandiv role="note" class="footnotes"
/spanp
/spansup82/span/sup&nbsp;`/spankbdfollowDown/span/kbd`, `/spankbdverify-infx/span/kbd` and `/spankbdancestors/span/kbd` are functions defined in the /spana href="https://web.archive.org/web/20220329012237/https://cloudflare-ipfs.com/ipfs/QmYDpsHgs8wjX4qedMUDzanErehZuUcpV8fp2dBFbr9c5r" target="_blank" Non-Interactive Proofs of Proof-of-Work/span/a (Kiayias, Miller, and Zindros; 2018) paper.
/span/p

/spanp
/spansup83/span/sup&nbsp; Note: when traversing the best-blocks μ-superDAG, we */spanemdo not care/span/em* about the status of on-main blocks. Thus, the on-main interlinks in off-main μ-superblocks are not important, and we should not count any differences there as inconsistencies with the on-main μ-superchain. Similar to NIPoPoWRs, the best-blocks proof only proves that a block */spanemexists/span/em*, and no conflicts are possible via different paths.
/span/p

/spanp
/spansup84/span/sup&nbsp; A definition is required as this word is an almost-neologism:
/spanbPosteritorial/span/b */spanemadjective/span/em*: having posterity; supported by its descendants, especially regarding well-known descendancy, epistemic antecedence, root-causal nature, etc; having a prototypical and defining quality that was widely inherited.
/spanemAside:/span/em Perhaps curiously, a prior candidate modifier was the latin phrase */spanemper posterās/span/em* — literally 'through what came after'. But also faux-translated as '[as] per post eras', which is roughly equivalent to the English phrase 'via posterity'. A downside of */spanemper posterās/span/em* is that, phonetically, it's very similar to 'preposterous', and Non-Interactive Proofs of Preposterous Proof of Work is not an apt name.
/span/p

/span/div

<!--..............................-->

/spanh6 id="autosec-305"/spanspan class="sectionnumber"4.8.7&#x2003;/span/spanBlock-DAG Interaction With The PoR Graph/span/h6

Up to now we have mostly considered block-DAG's in isolation; outside UT's broader context. Particularly, we should ask: */spanemdo the Axioms of /spana href="whitepaper.html#lateximage-whitepaper-63"Availability/span/aand /spana href="whitepaper.html#lateximage-whitepaper-63"Maximal Reflection/span/achange the game?/span/em*

Recall that, when defining the /spana href="whitepaper.html#lateximage-whitepaper-63"Axiom of Maximal Reflection/span/a, we made sure to recursively include blocks' ancestors, not just reflections. This was for two reasons. First, $L_i$should not explicitly reflect its own ancestors — links to parents */spanemare/span/em* reflections, but are the trivial case where the local and reflecting chains are the same, and the local and reflecting blocks are the same. Second, $L_i$should not skip accounting for R_k's implied reflection of its parent(s), so we treat parents like any other reflected block — we must be sure not to leave any gaps.

/spanspan class="uline"There is still one gap, though/span/span: Lblocks that are not an ancestor of L_i.

It should be clear that parent links are qualitatively different from reflections: child blocks attest to the validity of their parent blocks, and to continuity between these two blocks. The implicit 'PoR' between these blocks is also free, unlike explicit PoRs between blocks of different chains.

Unlike traditional chains, UT blocks are not only allowed to have multiple parents, but */spanemcan always include a valid chain-tip as an ancestor/span/em.* Recall the /spana href="whitepaper.html#lateximage-whitepaper-63"Axiom of Maximal Reflection/span/a's principle: */spanemA miner must reflect all possible blocks./span/em* Considering the symmetry between parents and reflected blocks, we can draft a new network principle: */spanemA miner should build on all possible chain-tips./span/em*

In effect, we want to ensure that a dishonest miner cannot evade the contradiction in refusing to build on a valid chain-tip whilst acknowledging it as available. If it is unavailable then the miner should not reflect the blocks that imply it is available; and if it is available and valid then it should be a parent of their draft block.

So far, this principle is convergent with honest miners building on */spanemvalid/span/em* blocks — including every chain-tip as a parent is what we expect honest miners to do by default. However, what are honest miners to do if there is an invalid block? We need to consider this because a block with a valid PoRs sub-block does not necessarily have a valid transactions sub-block.

We have already discussed the solution in section /spana href="whitepaper.html#lateximage-whitepaper-63"4.5/span/a: link back to, or */spanemflag/span/em*, that block as invalid */spanemwith an attached fraud proof/span/em*, and never use that block as a canonical ancestor. The transactions in that block aren't important — excepting the offending transaction, which is part of the fraud proof. And we can reconstruct the PoRs sub-block using O(1) data thanks to /spana href="whitepaper.html#lateximage-whitepaper-63"Maximal Reflection/span/a. Thus, an invalid block is only as large as its header, list of reflected PoR graph-tips, and fraud proof. That's O$(\log c)$*intheworstcase.*

<!--...............................-->
/spanspan class="paragraph" id="autosec-306"/spanspan class="sectionnumber"4.8.7.1&#x2003;/span/spanAxiom of Unified Ancestry/span/span

This is all that we need to plug the gap: ensure honest miners can always build on */spanemany/span/em* local chain-tip, valid or not. If the chain-tip is valid, the honest miner uses it as a parent. If it is invalid, the honest miner flags it as invalid. It is still an ancestor cryptographically, but not with respect to its transactions and chain-state.

/spandiv
class="tcolorbox"
style=" border: 1px solid #333333; background: #F2F2F2; "

/spandiv
class="tcolorboxtitle"
style=" border-top: 1px solid #333333; border-bottom: 1px solid #333333; background: #333333; color: #FFFFFF; "
Axiom of Unified Ancestry/span/div
/spandiv
class="tcolorboxupper"
style=" color: #000000; "
/spanemPrinciple:/span/em A miner must use all valid chain-tips as parents, and flag invalid ones./spanspan role="note" class="marginpar"**/spanbAxiom/span/b**/span/span

/span/div
/spandiv
class="tcolorboxlower"
style="border-top: 1px dashed #333333; color: #000000; "

/spanemPredicate:/span/em A block, L_i, is invalid unless, for all reflected blocks R_k: /spanul style="list-style-type:none"

/spanli
i. R_k is valid under this axiom; and
/span/li
/spanli
ii. each Lblock reflected by R_k: /spanul style="list-style-type:none"

/spanli
a. is an ancestor of L_i; or
/span/li
/spanli
b. is flagged as invalid by L_i; or
/span/li
/spanli
c. was flagged as invalid by an ancestor of L_i.
/span/li
/span/ul
/span/li
/span/ul
/span/div /span/div
This new axiom changes the game for block-DAGs.

<!--...............................-->
/spanspan class="paragraph" id="autosec-307"/spanspan class="sectionnumber"4.8.7.2&#x2003;/span/spanDoS Attacks/span/span

Previously, an attacker could delay transactions for $\sim A^2_{\text{blocks}}$ manyblocksbecausetheycouldevadethecontradictioninthePoRgraph.Now,theycannot.Thus,emptyblockDoSattacksagainstsimplex − chainblock − DAGsarelimitedto∼A_{blocks}inlength(onaverage)./spansup85/span/supIntermsofDoSpotential,thisis/spanemexactly/span/emthesameasifthesimplex − chainwereatraditionalchainandtheattackeralwaysbuiltonthebestblock,butonlymadeemptyblocks.Nowthatwe′veaddedthe/spanahref = ”whitepaper.html#lateximage − whitepaper − 63”AxiomofUnifiedAncestry/span/a,thealternativefortheattackerisproducingonlyinvalidblocks.

/spandiv role="note" class="footnotes"
/spanp
/spansup85/span/sup&nbsp; We will reduce this further in section /spana href="whitepaper.html#lateximage-whitepaper-63"4.9/span/a.
/span/p

/span/div
/spandiv
    class="tcolorbox"
    style=" border: 1px solid #404040; background: #ECECEC; "

/spandiv
    class="tcolorboxupper"
    style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbAside/span/b**/span/span Note that this axiom applies and is verified in the context of the PoR graph (not chain-state). /span/div /span/div
.7ex]0.5pt/spanspan style="width:3.4444pt; display:inline-block"<!---->/span/spanBegin DRAFT/spanspan style="width:3.4444pt; display:inline-block"<!---->/span/span.7ex]0.5pt

<!--................................-->
/spanspan class="paragraph" id="autosec-309"/spanspan class="sectionnumber"4.8.7.3&#x2003;/span/spanLimits on DAG Links/span/span

/spanul style="list-style-type:none"

/spanli
• inclusive protocol says we need to limit graph width and how far back things are linked.
/span/li
/spanli
• this matters for block rewards (distant off-main blocks should have the rewards reduced – makes coinbase UTXO a bit more complex).
/span/li
/spanli
• can't have a situation where someone makes old blocks for high rewards or something.
/span/li
/spanli
• however, at the simplex-chain level, WRT the max number of parents: this is determined by /spana href="whitepaper.html#lateximage-whitepaper-63"Axiom of Unified Ancestry/span/a.
/span/li
/spanli
• but the PoR graph is also a block-DAG; we could get a fork of the network for say 1hr, and then connectivity is resumed.
/span/li
/spanli
• we need to handle this case: /spanul style="list-style-type:none"

/spanli
– problem: too many blocks at once will cause lag/congestion and slow processing of the block that first reflects the fork (which might be bad for the miner).
/span/li
/spanli
– solution: limit the maximum number of newly reflected blocks that are not recent. e.g., added chain-slices must have maximum path length from tip to PoR graph of 10 blocks or fewer, and maximum of 20 new blocks overall. this means we'll progressively add chunks of the forked branch, limiting the impact.
/span/li
/span/ul
/span/li
/span/ul
.7ex]0.5pt/spanspan style="width:3.4444pt; display:inline-block"<!---->/span/spanEnd DRAFT/spanspan style="width:3.4444pt; display:inline-block"<!---->/span/span.7ex]0.5pt

<!--................................-->
/spanh6 id="autosec-310"/spanspan class="sectionnumber"4.8.8&#x2003;/span/spanNIPoPPoWs + NIPoPoWRs/span/h6

/spandiv
class="tcolorbox"
style=" border: 1px solid #404040; background: #F2F2F2; "

/spandiv
class="tcolorboxupper"
style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbWarning/span/b**/span/spanReminder: section /spana href="whitepaper.html#lateximage-whitepaper-63"4.8.8/span/a and sections on NIPoPoWs are speculative./span/div /span/div

In section /spana href="whitepaper.html#lateximage-whitepaper-63"4.8.6/span/a we devised NIPoPPoWs, modified NIPoPoWs that work with block-DAGs by accounting for posterity. That construction depended on a *best-blocks/span/em* proof in combination with an */spanemon-main only/span/em* proof. In the case of simplex-chain block-DAGs, we do not want to use the same best-blocks proof since it only accounts for */spanemlocal/span/em* chain-work and does not account for reflections.

Instead of a standard best-blocks NIPoPoWs, can we use NIPoPoWRs from **??**&nbsp;4.6.5.2? Functionally, both serve the same purpose: proving that a block */spanemexists/span/em*.

One potential problem is that the NIPoPPoW best-blocks proof substantially overlapped with the on-main only proof — that isn't the case for NIPoPoWRs as */spanemmost/span/em* of the μ-superblocks will be from reflecting chains.

Rather, we expect the best posteritorial μ-superchain to have a μlevel approximately $\log_2(N_1) lowerthanthatofthebestPoR\mu - superDAG(sincethereareN_1 \times asmanyblocks).Inotherwords,only1/N_1localblocksareatthesame\mu levelasthehighest\mu - superDAG.$

Let us take stock for a moment. Each axiom adds a constraint on what the simplex does consensus on, and each block's relationships to other blocks. What are those constraints on consensus created by our current axioms? /spanul
style="list-style-type:none"

/spanli
- /spana href="whitepaper.html#lateximage-whitepaper-63"Availability/span/a: the availability of blocks.

/span/li
/spanli
- /spana href="whitepaper.html#lateximage-whitepaper-63"Maximal Reflection/span/a: the order of blocks, and that there are no 'gaps' in the PoR graph.

/span/li
/spanli
- /spana href="whitepaper.html#lateximage-whitepaper-63"Unified Ancestry/span/a: the ancestry of blocks, and that there are no 'gaps' in each simplex-chain.

/span/li
/span/ul
These are properties that are verifiable */spanemsimplex-wide/span/em* — as part of tracking the PoR graph, each node can reconstruct each simplex-chain's */spanemlocal/span/em* interlink structure./spansup86/span/sup */spanemThus, the PoR graph knows the last on-main block for each simplex-chain./span/em*

So, can we use the same technique as we used before — tracing two proofs? Well, we will need to modify the interlink structure once again, but it seems in-principle possible.

Compared to NIPoPPoWs, we have greatly restricted the possibilities regarding to on-main blocks: /spana href="whitepaper.html#lateximage-whitepaper-63"Unified Ancestry/span/ameans that */spanemboth/span/em* on-main and off-main blocks contain reliable/spansup87/span/sup information regarding posterity. If a block has only one parent, then that parent must be on-main; off-main blocks need to use all available chain-tips as parents. If child blocks have only one parent, then the block is on-main. If the block has multiple parents, then one must be on-main, and we can always find the most recent common primary ancestor of those parents to find an on-main block. If child blocks have multiple parents and share the same primary parent, then we can swap to the on-main sibling whenever needed. If we get stuck, we can repeat any of these processes multiple times to increase our knowledge of that chain-segment — this means we can always determine on-main status via the PoR μ-superDAG. To prove posterity, we just need to show a large enough segment of the block-DAG so that we can, */spanemwith certainty,/span/em* identify which blocks within it are on-main.

From there, determining on-main status of ancestors is trivial.

So we no longer need two proofs, but we do need a bit more information than a stand-alone NIPoPoWR.

Since this method is almost the same as our previous NIPoPoWR method, we'll avoid creating a new acronym and just re-use the term 'NIPoPoWR' to refer to this version, too.

/spandiv role="note" class="footnotes"
/spanp
/spansup86/span/sup&nbsp; It might go without saying, but an improper interlink structure is a reason for a block to be invalid, so bad interlinks will be rejected by the PoR graph.
/span/p

/spanp
/spansup87/span/sup&nbsp; It's not 100% reliable, but it's good enough that we can always find an on-main block without much effort.
/span/p

/span/div
/spandiv
    class="tcolorbox"
    style=" border: 1px solid #404040; background: #ECECEC; "

/spandiv
    class="tcolorboxupper"
    style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbAside/span/b**/span/span Note: there is plenty of excess capacity available to us if the above is insufficient or insecure. We could, for example, use a NIPoPPoW to prove some on-main block, then prove */spanemeach/span/em* high level μ-superblock in the NIPoPPoW via NIPoPoWRs. /span/div /span/div
There is one other fact that we need to prove: that no fraud proof was ever recorded for the target block.

<!--...............................-->
/spanspan class="paragraph" id="autosec-312"/spanspan class="sectionnumber"4.8.8.1&#x2003;/span/spanIntegrating Fraud Proofs/span/span

NIPoPoWs (in general) exist in a unique context: we expect that a lot of time has passed since the target infix block was mined.
If PoR sub-blocks keep track of a set of */spanemall/span/em* fraudulent blocks (and this is enforced), then we can use a much later block (from any chain) to prove whether any fraud proof for any particular block was ever recorded. We don't care what the fraud proof actually was, since the PoR graph already came to consensus on the fact it exists. This can be done via a Merklix tree, or any other merklized structure that allows us to prove non-membership. Therefore, we can use one of the intermediate μ-superblocks to prove that no fraud proof was recorded for the target infix block. Since a long time has passed (at least some security parameter of our choosing), we can be confident that no fraud proof was possible, and thus that the block is valid.
We could also provide additional proofs (of non-membership) for e.g., all blocks used in the complete full NIPoPoWR, if we cared to. Blocks with invalid interlinks cannot be used as superblocks, though, so additional proofs should not be necessary.
This completes the sketch-construction of NIPoPoWRs.

<!--..............................-->
/spanh5 id="autosec-313"/spanspan class="sectionnumber"4.9&#x2003;/span/spanLowering Block Production Variance/span/h5

/spandiv
    class="tcolorbox"
    style=" border: 1px solid #777472; background: #FEE8F4; "

/spandiv
    class="tcolorboxtitle"
    style=" border-top: 1px solid #777472; border-bottom: 1px solid #777472; background: #777472; color: #FFFFFF; "
TODO: [2] **/spanb(LOW)/span/b**/span/div
/spandiv
    class="tcolorboxupper"
    style=" color: #000000; "
redraft 'lowering block prod variance' /span/div /span/div

Is it possible to */spanemdramatically/span/em* lower the variance of block production in PoW blockchains without altering incentive structures, compromising security, or changing the probability of generating a valid block?

Yes. The method relies on the */spanemstructure/span/em* of the network, rather than the consensus protocol itself. Particularly, the network must be structured such that miners' choices result in decreased block production variance — an emergent phenomenon. It's important that it is emergent and not synthetic (e.g., by increasing the block reward with time-since-last-block) because we don't want people to game the system. It's better to have a simple system with emergent properties than a complex system with those properties "designed in".

Say you have a network with 10 chains: $C_0, C_1, C_2, ..., C_9. If the networks are separate, then you have 10 groups of miners:$
$M_0, M_1, M_2, ..., M_9. They have to choose one chain to mine on, so the distribution of miners is expected to be approximately the distribution of normalized block rewards plus tx fees. The proportions of block rewards between C_i \&amp; C_j don't really matter, we expect the mining groups M_i \&amp; M_j to just sort themselves out due to market forces. For simplicity, though, t$

If the network has spare capacity (i.e., transactions are mostly cleared out with each block; the mempool for each chain is ~empty) then we have a situation like this:

Set t=0 immediately after a block is published on a chain. Then, as t progresses, transactions with fees should build up in the mempool, so TxFees $\propto t. The reward for mining a block is r +$ TxFees $for some block reward, r.$

The potential reward-over-time for a miner (t vs r + TxFees) looks like a sawtooth function with a y-axis offset. It builds as more transactions pile up, and drops back to the baseline reward after a block.

If the miners M$_0, ..., M_9 are capable of working on one of any \{C_0, ..., C_9\} (and they have identical ROI profiles to the other miners), then they're incented to work on the chain with the most transactions in the mempool. That means:$
$miners should, roughly, work the chain that has gone the longest without a block. What should we expect based on those incentives? Miners should work on each chain only in the final moments of the block production cycle. If block times were set to 60s, then they'd start mining at around the 54s mark because that's how they maximize their ROI.$

Why wouldn't they just keep mining on the same chain? Because in the time that they focused on one chain, another one passed that &gt;54s high-ROI threshold and thus has the best ROI potential per hash done.

We should thus expect that this configuration of chains actually */spanemsynchronizes/span/em* miners, resulting in block production that is somewhat regular and lower in variance.

/spandiv
    class="tcolorbox"
    style=" border: 1px solid #404040; background: #F2F2F2; "

/spandiv
    class="tcolorboxupper"
    style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbTerm/span/b**/span/span **/spanb/spana href="whitepaper.html#??"Miner Resonance/span/a/span/b**: The effect whereby block production */spanemvariance/span/em* is reduced when miners can (and do) collectively change which chain they are currently mining faster than blocks are produced for those chains, due to changes in network-wide incentivization. /span/div /span/div

The average hash-rate on each simplex-chain, as described above, is always the same regardless of which of the two miner strategies are used. However, the variance of block production on each of these chains won't be that of a chain with 60s block times, it'll be closer to that of a chain with 6s block times.

<!--................................-->
/spanh5 id="autosec-314"/spanspan class="sectionnumber"4.10&#x2003;/span/spanSimplex Security and the Confirmation Equivalence Conjecture/span/h5

<!--...............................-->
/spanh6 id="autosec-315"/spanspan class="sectionnumber"4.10.1&#x2003;/span/spanSimplex Security/span/h6

Simplexes are secure if attacking a single simplex-chain is as difficult as attacking the whole simplex (the network).

Say the attacker controls q proportion of the network's work-generation capacity (e.g., hash-rate for PoW chains), and honest nodes control p proportion, such that $p+q=1$. An attacker with $q < p$ can attempt doublespends, and should succeed some of the time (see section /spana href="whitepaper.html#lateximage-whitepaper-63"4.10.2/span/a), but never with certainty. The attacker should only be able to */spanemcontrol/span/em* the network's history if $q > p$.

Successfully attacking a single blockchain requires an attacker to publish an alternate chain-segment that the fork rule considers heavier than the corresponding public chain-segment. Those two segments will have a common ancestor, so the weight of the attacker's segment must be greater than the weight of the public chain-segment (both start at that common ancestor).

Simplex-chains evaluate block-weight as the sum of work done on that block, plus work done on reflecting blocks. So effecting a doublespend on one simplex-chain requires generating a chain-segment with more total block-weight (including reflections) than the public chain-segment.

It is not viable for the attacker to mine the attacking chain-segment in public (it is self-defeating; see section /spana href="whitepaper.html#lateximage-whitepaper-63"4.8/span/a), so they must mine it in private. Blocks mined in private will not gain reflections from honest miners on other simplex-chains, so any reflections contributing to the doublespend must be created by the attacker. The attacker's reflecting blocks (which reflect the attacking chain-segment) on other simplex-chains can be public, but the attacker's task is simpler and not detectable if the attacker mines reflecting blocks in private./spansup88/span/sup There is no viable way for the attacker to prevent honest miners from reflecting the honest chain-segment, including new blocks that extend it, nor to prevent the honest chain-segment from reflecting blocks from other simplex-chains (again, see section /spana href="whitepaper.html#lateximage-whitepaper-63"4.8/span/a).

The honest chain-segment, on the targeted simplex-chain, will not be reflected by the attacker (since that would be self-defeating). Similarly, the attacker's chain-segment will not be reflected by the honest network, since it's being mined in private.

Let r be the network-wide rate-of-work (hash-rate). Over some attack duration d, on */spanemaverage/span/em* we expect the attacker's chain-segment to weigh qrd and the honest chain-segment to weigh prd. Thus, for the attacker's chain-segment to */spanemreliably/span/em* win, it must be that $qrd > prd \implies q > p$.■

/spandiv role="note" class="footnotes"
/spanp

/spansup88/span/sup&nbsp; Additionally, if reflecting chains maintain projections as headers-only chains (i.e., construct a main chain via SPV rules), then it should be much harder for an attacker to succeed if reflections are mined in public.

/span/p

/span/div

<!--................................-->
/spanh6 id="autosec-317"/spanspan class="sectionnumber"4.10.2&#x2003;/span/spanConfirmation Equivalence Conjecture/span/h6

/spandiv
    class="tcolorbox"
    style=" border: 1px solid #404040; background: #F2F2F2; "

/spandiv
    class="tcolorboxupper"
    style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbTerm/span/b**/span/span **/spanb/spana href="whitepaper.html#??"Confirmation Equivalence Conjecture (CEC)/span/a/span/b**: The conjecture that, when using PoR and appropriately converting work, confirmations of reflecting chains can be treated as */spanemequivalent/span/em* to local confirmations of the same weight. See eq. /spanspan class="textup"(/spana href="whitepaper.html#lateximage-whitepaper-63"33/span/a)/span/span. /span/div /span/div

The Confirmation Equivalence Conjecture is intimately connected to why PoR works. If it were false, then PoR could not work, and neither would simplexes. Additionally, this identifies the root of UT's O(c) confirmation rate — the conjecture implies that confirmation rate is proportional to the number of mutually reflecting chains.

Can we test it? If so, how?

Consider a traditional PoW blockchain (e.g., Bitcoin, Eth1, etc). It's well known that the risk of a doublespend against a particular block is related to the number of */spanemconfirmations/span/em* that block has./spansup89/span/sup

However, this is not a linear relationship; rather, it is similar to */spanemexponential decay/span/em*. After the first few confirmations, each additional confirmation reduces the risk of a doublespend by approximately the same factor./spansup90/span/sup So */spanemsecurity takes /spanspan class="uline"time/span/span/span/em*, because confirmations take time.

Now, consider a simplex: the equivalence conjecture says that it doesn't matter whether confirmations come from the */spanemlocal/span/em* chain, or if they come from a */spanemreflecting/span/em* chain. So, a simplex-chain in a 2-chain simplex */spanemshould/span/em*, after C */spanemlocal/span/em* confirmations, be as secure as a standalone blockchain with 2C confirmations. This is true for larger simplexes, too: */spanemdoubling/span/em* the size of a simplex means we can */spanemhalve/span/em* the number of */spanemlocal/span/em* confirmations that we wait for — the chance of a doublespend succeeding should be approximately the same. More generally, a simplex-chain in an N-chain simplex will, after C */spanemlocal/span/em* confirmations, be as secure as a standalone blockchain after CN confirmations.

This is something we can test! (See section /spana href="whitepaper.html#lateximage-whitepaper-63"4.10.4/span/a for that.)

As blockchain architects, if the CEC is true, then UT allows us to maintain security by trading */spanemwaiting time/span/em* for */spanemmore simplex-chains!/span/em*

/spandiv role="note" class="footnotes"
/spanp
/spansup89/span/sup&nbsp; /spana href="https://web.archive.org/web/20220209100515/https://cloudflare-ipfs.com/ipfs/QmNUWmY94QUievK8ptoxsPyAQUsKvx1cjRyCgPcfmysAVv" target="_blank" Analysis of hashrate-based double-spending
(Meni Rosenfeld; 2012)/span/a
/span/p

/spanp
/spansup90/span/sup&nbsp; The exact factor depends on the attacker's hash-rate as a proportion of the network's (i.e., q).
/span/p

/span/div
/spandiv
class="tcolorbox"
style=" border: 1px solid #404040; background: #ECECEC; "

/spandiv
class="tcolorboxupper"
style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbAside/span/b**/span/span If you were looking for an essential aspect of how UT breaks the core conflict of the Trilemma&#x2026; /span/div /span/div

<!--..............................-->
/spanh6 id="autosec-319"/spanspan class="sectionnumber"4.10.3&#x2003;/span/spanGeneralizing Doublespends/span/h6

With a traditional blockchain, the network has no "memory" of work (mining) that has been done which did /spanemnot/span/em result in a valid block (i.e., the main chain does not record or account for that work). Specifically, the network gains no knowledge of how much work has been done /spanembetween/span/em blocks. This is almost self-evident: each block is the /spanemsole/span/em way that work can be /spanemadded/span/em to the chain. Naturally, there is no smaller unit of contributed work than an individual block — so there's no "memory"./spansup91/span/sup When an attacker publishes a heavier chain-segment which causes a reorganization, it is /spanemimmediately/span/em in the interests of miners to begin building on the attacker's best block. A heavier chain-segment is /spanemdecisively better/span/em in all cases. That may seem so obvious that it isn't worth mentioning, but it's actually a /spanemspecific case/span/em that only works for traditional blockchains. If there were some "memory", it might be worth miners /spanemcontinuing/span/em to mine their /spanemexisting/span/em block, /spaneminstead/span/em of reorganizing and building on the attacker's best block.
What kind of "memory" could there be? Are /spanemProofs of Reflection/span/em a "memory"?
As it turns out: yes.
Let the total chain-weight of the attacker's best block be called W$_A, and the total chain-weight of the honest network's best block be called W_H$.
Consider a miner of a non-DAG simplex-chain (chain L) who is working on a /spanemdraft block/span/em during an active attack (though, of course, the miner does not know about the attack). Specifically, let's consider /spanemjust before/span/em the attacker publishes their chain-segment. In the memory pool/spansup92/span/sup of that miner, there will be PoRs, and those PoRs are /spanemonly/span/em valid for the /spanemhonest/span/em chain-segment (most likely they are for the best known honest block, but don't need to be). Each L block, in and of itself, only adds ~w weight to the L chain. However, if that /spanemdraft block/span/em contains ~n pending PoRs, and each PoR provides ~w weight on average, then a valid block (created from that draft) will contribute, via PoRs, ~nw weight to the /spanemhonest chain-segment/span/em. What happens when the attacker's chain-segment is published (along with their reflecting chain-segments for other simplex-chains)? The miner has a choice: reorganize to the attacker's chain-segment and mine on top of a chain with weight W$_A, /spanemor/span/em continue mining on the honest chain-segment that weighs W_H + nw$.
That extra term, nw, is the "memory" we're looking for, and it makes all the difference. It is only safe for the attacker to publish their chain-segment /spanemafter/span/em
W$_A > W_H + nw --- only then is it /spanemdecisively better/span/em than the honest chain-segment. If, however, the attacker publishes their chain-segment when W_H < W_A < W_H + nw$:
$the best chain-segment for honest miners to mine is /spanemstill/span/em the honest chain-segment, not the attacker's$!
For non-DAG chains, miners don't actually have to change their behavior at all: their choice is the same, whether they are mining a simplex-chain /spanemor/span/em a traditional chain. That choice is: /spanemof all possible blocks to mine, which results in the greatest reward?/span/em If they build on the honest chain-segment, does their resulting block have more total chain-weight than if they were to build on the attacker's chain-segment?
There are some subtleties to consider if chains are DAGs or use GHOST: both chain-segments can be included as ancestors, but which should take priority? If the weight of PoRs is not taken into account (or applied after evaluating the order of parents), then the attacker's chain-segment takes priority when
W$_A > W_H. This rule conflicts with what we discussed above, so it must be that parents are ordered based on their chain-weights /spanemincluding/span/em any PoRs in that block --- the honest chain-segment should take priority when W_A < W_H + nw. Also, note that this is why the weight of PoRs is attributed to the /spanemreflected/span/em block (usually a parent) rather than the block that contains the PoRs. In essence, the inclusion of this "memory" requires that the fork rule take /spanemcontext/span/em into account --- particularly the context of the descendant block. This is not a paradox because the fork rule is /spanemonly/span/em used to choose the priority chain in the context of some descendant block (even if that block is an invalid draft)$.

/spandiv role="note" class="footnotes"
/spanp
/spansup91/span/sup&nbsp; We're not concerned with things like */spanemweak blocks/span/em* or super-block hybrid designs; though it's possible that doublespend methodology for those designs has some overlap with this section.
/span/p

/spanp
/spansup92/span/sup&nbsp; The */spanemmemory pool/span/em* is where unconfirmed transactions (and other things) are stored before they are included in a block. Each node has their own memory pool. A miner typically decides a block's contents by choosing the combination of transactions from their memory pool that results in the largest reward (i.e., transactions with the largest fees).
/span/p

/span/div
/spandiv
    class="tcolorbox"
    style=" border: 1px solid #404040; background: #ECECEC; "

/spandiv
    class="tcolorboxupper"
    style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbAside/span/b**/span/span In the context of the longest PoR chain, this property is made somewhat obvious. /span/div /span/div

<!--...............................-->
/spanh6 id="autosec-321"/spanspan class="sectionnumber"4.10.4&#x2003;/span/spanTesting the Confirmation Equivalence Conjecture/span/h6

Let's test the Confirmation Equivalence Conjecture.

<!--................................-->
/spanspan class="paragraph" id="autosec-322"/spanspan class="sectionnumber"4.10.4.1&#x2003;/span/spanHypothesis/span/span

The hypothesis is that: doublespends against an /spanemindividual simplex-chain/span/em in an N-chain simplex after C /spanemlocal/span/em confirmations are /spanemas hard/span/em as doublespends against a traditional blockchain after C$\cdot Nconfirmations.$
First, define P(q; c), where c is the number of confirmations and q is the proportion of global hash-rate controlled by the attacker, to be the function that returns the probability of an attempted doublespend succeeding on a traditional blockchain according to Rosenfeld's analytical solution/spansup93/span/sup:
/spanspan class="hidden" \(\seteqnumber{0}{}{29}\)/span/span <!--

$$\forall c \in \{1,2,\dots\} : P(q;c) = \begin{cases} 1 - \sum_{m=0}^{c} \binom{m+c-1}{m} (p^c q^m - p^m q^c), & \text{if } q < p \\ 1, & \text{otherwise} \end{cases} \tag{30}$$

```
-->
\begin{equation}
\forall c \in \{1, 2, \dots \}:  P(q; c) = \begin{cases} 1 - \sum _{m=0}^c \binom {m+c-1}{m}(p^c q^m - p^m q^c), &amp; \text {if } q &lt; p \\ 1, &amp; \text {otherwise} \end {cases} \label {eq:rosenfeld-analytical-p}
\end{equation}
```

Next, we will declare a new function, P$', andassumethat/spanemitexistsandismeasurable/span/em.Particularly : P'(q;c=\mathcal{C};N_1 = N) istheprobabilityofadoublespendattacksucceedingagainstasimplexwithNsimplex-chainsafter\mathcal{C}confirmations, givenanattackerwithqproportionofthenetworkhash-rate.Wedonotknowthedefinitionofthisfunction, butwecanmeasureitsoutput.$
As a base case, it is taken that:
/spanspan class="hidden" \(\seteqnumber{0}{}{30}\)/span/span <!--

$$P(q;c) \equiv P'(q;c;N_1 = 1) \tag{31}$$

```
-->
\begin{equation}
P(q; c) \equiv P^\prime (q; c; N_1 = 1) \label {eq:cec-base-case}
\end{equation}
```

Note that a simplex of 1 chain (the 0-simplex) /spanemis/span/em a traditional blockchain. So we can take Bitcoin for example: the probability of an attack succeeding after 6 confirmations is given by P(q; c = 6) $\approx P'(q;c=6;N_1=1)$.
One prediction of the hypothesis is that the probability of a successful attack against a simplex with N simplex-chains after C confirmations is equal to that of a successful attack on a traditional chain after C$\cdot Nconfirmations.Thatis:$
/spanspan class="hidden" \(\seteqnumber{0}{}{31}\)/span/span <!--

(32)

```
-->
\begin{equation}

\end{equation}
```

In fact, we can take this further. The relationship that exists is not only at the extremes, but between them too/spansup94/span/sup — it must be if confirmations are equivalent. This is the /spanemextended/span/em CEC:
/spanspan class="hidden" \(\seteqnumber{0}{}{32}\)/span/span <!--

$$\begin{split} \forall a \in \{1,\dots,N\} : P'\biggl(q; c = \frac{\mathcal{C}N}{a}; N_1 = a\biggr) & \text{ is approximately constant} \\ \implies P'\biggl(q; c = \frac{\mathcal{C}N}{a}; N_1 = a\biggr) & \approx P(q; c = \mathcal{C}N) \end{split} \tag{33}$$

```
-->
\begin{equation}
\begin{split} \forall a \in \{1,\dots ,N\}:  P^\prime \biggl (q; c = \frac {\mathcal {C}N}{a}; N_1 = a\biggr ) &amp;\text { is approximately constant} \label {eq:cec-extended-hypothesis} \\ \implies P^\prime \biggl (q; c = \frac {\mathcal {C}N}{a}; N_1 = a\biggr ) &amp;\approx P(q; c = \mathcal {C}N) \end {split}
\end{equation}
```

Notice that we don't need to know a closed form of
P$'(q,c,N_1) totesttheCEC : allcases/spanemthatwecanmeasure/span/emareequivalenttosome/spanemother, traditional/span/emcase, andwecanconvertbetweenthem./spansup95/span/supThisisthebasisofthemethodusedtotestthishypothesis.$
If eq. /spanspan class="textup"(/spana href="whitepaper.html#lateximage-whitepaper-63"33/span/a)/span/span is correct, it indicates that there exists a /spanemgeneralization/span/em of traditional blockchains (which have
N$_1 = 1); particularly, thatthereis/spanemanotherdimension/span/em(N_1; thesizeofthesimplex) whichisgeometricallyrelatedtothefirstdimension, c.$
Note: this experiment does /spanemnot/span/em test the conversion of chain-work — the same hashing algorithm is used for each simplex-chain and no conversion of work takes place.

`<!--...............................-->`
/spanspan class="paragraph" id="autosec-323"/spanspan class="sectionnumber"4.10.4.2&#x2003;/span/spanMethod/span/span

I have implemented a simulation of a blockchain network that is suitable for this experiment. Particularly:
/spanul style="list-style-type:none"

/spanli
- The implementation is modular. It supports multiple: attack strategies; blockchain data structures; hashing algorithms (for PoW); and fork rules.

/span/li
/spanli
- An executable is produced that can run a single instance of a simulated attack, and takes a variety of parameters. (This is looped through via bash script to obtain CSVs of the results.)

/span/li
/spanli
- The implementation supports both traditional blockchains (N$_1$ = 1)*andPoR*/*simplexes.*

/span/li
/spanli
- It is fast.

/span/li
/span/ul
The source code for this model blockchain network is contained in the /spana href="https://github.com/AmarooHQ/whitepaper/tree/master/experiments/por-sim-rs" target="_blank" `/spankbd/experiments/por-sim-rs//span/kbd`/span/a folder of /spana href="https://github.com/AmarooHQ/whitepaper" target="_blank" this paper's repository/span/a.
While the simulation supports blockchains with single-parent blocks and a "longest chain" fork rule, these were not used in this experiment. Rather, multi-parent blocks were used (which simulates both GHOST and block-DAGs), along with the weight-based fork rule — these are required by UT.
/spanfigure id="autoid-36" class="table "

/spandiv class="center"

/spandiv class="figurecaption"Table 4: Table of parameters necessary to generate the main CEC experimental results. Only those parameters marked as variable were altered while generating the main results. Some parameters are omitted as they are not directly relevant to this experiment.
/span/div

/spantable/spantr class="tbrule"
/spantd class="tdl"Parameter/span/td
/spantd class="tdl"Variable/span/td
/spantd class="tdp"Significance/span/td
/span/tr

/spantr class="hline"
/spantd class="tdl"q (ratio)/span/td
/spantd class="tdl"Yes/span/td
/spantd class="tdp"Attacker's proportion of the global hash-rate./span/td
/span/tr

/spantr
/spantd class="tdl"c (confirmations)/span/td
/spantd class="tdl"Yes/span/td
/spantd class="tdp"Work-equivalent of ‘c local confirmations’ waiting time./span/td
/span/tr

/spantr
/spantd class="tdl"N$_1(chains)/span/td$
$/spantdclass = "tdl"Yes/span/td$
$/spantdclass = "tdp"Thenumbersimplex - chains(N_1 = 1fortraditionalblockchains)./span/td$
$/span/tr$

B$_f^{-1}(ticks)/span/td$
$/spantdclass = "tdl"No/span/td$
$/spantdclass = "tdp"ArtificialunitsoftimeusedbytheDAA./span/td$
$/span/tr$

DAA$_N(blocks)/span/td$
$/spantdclass = "tdl"No/span/td$
$/spantdclass = "tdp"ThenumberofblocksoverwhichtheDAAoperates./span/td$
$/span/tr$

H (hashes)/span/td
/spantd class="tdl"No/span/td
/spantd class="tdp"The network-wide average hashes per chain per tick./span/td
/span/tr

/spantr class="tbrule"
/spantd class="tdl"/span/td
/spantd class="tdl"/span/td
/spantd class="tdp"/span/td
/span/tr/span/table /span/div/span/figure
When choosing values for
B$_f^{-1}andH, wewantthemtobelowenoughforthesimulationtobeefficient, butlargeenoughtoensurethattherearenoissueswithlimits - of - accuracy.Tothisend, bothwerechosentobe75.Thismeansthat, priortothedoublespendattackbeginning, theaveragedifficultyisaround5625.$
We also want to consider DAA$_N - - - butthereisn'tanissueof/spanemprecision/span/eminthiscase.TheDAAused/spansup96/span/supadjustseveryblock, solowervaluesofDAA_N meantheDAAismorereactivetoadropinhash -$
$rate.IftheDAAismorereactive, theattackersneedtodo/spanemalittlebitmorework/span/emsince, onaverage, eachoftheirblockswillweighlessthananhonestblockatanequivalentheight.Iftheattacker'sblocks(onaverage)weighless, thentheforkrulewillfavoranhonestchain-$
$segment(whichhas, onaverage, heavierblocks)andtheattackerwillneedatleastone/spanemextra/span/emblockfortheirchain -$
$segmentbefavored.Theonlywaytoovercomethis, forshorterattacksorattacksagainstsmallersimplexes, isfortheattackertobe/spanemalittlebitluckierthanusual/span/em.(Theeffectislesssignificantwithlongattacksorattacksagainstlargersimplexessince P' \to 0anyway.)$

/spandiv role="note" class="footnotes"
/spanp
/spansup93/span/sup&nbsp; /spana href="https://web.archive.org/web/20220209100515/https://cloudflare-ipfs.com/ipfs/QmNUWmY94QUievK8ptoxsPyAQUsKvx1cjRyCgPcfmysAVv" target="_blank" Analysis of hashrate-based double-spending (Meni Rosenfeld; 2012)/span/a
/span/p

/spanp
/spansup94/span/sup&nbsp; Since /spanspan class="textrm textmd textup textsc"/spanspan class="numerator"CN/span/span//spanspan class="denominator"a/span/span/span/span, where a is the number of simplex-chains, in eq. /spanspan class="textup"(/spana href="whitepaper.html#lateximage-whitepaper-63"33/span/a)/span/span is not necessarily an integer, we'll assume there exists a valid generalization of Rosenfeld's solution for c $\in \mathbb{R}_{>0}$.
/span/p

/spanp
/spansup95/span/sup&nbsp; This is not strictly true: Rosenfeld's solution requires an integer value for c. This is due to his solution assuming a static difficulty and the longest-chain fork rule; the */spanemconstruction/span/em* is consistent with Bitcoin (provided the difficulty doesn't adjust), but not with chains that recalculate difficulty every block. Additionally, when using */spanemwork/span/em* instead of */spanemheight/span/em* as the measure of an attack's success, fractional values of c have real meaning — especially with a reactive DAA and $\mathrm{N}_1 \gg 1.Bothfactorsturnouttobeunproblematicforus,though.Rosenfeld'ssolutionhasreachbeyondthoseassumptionsandobservedresultsconvergewithitforlargervaluesof\mathrm{DAA}_N.$
/span/p

/spanp
/spansup96/span/sup&nbsp; The simulation uses the algorithm named /spanspan class="textsc"DAA-2/span/span from /spana href="https://web.archive.org/web/20211018043918/https://cloudflare-ipfs.com/ipfs/Qmd8BE6xYCH58LNipE1zZ7BCftemN8hQWnfZJSJYq5XUE8" target="_blank" An Economic Analysis of Difficulty Adjustment Algorithms in Proof-of-Work Blockchain Systems (Noda, Okumura, Hashimoto; 2020)/span/a.
/span/p

/span/div
/spandiv
class="tcolorbox"
style=" border: 1px solid #404040; background: #ECECEC; "

/spandiv
class="tcolorboxupper"
style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbAside/span/b**/span/span This explanation — that a smaller $\mathrm{DAA}_N meanstheattackerneedstobeluckier---wasnotsomethingIexpected.Itissomewhatspeculative,andisincludedheretoexplainwhysmallersimplexes(3to10chains)appeartohavebettersecuritypropertiesthantheanalyticalsolutionpredicts,andwhyweshouldpreferlargervaluesof\mathrm{DAA}_N ifwewanttomatchexpectedresults./span/div$
/span/div
In essence, a shorter $\mathrm{DAA}_N meansthatP'approacheszerofaster.Sincewewantto/spanemmatch/span/emanalyticalresults,wewanta/spanemlarger/span/emvalueof\mathrm{DAA}_N forthisexperiment---thisisanalogoustoanassumptionofRosenfeld'ssolution:$
/spanemconstantdifficulty/span/em.
So that we match the analytical solution's assumptions, a value of $\mathrm{DAA}_N = 500waschosen.Themostimportantthingisthat\mathrm{DAA}_N \gg c,particularlyregardingtraditionalchains./spansup97/span/sup$

/spandiv role="note" class="footnotes"
/spanp
/spansup97/span/sup&nbsp; The differential effect seems to diminish as an attack goes on, so DAA$_N isparticularlyimportantfor/spanemlower/span/emvaluesofc.DAA_N \gg cmatterswhencissmall, butnotsomuchwhencislarge. Inotherwords:$
$ifthedoublespendoccursover600blocks, thevalueofDAA_N doesn'tmattermuch; butifthedoublespendoccursover20blocks, thenthevalueofDAA_N isasignificantvariable.$
$/span/p$

/span/div
/spandiv
class="tcolorbox"
style=" border: 1px solid #404040; background: #ECECEC; "

/spandiv
class="tcolorboxupper"
style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbAside/span/b**/span/span In practice, we should seriously consider smaller values of DAA$_N forthesecuritybenefit./span/div$ /span/div
With regards to the other values — q, c, and
N$_1 - - - theywillbevariedaspartoftheexperimentsothatwecangenerateeachdata-point. Foreachdata-point, we'llgatheratleast9,000samples(thisisindicatedby``n \geq 9000'' inthesefigures'legends, whereninthiscontextisthenumberofsamples).$
The goal is to generate multiple data series that the CEC predicts to align. To that end, the functions in **??**&nbsp;4.10.4.2 are measured and graphed on the same axes. If the CEC is true then we expect:
f$_1(x) \approx f_2(x) \approx f_3(x) \approx f_4(x) \approx f_5(x)./spanfigureid=$"autoid-37" $class="table"$
$/spandivclass="center"$

/spandiv class="figurecaption"Algorithm 3: Functions that will be simultaneously graphed to test the CEC. The main results are generated for q $\in \{0.4, 0.44\} andC = 5. TheCECpredicts: f_1(x) \approx f_2(x) \approx f_3(x) \approx f_4(x) \approx f_5(x).$
/span/div

/spantable/spantr class="tbrule"
/spantd class="tdl"Function/span/td
/spantd class="tdl"Description/span/td
/span/tr

/spantr class="hline"
/spantd class="tdl"f$_1(x) = P(q; c = Cx)/span/td$
$/spantdclass = ”tdl”Baseline : Rosenfeld'sanalyticalsolution(calculated)/span/td$
$/span/tr$

f$_2(x) = P'(q; c = Cx; N_1 = 1)/span/td$
$/spantdclass = ”tdl”Simulateddoublespendagainstatraditionalchain/span/td$
$/span/tr$

f$_3(x) = P'(q; c = C; N_1 = x)/span/td$
$/spantdclass = ”tdl”Simulateddoublespendagainstasimplex/span/td$
$/span/tr$

f$_4(x) = P'(q; c = 2C; N_1 = x/2)/span/td$
$/spantdclass = ”tdl”Simulateddoublespendagainstasimplex(viaCEC)/span/td$
$/span/tr$

f$_5(x) = P'(q; c = 4C; N_1 = x/4)/span/td$
$/spantdclass = ”tdl”Simulateddoublespendagainstasimplex(viaCEC)/span/td$
$/span/tr$

/span/td
/spantd class="tdl"/span/td
/span/tr/span/table /span/figure

<!--...............................-->
/spanspan class="paragraph" id="autosec-329"/spanspan class="sectionnumber"4.10.4.3&#x2003;/span/spanError Correction Iteration/span/span

Following the initial implementation of PoR in the simulator, results were promising but did not align with the theoretical prediction (based on Rosenfeld's analytical derivation of P(q; c)). At lower values of c and/or $N_1, theresultssuggestedthatthesimulationwas/spanemmore/span/emsecurethanexpected.AthighervaluesofN_1, P'didnotappeartoapproach0, insteadapproaching{\sim}0.05.Thishintedattheexistenceofpotentialerrorsinthesimulatorframework.$

Given the nature of writing a simulation (compared to writing a production blockchain), many simplifications were made. Some of these simplifications shouldn't matter, but some might. How do we know if a given simplification (or implementation detail) introduced errors that were responsible for unexpected results?

In this case, sources of error were brainstormed, and evaluated in turn based on their potential to explain the largest sources of error. Of those, 4 major sources of error were corrected, which incrementally brought the simulation results closer and closer to the expected results (based on the CEC prediction). The error correction steps (including interim results) are briefly documented in **??**. Those corrections were:

/spanul style="list-style-type:none"

/spanli
1. Accounting for draft reflected work (explained in section /spana href="whitepaper.html#lateximage-whitepaper-63"4.10.3/span/a)
/span/li
/spanli
2. Randomizing hash-rates over the simplex (p + q = 1 is globally maintained)
/span/li
/spanli
3. Implementing the attacker's "Bonus Block"
/span/li
/spanli
4. Increasing the number of blocks over which the DAA operates (DAA$_N$ = 100 → 500)
/span/li
/span/ul

Many possible sources of error were ruled out, too, like: choice of hashing algorithm (`/spankbdxxh3/span/kbd`); values of $B_f^{-1}, H, andotherparameters; andimplementationdetailslikeomittingcertainvalidationsteps.$

Changing the simulation based on early results is dangerous though — why should we believe these results are valid? The answer is simple: all error corrections were motivated by */spanemmaking the simulation more like a real blockchain/span/em*. No magic numbers were introduced, no values were scaled arbitrarily, simulations were not repeated to get better results, etc. Since we */spanemknow/span/em* that a simulation will naturally include simplifying assumptions (and omitted implementation details), we should expect that some of these simplifications might be significant. So focusing on, and correcting, those simplifications does not compromise the method. Rather, the fact that */spanemthese/span/em* corrections were the significant ones is consistent with the idea that */spanemthe simulation is consistent with real-world blockchain networks/span/em*; that the simulation */spanemworks/span/em*.

/spandiv
class="tcolorbox"
style=" border: 1px solid #404040; background: #FCD1E9; "

/spandiv
class="tcolorboxtitle"
style=" border-top: 1px solid #404040; border-bottom: 1px solid #404040; background: #404040; color: #FFFFFF; "
TODO [9]/span/div
/spandiv
class="tcolorboxupper"
style=" color: #000000; "
assumption: data is available: i.e., if a block is reflected then it is known. this forces the attacker to mine in private. that's an assumption, too: the miner mines in private. /span/div /span/div

/spandiv
class="tcolorbox"
style=" border: 1px solid #404040; background: #FCD1E9; "

/spandiv
class="tcolorboxtitle"
style=" border-top: 1px solid #404040; border-bottom: 1px solid #404040; background: #404040; color: #FFFFFF; "
TODO [10]/span/div
/spandiv
class="tcolorboxupper"
style=" color: #000000; "
Low values of c much more secure? or mb limitation of simulation. possible explanation: draft refl work that already exists but that the attacker cannot use? /span/div /span/div

/spandiv
class="tcolorbox"
style=" border: 1px solid #404040; background: #FCD1E9; "

/spandiv
class="tcolorboxtitle"
style=" border-top: 1px solid #404040; border-bottom: 1px solid #404040; background: #404040; color: #FFFFFF; "
TODO [11]/span/div
/spandiv
class="tcolorboxupper"
style=" color: #000000; "
attacker's draft refl work? /span/div /span/div

<!--................................-->
/spanspan class="paragraph" id="autosec-330"/spanspan class="sectionnumber"4.10.4.4&#x2003;/span/spanValidating the Model/span/span

/spanfigure id="autoid-38" class="figure "

```
/spandiv class="center"
/spana href="simulation/__results__trad__validation__9000__t=5__daa=500" target="__blank" /spanimg
     src="simulation/__results__trad__validation__9000__t=5__daa=500"
     class="inlineimage"
     alt="(image)"
/span/a
```

/spandiv class="figurecaption"Figure 27: A comparison of the measured probability of a successful doublespend against a traditional blockchain compared with Rosenfeld's analytical solution, for q $\in \{0.4, 0.44\}. Thatthemeasuredresultsconvergewiththeanalyticalsolutionistakentobevalidationofthesimulation--- i.e., it/spanemworks/span/emandis/spanemuseful/span/eminthecontextofthisexperiment.$

/span/div /span/div/span/figure

At a minimum, we should expect the simulation to faithfully replicate doublespends against /spanemtraditional/span/em chains — i.e., f$_1(x) \approx f_2(x). Resultsofthis, forq \in \{0.4, 0.44\}, areshowninfig. /spanahref=$"whitepaper.html$\#lateximage-whitepaper-63"27/span/a.$

fig. /spana href="whitepaper.html#lateximage-whitepaper-63"27/span/a shows that the simulation /spanemdoes/span/em align with Rosenfeld's solution, especially at values of q $\lesssim$ 0.40$. Itisconsistentwitheq. /spanspanclass = $"textup"$($/spanahref = $"whitepaper.html$\#lateximage-whitepaper-63"31/span/a)/span/span. Asqincreases, statisticalartifactsandotherinaccuracies/spanemdo/span/embecomemoresignificant, butnotenoughtocompromisetheresults--- thelargestdifferences(forq = 0.44andc = 5)arearound50partsperthousand(fig. /spanahref = $"whitepaper.html$\#lateximage-whitepaper-63"28/span/a)andtypicallylessthanthis. Therearemoresignificantdifferencesatq =$ 0.48$, /spansup98/span/suphoweverthisisclosetolimitingthresholdofblockchainsecurityingeneral(q < 0.5), soweshouldexpectlargererrorinthesimulationresults. Suchdifferencesaretakentobe/spanemnon-critical/span/em, i.e., theydonotcompromisetheresults.$

<!--................................-->
/spanspan class="paragraph" id="autosec-332"/spanspan class="sectionnumber"4.10.4.5&#x2003;/span/spanResults/span/span

In this section we will only look at results for q $\in \{0.4, 0.44\}, thoughresultsforq = 0.48werealsogenerated(amongothercombinationsofparameters).Resultsthatareomittedfromthissectionareavailablein$**??**.

First, do results of the simplest version of the CEC match predictions? fig. /spana href="whitepaper.html#lateximage-whitepaper-63"28/span/a graphs f$_1$, $f_2, f_3 andtheresultsareconsistentwithpredictions($**??**$)$.

/spanfigure id="autoid-39" class="figure "

```
/spandiv class="center"
/spana href="simulation/_results_trad_vs_1simplex_9000_qs=0.40-0.44_t=5_daa=500" target="_blank" /spanimg
    src="simulation/_results_trad_vs_1simplex_9000_qs=0.40-0.44_t=5_daa=500"
    class="inlineimage"
    alt="(image)"
/span/a
```

/spandiv class="figurecaption"Figure 28: A comparison of the measured probabilities, for q ∈ {0.4, 0.44}, *ofasuccessfuldoublespendbetween : atraditionalblockchain, asimplex, andRosenfeld′sanalyticalsolution.Thatthemeasuredresultsforasimplex(convertedviatheCEC)convergewiththeotherresultsistakentobeevidenceofO(n)securityandthattheC*

refuted.

/span/div /span/div/span/figure

While it appears that simplexes (for q=0.44, c=5, $N_1 \in [3, 10]$, *atleast)areslightlymoresecurethanexpected, thiseffectdiminishesas*N_1*increases.Asweareabouttosee, italsodiminishesascincreases.Forthisexperiment, itistakentobea/spanemnon − critical/span/emerror.*

Second, what about the */spanemgeneral case?/span/em* Does the extended CEC hold true, also? fig. /spana href="whitepaper.html#lateximage-whitepaper-63"29/span/a graphs
f_1, f_2, f_3, f_4, f_5*andtheseresultsarealsoconsistentwithpredictions(eq. /spanspanclass = ”textup”(/spanahref = ”*whitepaper.html*#lateximage − whitepaper − 63”33/span/a)/span/span).*

/spanfigure id="autoid-40" class="figure "

```
/spandiv class="center"
/spana href="./includes/ut/simulation/_results_cec_9000_qs=0.40-0.44_t=5_daa=500.svg" target="_blank" /spanimg
      src="./includes/ut/simulation/_results_cec_9000_qs=0.40-0.44_t=5_daa=500.svg"
      class="inlineimage"
      alt="(image)"
/span/a
```

/spandiv class="figurecaption"Figure 29: A comparison of the measured probabilities, for q $\in \{0.4, 0.44\}, ofasuccessfuldoublespendbetween : atraditionalblockchainandmultiplesimplexesconvertedviatheCEC.ThattheresultsconvergeaspredictedistakentomeanthattheextendedCECisnon-refuted.$

/span/div /span/div/span/figure

<!--................................-->
/spanspan class="paragraph" id="autosec-335"/spanspan class="sectionnumber"4.10.4.6&#x2003;/span/spanConclusions/span/span

These results are, for the purpose of this paper, taken to indicate the following statements are true/spansup99/span/sup. /spanul style="list-style-type:none"

/spanli
1. Sharing PoW security between blockchains without merged mining is possible.
/span/li
/spanli
2. PoR */spanemworks/span/em* as claimed when implemented correctly.
/span/li
/spanli
3. The */spanemConfirmation Equivalence Conjecture/span/em* is, */spanemgenerally and in principle/span/em*, true.
/span/li
/spanli
4. Simplexes are secure against a minority attacker and are thus O(n) secure (assuming no relevant bottlenecks).
/span/li
/span/ul

/spandiv role="note" class="footnotes"
/spanp
/spansup98/span/sup&nbsp; See **??**.
/span/p

/spanp
/spansup99/span/sup&nbsp; Experimental results never prove that something is correct, however, they are criticisms of */spanemcompeting/span/em* explanations; for example: we see that these experimental results */spanemconverge/span/em* as expected. This fact is NOT evidence that UT is secure, or that PoR works, or that the CEC is right. But, it is a */spanempre-emptive criticism/span/em* of ideas like: */spanemThe simulation does not work/span/em*, or */spanemPoR is broken/span/em*, or */spanemSimplexes aren't secure/span/em*, or */spanemSharing security between blockchains is impossible/span/em*. A decisive criticism of UT/PoR/CEC must therefore explain how these results are wrong — the explanation needs to include a */spanemrefutation/span/em* of these results. Such a refutation could be an explanation of how: the implementation was flawed, the methodology was flawed, or that UT cannot be effectively implemented as a production blockchain. No such explanations are currently known. In the */spanemabsence/span/em* of such criticisms, it is reasonable to work under the assumption that UT is secure, since */spanemPoR works/span/em* and */spanemthe CEC is true/span/em* are currently (to my knowledge) the only viable explanation for these results.
/span/p

/span/div

<!--..................................-->
/spanh6 id="autosec-337"/spanspan class="sectionnumber"4.10.5&#x2003;/span/spanClosed Form of P′*/span/h6*

Deriving a closed form of P′*fromfirstprincipleswouldbechallenging.Butperhapswedon′tneedto.First,let′sconsiderthe/spanemPoRgraph/span/emwithoutalltheadditionalcontextofPoRanddifferentsimplex − chains − − − /spanemwhatisit?/span/emItis, more − or − less, afairly − streamlinedblock − DAG.Ifweassumethereisnolatency, andthustherearenosiblingsor‘stale′blocks, thenweexpectaformsimilartoatraditionalblockchain(withaveryshortblockperiod).Fromthisperspective, itisfairlyobviousthat/spanspanclass = ”uline”weshouldexpect/span/spanP ≈ P′ − − − they′retryingtodescribethe/spanemsamething/span/em.*

The block-DAG nature of the PoR graph may seem complicate the derivation, too, until we notice that comparisons between */spanemlinearized slices/span/em* of a block-DAG work exactly the same as comparisons between traditional chain-slices: */spanemwhich is heavier?/span/em* Moreover, the */spanemorder/span/em* of blocks in the linearization */spanemdoesn't matter/span/em* since */spanemall block-weight is counted/span/em* (and addition is commutative). Only the final comparison between the attacker's slice and the honest slice matters.

Therefore, we can use **??** for the substitutions c $\to \mathcal{C}N$, *thenP* $\to P'$, *then*$\mathcal{C} \to c; N \to N_1 \to$ *ntoarriveataclosedform.Thesesubstitutionsareimportantbecausethey/spanemchangethearguments/span/emofPandP′withoutchangingthealgebraicdescription.Practically, ineq. /spanspanclass = ”textup”(/spanahref =* ”whitepaper.html*#lateximage − whitepaper − 63*”30*/span/a)/span/span, wearereplacingcwithcn./spanspanclass = ”hidde*
<!--

$$\forall c, n \in \{1, 2, \dots\} : P'(q; c; n) = \begin{cases} 1 - \sum_{m=0}^{cn} \binom{m+cn-1}{m} (p^{cn} q^m - p^m q^{cn}), & \text{if } q < p \\ 1, & \text{otherwise} \end{cases} \tag{34}$$

-->

```
\begin{align}
\forall c, n \in \{1, 2, \dots \}:  P^\prime (q; c; n) &amp;= \begin{cases} 1 - \sum _{m=0}^{c n} \binom {m+c n-1}{m}(p^{c n} q^m - p^m q^{c n}), &amp; \text {if } q &lt; p \\ 1, &amp; \text {otherwise} \end {cases} \label
{eq:cec-closed-1}
\end{align}
```

Like Rosenfeld's derivation, this closed form comes with some assumptions. For example, we inherit the assumptions that the block period of all simplex-chains is identical and that the difficulty remains constant through the attack. Experimentally we found that randomizing the attacker's hash-rate over the simplex was required to match the results of the CEC, so this appears to be an assumption of this closed form, too./spansup100/span/sup

/spandiv role="note" class="footnotes"
/spanp
/spansup100/span/sup&nbsp; Additionally, the simulation showed that attacks were less effective when the attacker's hash-rate was */spanemuniformly/span/em* distributed over the simplex. See **??** for more details.
/span/p

/span/div

`<!--................................-->`
/spanh5 id="autosec-339"/spanspan class="sectionnumber"4.11&#x2003;/span/spanIntra-Simplex Cross-Chain Transactions/span/h5

<!--...............................-->
/spanh6 id="autosec-340"/spanspan class="sectionnumber"4.11.1&#x2003;/span/spanIntroduction/span/h6

Our goal in this section is to find a method for cross-chain transactions between any two simplex-chains that is safe, fast, succinct, and reliable. /spanul style="list-style-type:none"

/spanli
- */spanemSafe/span/em*: an adversary must perform a 51% attack to execute an invalid cross-chain transaction.

/span/li
/spanli
- */spanemFast/span/em*: O(t)

$\leq O(\lambda) = O(1), where : tisthemaximumdelaybetweenacross - chaintransactionbeingconfirmedbyasimplex - chainandthepointatwhichthedestinationsimplex - chainallowsprocessingthattransaction; \lambda isasecurityparameter(e.g., \lambda = 4localconfirmations).$

/span/li
/spanli
- */spanemSuccinct/span/em*: for any cross-chain proof $\pi, O(\pi) < O(c) and |\pi| ispractical.$

/span/li
/spanli
- */spanemReliable/span/em*: the above properties hold for all transactions in all simplex-chains.

/span/li
/span/ul

Given everything we have discussed up to now, it's not clear how cross-chain transactions are possible. The problem is that miners on one simplex-chain do not validate the state transitions of other simplex-chains. Each simplex-chain, locally, has an O(/spanspan class="textrm textmd textup textulc"/spanspan class="numerator"n/span/span//spanspan class="denominator"c/span/span/span/span) share of the network's security (assuming there are O(c) many chains). An O(/spanspan class="textrm textmd textup textulc"/spanspan class="numerator"n/span/span//spanspan class="denominator"c/span/span/span/span) adversary could theoretically produce */spanemall/span/em* of the blocks for a single simplex-chain (e.g., by mining at a slight loss to push other miners out). The adversary could then create a very long chain of blocks that have valid headers, PoRs, etc, but also contain one or more fraudulent transactions. Perhaps the main chain eventually corrects to a non-fraudulent history (similar to section /spana href="whitepaper.html#lateximage-whitepaper-63"4.8.3/span/a) — but in that case we'd need to wait a */spanemvery/span/em* long time to be sure that such a correction */spanemmust/span/em* have happened. That violates the */spanemfast/span/em* goal, so this eventual-correction idea fails to meet our overall goal.

Are we stuck? For a blockchain network to be scalable, it must be the case that miners are */spanemnot/span/em* required to validate more than one chain! Have we come all this way, attempting to deal with the core conflict of the trilemma, only to find that we just moved the conflict?

<!--...............................-->
/spanh6 id="autosec-341"/spanspan class="sectionnumber"4.11.2&#x2003;/span/spanWhy does SPV work for Bitcoin?/span/h6

For the sake of simplicity, let's consider Bitcoin in isolation. Particularly, we'll assume that there are no other networks of similar size using the same PoW algorithm or compatible mining hardware.

A concise argument for the security of Bitcoin-SPV proofs is:

/spanul style="list-style-type:none"

/spanli
P.1 An SPV proof is a merkle branch proving that a transaction exists in a block.
/span/li
/spanli
P.2 All blocks are part of the same blockchain and history.
/span/li
/spanli
P.3 All transactions that occur are in some block's merkle tree.
/span/li
/spanli
P.4 If a transaction is invalid, then the block is invalid.
/span/li
/spanli
P.5 If a block builds on an invalid block, then it is invalid.
/span/li
/spanli
P.6 If a miner produces an invalid block, they get no block reward.
/span/li
/spanli
P.7 $\implies Honestminerswon't produceorbuildoninvalidblocks.$
/span/li
/spanli
P.8 $\implies Foranadversarytoproduceanapparentlyvalid(butfraudulent)proofleadingtoa/spanemmainchain/span/emblock, they'dneedtoout-competehonestminers(i.e., 51\%attackthenetwork).$
/span/li
/spanli
P.9 $\implies Onlyvalidtransactionsexistinthemainchain./spansup101/span/sup$
/span/li
/spanli
P.10 $\implies It'ssafetoacceptSPVproofsforblocksinthemainchain.$
/span/li
/span/ul

It appears that point /spana href="whitepaper.html#lateximage-whitepaper-63"8/span/a is our current sticking point for Simplex-SPV proofs.

But, we don't necessarily */spanemneed/span/em* to replicate point /spana href="whitepaper.html#lateximage-whitepaper-63"8/span/a if we could get to point /spana href="whitepaper.html#lateximage-whitepaper-63"9/span/a by other means.

/spandiv role="note" class="footnotes"
/spanp
/spansup101/span/sup&nbsp; This isn’t the full picture for some blockchains (e.g., Ethereum), even though SPV still works for them — a transaction that fails to execute can still be valid and produce a valid state transition. In these cases, an invalid state transition implies an invalid block, so it’s customary to prove that some state has the right values rather than that a transaction exists.
/span/p

/span/div

```
<!--................................-->
/spanh6 id="autosec-343"/spanspan class="sectionnumber"4.11.3&#x2003;/span/spanMethod/span/h6



/spandiv
       class="tcolorbox"
       style=" border: 1px solid #404040; background: #F2F2F2; "

/spandiv
       class="tcolorboxupper"
       style=" color: #000000; "
/spanspan role="note" class="marginpar"/spanbNote/span/b/span/spanThe details below are an overview of Amaroo’s cross-chain protocol. Full details comprise the Amaroo team’s forthcoming paper on the subject./span/div /span/div
```

<!--................................-->
/spanspan class="paragraph" id="autosec-344"/spanspan class="sectionnumber"4.11.3.1&#x2003;/span/spanContext/span/span

The purpose of UT's cross-chain protocol is to */spanemmaintain coherence/span/em* between simplex-chains. This means that all the essential properties we'd expect to hold for transactions on a single chain should hold between chains, too: coins are not created nor destroyed/spansup102/span/sup, coins can only be spent once, */spanemthe transaction executes entirely or not at all/span/em*, etc. There will obviously be some particular properties that are different, such as a delay between inputs being spent (on the sending chain) and outputs being created (on the receiving chain).

Before we address how cross-chain transactions will work, we first need to remember the context in which we're operating. Leading up to this point, we've constructed a network with some wildly different properties to traditional blockchains: ultra-fast confirmations, additional DoS and censorship resistance, shared security, higher order scaling, etc. These improvements are due to the specific combination of: */spanemProof of Reflection/span/em*, the use of chain-like block-DAGs in the PoR Graph and each simplex-chain, and the Axioms of /spana href="whitepaper.html#lateximage-whitepaper-63"Availability/span/a, /spana href="whitepaper.html#lateximage-whitepaper-63"Maximal Reflection/span/a, and /spana href="whitepaper.html#lateximage-whitepaper-63"Unified Ancestry/span/a. Each of these ideas, along with fraud proofs, will have a role to play in the cross-chain protocol described shortly.

Additionally, we are concerned about some possible attacks aimed at the cross-chain protocol. We are not necessarily concerned about how the protocol resolves during a 51% attack in this discussion since we already know that the network does not function in that context.

The first and most important thing to consider is */spanemUnder which circumstances can cross-chain transactions be invalidated?/span/em* If it is possible to invalidate a past cross-chain transaction, this could be used to perform a doublespend, or coin duplication, etc. Therefore, we should strive for a protocol that only permits such invalidation under a 51% attack (or worse). That way, we can be confident that the cross-chain protocol is not the weakest link in the chain, since local (intra-chain) transactions break down at that point, anyway.

The obvious way to invalidate a cross-chain transaction is to cause a reorganization on the origin chain. If the pivot of the reorganization is antecedent to the cross-chain transaction, then it is up to the attacker whether that transaction is included in the resulting canonical history. This outcome is self-evidently insecure. Therefore our question about cross-chain transaction invalidation has the same answer as questions like */spanemUnder which circumstances can a simplex-chain reorganization occur?/span/em* We are also concerned with the parameters around such a reorganization: */spanemIs it dependent on the number of confirmations?/span/em*, */spanemIf so, how many?/span/em*, */spanemWhat proportion of global hash-rate is required?/span/em*, etc.

Fortunately, we've done a lot of the legwork already to prove that the ordering of simplex-chain blocks is stable and works for cross-chain transactions. */spanemProof of Reflection/span/em* allows us to share security between simplex-chains. Block-DAGs and the PoR Graph provide some censorship resistance, and more importantly, are the basis for both the /spana href="whitepaper.html#lateximage-whitepaper-63"Axiom of Maximal Reflection/span/a and the /spana href="whitepaper.html#lateximage-whitepaper-63"Axiom of Unified Ancestry/span/a. These axioms go on to, respectively, bind the histories of each simplex-chain to one another and ensure that each chain's history spans all valid blocks of that chain in the PoR graph. The binding of histories is the basis for the */spanemcoherence/span/em* of the simplex, and PoR graph-spanning histories ensures that the ordering of blocks within each chain conforms to the ordering of blocks in the PoR graph. Therefore: to change the order of blocks within a simplex-chain requires that an attacker change the global ordering, which in turn requires a 51% attack. ■

Regarding network security, resisting this particular attack is paramount because this kind of reorganization */spanemaffects honest nodes/span/em*. By comparison, there are other attacks that do not affect honest nodes, but do affect, say, light clients./spansup103/span/sup These other attacks aren't as concerning provided that they only affect non-validating network participants. But! What happens when an invalid block is mined? It */spanemwill/span/em* be reflected, because miners of other simplex-chains do not validate the block. What we need is a method of */spanemerror correction/span/em* – one that is fast enough to ensure invalid blocks are dealt with before the cross-chain protocol allows those blocks to be processed. This role is filled by */spanemfraud proofs/span/em*.

In UT, */spanemfraud proofs/span/em* are automatically constructed by any full node during normal validation – the state transition function will return either a new state trie */spanemor/span/em* a fraud proof. Once a fraud proof is constructed, it is broadcast similar to a transaction. Then, */spanemany miner/span/em* from */spanemany simplex-chain/span/em* can validate and include the fraud proof */spanemas part of the PoR graph/span/em*. Due to /spana href="whitepaper.html#lateximage-whitepaper-63"Unified Ancestry/span/a, every simplex-chain calculates the expected chain-tips for every simplex-chain. When a fraud proof invalidates a block, the calculated chain-tips for a given simplex-chain are updated to exclude any invalid blocks from that chain's history. Thus, as long as fraud proofs are readily generated, broadcast, and included by other miners, the PoR graph knows which blocks are on-main valid blocks, and which are not.

There is, of course, some lag between an invalid block being mined, the fraud proof being generated, and the proof being included in the PoR graph. In a healthy network, the expected delay is $(\mathrm{N}_1 B_f)^{-1} + oseconds, whereoisalittleoverheadtoaccountforthefirstnodeofthatchaintoreceiveandprocesstheblock.Thisismuchlessthanablockperiod - -evenwhenaminorityofminersarehonest(particularlywhen N_1 \gg p^{-1})$.

<!--................................-->
/spanspan class="paragraph" id="autosec-345"/spanspan class="sectionnumber"4.11.3.2&#x2003;/span/spanConstruction/span/span

Typically, a cross-chain protocol will require two user interactions: one on the sending chain and one on the receiving chain. That is, there is some state modification on the first chain, and then, on the second, an acknowledgment of that and some associated action (like crediting an account). Based on the direction of information, order of actions, and the focus on those actions, we could describe these kinds of methods as */spanempush-pull/span/em* methods. Information is 'pushed' from the sending chain, and, at some later point, 'pulled' into the receiving one – conceptually, at least. In practice the 'pull' requires the user to publish a proof about the sending chain to the receiving chain.

By contrast, a */spanempush-push/span/em* method would be one where the receiving chain automatically processes the cross-chain output at the correct time. This means that no second action from the user is required. Such a system must therefore */spanemrequire/span/em* miners to process cross-chain transactions – but how do they know about transactions on other chains and whether they are valid? In our case, the /spana href="whitepaper.html#lateximage-whitepaper-63"Axiom of Availability/span/a means that all miners on all simplex-chains have all recent blocks from all other simplex-chains. Our use of fraud proofs means that all honest nodes quickly learn of invalid blocks. Thus, provided enough time has passed, we can be confident that miners know of all cross-chain transactions, and that they are, in fact, valid.

How long should we wait, and how do we measure how long we've waited? We need at least some delay to avoid a particularly lucky series of blocks from triggering cross-chain processing. Ideally, this processing should be as predictable as possible, and well distributed. We also want to avoid our delay method from being manipulated to cause earlier or later execution.

The delay duration itself is almost unconstrained: fraud proofs both propagate through the network well within one block period (at which point honest nodes are aware of them), and are included in the PoR graph within a block period, too. However, if we pick too short a period, then we risk some fragility in the network. We can technically recover if an invalid cross-chain transaction was processed before a corresponding fraud proof was recorded, but doing so robs honest miners of their block reward. So, whatever we pick, it needs to be something that is conservative enough for miners to be comfortable with.

Although it's somewhat arbitrary, 60 seconds (or 4 block periods at $\mathrm{B}_f =$
$/spanspanclass = "textrm\ textmd\ textup\ textulc"/spanspanclass = "numerator"1/span/span//spanspanclass = "denominator"15/span/span/span/spanHz)seemslikeareasonablechoice.Thereisplentyoftimeforthefraudprooftopropagateandberecorded,andtheriskofanhonestminer'sblockbeinginvalidatedislow.$

How do we measure this duration, then? We could use block timestamps, although that may open us up to some kind of time-warp attack. We could use the number of confirmations on the sending or receiving chain, but there's local block variance to consider. We could use the change in volume of the simplex (i.e., the number of confirmations over all simplex chains since some block), but we don't have any guarantees about the progression of either the sending or receiving chain. Each of these has disadvantages, but if we take these breakpoints together, then we have a reliable single condition.

We can now summarize; when drafting an Lblock, the Rblocks which are valid for cross-chain processing from the draft block's perspective are those which: /spanul style="list-style-type:none"

/spanli
1. Have not been processed before; and
/span/li
/spanli
2. Have a timestamp at least $\lambda blockperiodsinthepast; and$
/span/li
/spanli
3. Have a volume at least $\mathrm{N}_1\lambda blocksfewerthanthisblock'svolume; and$
/span/li
/spanli
4. Are in the PoRs history of this block's $\lambda - parent/spansup104/span/sup; and$
/span/li
/spanli
5. Are at least $\lambda localconfirmationsdeep.$
/span/li
/span/ul

These conditions are illustrated in fig. /spana href="whitepaper.html#lateximage-whitepaper-63"30/span/a.

/spanfigure id="autoid-41" class="figure "

/spandiv class="center"
/spana href="spv/xc_security_conditions_sag" target="_blank" /spanimg
src="spv/xc_security_conditions_sag"
class="inlineimage"
alt="(image)"
/span/a

/spandiv class="figurecaption"Figure 30: The conditions used to find Rblocks that are valid for cross-chain transaction processing from the perspective of L$_i. Particularly, wefindupto4distinctblocksvia4differentpaths. Notethattheseblockscanappearinanychronologicalorder, andthespecificblocksshowninthediagramarejustoneexampleofmanypossibili$
$parentofR_jresolvestoR_{j-\lambda}; andthe\lambda - parentofL_iresolvestoL_{i-\lambda}$.

/span/div /span/div/span/figure

There is one slight oversight in this formulation that we need to fix. How do we know that the transactions in those blocks to be processed are all consistent? If we naively collect blocks according to the above conditions, then we don't – there might be conflicting transactions in off-main blocks. To correct this, we first use the four conditions to find the 1–4 most recent corresponding blocks (some conditions may resolve to the same block). Next, for chain R, we find the on-main MRCA (most recent common ancestor) of those blocks – this is the most recent block that will be processed for cross-chain transactions. We repeat this process for the parent of the draft block to find its corresponding MRCA by the same rules. Let's call these blocks $R_\alpha andR_\beta respectively. Thesetofblockstoprocessis: R_\alpha andallon - mainblocksinitshistory/spanemminus/span/emR_\beta andallon - mainblocksinitshistory. Thisistheon - main``chain - slice'' fromR_\alpha (inclusive) toR_\beta (exclusive). Sincealloftheseblocksareon - main, anyvalidtransactionsfromoff - mainblocksbetweenR_\alpha andR_\beta willbeincludedinoneoftheon - mainblocks. Byinspection, wecanobservethatR_\beta forthecurrentblockwasR_\alpha fortheparentblock(oranearlierancestor). Byinduction, thisseriesofchain - slicescontinuesallthewaybacktothegenesisblock(beforewhichnochain - slicesexist). Therefore, everyon - mainblock(andthuseveryvalidtransactiononthatchain) isprocessedforcross - chaintransactionsattheearliestappropriatetime.$

The protocol for processing cross-chain transactions is now, simply: when drafting an Lblock, for each Rchain, miners gather the chain-slice (of Rblocks) that is valid for cross-chain processing, scan those blocks for incoming cross-chain outputs, and include those cross-chain outputs in the draft block.

/spandiv role="note" class="footnotes"
/spanp
/spansup102/span/sup&nbsp; That isn't to say burning coins isn't possible, but it should never be an unintended byproduct of a cross-chain transaction.
/span/p

/spanp
/spansup103/span/sup&nbsp; In this context light clients are not considered honest nodes (nor attacking nodes) since they do not contribute to global consensus.
/span/p

/spanp
/spansup104/span/sup&nbsp; Definition: the 1-parent is the parent; the 2-parent is the parent of the 1-parent; the (k+1)-parent is the parent of the k-parent.
/span/p

/span/div
/spandiv
class="tcolorbox"
style=" border: 1px solid #404040; background: #ECECEC; "

/spandiv
class="tcolorboxupper"
style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbAside/span/b**/span/span In practice, these conditions aren't ideal because updating the reflections of a draft block may introduce new blocks which are valid for cross-chain processing, in turn requiring a complete recalculation of that block's output state. To avoid this, we replace item /spana href="whitepaper.html#lateximage-whitepaper-63"4/span/a and item /spana href="whitepaper.html#lateximage-whitepaper-63"5/span/a with similar conditions based on the draft block's parent and $\lambda - 1. Thiskeepsthesetofblocksstablewhenupdatingthereflectionsofadraftblock./span/div$ /span/div

<!--................................-->
/spanspan class="paragraph" id="autosec-348"/spanspan class="sectionnumber"4.11.3.3&#x2003;/span/spanEvaluation/span/span

Safe Double-spending a cross-chain transaction requires a 51% attack on the entire network. Executing an invalid cross-chain transaction is not possible when there are honest nodes of that simplex-chain and honest miners of any simplex chain.

Fast Cross-chain transactions are processed based on conditions which are all O(1) with respect to time, and therefore the expected delay in execution is O(1).

Succinct No proof is explicitly recorded. If one did want to craft an explicit proof $\pi, thenitwouldbesimilarinsizetoanormaltransactionproof(assumingheadersfromthesurroundingPoRgraphareavailable).NormaltransactionproofsareO(\log c), therefore, O(\pi) = O(\log c) < O(c)$.

Reliable The consensus protocol requires that cross-chain transactions are processed if they are valid for cross-chain processing. Not doing so invalidates a block. Therefore, all cross-chain transactions on all simplex-chains will have these above properties.

Censorship Resistance This construction has a curious property. Say that an attacker, in an effort to perform an empty block DoS, is mining a simplex-chain at a loss, and has pushed out other miners. If you wish to make a local transaction, it could take much longer than a block period. An alternative is to make a cross-chain transaction to that simplex-chain from another simplex-chain instead. This will obviously take a minute or so to execute, but you will have a guarantee that it will be processed – the attacker cannot continue the attack without processing that cross-chain transaction. You could then 'rescue' your coins and send them to a more useful chain./spansup105/span/sup While this describes an unpleasant user experience, the takeaway is that it is yet another obstacle for an attacker to overcome.

/spandiv role="note" class="footnotes"
/spanp
/spansup105/span/sup&nbsp; This kind of operation would require some support from the transaction layer, e.g., a way to spend inputs indirectly. Such an output type would also need to contain instructions in case of a failure, since the sending chain cannot verify that the inputs to be spent (on the receiving chain) will be spendable when the cross-chain transaction is processed. Since we must scan blocks as part of the cross-chain processing, the branch taken should also be recorded in the block, alongside the corresponding output or transaction.
/span/p

/span/div

<!--.................................-->
/spanh6 id="autosec-350"/spanspan class="sectionnumber"4.11.4&#x2003;/span/spanDecoupled State Progression/span/h6

The key to cross-chain transactions working is the decoupling of state progression combined with the property that cross-chain transactions can be read from Rblocks without needing to validate the Rchain – if it exists, then it is valid. Since part of this protocol involves asynchronous communication between chains, we need a method of error correction to ensure cross-chain transactions are safe: fraud proofs. Crucially, we expect that honest nodes (particularly those on other chains) are never fooled by an invalid block for long — much less than a block period. On this basis we can pick a safety parameter $\lambda which (in combination with the conditions from earlier) provides enough buffer to ensure that the PoR graph remains coherent.$

This decoupling is the essential component of UT that makes O(c^2)$scaling possible, and is a direct result of how we use PoR to create the PoR graph. The procedural elements are shown in fig. /spanahref = $"whitepaper.html#$latex image - whitepaper - 63$"31$/span/a.$

/spanfigure id="autoid-42" class="figure "

/spandiv class="center"
/spana href="spv/network_scale_AL_split_sag" target="_blank" /spanimg
src="spv/network_scale_AL_split_sag"
class="inlineimage"
alt="(image)"
/span/a

/spandiv class="figurecaption"Figure 31: Flow chart of how the segmentation of state works in UT to achieve O(c^2)*capacity.“AL′′isshortfor“ActionLayer′′,aterminventedheresothatwecanpointtothevariousstagesofPoRgraphextensionandstateprogression.*AL^1*columnsindicateactionsthatvalidatingnodesofanysimplex−chainperformonthePoRgraph.The*AL^2*columnindicatesthatonlyvalidatingnodes/spanemofthatchain/span/emperformtheassociatedaction.*

/span/div /span/div/span/figure

`<!--................................-->`
/spanh6 id="autosec-352"/spanspan class="sectionnumber"4.11.5&#x2003;/span/spanFraud Proofs &amp; Bribe Attacks: Derivations &amp; Breakpoints/span/h6

Let’s assume that an attacker is willing to bribe miners to exclude a fraud proof. The goal of the attacker is to continue this attack beyond some breakpoint, so how much will that cost them over the course of the attack?

First, let’s establish a baseline: what does a healthy simplex look like? */spanemHealthy/span/em* meaning: no fraud proof is possible because all blocks and state-transitions are valid. We need to know what a healthy simplex (and its PoR graph) looks like because a successful attack will start off indistinguishable.

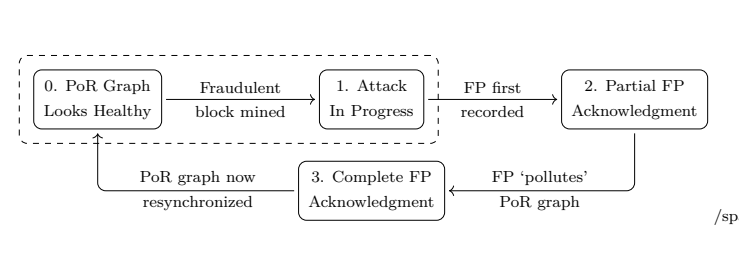
The defining quality of a healthy simplex is that additions to the PoR graph are highly interconnected. When new blocks are mined, they reflect all possible remote blocks and link back to all possible local blocks. Thus, there should be no partitions or asymmetric boundaries.

An */spanemasymmetric boundary/span/em* emerges when a group of miners contributes */spanemexclusively/span/em* to a */spanemsubgraph/span/em* of the full PoR graph (i.e., the subgraph that censors the fraud proof). The asymmetric boundary lies between the full (honest) PoR graph and the dishonest subgraph.

When an invalid block is mined, individual nodes of the network should rapidly become aware of the fraud proof — and we expect this if the P2P layer is working correctly. However, during an ongoing (successful) attack, that fraud proof is not recorded in the PoR graph. The fraud is ‘invisible’ when considering just the blockchain data itself, which means that SPV proofs and the like may be fraudulent.

When fraud is detected, the PoR graph should proceed through multiple phase changes until normality is restored (fig. /spana href="whitepaper.html#lateximage-whitepaper-63"32/span/a).

/spanfigure id="autoid-43" class="figure "


0. PoR Graph Looks Healthy
Fraudulent block mined
1. Attack In Progress
FP first recorded
2. Partial FP Acknowledgment
FP 'pollutes' PoR graph
3. Complete FP Acknowledgment
PoR graph now resynchronized

/spandiv class="center" /span/div

/spandiv class="figurecaption"Figure 32: The phase transitions of a PoR graph when fraud occurs. The dashed outline around phase 0 and 1 indicates that it's not possible to observe this phase change on-chain. Nodes connected to the P2P network should be aware of the fraud, but no chain has yet included the fraud proof in the PoR graph.

/span/div /span/figure

An attacker always fails if the network reaches phase 3. The attacker's primary goal is to prevent the network transitioning from phase 1 to phase 2.

The dynamics of the attack change for both the attacker and each miner during each phase.

For example, there is a qualitative difference between phase 1 and phase 2, and it is in the attacker's interest to prevent the network transitioning to phase 2 if possible. Therefore it is most valuable for the attacker if they can prevent the FP being recorded. It is also more profitable for miners, since the size of bribes grow quadratically.

Once the network has reached phase 2, it is only a matter of time before the network reaches phase 3, at which point we are back at the start.

If the network is kept in phase 1, then the PoR graph and (nearly all) chain data is indistinguishable from a healthy simplex.

```
<!--...............................-->
```

/spanspan class="paragraph" id="autosec-354"/spanspan class="sectionnumber"4.11.5.1&#x2003;/span/spanPhase 1 $\not\to Phase2ontheTargetChain/span/span$

The first honest block on the target chain must, by definition, link back to the fraudulent block as an invalid ancestor. Let's assume that there's a reward (C$_r H)forpublishingthefraudproof, whereC_ristheblockrewardofthetargetchain, andHistheproportionoftheblockrewardeachminergets.Theadditionalrewardforthisblockafterkattackingblocksis C_r Hk.Thus, afterH^{-1}blocks, the/spaneffective/span/emblockrewardforanhonestminerwillhavedoubled.$

A new honest block thus becomes heavily incentivized over time.

To mitigate this, the attacker must raise the difficulty on the target chain such that all other potential miners find it comparatively unprofitable.

Compared to mining honestly, the attacker forgoes the reward of C$_r Hkforthek^{\rm th}block.Thatrewardislosttoexcessiveworksecuringthetargetchainwhichwouldotherwiseearntheattackerthatmuchrevenuefrompublishingthefraudproof.Additionally, theattackerlosestheblockrewardseachtime---sotheselossesareC_r(1+Hk)perblockafterkblocks.LetC_r \cdot l_mrepresentthetotallossesduetomining, andzthenumberofblocksfortheattack: factoringoutl_minthiswayallowsustoanalyzethesituationindependentoftheblockreward.Inorderforthelossesl_mtobegreaterthantherewards, wemusthave:$
$/spanspanclass="hidden"\(\setminus seteqnumber\{0\}\{\}\{34\}\)/span/span$
<!--

$$\begin{aligned} l_m &> \sum_{k=0}^{z} \Bigl(1 + Hk\Bigr) \\ &= z + 1 + H \sum_{k=0}^{z} k \\ &= z + 1 + H \frac{z(z+1)}{2} \end{aligned} \tag{35}$$

```
-->

\begin{align}
l_m &amp;&gt; \sum \limits _{k=0}^z \Bigl ( 1 + H k \Bigr ) \nonumber \\ &amp;= z + 1 + H \sum \limits _{k=0}^z k \nonumber \\ &amp;= z + 1 + H \frac {z (z + 1)}{2} \label {eq:spv-atk-losses-mining}
\end{align}
```

Additionally, the rise in difficulty of these blocks presents a practical limit for the attacker. When k > q H$^{-1} N_1 theattackeris, /spanembydefinition/span/em, unabletokeepuptheattack.Thatisbecause, atthatpoint, theattacker'slossesareC_r(1 + H(qH^{-1}N_1)) = C_r(1 + qN_1).Assuminganevenhash-ratedistribution, C_r N_1(p+q)representsthe/spanemsum/span/emof/spanemall/span/emblockrewardsoverthenetwork.Therefore, themaximumoutput(measuredincoins)ofqproportionofthehash-rate, /spanembydefinition/span/em, isC_r N_1 q.Anattackermusthaveadditionalsupporttomaintainthisportionoftheattack---whentheminininglossesperblock(approxC_r Hk)exceedtheattacker'sminingcapacity(C_r N_1 q)./spanspanclass="hidden"\(\setminus seteqnumber\{0\}\{\}\{35\}\)/span/span$
<!--

$$\begin{aligned} C_r H z &< C_r N_1 q \\ z &< q H^{-1} N_1 && H \ge {N_1}^{-1} \\ z &< q {N_1}^2 && \text{(worst case)} \end{aligned}$$

```
-->

\begin{align}
\text {\hspace {8em}} &amp;&amp; C_r H z &amp;&lt; C_r N_1 q \nonumber \\ &amp;&amp; z &amp;&lt; q H^{-1} N_1 &amp; H \ge {N_1}^{-1} \nonumber \\ &amp;&amp; z &amp;&lt; q {N_1}^2 &amp; \text {(worst case)} \nonumber
\end{align}
```

If N$_1 = 100, thennoattackisviable(withouthelp)after5000blocks, unlessq > 0.5.Atthatpoint, atleast50\%ofthenetworkhash-rateneedstobededicatedtothetargetchaintomakeitunprofitableenoughtodissuadehonestminers.$

```
<!--.................................-->
```
/spanspan class="paragraph" id="autosec-355"/spanspan class="sectionnumber"4.11.5.2&#x2003;/span/spanPhase 1 $\not\to Phase2and\mathbb{B}(z)/span/span$

To prevent the network transitioning from phase 1 to phase 2, the attacker incurs a cost. We can define this cost $/spanemin\ terms\ of\ the\ block\ reward/span/em.$ Let the total cost to the attacker, in the k$^{\text{th}}round,beC_rZ_kcoins.(Z_kisthecostinblockrewards.)$

/spanul style="list-style-type:none"

/spanli
- Each miner (on each simplex-chain) needs bribing (but only if they make a block).

/span/li
/spanli
- Each round, the attacker must pay C$_rZ_kinbribes(whereZ_kisthebribeasamultipleoftheblockreward,andkistheround,startingat1).$

/span/li
/spanli
- These bribes are given to (N$_1-1)miners(wesubtract1toexcludethetargetchainfromthissplit).OnebribeperR-chain.$

/span/li
/spanli
- However, an attacker with q proportion of the hash-rate can recover q(N$_1-1)ofthesebribes,atmost,andthuslosesatleastp(N_1-1)ofthem.$

/span/li
/spanli
- A miner receives C$_rHkcoinsasarewardforpublishingafraudproofthatinvalidateskblocks.$

/span/li
/spanli
- Each round, miners can choose to publish the fraud proof, which earns them C$_rHkinrewards.$

/span/li
/spanli
- Thus, each round, miners have an opportunity cost of C$_rHkiftheydo/spanemnot/span/emtakethebribe.$

/span/li
/spanli
- This only makes rational sense if the miner's bribe is greater than their opportunity cost: C$_rZ_k/(N_1-1)>C_rHk;/spanemand/span/emtheminer is/spanemguaranteed/span/emtoreceivethebribe,regardlessofwhethertheattacksucceeds.$

/span/li
/spanli
- The attack lasts z rounds.

/span/li
/spanli
- Let C$_r\cdot l_bbethetotallossesduetobribes.$

/span/li
/span/ul

Therefore, for the bribes of round k to be more than the rewards on all chains, we must have: /spanspan class="hidden" \(\seteqnumber{0}{}{35}\)/span/span
```
<!--
```

$$Z_k > kH(N_1-1)$$

```
-->

\begin{align}
Z_k &amp;&gt; k H (N_1-1) \nonumber
\end{align}
```
Let l$_b=p(Z_1+\cdots+Z_z)denotetheexpectedcostoftheattackforthezrounds.Substitutingintheabove:/spanspanclass="hidden"\(\seteqnumber{0}{}{35}\)/span/span$
<!--

$$\begin{aligned} l_b &> pH(N_1-1)\sum_{k=1}^{z} k \\ l_b &> pH(N_1-1)\frac{z(z+1)}{2} \end{aligned} \tag{36}$$

```
-->

\begin{align}
l_b &amp;&gt; p H (N_1-1) \sum \limits _{k=1}^z k \nonumber \\ l_b &amp;&gt; p H (N_1-1) \frac {z (z+1)}{2} \label {eq:spv-atk-losses-bribes}
\end{align}
```

Let $\mathbb{B}$(z) denote the minimum cost of the attack for z rounds, given our current understanding. Combining eq. /spanspan class="textup"(/spana href="whitepaper.html#lateximage-whitepaper-63"35/span/a)/span/span with eq. /spanspan class="textup"(/spana href="whitepaper.html#lateximage-whitepaper-63"36/span/a)/span/span, and noting that p+q=1 yields:

/spanspan class="hidden" \(\seteqnumber{0}{}{36}\)/span/span `<!--`

$$\mathbb{B}(z) = l_b + (p+q)l_m > H(N_1p+q)\frac{z(z+1)}{2} + z + 1 \tag{37}$$

```
-->
\begin{equation}
\BribeCost (z)= l_b + (p+q)l_m &gt; H (N_1 p + q) \frac {z(z+1)}{2} + z + 1 \label {eq:spv-atk-losses-total}
\end{equation}
```

Observe that this is a lower bound on $\mathbb{B}$(z). To reduce complexity, we'll also define $\mathbb{B}$*/spanemin terms of the block reward/span/em* (i.e., in units of */spanemblock rewards/span/em* rather than */spanemcoins/span/em*). We can come back to it later if we need to.
/spanspan class="hidden" \(\seteqnumber{0}{}{37}\)/span/span
```
<!--
```

$$\mathbb{B}(z) = \eta\frac{z(z+1)}{2} + z + 1 \qquad \text{where } \eta = H(N_1p+q) \tag{38}$$

```
-->

\begin{align}
\BribeCost (z) &amp;= \eta \frac {z(z+1)}{2} + z + 1 &amp; &amp; \text {where } \eta = H(N_1 p + q) \label {eq:spv-bribe-cost-1}
\end{align}
```
We should also note the inverse of $\mathbb{B}$, which gives us a lower bound on the number of blocks for which we should wait, given a total outgoing value, v (expressed as a multiple of the block reward).   /spanspan class="hidden" \(\seteqnumber{0}{}{38}\)/span/span
```
<!--
```

$$\mathbb{B}^{-1}(v) = \frac{\sqrt{(\eta-2)^2+8\eta v}-\eta-2}{2\eta} \qquad \text{where } \eta = H(N_1p+q) \tag{39}$$

```
-->

\begin{align}
\BribeCost ^{-1}(v) &amp;= \frac {\sqrt { (\eta - 2)^2 + 8 \eta v } - \eta - 2}{2\eta } &amp; &amp; \text {where } \eta = H(N_1 p + q) \label {eq:spv-bribe-cost-inverse}
\end{align}
```

<!--................................-->
/spanspan class="paragraph" id="autosec-356"/spanspan class="sectionnumber"4.11.5.3&#x2003;/span/spanPhase 1 → *Phase2/span/span*

The network transitions from phase 1 to phase 2 as soon as a first miner produces a block that includes the fraud proof.

Once this happens, an asymmetry emerges: honest blocks (which acknowledge the fraud proof) produce proofs of reflection that are */spanemnot/span/em* usable by the dishonest miners. If the dishonest miners tried to use those PoRs, they'd produce invalid blocks unless they also acknowledge the fraud proof. Thus, using those PoRs effectively turns dishonest miners into honest miners.

```
<!--...............................-->
```

/spanspan class="paragraph" id="autosec-357"/spanspan class="sectionnumber"4.11.5.4&#x2003;/span/spanPhase 2 → *Phase3/span/span*

The duration of this transition depends on the proportion of honest miners: h. Let's define h via h + d = 1, where d is the proportion of dishonest miners, including the attacker./spansup106/span/sup Since d includes the attacker, we can say d > q, and h < p.

Similar to section /spana href="whitepaper.html#lateximage-whitepaper-63"4.8/span/a, the asymmetry between honest and dishonest miners means that, even for a small h, the chain-weight of the honest chain-tips will eventually dominate the chain-weight of dishonest chain-tips. After that point, each simplex-chain's main chain will include the fraud proof, and the dishonest chain-tips will never be able to out-weigh the honest ones.

Let us use a vector to represent the weight of the honest and dishonest chain segments (relative to their most recent common ancestor). This is purely for convenience so that we can analyze both at the same time. The top row is the weight of the honest chain segment, and the second row is the weight of the dishonest chain segment. At the start of the attack (t₀), $neitherchainsegmenthasanyblocks, sot_0 = \vec{0}$.

Each round (block period), we expect that h proportion of new blocks are honest, and d proportion are dishonest. So, after 1 block period, we expect:

/spanspan class="hidden" \(\seteqnumber{0}{}{39}\)/span/span `<!--`

$$t_1 = t_0 + \begin{bmatrix} N_1 h \\ N_1 d \end{bmatrix} = N_1 \begin{bmatrix} h \\ d \end{bmatrix}$$

```
-->
\begin{equation*}
t_1 = t_0 + \vecXY {N_1 h}{N_1 d} = N_1 \vecXY {h}{d}
\end{equation*}
```

After 2 block periods, the h proportion of new blocks that are honest can use PoRs via the dishonest blocks. Due to the asymmetry, the dishonest blocks can not.

/spanspan class="hidden" \(\seteqnumber{0}{}{39}\)/span/span `<!--`

$$t_2 = t_1 + \begin{bmatrix} N_1(h+d) \\ N_1 d \end{bmatrix} = N_1 \begin{bmatrix} h \\ d \end{bmatrix} + N_1 \begin{bmatrix} 1 \\ d \end{bmatrix} = N_1 \begin{bmatrix} (h+1) \\ 2d \end{bmatrix}$$

```
-->
\begin{equation*}
t_2 = t_1 + \vecXY {N_1 (h+d)}{N_1 d} = N_1 \vecXY {h}{d} + N_1 \vecXY {1}{d} = N_1 \vecXY {(h + 1)}{2 d}
\end{equation*}
```

Each round, the honest chain segments grow by the weight of N₁$blocks, butthedishonestchainsegmentsgrowbyN_1dblocks.Therefore, atthek^{\text{th}}round, k \geq 2 : /spanspanclass = "hidden"\backslash(\backslash seteqnumber\{0\}\{\}\{39\}\backslash)/span/span$
<!--

$$\begin{aligned} t_k &= t_{k-1} + N_1 \begin{bmatrix} 1 \\ d \end{bmatrix} = t_{k-2} + 2N_1 \begin{bmatrix} 1 \\ d \end{bmatrix} \\ \text{observe: } t_k &= t_{k-i} + iN_1 \begin{bmatrix} 1 \\ d \end{bmatrix} && \text{for some } i \in [1,k) \\ \text{set } i = k-1 \implies t_k &= t_1 + N_1(k-1) \begin{bmatrix} 1 \\ d \end{bmatrix} \\ t_k &= N_1 \begin{bmatrix} h \\ d \end{bmatrix} + N_1(k-1) \begin{bmatrix} 1 \\ d \end{bmatrix} \\ \therefore t_k &= N_1 \begin{bmatrix} h+k-1 \\ dk \end{bmatrix} \end{aligned} \tag{40}$$

```
-->

\begin{align}
t_k &amp;= t_{k-1} + N_1 \vecXY {1}{d} = t_{k-2} + 2 N_1 \vecXY {1}{d} \nonumber \\ \text {observe:  } t_k &amp;= t_{k-i} + i N_1 \vecXY {1}{d} &amp; \text {for some } i \in [1, k) \nonumber \\ \text {set } i = k - 1 \implies t_k &amp;=
t_1 + N_1 (k-1) \vecXY {1}{d} \nonumber \\ t_k &amp;= N_1 \vecXY {h}{d} + N_1 (k-1) \vecXY {1}{d} \nonumber \\ \therefore t_k &amp;= N_1 \vecXY {h + k - 1}{dk} \label {eq:h-to-d-chain-segs}
\end{align}
```

Thus, the honest chain-tips dominate when /spanspan class="hidden" \(\seteqnumber{0}{}{40}\)/span/span
`<!--`

$$\begin{aligned} h + k - 1 &> dk \\ k - dk &> 1 - h \\ k(1-d) &> 1 - h \\ hk &> 1 - h \\ k &> h^{-1} - 1 \end{aligned}$$

```
-->

\begin{align*}
h + k - 1 &amp;&gt; dk \\ k - dk &amp;&gt; 1 - h \\ k(1 - d) &amp;&gt; 1-h \\ h k &amp;&gt; 1-h \\ k &amp;&gt; h^{-1} - 1
\end{align*}
```

If h = /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"N₁$/span/span$/span/span, i.e., for each honest miner there are (N₁ − 1)$dishonestminers, thenthisinequalitybecomes : k > N_1 - 1$.

If only 1% of miners refuse the bribe (h = 0.01), and that the other 99% of miners continue to mine only blocks that do not include the fraud proof, then we have: k > 99.

If we assume an honest minority of substantial size, h = /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"3/span/span/span/span, then k > 2. That is, the honest chain-tips are expected to dominate after 2 block periods if /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"3/span/span/span/span of miners are honest.

/spandiv role="note" class="footnotes"
/spanp
/spansup106/span/sup&nbsp; Dishonest miners are those which will accept a bribe and participate in the fraud proof censorship attack, but are not necessarily controlled by the attacker directly. We assume that there are some miners that would accept a bribe for *this/span/em* attack, but not for *any/span/em* attack.
/span/p

/span/div

<!--................................-->
/spanh5 id="autosec-359"/spanspan class="sectionnumber"4.12&#x2003;/span/spanExpedited Transactions/span/h5

In Section /spana href="whitepaper.html#lateximage-whitepaper-63"4.7/span/a we saw that UT's confirmation rate was O(c), and that the expected confirmation */spanemtime/span/em* was O(c^{-1})*.Thisishowlongauserwaitsforatransactiontogofrom/spanemnewly/span/emto/spanemfully/span/emconfirmed.*

In those calculations, we did */spanemnot/span/em* account for the time */spanembetween/span/em* the initial */spanembroadcast/span/em* of a transaction and its */spaneminclusion/span/em* in a block. To account for the duration of the whole the transaction's journey, we need to consider: broadcast latency (ϕ)*,theblockperiodofthelocalchain,andtheconfirmationrateofthesimplexasawhole.*

`\[ \text {TimeToXConfirmations}(x) = \phi + \frac {1}{2 B_f} + \frac {x-1}{N_1 B_f} \]`

This process, end-to-end, is thus O($\phi + 1 + c^{-1}$) $= O(1) > O(c^{-1})$*.Canwedobetter?*

/spanemIdeally/span/em, we'd want the */spanemtime to first confirmation/span/em* (TTFC) to be as short as possible — currently it is the bottleneck, after all. Naively, our first idea might be to use cross-chain transactions somehow: there are $\sim$N$_1$*chainsthatarepotentially/spanemsending/span/emtransactions,sothatcouldprovidethenecessarycoefficienttobringdowntheTTFC.Thisintuitiondoeshavesomemerit :* *we′regoingtoneedtousethe/spanemotherchains/span/emifwewantatransactiontobe‘confirmed′/spanemduringtheperiodbetweenblocks/span/emofwhicheverchaintheuserisoperatingon.However,cross −* *chaintransactionsarewayslowerthanwe′reseekingastheyalreadywaitforsomenumberofconfirmations.*

We must also consider a */spanemcrucial difference/span/em* between local or cross-chain transactions and whatever solution we come up with: /spanspan class="uline"whether a transaction is spendable is only certain */spanemin the context of a block/span/em.*/span/span Transactions without contexts are just some signed data. Even with a spendability proof, */spanemthat proof is against a pre-existing block/span/em.* Therefore, if there */spanemwere/span/em* some way to confirm a transaction after the last block and before the next, */spanemwe cannot tell if the transaction is valid or not/span/em.*

Thus, whatever the involvement of other simplex-chains is, those chains cannot know */spanemfor sure/span/em* that a foreign transaction is */spanemguaranteed/span/em* to execute successfully. Requiring this would constitute */spanemforeign validation/span/em* which would mean an O(c^2)*validationload,sowe/spanemcannot/span/emhavesucharequirement.However,thereisacorollaryhere :* */spanspanclass = ”uline”expeditedtransactions/spanemdonotdepend/span/emonthe/spanemvalidity/span/emofforeignblocks/span/spanandarethusnotinvalidatedifafraudproofisproducedfortheforeignblockatsomelaterpoint.*

<!--...............................-->
/spanh6 id="autosec-360"/spanspan class="sectionnumber"4.12.1&#x2003;/span/spanProtocol Design/span/h6

This is enough to start sketching out expedited transactions. /spanul style="list-style-type:none"

/spanli
1. Rblocks must be able to include (specially flagged) Ltransactions.
/span/li
/spanli
2. Lminers must now scan all */spanemrelevant/span/em* Rblocks for expedited Ltransactions.
/span/li
/spanli
3. Lminers must then */spaneminclude and execute/span/em* those expedited transactions, and record whether each was successful or not. Duplicates are dropped.
/span/li
/span/ul

This is a rudimentary protocol and we will need to improve it before it can be used. For example: what if an Rminer includes a great many invalid Ltransactions? That would cause the Lminer to waste valuable block space, reducing overall revenue.

DoS and Griefing We will prevent any significant impact */spanemto miners/span/em* with an architecture that limits intentional DoS or griefing attacks.

The architectural change is simple: /spanul style="list-style-type:none"

/spanli
- A block is */spaneminvalid/span/em* if it includes more than one/spansup107/span/sup expedited transaction for any foreign chain.

/span/li
/span/ul

Additionally, we can note that to support expedited transactions, a miner must scan foreign blocks. We already do this for cross-chain transactions, and miners already have the blocks due to the /spana href="whitepaper.html#lateximage-whitepaper-63"Axiom of Availability/span/a. There is no need for miners to include the raw transaction */spanemagain/span/em* if it has already been recorded. Therefore, an Lminer only needs to reference this transaction by hash (similar to off-main transactions). This creates */spanema new asymmetry/span/em* which protects miners from griefing attacks: the attacking miner must use more block space than the victim.

Incentives and Fees A different kind of griefing is financial and might even happen without any malice: low-fee transactions. Miners normally have control over which transactions they include in their blocks and from this we get a functional fee market. However, if an Rminer can include an expedited low-fee Ltransaction, and especially if this happens on many Rchains at once, then Lminers are at the mercy of whatever transactions are chosen */spanemfor/span/em* them.

To resolve this, we will */spanemrequire/span/em*: /spanul style="list-style-type:none"

/spanli
- Expedited transactions must pay an */spanemequal/span/em* fee to */spanemboth/span/em* miners.

/span/li
/span/ul

Additionally, we should note that */spanemthe fee is dependent on the successful execution of the transaction/span/em*. The foreign miner is therefore incentivized to consider only transactions with */spanemcurrent/span/em* spendability proofs — the standard of proof is the most recent Lblock. (Since we are considering */spanemexpedited/span/em* transactions particularly, we tolerate more stringent requirements than we'd require for simply relaying transactions.)

Miscellaneous Since high-fee expedited transactions are likely, it's natural to ask whether miners of that chain can include them. There's no reason to forbid this, and it further increases that miner's rewards, so we will allow it.

If a chain is */spanemreally/span/em* popular, then it's conceivable that almost all of the transactions are expedited. If it's possible for a maximum block size to be exceeded with */spanemtoo many/span/em* expedited transactions then we'd have a deadlock on that chain. Therefore, when blocks are over the block size limit we need a special rule to allow such blocks.

To mitigate these issues, we will modify the protocol: /spanul style="list-style-type:none"

/spanli
- Blocks can include any number of expedited transactions as on-main for the local chain.

/span/li
/spanli
- Blocks are only invalidated by the block size limit if they include on-main transactions other than the coinbase.

/span/li
/span/ul

Synchronization Caveats We should note that synchronization of a single chain must change a little to work with expedited transactions. Particularly, along with the block itself, we will need some auxiliary data (the expedited transactions that were recorded in other chains). In terms of complexity, this will not be a problematic increase: we have at most $\mathrm{N}_1 expeditedtransactionsperblock, and O(N_1) = O(c). Practically, thiscanmeansynchronizingLchainnodesmustdownloadupto{\sim}5\times asmuchforperiodsofhightransactionvolume(we'llderivethisshortly). On the flipside, expeditedtransactionscanbeprunedfromtheRchainthattheywereincludedin, theverifyingRnodeonlyneedstoknowenoughtorecalculatetheblockrootcorrectly.$

The LMiner's Perspective A good feature increases the value of the network and it is in the rational self-interest of network participants to adopt it. Therefore, we can judge this feature from a miner's perspective to make sure it is preferable.

Miner's are concerned about making and re-making blocks. They want to be able to create them regularly and keep them up to date with minimal computational effort. Since we require (for fraud proofs) that the */spanemoutput state/span/em* from each transaction is recorded in the block, any insertion of a transaction means re-executing transactions after the point of insertion to calculate new state roots. For simple transactions this is not too bad, but more complex transactions might have non-negligible overhead.

In general, we'd like the frequency of changes to be lowest for the first transactions and increase as we go. Ideally the way state is calculated should account for this so that miners don't unnecessarily have to recalculate state.

In terms of the transactions themselves, the cost of including an expedited transaction is two hashes: the transaction ID and the resulting state root. Since transactions are typically 250 bytes or more, we can estimate a $5\times reductioninblocksizeusedforasimilartransactionfee. Althoughnewexpeditedtransactionscanarrivewithanyforeignblock(whichhappensfrequently), noteveryforeignblockwillhaveone, andtheywon'tchangeorder(oratleastnotoften)whensubsequentnewonesarrive. Thatsoundsgoodprovidedthattheoverheadismanageable.$

Additionally, having the option to include expedited transactions for foreign chains is welcome. They'll have a decent fee and don't require recalculating any state. That said, a miner might miss out on some fees if they include an expedited transaction that some other miner already included in a simultaneous block. Choosing semi-randomly might help avoid collisions, but in the meantime the severity of this and efficacy of mitigation remains to be seen.

Overall, this is */spanemwin–win/span/em* provided that expedited transactions are not too complex, the overhead of recalculating state is not too high, and an intelligent strategy can be found for which foreign expedited transactions to include.

/spandiv role="note" class="footnotes"
/spanp

/spansup107/span/sup&nbsp;We could set this limit higher, but let’s stick with one for now.

/span/p

/span/div

`<!--................................-->`
/spanh6 id="autosec-362"/spanspan class="sectionnumber"4.12.2&#x2003;/span/spanSecurity of Confirmations/span/h6

Expedited transactions now have /spanspan class="uline"multiple stages/span/span of confirmation: first on the foreign chain, then on the local chain when they are actually executed. The first stage guarantees that the transaction will be executed, but not that it will succeed. Even a spendability proof against the prior *L*block is not sufficient since expedited transactions consume inputs, and we don't know which transactions will be processed */spanembefore/span/em* the expedited transaction is executed.

/spandiv
    class="tcolorbox"
    style=" border: 1px solid #404040; background: #ECECEC; "

/spandiv
    class="tcolorboxupper"
    style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbAside/span/b**/span/span This kind of confirmation is not unprecedented; off-main transactions are in a similar superposition-like state before they are executed in an on-main block. /span/div /span/div

Thus, we are left with the question: */spanemAre these first-stage confirmations equivalent to second-stage confirmations?/span/em* One test is against double-spends: if they are easier to double-spend, then they are not equivalent.

Double-Spend Resistance If the attacker controls the inputs, then an attack depends on their ability to execute a conflicting transaction before the one in question. Therefore, */spanemif/span/em* expedited transactions are */spanemsimultaneous or after/span/em /spanemon-main/span/em* transactions they /spanspan class="uline"would be vulnerable/span/span. However, if expedited transactions are */spanembefore/span/em* on-main transactions, then the inputs would have to be spent another way. Off-main transactions are executed before on-main ones, but off-main parents are always near-simultaneous with their siblings (due to the /spana href="whitepaper.html#lateximage-whitepaper-63"Axiom of Unified Ancestry/span/a) — we can therefore trivially calculate any conflicts./spansup108/span/sup Cross-chain transactions for a block are determined well in advance due to the delays built into the protocol, so any conflict here will be detectable and we can predict the outcome./spansup109/span/sup Other expedited transactions might conflict too, but we already know which those are (or will in short order), so again we can predict the outcome. There is one other candidate that we haven't thought about */spanemat all/span/em* so far: some kind of automated transaction that is executed at the start of a block. However, since this would be entirely determined by the prior block, any outputs removed at this stage are already well known. These are all of the cases for transactions that may execute prior to the expedited transaction, and none allow for an advantage to the attacker provided we are watching for it. Therefore, expedited transactions are /spanspan class="uline"not vulnerable/span/span to double-spends, */spanemprovided/span/em* that they are executed */spanembefore/span/em* on-main transactions.

/spandiv role="note" class="footnotes"
/spanp
/spansup108/span/sup&nbsp; An attacker might create a conflicting off-main block after the expedited transaction is first confirmed. This could spend the expedited transaction's inputs; to mitigate this vector, we require that expedited transactions are executed *before/span/em* any off-main transactions.
/span/p

/spanp
/spansup109/span/sup&nbsp; Cross-chain transactions are technically outputs rather than transactions, so they never fail or conflict. However, it would be useful to send cross-chain transactions that can *try/span/em* to spend some inputs to new outputs. To ensure that such an output always executes successfully, such an output must include another output that is used if any of the inputs are not present or are invalid. The behavior of this output type, in english, is: try to spend the given inputs using these signatures; if this succeeds, insert the first set of outputs; otherwise, insert the second set of outputs. Care should be taken to ensure conservation of tokens depending on the branch taken.
/span/p

/span/div
/spandiv
    class="tcolorbox"
    style=" border: 1px solid #404040; background: #ECECEC; "

/spandiv
    class="tcolorboxupper"
    style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbAside/span/b**/span/span As sometimes happens when design work and development are simultaneous, some of the planning document may be written with the hindsight of solutions already implemented. The idea of automatic */spanemsystem/span/em* transactions is an example — this has already been implemented in the Amaroo client. That's why I know they are deterministic and depend only on the parent block. The reason that they are at the start of a block is that they might contain heavy state operations that are inconvenient to do at the end. Having them at the start gives the same */spanemresult/span/em* as if they were at the end of the prior block, but, this way, the associated state calculations only ever have to be done once. /span/div
/span/div

When Confirmations Differ The primary difference between the first and second stage of confirmation is that the first stage is */spanemnot/span/em* guaranteed to succeed.

Therefore, if we are to be */spanemabsolutely certain/span/em* that a transaction will succeed, then we */spanemshould/span/em* discriminate. Particularly, we need to wait for a local confirmation to actually process the block at minimum. At that point, however, we can */spanemin hindsight/span/em* treat the confirmations as equivalent since they are double-spend resistant.

If the transaction is less significant (e.g., paying for groceries) then we don't need to worry too much. Such transactions are more confirmed than zero-confirmation transactions, and less than it will be after the next block.

Light-clients are a special case: if a local node is not available to validate blocks, then it is possible that we have an invalid block somewhere and we don't yet know about the fraud proof. In these cases, we need to wait as long as the cross-chain protocol waits to ensure that no fraud proof exists — about 60 seconds in practice. After this, the confirmations of both stages can be treated as normal.

<!--................................-->
/spanspan class="paragraph" id="autosec-364"/spanspan class="sectionnumber"4.12.2.1&#x2003;/span/spanTransaction Execution Order/span/span

For clarity, here is the execution order of transactions in a block: /spanul style="list-style-type:none"

/spanli
1. Automated System Transactions (deterministic with on-main parent)
/span/li
/spanli
2. Cross-chain Transactions
/span/li
/spanli
3. Expedited Transactions
/span/li
/spanli
4. Off-main Transactions
/span/li
/spanli
5. On-main Transactions
/span/li
/span/ul

`<!--................................-->`
/spanh6 id="autosec-365"/spanspan class="sectionnumber"4.12.3&#x2003;/span/spanC' and Time to First Confirmation/span/h6

We can now calculate the time to first confirmation (TTFC) for expedited transactions.

The TTFC is the time it takes for an expedited transaction (with an appropriate fee) to be broadcast and confirmed on any foreign chain (of which there are N_1).*/spanspanclass = "hidden"\(\seteqnumber{0}{}{40}\)/span/span*
<!--

$$\begin{aligned}\text{TTFC} &= \phi + \frac{1}{N_1 B_f}\\ O(\text{TTFC}) &= O(c^{-1}) + O(c^{-1}) = O(c^{-1})\end{aligned}$$

`-->`

```
\begin{align*}
\text {TTFC} &amp;= \phi + \frac {1}{N_1 B_f} \\ O(\text {TTFC}) &amp;= O(c^{-1}) + O(c^{-1}) = O(c^{-1})
\end{align*}
```
For a 150-simplex with 15 second block periods, TTFC ≈ 100 ms.*Thisisontheorderof*ϕ*andaboutascloseto‘instant’aswecanget.*

We can now see that C' is accurate */spanemfrom the /spanspan class="uline"moment/span/span an expedited transaction is broadcast/span/em.* For normal on-main transactions, C' is accurate */spanemafter/span/em* the first confirmation.

Additionally, for expedited transactions:

\[`\text {TimeToXConfirmations}(x) = \phi + \frac {x}{N_1 B_f}` \]

`<!--................................-->`
/spanh6 id="autosec-366"/spanspan class="sectionnumber"4.12.4&#x2003;/span/spanEffect on Chain-Capacity/span/h6

There is a /spanem*significant/span/em* effect of expedited transactions that must be explored: the /spanem*effective capacity/span/em* of a chain is increased with demand, and the /spanem*overall/span/em* network capacity is slightly decreased.

Let us consider an exceptionally popular chain, L, with a growing demand for transactions.

To start with, the chain is at 100% utilization and fees are roughly equivalent to any other chain. As fee pressure increases, the first breakpoint for expedited transactions will be hit: the fee is more than
$2\times thelowestfeeofanyotherchain.Atthispoint,itisprofitableforthatRminertoincludetheexpeditedtransactionoverlocaltransactions.Additionally,transactionswiththisfeeorhighershouldbecountedastakingupabout/spanspanclass = "textrm\ textmd\ textup\ textulc"/spanspanclass = "numerator"1/span/span//spanspanclass = "denominator"5/sp$

The next major breakpoint to consider is when the average fee passes $2\times theaveragelowestfeeover/spanemall/span/emotherchains.Whenthishappens,theminersofhalfofallsimplex-$
$chainscanincreasetheirtotalfeeintakebyincludingexpeditedLtransactions.Onthe(perhapsarbitrary)assumptionthat50\%ofallLblockspaceisusedbyexpeditedtransactions,wehavetraded50\%ofournaivecapacity(fornormalon-$
$maintransactions)foranequivalent250\%ofournaivecapacityintheformofexpeditedtransactions.Theeffectivecapacityistherefore300\%ofthenaivecapacity.$

As the fee pressure increases further, other chains will become saturated with expedited transactions and there might be no space left for normal on-main transactions. This will barely register for other chains since only one expedited Ltransaction can be included in any
Rblock. Let's also assume there are no duplicate expedited transactions. Since, under these conditions, we expect $\sim N_1 expeditedtransactionsperLblock,wecanuse\textbf{??}tosay: /spanspanclass = "hidden"\backslash(\backslash seteqnumber\{0\}\{\}\{40\}\backslash)/span/span$
`<!--`

$$\text{ExpTxsPerBlock}_{\text{max}} = N_1 = \frac{k_1}{2B_f B_h} \tag{41}$$

`-->`

```
\begin{align}
\text {ExpTxsPerBlock}_\text {max} &amp;= N_1 = \frac {k_1}{2 B_f B_h} \label {eq:expedited-txs-per-block-max}
\end{align}
```

For our typical lower-end parameters ($k_1 = 3000, B_f = /spanspanclass = "textrm\ textmd\ textup\ textulc"/spanspanclass = "numerator"1/span/span//spanspanclass = "denominator"15/span/span/span/span, B_h = 112) : \text{ExpTxsPerBlock}_{\text{max}} \approx$
$200.Bycomparison,ablockwouldtypicallycontainaboutk_1(2B_f\cdot \text{Tx}_{\text{avg}})^{-1}\approx$
$90transactions.Undertheseparameters,wecan'tevenuseallof the available block space!The200expeditedtransactionsconsumethespaceof\sim 40normaltransactions,sooverallwecouldexpectaround250transactionsperblockatmost(80\%ofwhichwouldbeexpedited).$

This is a little bit of a surprise (that the maximum increase is so low), but if we look to eq. /spanspan class="textup"(/spana href="whitepaper.html#lateximage-whitepaper-63"41/span/a)/span/span we can observe a symmetry with the formula for naive capacity:

`\[ \text {NormTxsPerBlock}_\text {max} = \frac {k_1}{2 B_f \cdot \text {Tx}_\text {avg}} \]`

In effect, we are capped based on the ratio of $\text{Tx}_{\text{avg}} to B_h.Oneconsequenceisthat,atleastinsomecases,wecanprobablyrelaxtheone-per-chainrequirement.$

Alternate case: $\textbf{UT}_{1+\textbf{HOT}}$ What if $\text{B}_h weremuchsmaller,though?Ifwe/spanemeffectively/span/emhaveB_h \le g(thesizeofahash),thenwehave:$

`\[ \text {ExpTxsPerBlock}_\text {max} \ge \frac {k_1}{2 B_f g} \approx 700 \]`

/spandiv
  class="tcolorbox"
  style=" border: 1px solid #404040; background: #ECECEC; "

/spandiv
  class="tcolorboxupper"
  style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbAside/span/b**/span/spanAhh, that's more like it. /span/div /span/div

This will use the equivalent block space of approximately 140 normal transactions – more than we could otherwise handle. This will increase the block size beyond its normal limit, but not by that much. In this case, /spanspan
class="textrm textmd textup textulc"/spanspan class="numerator"140/span/span//spanspan class="denominator"90/span/span/span/span $\approx 1.56, andsincethisisonlyconsideringthetransaction-halfoftheblock,thecompleteblockisonly\sim 1.28\times largerthannormal.$

Even in the most extreme case, this is clearly still within O(c) block-size limits. That said, a /spanem*synchronizing/span/em* node will need to download around /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"(700 + 140 +
90)/span/span//spanspan class="denominator"(90 ×2)/span/span/span/span $\approx 5\times moredatatosynchronizethechainoversuchperiods.Ifweareinasituationmorelikethisthanthepreviousone,weshouldkeeptheone-per-chainrequirement.$

/spandiv
  class="tcolorbox"
  style=" border: 1px solid #404040; background: #ECECEC; "

/spandiv
  class="tcolorboxupper"
  style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbAside/span/b**/span/span In practice, these formulas are a rough estimate and should be taken as such. The main use of this analysis is to set ballpark expectations and plan accordingly. Real-world figures will be heavily dependent on a variety of implementation details. /span/div /span/div

The take-away from all this is that expedited transactions provide a /spanspan class="uline"/spanem*capacity buffer/span/em*/span/span for a chain when demand is high, and have the effect of */spanem/spanspan class="uline"distributing fee expenditure and transaction load/span/span/span/em* over the simplex.

.7ex]0.5pt/spanspan style="width:3.4445pt; display:inline-block"`<!---->`/span/spanBegin DRAFT/spanspan style="width:3.4445pt; display:inline-block"`<!---->`/span/span.7ex]0.5pt

.7ex]0.5pt/spanspan style="width:3.4445pt; display:inline-block"`<!---->`/span/spanEnd DRAFT/spanspan style="width:3.4445pt; display:inline-block"`<!---->`/span/span.7ex]0.5pt

`<!--.................................-->`
/spanh5 id="autosec-367"/spanspan class="sectionnumber"4.13&#x2003;/span/spanInitial Configuration/span/h5

The current plan for the initial configuration of UT and the Amaroo network:

/spanul style="list-style-type:none"

/spanli
- **/spanbBlock Size:/span/b** $\mathrm{k}_1 \in [3000, 6000] B/s.Anexactvaluewillbechosenclosertogenesis.$

/span/li
/spanli
- **/spanbBlock Frequencies:/span/b** $\mathrm{B}_f = /spanspanclass = "textrm\ textmd\ textup\ textulc"/spanspanclass = "numerator"1/span/span//spanspanclass = "denominator"15/span/span/span/spanHzforallsimplex - chains.$

/span/li
/spanli
- **/spanbBlock Rewards:/span/b** $\mathrm{B}_r = /spanspanclass = "textrm\ textmd\ textup\ textulc"/spanspanclass = "numerator"/spanspanclass = "textsc"BaseReward/span/span/span/span//spanspanclass = "denominator"N_1/span/span/span/spancoins.Thatis : accordingtotheissuanceschedule(whichdetermines/spanspanclass = "textsc"BaseReward/span/span), eachblock(fromanysimplex - chain)providesaproportionalshareofthebasereward.$

/span/li
/spanli
- **/spanbConvertible Context:/span/b** Single Root Token (SRT; section /spana href="whitepaper.html#lateximage-whitepaper-63"2.4.1/span/a).

/span/li
/spanli
- **/spanbDifficulty Adjustment:/span/b** /SPANSPAN CLASS="TEXTSC"DAA-2/SPAN/SPAN as in /spana href="https://web.archive.org/web/20211018043918/https://cloudflare-ipfs.com/ipfs/Qmd8BE6xYCH58LNipE1zZ7BCftemN8hQWnfZJSJYq5XUE8" target="_blank" An Economic Analysis of Difficulty Adjustment Algorithms in Proof-of-Work Blockchain Systems (Noda, Okumura, Hashimoto; 2020)/span/a, modified to work for block-DAGs. The exact window size $(\mathrm{DAA}_N)isTBDbut\mathrm{DAA}_N = 100(\sim 25minutes)satisfiesourrequirements.section\ /spanahref = "\mathrm{whitepaper.html}\#lateximage - whitepaper - 63"4.10.4/span/ahasimportantcontributions.$

/span/li
/spanli
- **/spanbPoW Algorithms:/span/b** TBD, but the goals are for diversity in the design of mining rig hardware, avoiding imbalance due to the current distributions of mining hardware, attracting existing under-utilized hardware, and providing */spanemmultiple/span/em* different paths for technology development. Additionally, PoW algorithms */spanemwill/span/em* be used by multiple simplex-chains (miner resonance would not happen, otherwise). One of the PoW algorithms will be deliberately compatible with Bitcoin's double-SHA256 and Bitcoin ASICs.

/span/li
/spanli
- **/spanbUT$_1$ Consensus Primitives:/span/b** PoW + PoR; multiple hashing algorithms.

/span/li
/spanli
- **/spanbUT Variant:/span/b** Conservative version of +HOT. Capacity is somewhere between +HO and +HOT; the current truncation method is lossless.

/span/li
/spanli
- **/spanbUT$_1$ Transaction Capabilities:/span/b** UTXOs by default; secondary EUTXO and EVM/WASM subsystems. Special systems will allow for interoperability between subsystems.

/span/li
/spanli
- **/spanbSimplex-Chains:/span/b** Mostly homogenous in architecture, though each simplex-chain can differ in these ways: the PoW algorithm, which external (non-Amaroo) chains are imaged, and the specific integrations with those external chains.

/span/li
/spanli
- **/spanbIntra-Simplex Cross-Chain Security:/span/b** $\lambda = 4, whichcorrespondsto \sim 60seconds.$

/span/li
/spanli
- **/spanbUT$_2$ and Dapp-Chains:/span/b** Limited support initially with a focus on PoA dapp-chains; secure and universal dapp-chains will require further development. Constrained by and dependent on further developments of suitable PoS + PoR consensus methods.

/span/li
/spanli
- **/spanbSimplex Size:/span/b** For reasons discussed in section /spana href="whitepaper.html#lateximage-whitepaper-63"6/span/a, the initial limit on the number of simplex-chains will be 50% of maximal $\mathrm{N}_1.Thiscorrespondsto \sim 75\%ofmaximalTPScapacity.Thesimplexwillstartwithfewerchainsatgenesis, withmoreaddedlater.Genesisestimate : between15and30simplex - chains(\Sigma\ \mathrm{TPS} \in [150, 700]).$

/span/li
/span/ul

.7ex]0.5pt/spanspan style="width:3.4445pt; display:inline-block"`<!---->`/span/spanBegin DRAFT/spanspan style="width:3.4445pt; display:inline-block"`<!---->`/span/span.7ex]0.5pt

```
<!--................................-->
/spanh5 id="autosec-368"/spanspan class="sectionnumber"4.14&#x2003;/span/spanReflection: Incentive and Censorship/span/h5



/spandiv
       class="tcolorbox"
       style=" border: 1px solid #404040; background: #FCD1E9; "

/spandiv
       class="tcolorboxtitle"
       style=" border-top: 1px solid #404040; border-bottom: 1px solid #404040; background: #404040; color: #FFFFFF; "
TODO [12]/span/div
/spandiv
       class="tcolorboxupper"
       style=" color: #000000; "
  is a refl censorship attack possible? meaningful? explore. (NB: I don't think there's a viable strategy here, which is why I haven't prioritized writing this out.)   /span/div /span/div


/spandiv
       class="tcolorbox"
       style=" border: 1px solid #404040; background: #FCD1E9; "

/spandiv
       class="tcolorboxtitle"
       style=" border-top: 1px solid #404040; border-bottom: 1px solid #404040; background: #404040; color: #FFFFFF; "
TODO [13]/span/div
/spandiv
       class="tcolorboxupper"
       style=" color: #000000; "
   Add a nash equilibrium diagram + explanation to show that it's always in the interest of miners to publish headers — intuition: including headers means that the /spanemother chain's miner/span/em has an incentive to include your header. that means that the next miner (on your chain) will be able to build on a heavier chain if they reflect that other chain's next header — so that next miner (on the local chain) has an incentive to include that other chain's next header. If the original miner (who might chose not to publish the most recent header of that other chain) censors that reflection, then they disadvantage themselves relative to their competitors (other miners of that simplex-chain). Thus, it's never helpful to a miner to censor reflections (esp if we enforce the limit on
k_bandk_tx).Itdoesn'thelphonestminers,anditmakesanattackerschain−segmentlesscompetitive./span/div /span/div

Does a miner ever benefit from withholding reflections?

.7ex]0.5pt/spanspan style="width:3.4445pt; display:inline-block"<!---->/span/spanEnd DRAFT/spanspan style="width:3.4445pt; display:inline-block"<!---->/span/span.7ex]0.5pt

.7ex]0.5pt/spanspan style="width:3.4445pt; display:inline-block"<!---->/span/spanBegin DRAFT/spanspan style="width:3.4445pt; display:inline-block"<!---->/span/span.7ex]0.5pt


/spandiv
       class="tcolorbox"
       style=" border: 1px solid #BC842C; background: #FEE8F4; "

/spandiv
       class="tcolorboxtitle"
       style=" border-top: 1px solid #BC842C; border-bottom: 1px solid #BC842C; background: #BC842C; color: #FFFFFF; "
[TODO: [2] &#x2003;&#x2003;/spanb(MED)/span/b/span/div
/spandiv
       class="tcolorboxupper"
       style=" color: #000000; "
 (DraftOnly) Summary of practical considerations (section /spana href="whitepaper.html#lateximage-whitepaper-63"4/span/a) /span/div /span/div

.7ex]0.5pt/spanspan style="width:3.4445pt; display:inline-block"<!---->/span/spanEnd DRAFT/spanspan style="width:3.4445pt; display:inline-block"<!---->/span/span.7ex]0.5pt
```

<!--...............................-->
/spanh4 id="autosec-369"/spanspan class="sectionnumber"5&#x2003;/span/spanScaling Complexity Analysis of */spanemUltra Terminum/span/em*/span/h4

UT has two primary methods of scaling: horizontally via mutual PoR (simplex-chains), and vertically via one-way PoR (dapp-chains). Horizontal scaling via PoR is novel. Dapp-chains are similar to many of the sharding and pseudo-sharding ideas proposed for other networks (Polkadot, Eth2, etc), though there are fewer restrictions on dapp-chains in UT compared to other designs. Additionally, dapp-chains in UT are secured by the entire simplex. In the case of PoS dapp-chains, this provides */spanemadditional/span/em* security compared to ‘naked’ PoS chains. Hosting dapp-chains on many simplex-chains also provides greater system-wide maximum capacity than a network built upon a single base-chain.

A common method of sharding is to */spanemnest/span/em* blockchains. For example, Ethereum 2 has */spanemThe Beacon Chain/span/em* — its root-chain (the single base-chain of a network).

/spandiv
class="tcolorbox"
style=" border: 1px solid #404040; background: #ECECEC; "

/spandiv
class="tcolorboxupper"
style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbQuote/span/b**/span/span /spanspan role="note" class="marginpar"**/spanbQuote/span/b**/span/span The Beacon Chain will conduct or coordinate the expanded network of shards and stakers. But it won’t be like the Ethereum mainnet of today. It can’t handle accounts or smart contracts.

.7ex]0.5pt

/spandiv class="flushright" */spani— /spana href="https://web.archive.org/web/20210225022359/https://ethereum.org/en/eth2/beacon-chain/" target="_blank" The Beacon Chain | ethereum.org/span/a (2021)/span/i* /span/div /span/div /span/div

This type of configuration, where a base-chain facilitates child-chains, is referred to as */spanemnesting/span/em* in this section and in the context of UT's architecture and complexity. Base-chains are at the first level of nesting. The shards of Ethereum 2 are */spanema level of nesting/span/em* above the Beacon Chain, i.e., nesting level 2. UT's dapp-chains are also at nesting level 2.

/spandiv
class="tcolorbox"
style=" border: 1px solid #404040; background: #F2F2F2; "

/spandiv
class="tcolorboxupper"
style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbTerm/span/b**/span/span **/spanb/spana href="whitepaper.html#??"Base-chain/span/a/span/b**: A chain that has no parent-chains; i.e., is at the base nesting level. /span/div /span/div

Sometimes (but not always) people use terms like */spanemlayer 2/span/em* to describe this sort of nesting, though such usage of */spanemlayer 2/span/em* is ambiguous and potentially misleading. It easily confuses nesting with off-chain scaling methods (such as payment channels, rollups, or ephemeral 'child' blockchains, e.g., Plasma), and it potentially misleads readers about the security properties of nested blockchains. Nested blockchains */spanemcan/span/em* faithfully inherit the security properties of their parent-chains, which is not the case for layer 2 solutions prior to finalization.

Furthermore, terms like */spanemlayer x/span/em* cannot accurately describe UT's design. Consider a PoS dapp-chain on UT. Would that dapp-chain be */spanemlayer 1/span/em* or */spanemlayer 2/span/em*? It would be misleading to call it */spanemlayer 2/span/em* whilst */spanemdirectly comparable/span/em* chains (like Ethereum 2, Polkadot, or Cardano) are called */spanemlayer 1/span/em*. Such PoS UT dapp-chains have */spanemall/span/em* the security qualities of an equivalent stand-alone PoS chains, */spanemand more/span/em*. If they were called */spanemlayer 1/span/em* chains, then what is the simplex — */spanemlayer 0/span/em*? It is clear that the common idea behind */spanemlayer 1/2/span/em* scaling does not have sufficient capacity to accurately describe UT's simplex- and dapp-chains; it is inadequate.

`<!--...............................-->`
/spanh5 id="autosec-371"/spanspan class="sectionnumber"5.1&#x2003;/span/spanAnalysis Methodology/span/h5

The following derivations focus on */spanemthroughput/span/em* of particular blockchain designs and scaling configurations. These derivations will let us evaluate the complexity of each design.

Raw throughput of a network, $\mathrm{T}_i, ismeasuredinbytes/sec(B/s)forsomelevelofnesting, i.T_idirectlycorrespondstoadesign'smaximumtransactionspersecond(\mathrm{TPS}_i), where\mathrm{Tx}_{\mathrm{avg}}istheaveragesizeofatransaction, via:$

/spanspan class="hidden" \(\seteqnumber{0}{}{41}\)/span/span `<!--`

`-->`
`\begin{equation*}`

`\end{equation*}`

The raw B/s throughput of a chain at the $\mathrm{i}^{\mathrm{th}}levelofnestingisdenotedbyk_i.NotethatT_iisa/spanemcalculated/span/emvalue, butk_iisa/spanemparameter/span/emthatmaybechosen.Anincreasetok_iiseffectivelyanincreaseinmaximumblocksize.$

We will also derive relationships between the maximum number of chains at a level of nesting, $\mathrm{N}_i, andthemaximumnetworkthroughputatthatlevelofnesting, T_i.Formostexistingblockchaindesigns, N_1 = 1.$

With regards to simplexes, we are particularly concerned with the complexity of a */spanemmaximal/span/em* simplex — i.e., the simplex with the highest TPS possible.

/spandiv
    class="tcolorbox"
    style=" border: 1px solid #404040; background: #F2F2F2; "

/spandiv
    class="tcolorboxupper"
    style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbTerm/span/b**/span/span **/spanb/spana href="whitepaper.html#??"Maximal Simplex/span/a/span/b**: A simplex with the maximum TPS under given O(c) constraints. /span/div /span/div

Additionally, $\mathrm{O}(\mathrm{k}_i) is/spanemdefined/span/emasO(k_i) \equiv O(c).ThisisreasonableprovidedtherearenoO(c)bottlenecks, e.g., networkbandwidth, CPUthroughput, memoryrequirements, etc(section\ /spanahref = $"whitepaper.html#sec:a-principle-of-scaling"1.1.2$/span/a).$

<!--...............................-->
/spanh5 id="autosec-372"/spanspan class="sectionnumber"5.2&#x2003;/span/spanComplexity of O(c) Chains/span/h5

Example: Bitcoin.

The */spanemraw throughput/span/em*, $\mathrm{k}_1, can be calculated for existing chains (e.g., Bitcoin) via the product of the maximum block size, B_{\max} (in bytes), and the block production frequency, B_f (in hertz, or s^{-1})$:

/spanspan class="hidden" \(\seteqnumber{0}{}{41}\)/span/span <!--

-->
\begin{equation*}

\end{equation*}

The */spanemthroughput/span/em*, $\mathrm{T}_1, of an O(c) chain is equivalent to its raw throughput$:

/spanspan class="hidden" \(\seteqnumber{0}{}{41}\)/span/span <!--

-->
\begin{equation*}

\end{equation*}

The complexity order of the network is given by $\mathrm{O}(\mathrm{T}_1) = O(k_1) = O(c) as expected.$

Care should be taken to account for protocol extensions like */spanemSegregated Witness/span/em* that effectively reduce the size of transactions (in SegWit's case, by ∼/spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"4/span/span/span/span).

For Bitcoin — given $\mathrm{k}_1 \approx 1700 B/s, and transaction size \mathrm{Tx}_{\mathrm{avg}} = 500 \cdot /spanspanclass = "textrm textmd textup textulc"/spanspanclass = "numerator"3/span/span//spanspanclass = "denominator"4/span/span/span/spanB --- the maximum TPS is given by$:

/spanspan class="hidden" \(\seteqnumber{0}{}{41}\)/span/span <!--

-->
\begin{equation*}

\end{equation*}

This is what we expect based on the measured real-world performance of Bitcoin.

`<!--................................-->`
/spanh5 id="autosec-373"/spanspan class="sectionnumber"5.3&#x2003;/span/spanOptimistic Complexity of O(c^2)$Chains/span/h5$

Examples: Ethereum 2, Polkadot.

Suppose the root-chain has a throughput of
$\mathrm{k}_1B/sanditcansupportuptoN_2nestedchains.ThosenestedchainshaveheadersofD_hbytesthatareproducedatafrequencyofD_f(s^{-1}).If/spanemall/span/emheadersofnestedchainsarerecordedinthehostchain,theneachnestedchainconsumes/spanematleast/span/emD_f\cdot D_hB/softheroot-chain'scapacity.$

Thus, N$_2isgivenby$:

/spanspan class="hidden" \(\seteqnumber{0}{}{41}\)/span/span `<!--`

(42)

`-->`
`\begin{equation}`

`\end{equation}`

NB: For blockchains of this design: N$_1$ = 1.

If each nested chain has a throughput capacity of k$_2B/s, then$:

/spanspan class="hidden" \(\seteqnumber{0}{}{42}\)/span/span `<!--`

(43)

`-->`
`\begin{equation}`

`\end{equation}`

Thus O(T$_2$) = $O(c^2)asexpected.$

`<!--................................-->`
/spanh6 id="autosec-374"/spanspan class="sectionnumber"5.3.1&#x2003;/span/spanEffective Header Size/span/h6

It's typical, though, that the headers of nested chains, alone, are not sufficient: additional data is required. When such data is required to be recorded on-chain (i.e., it cannot be deterministically regenerated), then the */spanemeffective/span/em* header size is the size of the raw header, plus the size of any auxiliary data.

/spandiv
    class="tcolorbox"
    style=" border: 1px solid #404040; background: #ECECEC; "

/spandiv
    class="tcolorboxupper"
    style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbAside/span/b**/span/span Note that, for the calculations in this paper, 'Ethereum 2' means the sharded beacon chain design as written in 2021, not the rollup-centric /spana href="https://web.archive.org/web/20250105115555/https://ethereum.org/en/roadmap/danksharding/" target="_blank" danksharding/span/a approach being pursued as of January 2025. /span/div /span/div

For example, in an */spanemEthereum 2/span/em* beacon block, each shard has a header size of 280 B, but there is additional overhead. A reasonable lower-bound is that each header has an */spanemeffective/span/em* minimum header size of 312 B./spansup110/span/sup

In the case of */spanemPolkadot/span/em*, it is /spana href="https://github.com/AmarooHQ/polkadot-effective-dh/blob/5cd0f0d21ff1cd3c57d1c2af70aaf6d8ee19dc11/main.js" target="_blank" measurable/span/a/spansup111/span/sup that a typical minimum of 819 B is used in the `/spankbdparaInclusion.candidateBacked/span/kbd` extrinsic (i.e., the transaction type that records parachain headers). So, a lower-bound on the effective header size of a parachain is 819 B (this does not include */spanembitfields/span/em*/spansup112/span/sup).

In those situations, with regards to these capacity derivations, one can use the */spanemeffective/span/em* header size as a replacement for the */spanemraw/span/em* header size.

/spandiv role="note" class="footnotes"
/spanp
/spansup110/span/sup&nbsp; As of late September 2021, the Ethereum 2 /spana href="https://github.com/ethereum/consensus-specs/blob/296f9bab81566e2a11dd0ce3de806ff191e926bb/specs/sharding/beacon-chain.md\#beaconblockbody" target="_blank" sharding spec/span/a has capacity for 2:1 attestations to shards per block (with 64 shards), but only 32 B of each attestation is dedicated to sharding. The spec also has capacity for 4:1 shard headers to shards per block. It seems reasonable that capacity which exists will be used within reason. Thus a reasonable lower-bound for the effective header-size of shards is taken via:
$1 \times headerspershardperblock, 1 \times attestationspershardperblock(whichdonotcounttowardseffectiveheader - size), and 1 \times 32Bperattestationperblock.Shardshaveheadersof 280B, sotheminimumeffectiveheadersizeistakentobe 312B.(note:$
$arequireddependencyofthereferencedshardingspecisthe/spanahref = "https://github.com/ethereum/consensus - specs/blob/296f9bab81566e2a11dd0ce3de806ff191e926bb/specs/merge/beacon - chain.md\#beaconblockbody" target = "\_blank" October 2021 mergespec/span/aand/spanahref = "https://github.com/ethereum/consensus - sp$
/span/p

/spanp
/spansup111/span/sup&nbsp; Whilst some parachain headers exist that are smaller than 819 B, it's not really significant for this analysis (a reduction of 10% wouldn't change much). We're already ignoring bitfields, and 819 B seems optimistic if we're interested in the *spanemaverage/span/em* parachain header size, since many are larger. All-in-all, I guess that 819 B is a bit generous, and (ideally) all claims about existing chains in this paper err on the side of generosity.
/span/p

/spanp
/spansup112/span/sup&nbsp; Bitfields is a Polkadot term — it's a list of hundreds of signatures, totalling > 14 kB per block on the current Kusama testnet (October 3rd2021).
/span/p

/span/div

```
<!--...............................-->
```
/spanh5 id="autosec-376"/spanspan class="sectionnumber"5.4&#x2003;/span/spanComplexity of UT$_1$/span/h5

There is no single root-chain for a collection of mutually reflecting blockchains (i.e., a simplex), so $N_1 \neq 1. WhatisN_1then?Inasimplex, eachchainhask_1B/scapacity, butthisissplitbetweenreflectionsandtransactions. Atthisfoundationallevel(wherethereisnonestingyet), headersareB_hbyteswithafrequencyofB_fHz. ThereareN_1simplex-chains.$

For the purpose of section /spana href="whitepaper.html#lateximage-whitepaper-63"5/span/a we will generally /spanemnot/span/em consider the impact of /spanemexplicitly/span/em including PoRs along with block headers (i.e., the +PoRs UT variants). The methods we use here are easily generalized to account for those variants, and associated analysis can be found in **??**. Unless otherwise stated, section /spana href="whitepaper.html#lateximage-whitepaper-63"5/span/a analyzes the UT$_{+OP}$ variant.

Reflecting a single simplex-chain requires $B_f \cdot B_hB/sofcapacity, andeachsimplex-chainmustreflectN_1 - 1 \approx N_1othersimplex-chains. Thismeansthatasimplex-chainmustreserveN_1 \cdot B_f \cdot B_hB/sofitscapacityforreflections, denotedbyk_{1,B} = N_1 \cdot B_f \cdot B_h. Additionally, simplex-chainsmustreservesomecapacityfortransactions, k_{1,tx}.$

Since simplex-chains must split their capacity between reflections and transactions, set:

/spanspan class="hidden" \(\seteqnumber{0}{}{43}\)/span/span `<!--`

$$\begin{aligned} k_1 &= k_{1,tx} + k_{1,B} \\ k_{1,tx} &= k_1 - k_{1,B} \end{aligned} \tag{44}$$

```
-->
\begin{equation}
\begin{split} \label {eq:k1-reflection-defn} k_1 &amp; = k_{1,tx} + k_{1,B} \\ k_{1,tx} &amp; = k_1 - k_{1,B} \end {split}
\end{equation}
```

Given $k_{1,B} = N_1 \cdot B_f \cdot B_h$:

/spanspan class="hidden" \(\seteqnumber{0}{}{44}\)/span/span `<!--`

(45)

```
-->
\begin{equation}

\end{equation}
```

Since each simplex-chain reserves $k_{1,tx}B/sfortransactions, thetotalthroughputreservedfortransactionswillbeN_1 \cdot k_{1,tx}. Thus: /spanspanclass = "hidden"\(\seteqnumber{0}{}{45}\)/span/span$
<!--

$$\begin{aligned} T_1 &= N_1 \cdot k_{1,tx} && (46) \\ &= N_1(k_1 - N_1 \cdot B_f \cdot B_h) \\ &= N_1 \cdot k_1 - {N_1}^2 \cdot B_f \cdot B_h && (47) \end{aligned}$$

```
-->

\begin{align}
T_1 &amp; = N_1 \cdot k_{1,tx} \label {eq:reflection-t1-start} \\ &amp; = N_1(k_1 - N_1 \cdot B_f \cdot B_h) \notag \\ &amp; = N_1 \cdot k_1 - {N_1}^2 \cdot B_f \cdot B_h \label {eq:reflection-t1-in-terms-of-n1}
\end{align}
```

The optimal number of simplex-chains will maximize throughput. We can find that maxima via:

/spanspan class="hidden" \(\seteqnumber{0}{}{47}\)/span/span `<!--`

$$\frac{dT_1}{dN_1} = k_1 - 2 \cdot N_1 \cdot B_f \cdot B_h$$

```
-->
\begin{equation*}
\begin{split} \frac {dT_1}{dN_1} &amp; = k_1 - 2 \cdot N_1 \cdot B_f \cdot B_h \end {split}
\end{equation*}
```

At dT$_1 \frac{}{dN_1=0:}$

/spanspan class="hidden" \(\seteqnumber{0}{}{47}\)/span/span `<!--`

(48)

```
-->
\begin{equation}

\end{equation}
```

Thus $O(N_1) = O(k_1) = O(c)$.

From eq. /spanspan class="textup"(/spana href="whitepaper.html#lateximage-whitepaper-63"47/span/a)/span/span and substituting $N_1 from$ **??** :

/spanspan class="hidden" \(\seteqnumber{0}{}{48}\)/span/span `<!--`

(49)

```
-->
\begin{equation}

\end{equation}
```

Thus $O(T_1) = O({k_1}^2) = O(c^2)$.

What are $k_{1,B}andk_{1,tx}intermsofk_1? Fromeq. /spanspanclass = "textup"(/spanahref = $"whitepaper.html$\#lateximage-whitepaper-63"46/span/a)/span/spanand$ **??** :

/spanspan class="hidden" \(\seteqnumber{0}{}{49}\)/span/span `<!--`

```
-->
\begin{equation*}

\end{equation*}
```

Substituting $N_1 from$ **??** $gives$:

/spanspan class="hidden" \(\seteqnumber{0}{}{49}\)/span/span `<!--`

```
-->
\begin{equation*}

\end{equation*}
```

Thus, from the definition of $k_1 ineq. /spanspanclass = "textup"(/spanahref = $"whitepaper.html$\#lateximage-whitepaper-63"44/span/a)/span/span, wefindthatmaximalthroughputrequireshalfoftheblocktobeusedforreflections$:

/spanspan class="hidden" \(\seteqnumber{0}{}{49}\)/span/span `<!--`

```
-->
\begin{equation*}

\end{equation*}
```

<!--................................-->
/spanh5 id="autosec-377"/spanspan class="sectionnumber"5.5&#x2003;/span/spanDapp-Chains and the Complexity of UT$_2$ and UT$_3$/span/h5

<!--................................-->
/spanh6 id="autosec-378"/spanspan class="sectionnumber"5.5.1&#x2003;/span/spanDapp-Chains/span/h6

If a system supports nested chains, then we can say that for some throughput, T$_i, atnestingleveli, the(i+1)^{th} nestinglevelcansupportN_{i+1}nestedchainsvia$:

/spanspan class="hidden" \(\seteqnumber{0}{}{49}\)/span/span <!--

(50)

```
-->
\begin{equation}

\end{equation}
```

Therefore, via the same logic used for **??**:

/spanspan class="hidden" \(\seteqnumber{0}{}{50}\)/span/span <!--

$$T_{i+1} = T_i \cdot \frac{k_{i+1}}{D_f \cdot D_h} \tag{51}$$

```
-->
\begin{equation}
\label {eq:throughput-iter} T_{i+1} = T_i \cdot \frac {k_{i+1}}{D_f \cdot D_h}
\end{equation}
```

Note that this relationship only holds for the traditional sharding model of securing sharded chains via their inclusion in a parent-chain, e.g., UT's dapp-chains (UT$_2$) and dapp-dapp-chains (UT$_3$), or existing O(c$^2)designs$.

Combining these yields:

/spanspan class="hidden" \(\seteqnumber{0}{}{51}\)/span/span <!--

$$N_{i+1} = \frac{T_{i+1}}{k_{i+1}} \tag{52}$$

```
-->
\begin{equation}
\label {eq:simple-scaling} N_{i+1} = \frac {T_{i+1}}{k_{i+1}}
\end{equation}
```

```
<!--..................................-->
```
/spanh6 id="autosec-379"/spanspan class="sectionnumber"5.5.2&#x2003;/span/spanUT with Dapp-Chains (UT$_2$)/span/h6

Starting with **??** and building on eq. /spanspan class="textup"(/spana href="whitepaper.html#lateximage-whitepaper-63"51/span/a)/span/span:

/spanspan class="hidden" \(\seteqnumber{0}{}{52}\)/span/span `<!--`

$$\begin{aligned} T_1 &= \frac{{k_1}^2}{4 \cdot B_f \cdot B_h} \\ \therefore T_2 &= \frac{{k_1}^2 \cdot k_2}{4 \cdot B_f \cdot B_h \cdot D_f \cdot D_h} \end{aligned} \tag{53}$$

```
-->
\begin{equation}
\begin{split} \label {eq:throughput-c-3} T_1 &amp; = \frac {{k_1}^2}{4 \cdot B_f \cdot B_h} \\ \therefore T_2 &amp; = \frac {{k_1}^2 \cdot k_2}{4 \cdot B_f \cdot B_h \cdot D_f \cdot D_h} \end {split}
\end{equation}
```

Thus O(T$_2$) $= O(c^3)$.

The maximum number of dapp-chains is given by:

/spanspan class="hidden" \(\seteqnumber{0}{}{53}\)/span/span `<!--`

$$\begin{aligned} N_2 &= \frac{T_2}{k_2} \\ &= \frac{{k_1}^2}{4 \cdot B_f \cdot B_h \cdot D_f \cdot D_h} \end{aligned}$$

```
-->
\begin{equation*}
\begin{split} N_2 &amp; = \frac {T_2}{k_2} \\ &amp; = \frac {{k_1}^2}{4 \cdot B_f \cdot B_h \cdot D_f \cdot D_h} \end {split}
\end{equation*}
```

When D$_h = B_h and D_f = B_f, note that N_2 = {N_1}^2$.

```
<!--...............................-->
```
/spanh6 id="autosec-380"/spanspan class="sectionnumber"5.5.3&#x2003;/span/spanUT with Dapp-Dapp-Chains (UT$_3$)/span/h6

If we say each dapp-chain hosts shards or more dapp-chains (e.g., as a dapp-chain version of Eth2 or Polkadot would), then via eq. /spanspan class="textup"(/spana href="whitepaper.html#lateximage-whitepaper-63"51/span/a)/span/span and eq. /spanspan class="textup"(/spana href="whitepaper.html#lateximage-whitepaper-63"53/span/a)/span/span,

/spanspan class="hidden" \(\seteqnumber{0}{}{53}\)/span/span `<!--`

$$\begin{split} T_3 &= \frac{T_2}{D_f \cdot D_h} \cdot k_3 \\ &= \frac{{k_1}^2 \cdot k_2 \cdot k_3}{4 \cdot B_f \cdot B_h \cdot {D_f}^2 \cdot {D_h}^2} \end{split} \tag{54}$$

```
-->
\begin{equation}
\label {eq:throughput-c-4} \begin{split} T_3 &amp; = \frac {T_2}{D_f \cdot D_h} \cdot k_3 \\ &amp; = \frac {{k_1}^2 \cdot k_2 \cdot k_3}{4 \cdot B_f \cdot B_h \cdot {D_f}^2 \cdot {D_h}^2} \end {split}
\end{equation}
```

Thus O(T$_3$) = $O(c^4)$.

Note that the derivation of T$_3 presumesthattheparameters D_f and D_h arethesameforbothdapp-chainsanddapp-dapp-chains.$

Via eq. /spanspan class="textup"(/spana href="whitepaper.html#lateximage-whitepaper-63"52/span/a)/span/span and eq. /spanspan class="textup"(/spana href="whitepaper.html#lateximage-whitepaper-63"54/span/a)/span/span:

/spanspan class="hidden" \(\seteqnumber{0}{}{54}\)/span/span `<!--`

$$\begin{split} N_3 &= \frac{T_3}{k_3} \\ &= \frac{{k_1}^2 \cdot k_2}{4 \cdot B_f \cdot B_h \cdot {D_h}^2 \cdot {D_f}^2} \end{split}$$

```
-->
\begin{equation*}
\begin{split} N_3 &amp; = \frac {T_3}{k_3} \\ &amp; = \frac {{k_1}^2 \cdot k_2}{4 \cdot B_f \cdot B_h \cdot {D_h}^2 \cdot {D_f}^2} \end {split}
\end{equation*}
```

```
<!--................................-->
/spanh5 id="autosec-381"/spanspan class="sectionnumber"5.6&#x2003;/span/spanComplexity of Cross-Chain SPV Proofs &amp; Proofs of Reflection/span/h5
```

```
<!--................................-->
```

/spanh6 id="autosec-382"/spanspan class="sectionnumber"5.6.1&#x2003;/span/spanCross-Chain SPV Proofs/span/h6

Each chain — at full capacity — operates with order O(c) by definition. Thus its state has order O(c) also. The size of SPV proofs scale logarithmically with the set you're proving membership of, e.g., the number of transactions, or size of the chain's state, etc. Thus, SPV proofs scale with order O($\log_2 c$).

For a given O(c^j)$; j \in \{2,3,4\} configuration of UT (i.e., \mathrm{UT}_1, \mathrm{UT}_2, \mathrm{UT}_3), a chain can process SPV proofs of state on another chain. For j = 4, the furthest that a transaction can occur from its host simplex-chain is in the 3rd level of nesting (i.e., a dapp-dapp-chain). It would require j-1 SPV proofs to ``ascend'' from the host simplex-chain to a dapp-dapp-chain. However, given that full nodes of a dapp-dapp-chain are required to be full nodes of both the host dapp-chain and the host simplex-chain, transactions in that dapp-dapp-chain do not need to provide SPV proofs of state in either of those host chains --- full nodes already have those details. That is: transactions which ``descend'' the levels of nesting can do so with O(1) cost. SPV proofs are only required when transactions ``ascend'' the levels of nesting to other simplex-, dapp-, or dapp-dapp-chains.$

Thus, the maximum number of SPV proofs required to prove state anywhere in a UT simplex is j.

Since j is constant, cross-chain SPV proofs therefore have order:

/spanspan class="hidden" \(\seteqnumber{0}{}{54}\)/span/span `<!--`

$$O(j \cdot \log_2 c) = O(\log_2 c) \tag{55}$$

```
-->
\begin{equation}
O(j \cdot \log _2 c) = O(\log _2 c) \label {eq:spv-complexity}
\end{equation}
```

<!--................................-->
/spanh6 id="autosec-383"/spanspan class="sectionnumber"5.6.2&#x2003;/span/spanProofs of Reflection/span/h6

A simplex-chain reflects $\mathrm{N}_1 - 1 \approx N_1 othersimplex - chains. AmerkletreeofreflectedheadershasorderO(N_1) = O(k_1) = O(c) andacorrespondingproofsizeoforderO(\log_2 N_1) = O(\log_2 k_1) = O(\log_2 c). Sincethoseothersimplex - chainsalsohave{\sim}N_1 reflections, provingreflectioninthoseother{\sim}N_1 simplex - chainsrequires{\sim}N_1 merklebranches. Thus, thefullsetofreflectionproofs, persimplex - chain, isO(N_1 \cdot \log_2 N_1) = O(c \cdot \log_2 c)$.

Note: In a production system, these proofs can be excluded from blocks by treating them as droppable witnesses; see section /spana href="whitepaper.html#lateximage-whitepaper-63"4.2/span/a.

`<!--................................-->`
/spanh5 id="autosec-384"/spanspan class="sectionnumber"5.7&#x2003;/span/spanTPS Complexity Comparison/span/h5

k: raw per-chain throughput (bytes/s)/spanbr /
$\mathrm{B}_f : simplexblockfrequency(s^{-1})/spanbr/$
$B_h : simplexblockheadersize(bytes)/spanbr/$
$D_f = B_f : dapp - chainblockfrequency(s^{-1})/spanbr/$
$D_h = B_h : dapp - chainblockheadersize(bytes)/spanbr/$

NB: For the purposes of table /spana href="whitepaper.html#lateximage-whitepaper-63"6/span/a and on, the average transaction size is taken to be 250 bytes.

/spanfigure id="autoid-44" class="table "

/spandiv class="center"

/spandiv class="figurecaption"Table 6: A comparison of the maximum theoretical transaction throughput (transactions per second; TPS) with parameters k, B$_f$, $B_h forO(c), ShardedO(c^2)andthedifferent\mathrm{UT}_{+\mathrm{OP}}scalingconfigurations.Notethatthe/spanemShardedO(c^2)/span/emcolumnisthetheoreticaloptimallimitforshardedsystemswhereallheade$
chain.

/span/div

/spantable/spantr class="tbrule"
/spantd class="tdl"k, B$_f$, $B_h/span/td$
$/spantdclass = ”tdr”O(c)/span/td$
$/spantdclass = ”tdr”ShardedO(c^2)/span/td$
$/spantdclass = ”tdr”\mathrm{UT}_{1+\mathrm{OP}}TPS/span/td$
$/spantdclass = ”tdr”\mathrm{UT}_{2+\mathrm{OP}}TPS/span/td$
$/spantdclass = ”tdr”\mathrm{UT}_{3+\mathrm{OP}}TPS/span/td$
$/span/tr$

3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 112/span/td
/spantd class="tdr"12/span/td
/spantd class="tdr"4,821/span/td
/spantd class="tdr"1,205/span/td
/spantd class="tdr"484,295/span/td
/spantd class="tdr"1.95×10$^8/span/td$
$/span/tr$

3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 84/span/td
/spantd class="td"12/span/td
/spantd class="td "6,428/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "1,607/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "860,969/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "4.61×10$^8/span/td$
$/spantdclass = ”tdat”/span/td$
$/spantdclass = ”tdbang”/span/td$
$/span/tr$

3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"30/span/span/span/span, 112/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "12/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "9,642/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "2,410/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "1,937,181/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "1.56×10$^9/span/td$
$/spantdclass = ”tdat”/span/td$
$/spantdclass = ”tdbang”/span/td$
$/span/tr$

3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"30/span/span/span/span, 84/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "12/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "12,857/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "3,214/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "3,443,877/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "3.69×10$^9/span/td$
$/spantdclass = ”tdat”/span/td$
$/spantdclass = ”tdbang”/span/td$
$/span/tr$

3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"60/span/span/span/span, 112/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "12/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "19,285/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "4,821/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "7,748,724/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "1.25×10$^{10}/span/td$
$/spantdclass = ”tdat”/span/td$
$/spantdclass = ”tdbang”/span/td$
$/span/tr$

3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"60/span/span/span/span, 84/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "12/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "25,714/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "6,428/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "13,775,510/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "2.95×10$^{10}/span/td$
$/spantdclass = ”tdat”/span/td$
$/spantdclass = ”tdbang”/span/td$
$/span/tr$

/span/tr/span/table /span/div/span/figure

More detailed comparison tables can be found in **??**.

%

```
<!--................................-->
/spanh5 id="autosec-387"/spanspan class="sectionnumber"5.8&#x2003;/span/spanBandwidth Complexity/span/h5
```

```
<!--................................-->
```
/spanh6 id="autosec-388"/spanspan class="sectionnumber"5.8.1&#x2003;/span/spanFull Node/span/h6

What data must a full node download? A full node must be able to */spanemcompletely validate/span/em* a */spanemsingle chain/span/em*. For a simplex-chain, provided that the PoRs and corresponding headers remain available (which they always do/spansup113/span/sup), this means it must download: all blocks for that simplex-chain, and any auxiliary data necessary to verify PoRs. Note that bandwidth requirements differ based on which UT protocol variant is used.

For UT$_{+\mathrm{OP}}$, the data a full node requires are: each block, the headers of all reflecting chains, and the missing branches for all PoRs. Network-wide, headers consume $\mathrm{N}_1 \cdot B_f \cdot B_h B/s. For a single chain, PoRs use N_1 \cdot B_f \cdot g \cdot \lceil \log_2 N_1 \rceil B/s of capacity, where g is the digest size of the hash used for merkle trees (usually 32 bytes). Let's denote the total bandwidth required for a full node \Delta s. In the worst case (\mathrm{UT}_{+\mathrm{HO}} and \mathrm{UT}_{+\mathrm{HOT}}), where both headers and PoRs must be downloaded: /span span class = "hidden" \(\seteqnumber{0}{}{55}\) /span/span$
<!--

$$
\begin{aligned}
\Delta s &= k_1 + (N_1 \cdot B_f \cdot B_h) + (N_1 \cdot B_f \cdot g \cdot \lceil \log_2 N_1 \rceil) \\
&= k_1 + N_1 \cdot B_f \cdot (B_h + g \cdot \lceil \log_2 N_1 \rceil)
\end{aligned}
$$

```
-->

\begin{align}
\Delta s &amp; = k_1 + (N_1 \cdot B_f \cdot B_h) + (N_1 \cdot B_f \cdot g \cdot \ceil {\log _2 N_1}) \nonumber \\ &amp; = k_1 + N_1 \cdot B_f \cdot ( B_h + g \cdot \ceil {\log _2 N_1}) \nonumber
\end{align}
```

Thus, with */spanemomitted proofs/span/em*, $\mathrm{O}(\Delta s) = O(k_1 + N_1 \cdot \log_2 N_1) = O(c \cdot \log_2 c)$.

However, with */spanemexplicit PoRs/span/em* (variants including +PoRs), $\Delta s \leq k_1 + N_1 \cdot B_f \cdot B_h = O(k_1) = O(c)$.

/spandiv role="note" class="footnotes"
/spanp
/spansup113/span/sup&nbsp; The necessary PoRs are, at the very least, part of other simplex-chains, so “always do” assumes that simplex-chains themselves remain available, excluding planned shutdown. Since all blockchain networks $/spanemdepend/span/em$ on the availability of their chains, this is a safe assumption.
/span/p

/span/div
/spandiv
class="tcolorbox"
style=" border: 1px solid #404040; background: #ECECEC; "

/spandiv
class="tcolorboxupper"
style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbAside/span/b**/span/span The term g
$\cdot\lceil\log_2 N_1\rceil intheseequationsisthesizeofamerklebranchforaPoR.Whatifverkletreesareusedinstead?Withabranchingfactorof256,32bytecommitmentsandproofs,and1bytelocationspecifiers,thistermshouldbereplacedwith(1+32)\cdot$
$\max(1,\log_{256} N_1).Thecomplexityofthesetwotermsisthesame---O(\log c)---$
$butinpracticeverklePoRsarelessthanhalfthesizeofmerklePoRs.Forthisreason,thenumericalcalculationsusedinthetablesthroughoutthispaperassumethattheUTimplementationusesverkletrees./span/div$ /span/div

```
<!--................................-->
```
/spanh6 id="autosec-390"/spanspan class="sectionnumber"5.8.2&#x2003;/span/spanPoR Graph/span/h6

We found the bandwidth requirements for fully reconstructing the PoR graph before in section /spana href="whitepaper.html#lateximage-whitepaper-63"4.6.1/span/a involving the measure of propagation delay,
$\phi seconds.Let'scallthetotalbandwidthrequirementsforthePoRgraph\Delta randsubstitutein\textbf{??}./spanspanclass = "hidden"\backslash(\backslash seteqnumber\{0\}\{\}\{55\}\backslash)/span/span$
<!--

$$\begin{aligned}
\Delta r &= N_1 B_f (B_h + g(1 + N_1 B_f \phi)) \\
&= \frac{k_1}{2B_h}\Bigl(B_h + g\Bigl(1 + \frac{k_1\phi}{2B_h}\Bigr)\Bigr) \\
&= \frac{k_1}{2}\Bigl(1 + \frac{g}{B_h} + \frac{gk_1\phi}{2{B_h}^2}\Bigr) \tag{56}\\
\implies O(\Delta r) &= O(c^2\phi)
\end{aligned}$$

```
-->

\begin{align}
{} \Delta r &amp;= N_1 B_f (B_h + g(1 + N_1 B_f \phi )) \nonumber \\ &amp;= \frac {k_1}{2 B_h} \Bigl (B_h + g\Bigl (1 + \frac {k_1 \phi }{2 B_h}\Bigr )\Bigr ) \nonumber \\ &amp;= \frac {k_1}{2}\Bigl (1 + \frac {g}{B_h} + \frac {g {k_1}
\phi }{2 {B_h}^2}\Bigr ) \label {eq:por-graph-delta-r} \\ \implies O(\Delta r) &amp;= O(c^2\phi ) \nonumber
\end{align}
```
As discussed in section /spana href="whitepaper.html#lateximage-whitepaper-63"4.6.1/span/a it is unclear whether
$O(c^2\phi) = O(c).However, wedon'tneedtoknowO(\Delta r)toestimatesomerealworldvaluesof\Delta r.Practically, \Delta rlookstobebetweenk_1and3k_1fork_1 = 3000 - - - but, thisissensitiveto\phi.$

If we try to restrict $\Delta r < k_1, thenwefindthatwerequirealargeB_horsmallk_1 : /spanspanclass = "hidden"\backslash(\backslash seteqnumber\{0\}\{\}\{56\}\backslash)/span/span$
<!--

$$\begin{aligned}
k_1 &> \frac{k_1}{2}\Bigl(1 + \frac{g}{B_h} + \frac{gk_1\phi}{2{B_h}^2}\Bigr) \\
2{B_h}^2 &> 2B_h g + gk_1\phi \\
k_1 &< 2B_h(B_h - g)(g\phi)^{-1}
\end{aligned}$$

```
-->

\begin{align*}
k_1 &amp;&gt; \frac {k_1}{2}\Bigl (1 + \frac {g}{B_h} + \frac {g {k_1} \phi }{2 {B_h}^2}\Bigr ) \\ 2 {B_h}^2 &amp;&gt; 2 B_h g + g k_1 \phi \\ k_1 &amp;&lt; 2 B_h (B_h - g) (g \phi )^{-1}
\end{align*}
```
If $k_1 = 3000, \phi = 1, wehaveB_h \geq 235 - - - muchlargerthanwhatwe'veassumedtobetheminimumsofar.Achieving\Delta r < 3000requires\phi \lesssim 0.2giventheusualparameters.$

Regarding the terms from eq. /spanspan class="textup"(/spana href="whitepaper.html#lateximage-whitepaper-63"56/span/a)/span/span within the parentheses, it is clear that 1 +
$g/B_h willbeavaluearound1.3, giveortake.Whatdoweexpectthevalueofgk_1\phi/(2{B_h}^2)tobe?IfB_h \in [50, 500], and\phi \in [0.05, 2.0], thenwecanseethatthecoefficientofk_1is, atmost, 0.0128(and3.2 \cdot 10^{-7}atleast).Thus, thislasttermdominates(1 + g/B_h)whengk_1\phi \gtrsim 2{B_h}^2.Tokeep\Delta rpracticable, weshouldtherefore gotosomeefforttokeepgk_1\phi \lesssim 2{B_h}^2z(wherezisaconstantnear1thatwecanusetoaddsomeacceptabletoleranceinlieuofimproving\phi).$

/spana href="whitepaper.html#lateximage-whitepaper-63"Axiom of Unified Ancestry/span/aOptimization Using the /spana href="whitepaper.html#lateximage-whitepaper-63"Axiom of Unified Ancestry/span/a, we do not need to reference a block's parents directly, since they are implied by the PoR graph and the absence of fraud proofs. This results in a small optimization where the parent hash is replaced with the best tip of the longest PoR chain. Regarding
$\Delta r, g(1 + N_1 B_f \phi) \rightarrow gN_1 B_f \phi andthus : /spanspanclass = "hidden"\backslash(\backslash seteqnumber\{0\}\{\}\{56\}\backslash)/span/span$
<!--

$$\begin{aligned}
\Delta r &= N_1 B_f (B_h + gN_1 B_f \phi) \\
&= \frac{k_1}{2}\Bigl(1 + \frac{gk_1\phi}{2{B_h}^2}\Bigr)
\end{aligned}$$

```
-->

\begin{align}
{} \Delta r &amp;= N_1 B_f (B_h + g N_1 B_f \phi ) \nonumber \\ &amp;= \frac {k_1}{2}\Bigl (1 + \frac {g {k_1} \phi }{2 {B_h}^2}\Bigr ) \nonumber
\end{align}
```
A maximal simplex of reasonable parameters has $\Delta r \approx 7{,}200B/s.Restricting\Delta r < k_1nowimpliesk_1 < 2{B_h}^2(g\phi)^{-1}.Ifk_1 = 3000, \phi = 1, wehaveB_h \geq 219.$

```
<!--................................-->
```
/spanh6 id="autosec-391"/spanspan class="sectionnumber"5.8.3&#x2003;/span/spanComplete Simplex/span/h6

/spandiv
    class="tcolorbox"
    style=" border: 1px solid #404040; background: #ECECEC; "

/spandiv
    class="tcolorboxupper"
    style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbAside/span/b**/span/span This is also the bandwidth required by all active miners due to the /spana href="whitepaper.html#lateximage-whitepaper-63"Axiom of Availability/span/a. Miners do not verify the state transitions of each block, but do verify the integrity of the data structure and PoR graph (which is very cheap by comparison). /span/div /span/div

What about the bandwidth required to verify */spanemthe entire simplex/span/em*?

If miners temporarily keep the blocks of every simplex-chain (so that they can regenerate PoRs and verify that reflected headers correspond to existent blocks) then what is the complexity and burden of this?

The amount of network bandwidth,
$\Delta S, requiredtodownloadallblocks(astheyareproduced)acrossallsimplex-chainsistheoveralltransactionthroughput,(N_1 \cdot k_{1,tx}), plusthedatanecessarytoreconstructthePoRgraph:\Delta r.Anyauxiliarydatacanbedeterministicallyregenerated.$

/spanspan class="hidden" \(\seteqnumber{0}{}{56}\)/span/span `<!--`

(57)

```
-->
\begin{equation}

\end{equation}
```

If this were more than just downloading all the blocks, then we'd do that instead:

/spanspan class="hidden" \(\seteqnumber{0}{}{57}\)/span/span `<!--`

$$
\begin{aligned}
\Delta S \le N_1 \cdot k_1 &= \frac{{k_1}^2}{4 \cdot B_f \cdot B_h} \\
\implies O(\Delta S) &= O(c^2)
\end{aligned}
$$

```
-->
\begin{equation}
\begin{split} \Delta S &amp;\le N_1 \cdot k_1 = \frac {{k_1}^2}{4 \cdot B_f \cdot B_h} \nonumber \\ \implies O(\Delta S) &amp;= O(c^2) \nonumber \end {split}
\end{equation}
```

It is clear that
$\Delta S hasorder O(c^2), buthowbadisthis?For k_1 = 3000, B_f = /spanspanclass = "textrm\ textmd\ textup\ textulc"/spanspanclass = "numerator"1/span/span//spanspanclass = "denominator"60/span/span/span/span, and B_h = 112 : \Delta S \approx 1.2MB/s. Withthosefigures:$
$N_1 \approx 800simplex-chains, N_2 \approx 645,000dapp-chains, andmaximumtpsof{\sim}7.7 \times 10^6atnestinglevel2.Decreasingblocktimesto15scorrespondinglydecreasesthebandwidthrequirementsto0.3MB/s forasimplexwith{\sim}200chains,{\sim}40,000dapp-chains, and{\sim}484,000maxtps.$

While O(c^2)$bandwidthscalingisnotideal, it'sclearthat---especiallyintheearlydaysofaUTsimplexwhentherearefewersimplex-chains---therearetolerableconfigurationsavailable; i.e., thereis/spanemexcesscapacity/span/em.$

/spanfigure id="autoid-45" class="table "

/spandiv class="center"

/spandiv class="figurecaption"Table 7: Chain-capacity and bandwidth requirements for $\mathrm{UT}_{+\mathrm{OP}}$: $\mathrm{N}_1, N_2, N_3, \Delta S, \Delta r, and \mathbb{C}' for various parameters.$

/span/div

/spantable/spantr class="tbrule"
/spantd class="tdl"k, B$_f, B_h/span/td$
$/spantdclass = ”tdr”N_1/span/td$
$/spantdclass = ”tdr”N_2/span/td$
$/spantdclass = ”tdr”N_3/span/td$
$/spantdclass = ”tdr”\Delta S(B/s)/span/td$
$/spantdclass = ”tdr”\Delta r(B/s)/span/td$
$/spantdclass = ”tdr”\mathbb{C}'(Hz)/span/td$
$/span/tr$

3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 112/span/td
/spantd class="tdr"200/span/td
/spantd class="tdr"40,357/span/td
/spantd class="tdr"16,215,243/span/td
/spantd class="tdr"306,137/span/td
/spantd class="tdr"4,798.5/span/td
/spantd class="tdr"13.4/span/td
/span/tr

/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 84/span/td
/spantd class="tdr"267/span/td
/spantd class="tdr"71,747/span/td
/spantd class="tdr"38,436,133/span/td
/spantd class="tdr"408,959/span/td
/spantd class="tdr"7,173.5/span/td
/spantd class="tdr"17.9/span/td
/span/tr

/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"30/span/span/span/span, 112/span/td
/spantd class="tdr"401/span/td
/spantd class="tdr"161,431/span/td
/spantd class="tdr"1.30×10$^8/span/td$
$/spantdclass = ”tdr”607,477/span/td$
$/spantdclass = ”tdr”4,798.5/span/td$
$/spantdclass = ”tdr”13.4/span/td$
$/span/tr$

3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"30/span/span/span/span, 84/span/td
/spantd class="tdr"535/span/td
/spantd class="tdr"286,989/span/td
/spantd class="tdr"3.07×10$^8/span/td$
$/spantdclass = ”td”810,744/span/td$
$/spantdclass = ”td”7,173.5/span/td$
$/spantdclass = ”tdat”/span/td$
$/spantdclass = ”tdbang”/span/td$
$/spantdclass = ”td”17.9/span/td$
$/spantdclass = ”tdat”/span/td$
$/spantdclass = ”tdbang”/span/td$
$/span/tr$

3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"60/span/span/span/span, 112/span/td
/spantd class="td"803/span/td
/spantd class="td "645,727/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "1.04×10$^9/span/td$
$/spantdclass = ”tdat”/span/td$
$/spantdclass = ”tdbang”/span/td$
$/spantdclass = ”td”1,210,155/span/td$
$/spantdclass = ”tdat”/span/td$
$/spantdclass = ”tdbang”/span/td$
$/spantdclass = ”td”4,798.5/span/td$
$/spantdclass = ”tdat”/span/td$
$/spantdclass = ”tdbang”/span/td$
$/spantdclass = ”td”13.4/span/td$
$/spantdclass = ”tdat”/span/td$
$/spantdclass = ”tdbang”/span/td$
$/span/tr$

3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"60/span/span/span/span, 84/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "1,071/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "1,147,959/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "2.46×10$^9/span/td$
$/spantdclass = ”tdat”/span/td$
$/spantdclass = ”tdbang”/span/td$
$/spantdclass = ”td”1,614,316/span/td$
$/spantdclass = ”tdat”/span/td$
$/spantdclass = ”tdbang”/span/td$
$/spantdclass = ”td”7,173.5/span/td$
$/spantdclass = ”tdat”/span/td$
$/spantdclass = ”tdbang”/span/td$
$/spantdclass = ”td”17.9/span/td$
$/spantdclass = ”tdat”/span/td$
$/spantdclass = ”tdbang”/span/td$
$/span/tr$

/span/tr/span/table /span/div/span/figure

<!--..............................-->
/spanh5 id="autosec-394"/spanspan class="sectionnumber"5.9&#x2003;/span/spanThe Impact of Header Size/span/h5

eq. /spanspan class="textup"(/spana href="whitepaper.html#lateximage-whitepaper-63"51/span/a)/span/span shows that UT's throughput is inversely proportional to the size of headers, $\mathrm{D}_h, forthatgivendepthofnesting(thisistruefor\mathrm{UT}_1, too).Italsoshowsthatthroughputisinverselyproportionaltotheblockfrequency, D_f, andproportionaltochosenrawthroughput, k.$

Of these three values (header size, block frequency, and raw throughput), header size is the only value we */spanemcannot/span/em* choose arbitrarily. To maintain overall throughput, doubling the header size requires one of: halving the block production frequency (i.e., doubling the block target time), doubling the chain's raw throughput, or some combination of those two options. One such combination would be to decrease the block production frequency by a factor of /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"$\sqrt{2}$*/span/span*/span/span and increase the raw throughput by a factor of $\sqrt{2}$.

Changing all header sizes by some factor has different effects for different UT configurations. For UT$_1$, the effect on throughput is linearly proportional to the factor; doubling the header sizes reduces overall throughput by a factor of 2. However, for UT$_2$, the effect is quadratically proportional to the factor; doubling the header sizes will reduce overall throughput by a factor of 4! The relationship is even worse for UT$_3$, where the effect is cubicly proportional.

It is worth noting, though, that different header schemes can be used in each level of nesting. This means that if, say, dapp-chains need larger headers than simplex-chains, then there isn't a negative effect on the capacity of the simplex (i.e., the level(s) beneath).

This effect is not unique to UT, though. In general, any system of sharding is also affected in this manner when the headers of a child-chain are included in the parent-chain's blocks.

Practically, this effect means that a decrease to the size of headers has */spanemincreasing/span/em* marginal benefit. Compared to O(c) blockchains (e.g., Bitcoin), efficient header schemes are far more important for UT and sharded blockchain networks.

```
/spandiv
      class="tcolorbox"
      style=" border: 1px solid #BC842C; background: #FEE8F4; "

/spandiv
      class="tcolorboxtitle"
      style=" border-top: 1px solid #BC842C; border-bottom: 1px solid #BC842C; background: #BC842C; color: #FFFFFF; "
[TODO: [3] &#x2003;&#x2003;/spanb(MED)/span/b/span/div
/spandiv
      class="tcolorboxupper"
      style=" color: #000000; "
 some discussion to replace +HOT stuff. /span/div /span/div
```

`<!--................................-->`
/spanh5 id="autosec-395"/spanspan class="sectionnumber"5.10&#x2003;/span/spanComparison of UT Variants/span/h5

table /spana href="whitepaper.html#lateximage-whitepaper-63"8/span/a and table /spana href="whitepaper.html#lateximage-whitepaper-63"10/span/a show a comparison between UT variants. For comparisons over a range of parameters, see **??**.

/spanfigure id="autoid-46" class="table "

/spandiv class="center"

/spandiv class="figurecaption"Table 8: Comparison of UT variants' capacities with parameters: k = 3000 B/s; B$_f$ $= /spanspanclass = "textrm\ textmd\ textup\ textulc"/spanspanclass = "numerator"1/span/span//spanspanclass = "denominator"15/span/span/span/span; B_h = 84 bytes; 250 byte transactions. “E.B''_h means the /spanem effective/span/em header-size. Simplex-headers /spanem for the purposes of PoR/span/em can shrink to less than the actual header-size due to the data that is excluded under the corresponding scaling configuration.$

/span/div

/spantable/spantr class="tbrule"
/spantd class="tdl"/span/td
/spantd class="tdr"N$_1$/span/td
/spantdclass = ”tdr”Σ TPS$_1$/span/td
/spantdclass = ”tdr”N_2/span/td
/spantdclass = ”tdr”Σ TPS$_2$/span/td
/spantdclass = ”tdr”$\mathbb{C}'(Hz)$/span/td
/spantdclass = ”tdr”$E.B_h(B)$/span/td
/spantdclass = ”tdr”$PoR(B)$/span/td
/span/tr

UT$_{+\mathrm{PoRs}}$/span/td
/spantd class="tdr"192/span/td
/spantd class="tdr"1,153/span/td
/spantd class="tdr"51,510/span/td
/spantd class="tdr"618,130/span/td
/spantd class="tdr"12.8/span/td
/spantd class="tdr"117/span/td
/spantd class="td"33/span/td
/span/tr

/spantr
/spantd class="tdl"UT$_{+\mathrm{PoRTs}}$/span/td
/spantd class="tdr"227/span/td
/spantd class="tdr"1,363/span/td
/spantd class="tdr"77,479/span/td
/spantd class="tdr"929,750/span/td
/spantd class="tdr"15.1/span/td
/spantd class="tdr"99/span/td
/spantd class="tdr"33/span/td
/span/tr

/spantr
/spantd class="tdl"UT$_{+\mathrm{HOPoRs}}$/span/td
/spantd class="tdr"273/span/td
/spantd class="tdr"1,940/span/td
/spantd class="tdr"86,648/span/td
/spantd class="tdr"1,039,779/span/td
/spantd class="tdr"18.2/span/td
/spantd class="tdr"67/span/td
/spantd class="tdr"35/span/td
/span/tr

/spantr
/spantd class="tdl"UT$_{+\mathrm{HOPoRTs}}$/span/td
/spantd class="tdr"326/span/td
/spantd class="tdr"2,319/span/td
/spantd class="tdr"131,797/span/td
/spantd class="tdr"1,581,574/span/td
/spantd class="tdr"21.7/span/td
/spantd class="tdr"56/span/td
/spantd class="tdr"40/span/td
/span/tr

/spantr
/spantd class="tdl"UT$_{+\mathrm{OP}}$/span/td
/spantd class="tdr"267/span/td
/spantd class="tdr"1,607/span/td
/spantd class="tdr"71,747/span/td
/spantd class="tdr"860,969/span/td
/spantd class="tdr"17.9/span/td
/spantd class="tdr"84/span/td
/spantd class="tdr"35/span/td
/span/tr

/spantr
/spantd class="tdl"UT$_{+\mathrm{OPT}}$/span/td
/spantd class="tdr"340/span/td
/spantd class="tdr"2,045/span/td
/spantd class="tdr"116,219/span/td
/spantd class="tdr"1,394,628/span/td
/spantd class="tdr"22.7/span/td
/spantd class="tdr"66/span/td
/spantd class="tdr"41/span/td
/span/tr

/spantr
/spantd class="tdl"UT$_{+\mathrm{HO}}$/span/td
/spantd class="tdr"703/span/td
/spantd class="tdr"4,218/span/td
/spantd class="tdr"188,337/span/td
/spantd class="tdr"2,260,044/span/td
/spantd class="tdr"46.9/span/td
/spantd class="tdr"32/span/td
/spantd class="tdr"54/span/td
/span/tr

/spantr
/spantd class="tdl"UT$_{+\mathrm{HOT}}$/span/td
/spantd class="tdr"1,406/span/td
/spantd class="tdr"8,437/span/td
/spantd class="tdr"479,403/span/td
/spantd class="tdr"5,752,840/span/td
/spantd class="tdr"93.8/span/td
/spantd class="tdr"16/span/td
/spantd class="tdr"60/span/td
/span/tr

/spantr class="tbrule"
/spantd class="tdl"/span/td
/spantd class="tdr"/span/td
/spantd class="tdr"/span/td
/spantd class="tdr"/span/td
/spantd class="tdr"/span/td
/spantd class="tdr"/span/td
/spantd class="tdr"/span/td
/spantd class="tdr"/span/td
/span/tr/span/table /span/div/span/figure

/spanfigure id="autoid-47" class="table "

/spandiv class="center"

/spandiv class="figurecaption"Table 9: Comparison of UT variants' capacities; as in table /spana href="whitepaper.html#lateximage-whitepaper-63"8/span/a with k = 20000 B/s.
/span/div

/spantable/spantr class="tbrule"
/spantd class="tdl"/span/td
/spantd class="tdr"N$_1$/span/td
/spantdclass = ”tdr”Σ TPS$_1$/span/td
/spantdclass = ”tdr”N_2/span/td
/spantdclass = ”tdr”Σ TPS$_2$/span/td
/spantdclass = ”tdr”ℂ′(Hz)/span/td
/spantdclass = ”tdr”$E.B_h(B)$/span/td
/spantdclass = ”tdr”PoR(B)/span/td
/span/tr

UT$_{\text{+PoRs}}$/span/td
/spantd class="tdr"1,023/span/td
/spantd class="tdr"42,256/span/td
/spantd class="tdr"1,886,448/span/td
/spantd class="tdr"1.51×10^8/span/td
/spantdclass = ”tdr”68.2/span/td
/spantdclass = ”tdr”142/span/td
/spantdclass = ”td”58/span/td
/span/tr

UT$_{\text{+PoRTs}}$/span/td
/spantd class="tdr"1,168/span/td
/spantd class="tdr"48,009/span/td
/spantd class="td"2,727,810/span/td
/spantd class="td "2.18×10^8/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”77.9/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”125/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”59/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/span/tr

UT$_{\text{+HOPoRs}}$/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "1,535/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "64,614/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "2,884,575/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "2.31×10^8/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”102.3/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”93/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”61/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/span/tr

UT$_{\text{+HOPoRTs}}$/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "1,791/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "77,078/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "4,379,483/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "3.50×10^8/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”119.4/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”77/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”61/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/span/tr

UT$_{\text{+OP}}$/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "1,785/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "71,428/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "3,188,775/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "2.55×10^8/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”119.0/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”84/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”61/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/span/tr

UT$_{\text{+OPT}}$/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "2,272/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "90,909/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "5,165,289/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "4.13×10^8/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”151.5/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”66/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”62/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/span/tr

UT$_{\text{+HO}}$/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "4,687/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "187,500/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "8,370,535/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "6.70×10^8/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”312.5/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”32/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”64/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/span/tr

UT$_{\text{+HOT}}$/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "9,375/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "375,000/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "21,306,818/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "1.70×10^9/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”625.0/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”16/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”65/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/span/tr

/span/tr/span/table /span/div/span/figure

/spanfigure id="autoid-48" class="table "

/spandiv class="center"

/spandiv class="figurecaption"Table 10: Comparison of UT variants' network and storage requirements with parameters: k = 3000 B/s; B$_f$ = /spanspanclass = "textrm textmd textup textulc"/spanspanclass = "numerator"1/span/span//spanspanclass = "denominator"15/span/span/span/span; B_h = 84bytes; 250bytetransactions.DTS$_{5yrs}$: Thedaystosync5yearsofasimplex − chain′shistory, includingPoRs, withafullyutilized10MB/snetworkconnection.

/span/div

/spantable/spantr class="tbrule"
/spantd class="tdl"/span/td
/spantd class="tdr"$\Delta s(B/s)/span/td$
$/spantdclass = "tdr"$DTS$_{5yrs}/span/td$
$/spantdclass = "tdr"Chain - GB/yr/span/td$
$/spantdclass = "tdr"\Delta r(B/s)/span/td$
$/spantdclass = "tdr"\Delta S(B/s)/span/td$
$/spantdclass = "tdr"\Sigma$DTS$_{5yrs}/span/td$
$/span/tr$

UT$_{+\mathrm{PoRs}}$/span/td
/spantd class="tdr"3,000/span/td
/spantd class="tdr"0.55/span/td
/spantd class="tdr"94.7/span/td
/spantd class="tdr"4,106/span/td
/spantd class="tdr"292,567/span/td
/spantd class="td"53/span/td
/span/tr

/spantr
/spantd class="tdl"UT$_{+\mathrm{PoRTs}}$/span/td
/spantd class="tdr"3,000/span/td
/spantd class="tdr"0.55/span/td
/spantd class="tdr"94.7/span/td
/spantd class="tdr"3,073/span/td
/spantd class="tdr"343,981/span/td
/spantd class="tdr"62/span/td
/span/tr

/spantr
/spantd class="tdl"UT$_{+\mathrm{HOPoRs}}$/span/td
/spantd class="tdr"4,528/span/td
/spantd class="tdr"0.83/span/td
/spantd class="tdr"142.9/span/td
/spantd class="tdr"7,411/span/td
/spantd class="tdr"492,641/span/td
/spantd class="tdr"89/span/td
/span/tr

/spantr
/spantd class="tdl"UT$_{+\mathrm{HOPoRTs}}$/span/td
/spantd class="tdr"4,434/span/td
/spantd class="tdr"0.81/span/td
/spantd class="tdr"139.9/span/td
/spantd class="tdr"5,560/span/td
/spantd class="tdr"585,471/span/td
/spantd class="tdr"106/span/td
/span/tr

/spantr
/spantd class="tdl"UT$_{+\mathrm{OP}}$/span/td
/spantd class="tdr"3,617/span/td
/spantd class="tdr"0.66/span/td
/spantd class="tdr"114.2/span/td
/spantd class="tdr"7,173/span/td
/spantd class="tdr"408,959/span/td
/spantd class="tdr"74/span/td
/span/tr

/spantr
/spantd class="tdl"UT$_{+\mathrm{OPT}}$/span/td
/spantd class="tdr"3,939/span/td
/spantd class="tdr"0.72/span/td
/spantd class="tdr"124.3/span/td
/spantd class="tdr"5,995/span/td
/spantd class="tdr"517,359/span/td
/spantd class="tdr"94/span/td
/span/tr

/spantr
/spantd class="tdl"UT$_{+\mathrm{HO}}$/span/td
/spantd class="tdr"9,472/span/td
/spantd class="tdr"1.73/span/td
/spantd class="tdr"298.9/span/td
/spantd class="tdr"40,593/span/td
/spantd class="tdr"1,095,281/span/td
/spantd class="tdr"200/span/td
/span/tr

/spantr
/spantd class="tdl"UT$_{+\mathrm{HOT}}$/span/td
/spantd class="tdr"14,822/span/td
/spantd class="tdr"2.71/span/td
/spantd class="tdr"467.8/span/td
/spantd class="tdr"78,000/span/td
/spantd class="tdr"2,187,375/span/td
/spantd class="tdr"399/span/td
/span/tr

/spantr class="tbrule"
/spantd class="tdl"/span/td
/spantd class="tdr"/span/td
/spantd class="tdr"/span/td
/spantd class="tdr"/span/td
/spantd class="tdr"/span/td
/spantd class="tdr"/span/td
/spantd class="tdr"/span/td
/span/tr/span/table /span/div/span/figure

/spanfigure id="autoid-49" class="table "

/spandiv class="center"

/spandiv class="figurecaption"Table 11: Comparison of UT variants' network and storage requirements; as in table /spana href="whitepaper.html#lateximage-whitepaper-63"10/span/a with k = 20000 B/s.
/span/div

/spantable/spantr class="tbrule"
/spantd class="tdl"/span/td
/spantd class="tdr"$\Delta s(B/s)/span/td$
$/spantdclass = ”tdr”$DTS$_{5yrs}/span/td$
$/spantdclass = ”tdr”Chain - GB/yr/span/td$
$/spantdclass = ”tdr”\Delta r(B/s)/span/td$
$/spantdclass = ”tdr”\Delta S(B/s)/span/td$
$/spantdclass = ”tdr”\Sigma$DTS$_{5yrs}/span/td$
$/span/tr$

UT$_{+\mathrm{PoRs}}$/span/td
/spantd class="tdr"20,000/span/td
/spantd class="tdr"3.65/span/td
/spantd class="tdr"631.2/span/td
/spantd class="tdr"82,331/span/td
/spantd class="tdr"10,646,442/span/td
/spantd class="td"1,944/span/td
/span/tr

/spantr
/spantd class="tdl"UT$_{+\mathrm{PoRTs}}$/span/td
/spantd class="tdr"20,000/span/td
/spantd class="tdr"3.65/span/td
/spantd class="tdr"631.2/span/td
/spantd class="tdr"54,890/span/td
/spantd class="tdr"12,057,258/span/td
/spantd class="tdr"2,201/span/td
/span/tr

/spantr
/spantd class="tdl"UT$_{+\mathrm{HOPoRs}}$/span/td
/spantd class="tdr"28,596/span/td
/spantd class="tdr"5.22/span/td
/spantd class="tdr"902.4/span/td
/spantd class="tdr"179,424/span/td
/spantd class="tdr"16,333,048/span/td
/spantd class="tdr"2,982/span/td
/span/tr

/spantr
/spantd class="tdl"UT$_{+\mathrm{HOPoRTs}}$/span/td
/spantd class="tdr"27,880/span/td
/spantd class="tdr"5.09/span/td
/spantd class="tdr"879.8/span/td
/spantd class="tdr"123,841/span/td
/spantd class="tdr"19,393,568/span/td
/spantd class="tdr"3,541/span/td
/span/tr

/spantr
/spantd class="tdl"UT$_{+\mathrm{OP}}$/span/td
/spantd class="tdr"27,307/span/td
/spantd class="tdr"4.99/span/td
/spantd class="tdr"861.7/span/td
/spantd class="tdr"240,566/span/td
/spantd class="tdr"18,097,709/span/td
/spantd class="tdr"3,305/span/td
/span/tr

/spantr
/spantd class="tdl"UT$_{+\mathrm{OPT}}$/span/td
/spantd class="tdr"29,454/span/td
/spantd class="tdr"5.38/span/td
/spantd class="tdr"929.5/span/td
/spantd class="tdr"196,078/span/td
/spantd class="tdr"22,923,351/span/td
/spantd class="tdr"4,186/span/td
/span/tr

/spantr
/spantd class="tdl"UT$_{+\mathrm{HO}}$/span/td
/spantd class="tdr"66,351/span/td
/spantd class="tdr"12.12/span/td
/spantd class="tdr"2,093.9/span/td
/spantd class="tdr"1,598,750/span/td
/spantd class="tdr"48,473,750/span/td
/spantd class="tdr"8,852/span/td
/span/tr

/spantr
/spantd class="tdl"UT$_{+\mathrm{HOT}}$/span/td
/spantd class="tdr"102,016/span/td
/spantd class="tdr"18.63/span/td
/spantd class="tdr"3,219.4/span/td
/spantd class="tdr"3,176,250/span/td
/spantd class="tdr"96,926,250/span/td
/spantd class="tdr"17,701/span/td
/span/tr

/spantr class="tbrule"
/spantd class="tdl"/span/td
/spantd class="tdr"/span/td
/spantd class="tdr"/span/td
/spantd class="tdr"/span/td
/spantd class="tdr"/span/td
/spantd class="tdr"/span/td
/spantd class="tdr"/span/td
/span/tr/span/table /span/div/span/figure

<!--...............................-->
/spanh4 id="autosec-404"/spanspan class="sectionnumber"6&#x2003;/span/spanUT$_{\aleph}$: *TilingSimplexes/span/h4*

In a normal simplex, */spanemall/span/em* simplex-chains mutually reflect */spanemall other/span/em* simplex-chains. This method (UT$_1$) is useful, but, without dapp-chains, it is limited to O(c^2)*scalability*.

Can we do better than O(c^2)*scalability, though?Is/spanspanclass* = ”*uline*”*O*(*n*)*/span/spanscalabilitypossible*?

When analyzing simplexes in section /spana href="whitepaper.html#lateximage-whitepaper-63"5/span/a, we saw that each chain in a simplex reserves */spanspanclass* = ”*textrm textmd textup textulc*”*/spanspanclass* = ”*numerator*”1*/span/span//spanspanclass* = ”*denominator*”2*/span/span/span/span*of its capacity for reflections (with other simplex-chains) and */spanspanclass* = ”*textrm textmd textup textulc*”*/spanspanclass* = ”*numerator*”1*/span/span//spanspanclass* = ”*denominator*”2*/span/span/span/span*of its capacity for transactions and dapp-chain headers. We could, however, use some of this */spanemcapacity for reflections/span/em* for a slightly different purpose. If we were to divide the */spanemcapacity for reflections/span/em* into multiple partitions, then we would have a */spanemsmaller simplex/span/em*, but could also use this new */spanemexcess capacity/span/em* for something else. Let’s consider a simplex where /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"3/span/span//spanspan class="denominator"4/span/span/span/span of its capacity for reflections is reserved as */spanemexcess capacity/span/em*. Here, */spaneminternal reflections/span/em* are the mutual reflections between all simplex-chains that are part of the same simplex.

/spandiv class="center"
/spana href="content/50-tiling/00-10-tile-capacity_sag" target="_blank" /spanimg
src="content/50-tiling/00-10-tile-capacity_sag"
class="inlineimage"
alt="(image)"
/span/a /span/div

What can we use that */spanemexcess capacity/span/em* for?

To answer this, we need to introduce a new */spanemtype/span/em* of simplex: a simplex */spanemtile/span/em*. These are smaller simplexes that, unlike */spanemmaximal simplexes/span/em*, explicitly reserve some reflection capacity for the purpose of mutually reflecting simplex-chains */spanemin other simplex-tiles/span/em*. Reflections with simplex-chains in other tiles are called */spanemexternal reflections/span/em*.

/spandiv
class="tcolorbox"
style=" border: 1px solid #404040; background: #F2F2F2; "

/spandiv
class="tcolorboxupper"
style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbTerm/span/b**/span/span **/spanb/spana href="whitepaper.html#??"Simplex-Tile/span/a/span/b**: A simplex which partitions its capacity for mutual reflections such that each simplex-chain mutually reflects all other simplex-chains in that tile, and all other simplex-chains in */spanemneighboring/span/em* (or */spanemadjacent/span/em*) tiles. /span/div /span/div

We can now replace the */spanemExcess Reflection Capacity/span/em* in our simplex-chain capacity diagram like this:

```
/spandiv class="center"
/spana href="content/50-tiling/00-20-tile-capacity2_sag" target="_blank" /spanimg
       src="content/50-tiling/00-20-tile-capacity2_sag"
       class="inlineimage"
       alt="(image)"
/span/a /span/div
```

Simplex-tiles will be the foundational unit from which we construct */spanemSimplex Tilings/span/em*.

<!--................................-->
/spanh5 id="autosec-407"/spanspan class="sectionnumber"6.1&#x2003;/span/spanSimplex Tilings/span/h5

We connect simplex-tiles together, via mutual reflection, to create a simplex tiling. For the sake of brevity, when we say that a simplex-tile reflects another tile (or that a tile is */spanemconnected/span/em* to another tile), we mean that all simplex-chains in the first tile mutually reflect all simplex-chains in the second tile. A simplex tiling is thus a */spanemgraph/span/em* of interconnected tiles. Connected tiles are */spanemneighbors/span/em* and are */spanemadjacent/span/em* to one another in the tiling.

/spandiv
    class="tcolorbox"
    style=" border: 1px solid #404040; background: #F2F2F2; "

/spandiv
    class="tcolorboxupper"
    style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbTerm/span/b**/span/span **/spanb/spana href="whitepaper.html#??"Simplex Tiling/span/a/span/b**: An interconnected graph of mutually reflecting simplex-tiles. /span/div /span/div

Before we look at some complex (and useful) tilings, let's consider three examples that are some of the simplest tilings possible: fig. /spana href="whitepaper.html#lateximage-whitepaper-63"33/span/a. The examples contain 2, 3, and 4 simplex-tiles respectively.

/spandiv
    class="tcolorbox"
    style=" border: 1px solid #404040; background: #ECECEC; "

/spandiv
    class="tcolorboxupper"
    style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbAside/span/b**/span/span Note: we are not yet concerned with the security properties of these examples — we need to build up to that first. /span/div /span/div

/spanfigure id="autoid-50" class="figure "

```
/spandiv class="minipage" style="justify-content:flex-start ; "

/spana href="tiling_alt_s5_d0_sag" target="_blank" /spanimg
      src="tiling_alt_s5_d0_sag"
      class="inlineimage"
      alt="(image)"
/span/a

/spandiv class="figurecaption"(a) 2 simplex-tiles

/span/div

/span/div

&#x2003;&#x2003;
```

```
/spandiv class="minipage" style="justify-content:flex-start ; "

/spana href="tiling_a3_v3_s5_d0_sag" target="_blank" /spanimg
    src="tiling_a3_v3_s5_d0_sag"
    class="inlineimage"
    alt="(image)"
/span/a

/spandiv class="figurecaption"(b) 3 simplex-tiles

/span/div

/span/div

&#x2003;&#x2003;
```

/spandiv class="minipage" style="justify-content:flex-start ; "

/spana href="tiling_a4_v3_s5_d0_sag" target="_blank" /spanimg
src="tiling_a4_v3_s5_d0_sag"
class="inlineimage"
alt="(image)"
/span/a

/spandiv class="figurecaption"(c) 4 simplex-tiles

/span/div

/span/div

/spandiv class="figurecaption"Figure 33: Trivial simplex tilings — these are all equivalent to a single simplex as each simplex-chain reflects all other simplex-chains in the network.

/span/div /span/figure

In those examples, notice that each simplex-tile reflects each other tile. This means that each simplex-chain is mutually reflecting */spanemall/span/em* other simplex-chains (in all other tiles), so each of these tilings is equivalent to a standalone simplex. These tilings aren't particularly useful to us in practice, though, because we don't have a way to increase network-wide capacity beyond O(c^2).

If we want O(n) capacity, then we'll need to come up with a new pattern for how to connect tiles together and how to add new tiles. Specifically: the pattern must */spanemalways/span/em* have room for more simplex-tiles, and each tile added must, on average, add */spanemmore/span/em* than 1 new spot for additional tiles./spansup114/span/sup

/spandiv role="note" class="footnotes"
/spanp
/spansup114/span/sup&nbsp; There are tilings where a tile consumes more than one free spot. For those tilings, on average each tile must add more spots than it consumes.
/span/p

/span/div

`<!--...............................-->`
/spanh5 id="autosec-412"/spanspan class="sectionnumber"6.2&#x2003;/span/spanTile Valence/span/h5

The */spanemvalence/span/em* (v) of a tile is the number of other tiles that it can connect to. In our example earlier, we split our simplex-tile's capacity for reflections into 4 equal parts — each part was 12.5% of each simplex-chain's */spanemtotal/span/em* capacity. Since 1 of those parts is reserved for internal reflections, and 3 parts are reserved for external reflections, that simplex-tile has a valence of v=3. If, instead, we split our simplex-tile's capacity for reflections into */spanem5/span/em* equal parts, then each part would be 10% of each simplex-chain's */spanemtotal/span/em* capacity, and the tile would have v=4.

We don't */spanemhave/span/em* to use a valence of 3 — we can split up a tile's capacity for reflections any way that we like. Each tile could even have different valences, too, but this makes designing a tiling more complex. For the sake of simplicity, we will only consider tilings where each tile has the same capacity and the same valence. When we say that a */spanemtiling/span/em* (rather than a tile) has a valence of v, we mean that */spanemeach tile in the tiling/span/em* has a valence of v.

If a tiling has a valence of v=0, then all capacity for reflections is reserved for internal reflections — so this is just a normal simplex.

If a tiling has a valence of v=1, then the tiling is limited to 2 tiles at most — there is only one solution. We end up with a tiling that's equivalent to a normal simplex, though, since all simplex-chains */spanemmust/span/em* reflect all other simplex-chains. The complete tiling with v=1, which is the only possible tiling, is shown in fig. /spana href="whitepaper.html#lateximage-whitepaper-63"33a/span/a.

If a tiling has a valence of v=2, then there is a countably infinite number of solutions. The trivial solution (which has 3 tiles) is where each simplex-tile is connected to all other simplex-tiles — this is shown in fig. /spana href="whitepaper.html#lateximage-whitepaper-63"33b/span/a. We can create a tiling of v=2 with 4 tiles, too: A ↔ *B* ↔ *C* ↔ *D* ↔ *A.Thisispartofaclassofsolutions* : *aloop.Withv* = 2, *wecanconstructaloopofarbitrarylength.Thelastsolutionisalongchainoftilesthatneverloopsbackaround*; *seefig.* */spanahref* = "whitepaper.html#*lateximage* − *whitepaper* − 63"34*/span/a.Thisisourfirst/spanemunbounded/span/emsolution* − − − *it′sasolutionwherewecanalwaysaddmoretiles.Theproblemwiththissolutionisthat*, *althoughitmayhaveO*(*n*)*capacity*, *the/spanemdistance/span/embetweentilesis/spanemalso/span/emO*(*n*).*ThisisnotpracticalsinceO*(*n*)*SPV proofswouldbeneededtoprovestateonthefarsideofthetiling.*(*Theclassofsolutionsthatareloopsisimpracticalforthesamereas*

/spanfigure id="autoid-51" class="figure "

```
/spandiv class="center"
/spana href="tiling_v2_s5_d6_sag" target="_blank" /spanimg
       src="tiling_v2_s5_d6_sag"
       class="inlineimage"
       alt="(image)"
/span/a
```

/spandiv class="figurecaption"Figure 34: The only unbounded tiling at v=2. Although it is unbounded (we can always keep adding tiles), the */spanemdistance/span/em* between those tiles is proportional to the size of the network — so this isn't scalable.

/span/div /span/div/span/figure

Things start to get interesting when v > 2.

If a tiling has a valence of v≥ 3, *thentherearemanypossiblepatternsofsolutions.Wewillfocusononeparticularclass : thepatternsoftilingthatarealso/spanemtrees/span/em.*

<!--................................-->
/spanh5 id="autosec-414"/spanspan class="sectionnumber"6.3&#x2003;/span/spanTree-Tilings/span/h5

The tree-tilings that we'll consider are straight forward to construct. First, we start with a */spanemroot tile/span/em*. Then, when we need more capacity, we create new tiles that are children of the root tile. Then, when each of these child-tiles is nearing maximum utilization, new children are created as children of */spanemthat/span/em* tile — they are grandchildren of the root tile. We can continue in this manner indefinitely.

How many children, at most, should we create under each tile? The root tile has */spanemcapacity/span/em* for v children, and each other tile has capacity for v-1 children and 1 parent. We */spanemcould/span/em* allow the root tile to have up to v children — and it is possible to create a tree-tiling like this — however, we're not going to do that here. Instead, we'll limit the root tile to v-1 children to match all other tiles./spansup115/span/sup This means that the tilings we'll consider will be n-ary trees — if v=3 then the tiling is a binary tree; if v=4 then it's a ternary tree; if v=5 then it's a quaternary tree, etc. Binary tree-tilings of depths 1, 2, and 3 are shown in fig. /spana href="whitepaper.html#lateximage-whitepaper-63"35/span/a. Note that, although fig. /spana href="whitepaper.html#lateximage-whitepaper-63"35/span/a constructs the tiling as a balanced tree, there's no requirement that a tree-tiling needs to be balanced.

/spanfigure id="autoid-52" class="figure "

```
/spandiv class="minipage" style="justify-content:flex-start ; "


/spana href="tiling_v3_nary_s5_d1_sag" target="_blank" /spanimg
     src="tiling_v3_nary_s5_d1_sag"
     class="inlineimage"
     alt="(image)"
/span/a


/spandiv class="figurecaption"(a) 1st iteration. 3 tiles.

/span/div

/span/div

&#x2003;&#x2003;
```

```
/spandiv class="minipage" style="justify-content:flex-start ; "

/spana href="tiling_v3_nary_s5_d2_sag" target="_blank" /spanimg
    src="tiling_v3_nary_s5_d2_sag"
    class="inlineimage"
    alt="(image)"
/span/a

/spandiv class="figurecaption"(b) 2nd iteration. 7 tiles.

/span/div

/span/div

&#x2003;&#x2003;
```

/spandiv class="minipage" style="justify-content:flex-start ; "

```
/spana href="tiling_v3_nary_s5_d3_sag" target="_blank" /spanimg
    src="tiling_v3_nary_s5_d3_sag"
    class="inlineimage"
    alt="(image)"
/span/a
```

/spandiv class="figurecaption"(c) 3rd iteration. 15 tiles.

/span/div

/span/div

/spandiv class="figurecaption"Figure 35: Constructing a binary tree-tiling: the first 3 iterations.

/span/div /span/figure

Let's use the term */spanemdepth/span/em* (d) to describe how far a tile is from the root tile. The root tile has d=0, children of the root tile have d=1, grandchildren of the root tile have d=2, etc.

This scaling configuration is $\mathrm{UT}_{\aleph}$, and each of UT_1, UT_2, and UT_3 have a corresponding tiling configuration: $\mathrm{UT}_{\aleph 1}$, $\mathrm{UT}_{\aleph 2}$, and $\mathrm{UT}_{\aleph 3}$.

```
/spandiv role="note" class="footnotes"
/spanp
/spansup115/span/sup&nbsp; Exactly why we should do this will be discussed in section /spana href="whitepaper.html#lateximage-whitepaper-63"6.5/span/a.
/span/p

/span/div
```

`<!--................................-->`
/spanh5 id="autosec-419"/spanspan class="sectionnumber"6.4&#x2003;/span/spanRecursive */spanemProof of Reflection/span/em*/span/h5

So far, we've defined the 'shape' of our tiling, but we don't know much about whether it is (or even can be) secure. Before we cover that in section /spana href="whitepaper.html#lateximage-whitepaper-63"6.5.2/span/a, we need to cover a new idea that we are currently missing: */spanemrecursive Proofs of Reflection/span/em*. The reason for covering recursive PoR first will become obvious, but for now, notice that the leaf tiles in fig. /spana href="whitepaper.html#lateximage-whitepaper-63"35c/span/a are 3 reflections away from the root tile. Eventually, we will need to answer at least these two questions: */spanemCan the chains in those leaf tiles be secured by the root tile?/span/em* and */spanemHow do we do this?/span/em* Recursive PoR provides the foundation that we need to answer those questions and analyze the security of our constructed tiling.

Let us begin with the simplest case of recursive PoR: 3 chains that do not form a simplex.

```
<!--................................-->
```

/spanh6 id="autosec-420"/spanspan class="sectionnumber"6.4.1&#x2003;/span/spanSimple Recursive PoR: $L \leftrightarrow M \leftrightarrow R$/span/h6

Say we have three chains: L, M, and R. They are doing mutual reflection like this: $L \leftrightarrow M \leftrightarrow R$. That is: Lreflects Mbut not R; Mreflects both $L$and R; and Rreflects Mbut not L.

With what we already know, it's clear that $M$is as secure as all three combined. However, that isn't the case for $L$and R. How can we make $L$and Ras secure as M/spanemwithout/span/em having them mutually reflect one another? Is this even reasonable?

Let's consider chain L — what happens if L's local chain-work is, say,
$3\times thatofchainM?Thetotalchain-workthatLtakesintoaccountis/spanspanclass="textrm textmd textup textulc"/spanspanclass="numerator"4/span/span//spanspanclass="denominator"3/span/span/span/spanofitslocalchain-work,soanattackerwouldneed/spanspanclass="textrm textmd textup textulc"/spanspanclass="numerator"2/span/span//spanspanclass="denominator"3/span/span/span/spanworthofL'slocalchain-worktocontrolL'shistory.Thisismorethan50\%ofL'slocalchain-work,but,dependingonR,it'swithinreasonthatanattackermightachievetherequiredhash-rate.$

Case 1: If R's local chain-work is large (say, $3\times thatofM),thenMisalmosttwiceashardtoattackasL./spansup116/span/supThisisaproblematicsituation: itmeansthatanattackermightbeabletoperformadoublespendagainstLeventhoughtheycouldnotperformoneagainstM.Therefore,thiscasewouldnotbeO(n)secure./spansup117/span/sup$

Case 2: However, if R's local chain-work is small (say, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"3/span/span/span/span that of M), then $R$is not particularly significant. Thus, $M$is not substantially harder to attack than L, and an attacker could attack both $L$and Msimultaneously (optionally, Rtoo). But, R/spanemis/span/em substantially easier to attack than either Mor L. The total chain-work that Rtakes into account is only /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"4/span/span//spanspan class="denominator"13/span/span/span/span that of the whole network. So, this case is /spanemalso/span/em not O(n) secure, even though L/spanemis/span/em O(n) secure./spansup118/span/sup

These two problem cases are what recursive PoR solves.

Notice that in both cases, $M$is O(n) secure. Given this, chains $L$and Rshould be able to use the same work that chain Mdoes to reach the same level of security.

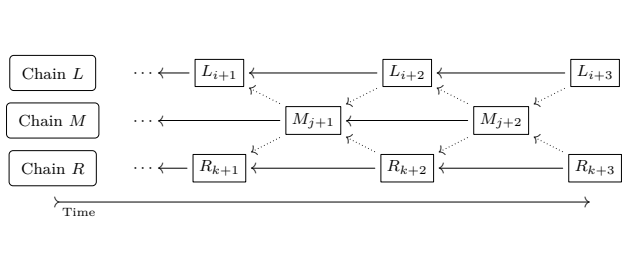
The first problem is that $L$and Rcannot use the same /SPANSPAN CLASS="TEXTSC"WEIGHTOF/SPAN/SPAN algorithm as before (although, Mcan). Particularly, $L$and R's /SPANSPAN CLASS="TEXTSC"WEIGHTOF/SPAN/SPAN must count: not only their local chain-work and that contributed by reflected Mblocks, but also some additional weight based on reflections from /spanemother/span/em chains. Each Mblock might record the total chain-work that it adds to M(including PoRs), but $L$and R's new /SPANSPAN CLASS="TEXTSC"WEIGHTOF/SPAN/SPAN can't use this value directly. Firstly, that value includes past local chain-work that reflected M(which includes both $L$and Rblocks). But more importantly, we don't want to force $L$and $R$to verify /spanemevery/span/em PoR that contributes to M, just the ones that matter. So how can $L$and Rnodes know which work to count?

There is a trivial solution to this problem: require $L$and Rnodes to download the headers of /spanemall/span/em relevant chains, verify relevant PoRs, and track the origin of those PoRs./spansup119/span/sup Nodes must track the origin of work contributed via PoR to avoid double-counting work. An easy example of this is: Lnodes should not count M's PoRs where an Lblock is the reflecting block; that work is already directly included in L's local chain-work.

Chain Lcan only count work contributed by R/spanemafter/span/em a multi-stage PoR can be constructed. For example, let's consider the (rather orderly) chain-segments in fig. /spana href="whitepaper.html#lateximage-whitepaper-63"36/span/a.

/spanfigure id="autoid-53" class="figure "

/spanspan class="hidden" \(\seteqnumber{0}{}{57}\)/span/span `<!--`



```
-->
\begin{equation*}
\xyMChains { \xyL {\text {Chain \cL }} &amp; \cdots &amp; \xBL 1 \ar [l] &amp; &amp; \xBL 2 \ar [ll] \arf [ld] &amp; &amp; \xBL 3 \ar [ll] \arf [ld] \\ \xyL {\text {Chain \cM }} &amp; \cdots &amp; &amp; \xBM 1 \ar [ll] \arf [ld] \arf [lu]
&amp; &amp; \xBM 2 \ar [ll] \arf [lu] \arf [ld] &amp; \\ \xyL {\text {Chain \cR }} &amp; \cdots &amp; \xBR 1 \ar [l] &amp; &amp; \xBR 2 \ar [ll] \arf [lu] &amp; &amp; \xBR 3 \ar [ll] \arf [lu] \\ \xyTimeHoriz {rrrrrr}
&amp;&amp;&amp;&amp;&amp;&amp; }
\end{equation*}
```

/spandiv class="figurecaption"Figure 36: A diagram of chain-segments demonstrating the simplest case of recursive PoR. Solid arrows indicate the usual child-parent relationship, and dotted arrows ($\dashleftarrow$) indicate reflection.

/span/div /span/figure

We can see that R_{k+2} /spanspan class="uline"/spanemimplicitly/span/em/span/span reflects L_{i+1} via R_{k+2}'s /spanspan class="uline"/spanemexplicit/span/em/span/span reflection of M_{j+1}. We can use this implicit reflection because the network already depends on all the PoRs that are sufficient to prove it. L_{i+2}'s PoR for L_{i+1} will prove $L_{i+1} \dashleftarrow M_{j+1}$ (i.e., that L_{i+1} was reflected by M_{j+1}). M_{j+2}'s PoR (via R) for M_{j+1} will prove $M_{j+1} \dashleftarrow R_{k+2}$. We combine these two for the full path by which R_{k+2} implicitly reflects L_{i+1}: $L_{i+1} \dashleftarrow M_{j+1} \dashleftarrow R_{k+2}$.

Similarly, we know that L_{i+3} implicitly reflects R_{k+2}: $R_{k+2} \dashleftarrow M_{j+2} \dashleftarrow L_{i+3}$.

Thus, for L_{i+3}, we can construct a full recursive PoR of L_{i+1} via R_{k+2} using the proofs for each link between them: $L_{i+1} \dashleftarrow M_{j+1} \dashleftarrow R_{k+2} \dashleftarrow M_{j+2} \dashleftarrow L_{i+3}$.

Since Lnodes already know about all the $M$and Rblocks (and the relevant PoRs), the full recursive PoR /spanemdoes not need to be explicitly recorded/span/em. It is already known to all Lnodes, since each part of the recursive PoR is available in one of the L, M, or Rchains (it is explicitly recorded if a +PoRs UT variant is used, and recalculated by the node otherwise).

In section /spana href="whitepaper.html#lateximage-whitepaper-63"2.7/span/a, we discussed the idea that a PoR can only contribute to local blocks that are in the reflecting block's past. By inspection, we can see that the most recent Rblock (the reflecting block) in L_{i+3}'s past is R_{k+2}, and the most recent Lblock (the local block) in R_{k+2}'s past is L_{i+1}. Thus, we can see that our method for attributing reflected work — following the arrows — works for recursive PoR, too.

There is something important to point out here: /spanemlatency/span/em. Notice that these 3 blocks (in chronological order) are required for this recursive PoR to exist: M_{j+1}, R_{k+2}, and M_{j+2}. A non-recursive PoR only needs to wait for 1 reflecting block to be produced (after which the PoR counts as /spanemdraft reflected work/span/em). So, in this case, the expected delay will be
$3\times normal(assumingallchainshavethesameblockperiods).Ingeneral(ifonlyonePoRpathbetweentwogivenblockswillexist)thisdelay,measuredinblockperiods,isequaltothelengthofthefullPoR(excludingLblocks),whichequals2d-1,withdbeingthedistance(inPoRs)betweenthechains.$

Recapping the first problem: $L$and Rnodes need an expanded /SPANSPAN CLASS="TEXTSC"WEIGHTOF/SPAN/SPAN function with access to /spanemthe origin/span/em of implicitly reflected work, and knowledge of /spanemwhere/span/em that work has already been counted. Nodes can do this relatively efficiently, for all relevant chains, by downloading the headers and verifying the PoRs. This method does not require recursive PoRs to be explicitly recorded in full since the components of these PoRs are already available and have already been individually verified. Our existing rules about how to attribute the weight of PoRs (follow the arrows) still works.

Our solution to the first problem is promising, but we have used an implicit assumption: there is only one viable path for PoRs between $L$and R. What if there are /spanemmultiple paths/span/em between $L$and R? This is the second problem.

/spandiv role="note" class="footnotes"
/spanp
/spansup116/span/sup&nbsp; *M*is /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"7/span/span//spanspan class="denominator"4/span/span/span/span×*hardertoattackthanL*.
/span/p

/spanp
/spansup117/span/sup&nbsp; If something is “O(n) secure”, that means that it is */spanemas difficult to attack as attacking the entire network/span/em.* For a particular case to be O(n) secure, each chain in that case must be O(n) secure.
/span/p

/spanp
/spansup118/span/sup&nbsp; *L*is secured by /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"12/span/span//spanspan class="denominator"13/span/span/span/span of the network. While that is less than 100%, it is not substantially easier to attack than *M*, so, arguably, the security of *L*and *M*is of the same order.
/span/p

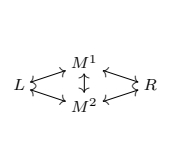
/spanp
/spansup119/span/sup&nbsp; While this solution is trivial, it is also limited. It’s fine for a few chains, but in the case of a full tiling, especially with a few layers, we will hit bandwidth limitations at some point. Discussed more in section /spana
href="whitepaper.html#lateximage-whitepaper-63"6.4.3/span/a.
/span/p

/span/div

<!--...............................-->
/spanh6 id="autosec-422"/spanspan class="sectionnumber"6.4.2&#x2003;/span/spanRecursive PoR with Multiple Paths/span/h6

Say that we now have four chains: L, M^1, M^2, and Rdoing mutual PoR like so:

/spanspan class="hidden" \(\seteqnumber{0}{}{57}\)/span/span <!--



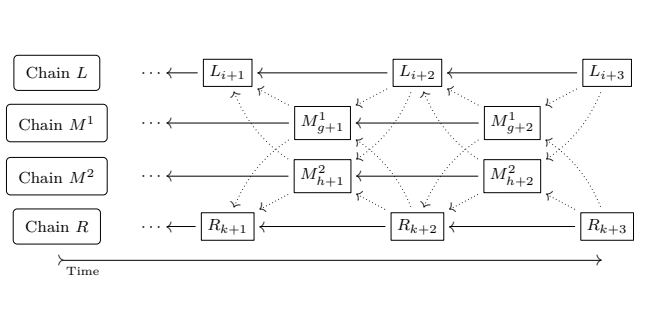
```
-->
\begin{equation*}
\xymatrix @C=2em@R=-.15em{ &amp; {\cMone } \ar @{&lt;-&gt;}[dd] &amp; \\ {\cL } \ar @{&lt;-&gt;}[ur] &amp; &amp; {\cR } \ar @{&lt;-&gt;}[ul] \\ &amp; {\cMtwo } \ar @{&lt;-&gt;}[ul] \ar @{&lt;-&gt;}[ur] &amp; }
\end{equation*}
```

Both $M^1$and $M^2$are already O(n) secure. Assuming that $L$and Rnodes download and verify all PoRs, how can $L$and Rcount each other's work?

First, let's consider when Lcan */spanemdefinitely/span/em* count */spanemall/span/em* the work contributed by a Rblock.

/spanfigure id="autoid-54" class="figure "

/spanspan class="hidden" \(\seteqnumber{0}{}{57}\)/span/span <!--



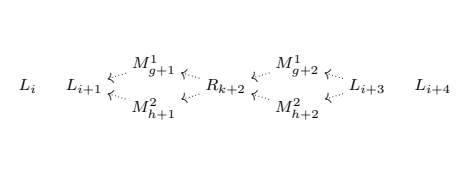
```
-->
\begin{equation*}
\xyMChains { \xyL {\text {Chain \cL }} &amp; \cdots &amp; \xBL 1 \ar [l] &amp; &amp; \xBL 2 \ar [ll] \arf [ld] \arfbu [ldd] &amp; &amp; \xBL 3 \ar [ll] \arf [ld] \arfbu [ldd] \\ \xyL {\text {Chain \cMone }} &amp; \cdots &amp; &amp;
\xBMone 1 \ar [ll] \arfbd [ldd] \arf [lu] &amp; &amp; \xBMone 2 \ar [ll] \arf [lu] \arfbd [ldd] &amp; \\ \xyL {\text {Chain \cMtwo }} &amp; \cdots &amp; &amp; \xBMtwo 1 \ar [ll] \arf [ld] \arfbu [luu] &amp; &amp; \xBMtwo 2 \ar [ll] \arfbu
[luu] \arf [ld] &amp; \\ \xyL {\text {Chain \cR }} &amp; \cdots &amp; \xBR 1 \ar [l] &amp; &amp; \xBR 2 \ar [ll] \arf [lu] \arfbd [luu] &amp; &amp; \xBR 3 \ar [ll] \arf [lu] \arfbd [luu] \\ \xyTimeHoriz {rrrrrr}
&amp;&amp;&amp;&amp;&amp;&amp; }
\end{equation*}
```

/spandiv class="figurecaption"Figure 37: Note that reflections between $M^1$and $M^2$are not shown.

/span/div /span/figure

As before, we'll look at valid PoRs for L_{i+3} that prove L_{i+1} was reflected in R_{k+2}. The exact reflections that exist will be analogous to our prior example, too. See fig. /spana href="whitepaper.html#lateximage-whitepaper-63"37/span/a for the chain-diagram. Notice that there are 4 possible paths from L_{i+3} to L_{i+1}:

/spanspan class="hidden" \(\seteqnumber{0}{}{57}\)/span/span <!--

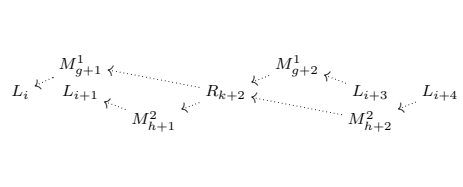


```
-->
\begin{equation*}
\xymatrix @C=1em@R=-.5em{ &amp; &amp; \BMone 1 \arf [ld] &amp; &amp; \BMone 2 \arf [ld] &amp; \\ \BL {} &amp; \BL {1} &amp; &amp; \BR {2} \arf [lu] \arf [ld] &amp; &amp; \BL {3} \arf [lu] \arf [ld] &amp; \BL {4} \\ &amp; &amp; \BMtwo 1
\arf [lu] &amp; &amp; \BMtwo 2 \arf [lu] &amp; \\ }
\end{equation*}
```

Here is what we can say about this case: if all 4 of these PoR paths exist, then R_{k+2} */spanemdefinitely/span/em* reflects L_{i+1}, and its weight can be included in L_{i+3}'s total chain-weight.

But, what if only some of those paths are available? Consider this alternative case:

/spanspan class="hidden" \(\seteqnumber{0}{}{57}\)/span/span <!--



```
-->
\begin{equation*}
\xymatrix @C=1em@R=-.15em{ &amp; \BMone 1 \arff [ld] &amp; &amp; &amp; \BMone 2 \arff [ld] &amp; \\ \BL {} &amp; \BL {1} &amp; &amp; \BR {2} \arff [llu] \arff [ld] &amp; &amp; \BL {3} \arff [lu] &amp; \BL {4} \arf [ld] \\ &amp; &amp;
\BMtwo 1 \arff [lu] &amp; &amp; &amp; \BMtwo 2 \arff [llu] \\ }
\end{equation*}
```

In this case, multiple Lblocks are implicitly reflected in R_{k+2}, and L_{i+3} has two possible recursive PoRs of this: L_{i+1} via M^2_{h+1}; and L_i via M^1_{g+1}. Are both valid? Is either preferable? If L_{i+3} includes the full weight of R_{k+2} in its chain-weight, then what happens when L_{i+4} is mined and the other two PoR paths via M^2_{h+2} become available?

We */spanemcan/span/em* be sure that R_{k+2}'s weight can be fully included in L's chain-weight */spanemafter/span/em* L_{i+4} is mined (since all PoR paths now exist). However, it's not so clear what we should do when not all PoR paths yet exist.

Let us approach this from these principles: /spanul style="list-style-type:none"

/spanli
- A reflecting block can only contribute work once — we should not count work more than once.

/span/li
/spanli
- A PoR contributes as much work as an attacker would need to perform to create a competing PoR of the same weight.

/span/li
/span/ul

Firstly, we can say that the total work contributed by all possible PoR paths is equal to the sum of local work contributed by each unique block in those paths to their respective chains. That is: no work is counted twice, and all blocks in a PoR path contribute work.

Secondly, consider that the work contributed by an individual PoR is equal to the amount of work that an attacker would need to do to produce an equal and competing PoR. This means that */spanemany/span/em* PoR path proving that R_{k+2} reflects L_{i+1} is worth */spanemat least/span/em* as much as R_{k+2}. Additionally, a recursive PoR is, in isolation, worth */spanemexactly/span/em* as much as the sum of weights of each block in that PoR (excluding local blocks).

Thus, we can safely add R_{k+2}'s */spanemfull/span/em* weight via L_{i+3} (and that weight is attributed to L_{i+1} since it is the most recent Lblock in R_{k+2}'s history). An attacker would need to produce at least as much work as R_{k+2} to perform doublespend against L*/spanemafter/span/em* R_{k+2}'s reflection of L_{i+1} has been counted (assuming the doublespend competes with L_{i+1}).

Therefore, only the */spanemfirst/span/em* available recursive PoR via R_{k+2} matters. The number of paths from R_{k+2} to past Lblocks is not relevant: L_i is in the history of L_{i+1}, and R_{k+2} can only count once. Additionally, the number of paths from Lblocks to R_{k+2} is */spanemalso/span/em* not relevant, since R_{k+2} is in the history of all such blocks. Having multiple paths does provide some benefit (as there must be some new reflecting blocks, at least initially), but there will almost always be shorter PoRs that provide the same benefit.

Another way to look at multiple paths is that an attacker (who is attacking L, M^1, M^2, and R) does not have to do any additional work to generate multiple redundant paths (besides the additional work that is directly involved in creating new blocks).

Notice that we do */spanemnot/span/em* require that a recursive PoR use the same chains in each half of the PoR. Of the 4 possible PoRs, the preferable one is: $L_{i+1} \leftarrow M^2_{h+1} \leftarrow R_{k+2} \leftarrow M^1_{g+2} \leftarrow L_{i+3}$./spansup120/span/sup We go from L_{i+3} to R_{k+2} via M^1, and from R_{k+2} to L_{i+1} via M^2.

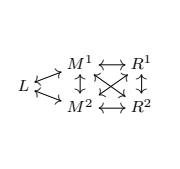
We also can now reason about paths that are longer than the shortest possible path. For example, consider that $M^1$and M^2reflect each other (although this is not shown in our chain-segment examples). There could be PoR paths that include multiple blocks from $M^1$and M^2(e.g., $L_i \leftarrow M^1_{g+1} \leftarrow M^2_{h+1} \leftarrow R_{k+2} \leftarrow M^1_{g+2} \leftarrow L_{i+3}$). Whilst these kind of paths are valid, they are not that useful: in the example, we expect that M^2_{h+1} already reflects an Lblock, either L_i directly or a more recent Lblock.

Finally, let's consider the issue of */spanemlatency/span/em* again. Now that there are two possible chains to use going from $L$to $R$and vice versa, how long should we expect to wait for draft reflected work via $R$to appear?

After the first Lblock is generated, we need to wait for a block from either M^1or M^2. That takes 0.5 block periods on average. Next, we wait for a block from R(1 block period on average), and then for another block from either M^1or M^2(0.5 block periods). So the expected waiting time is 2 block periods.

If we had two possible Rchains ($R^1$and R^2), then we'd expect to wait only 1.5 block periods for draft reflected work from */spanemeither/span/em* R^1or R^2. That configuration would look like this:

/spanspan class="hidden" \(\seteqnumber{0}{}{57}\)/span/span <!--



```
-->
\begin{equation*}
\xymatrix @C=1.5em@R=-.15em{ &amp; \cMone \ar @{&lt;-&gt;}[dd] \ar @{&lt;-&gt;}[r] &amp; \cRone \ar @{&lt;-&gt;}[dd] \\ \cL \ar @{&lt;-&gt;}[ur] \ar @{&lt;-&gt;}[dr] &amp; &amp; \\ &amp; \cMtwo \ar @{&lt;-&gt;}[r] \ar @{&lt;-&gt;}[uur]
&amp; \cRtwo \ar @{&lt;-&gt;}[uul] }
\end{equation*}
```

In general, for a recursive PoR of length 2d-1, where each reflection could be provided by $\mathrm{N}_1(v+1)^{-1} possible chains, the expected latency of draft reflected work is N_1{}^{-1}(v+1)(2d-1) block periods. We divide N_1 by v+1 because each simplex-tile's capacity for reflections is divided equally between v other tiles and itself. This is directly relevant to how fast work propagates within a tiling.$

Note that this does not work for a standalone simplex (d=0
$\implies 2d-1=-1) because we are looking particularly at when recursive PoRs become available (i.e., a draft local block could include them for the first time). In a two-simplex tiling, though (which is equivalent to a standalone simplex), we can see that that d=1 \implies 2d-1 = 1 (each PoR is of length 1), and v=1, so the expected latency of draft reflected work from the other tile is 2\cdot N_1{}^{-1} block periods, which, in seconds, is 2\cdot(N_1 B_f)^{-1} = 2\cdot \mathbb{C}'^{-1} (as expected).$

For a leaf tile at d=3 where each tile has 5 chains and v=3, as in fig. /spana href="whitepaper.html#lateximage-whitepaper-63"35c/span/a, the expected latency is 1 block period./spansup121/span/sup If, instead, each tile had 50 chains, the expected latency is 0.1 block periods. We will revisit this in section /spana href="whitepaper.html#lateximage-whitepaper-63"6.5.4/span/a.

/spandiv role="note" class="footnotes"
/spanp
/spansup120/span/sup&nbsp; This recursive PoR path is preferable because it is from the */spanemfirst possible/span/em* Lblock to enable a PoR via R_{k+2}, and goes to the */spanemmost recent/span/em* Lblock in R_{k+2}'s history.
/span/p

/spanp
/spansup121/span/sup&nbsp; Note: $\mathrm{N}_1 = 20inthisexample, andN_1 = 200inthenext.$
/span/p

/span/div

`<!--................................-->`
/spanh6 id="autosec-424"/spanspan class="sectionnumber"6.4.3&#x2003;/span/spanData Availability/span/h6

The /spana href="whitepaper.html#lateximage-whitepaper-63"Axiom of Availability/span/a states that miners must download all blocks from all simplex-chains before reflecting the corresponding header. This works fine for small examples like those we've just considered, but, in a large/spansup122/span/sup tiling, the sheer volume of data will become a problem at some point.

I don't have a solution for this. In principle, it's not clear that it's even */spanempossible/span/em* to solve without resorting to some form of centralization or group effort. The immediate problem with those kinds of solutions is that even if someone else has the blocks, they always have the option of refusing to share them later — verifying that the blocks exist is not enough. These kinds of solutions are substantially different to all the other solutions in this paper and we'll leave these problems for future research.

The good news is that this problem is not tiling specific. Every other network dealing with a lot of data has a similar challenge to solve, and they will hit this wall before UT due to higher validation requirements. It's plausible that if a solution is found, it could work for tiling, too.

Could we use PoS in the outer tiles to mitigate this somehow, similar to dapp-chains? Perhaps, but tiling is a different context to dapp-chains. For one, using a single root token over the entire tiling means that we need stronger guarantees about conservation of coins, cross-chain transactions, etc. Dapp-chains will be created by users for some purpose – we know from the start that there is a party invested in that chain's ongoing operation. Dapp-chains are also somewhat sandboxed and the disappearance of a dapp-chain does not result in a network failure. Simplex-chains in tiles have neither of those properties, and a failure could have disastrous consequences (e.g., if some tiles are severed from the main tiling). So, while PoS could play a role in a solution, it's not obvious what that solution would be.

/spandiv role="note" class="footnotes"
/spanp
/spansup122/span/sup&nbsp; Particularly, a tiling that has significantly more capacity than a standalone simplex with equivalent parameters.
/span/p

/span/div

`<!--...............................-->`
/spanh5 id="autosec-426"/spanspan class="sectionnumber"6.5&#x2003;/span/spanTree-Tiling Security/span/h5

What does it mean for a tiling to be secure? Well, we have our broad definition from the trilemma: the network is secure against an O(n) attacker. To answer this more specifically, though, we need to first answer: */spanemWhat does a doublespend against a tiling look like?/span/em* Currently, we don't know enough to answer this — we next need answers to questions like */spanemWhich PoRs do simplex-chains in a given tile use?/span/em* and */spanemHow do we govern the difficulties of simplex-chains in different tiles?/span/em*

To answer these questions, and then evaluate the security of tree-tiling, we first need to define some key concepts. After that we will construct a security architecture so that we can answer the above questions. Finally, we will analyze this architecture to find out if it's secure.

`<!--................................-->`
/spanh6 id="autosec-427"/spanspan class="sectionnumber"6.5.1&#x2003;/span/spanKey Concepts/span/h6

`<!--................................-->`
/spanspan class="paragraph" id="autosec-428"/spanspan class="sectionnumber"6.5.1.1&#x2003;/span/spanLocal Chain-work, Tile-work, and Unit Tile/span/span

We're familiar with */spanemlocal chain-work/span/em* (that's work directly contributed by a single simplex-chain).

/spanemTile-work/span/em is the work that is contributed from simplex-chains that are part of a given tile. */spanemTile-work/span/em* is the sum of the local chain-work of each constituent simplex-chain.

We could talk about tile-work directly in terms of */spanemnumber of hashes/span/em*, but this isn't necessary. Instead, we'll normalize tile-work via a */spanemunit tile/span/em*. If some tile's tile-work is $3\times thatofthe/spanemunittile/span/em, thenwewritedownthattile - workas3insteadofsomenumberofhashes.Thiswillsimplifyouranalysis.$

<!--...............................-->
/spanspan class="paragraph" id="autosec-429"/spanspan class="sectionnumber"6.5.1.2&#x2003;/span/spanThe Core/span/span

/spanemThe core/span/em of a tree-tiling is */spanemthe root tile and its children/span/em*. It's called */spanemthe core/span/em* because all nodes in the network verify the work and PoRs of all simplex-chains that are part of the core. Additionally, the core is where the vast majority of chain-work is done.

<!--...............................-->

/spanh6 id="autosec-430"/spanspan class="sectionnumber"6.5.2&#x2003;/span/spanSecurity Architecture/span/h6

The literal keystone of our tiling is the root tile, which is at depth d = 0. Most of the network's total chain-work will be produced by chains in the root tile. We'll guarantee this for d $\geq 1(i.e., allothertiles)viatheRAAbyenforcingan/spanemupperlimit/span/emoneachtile'stile - work./spansup123/span/supThisupperlimitwillexponentiallydecreaseasthedepthofthetilingincreases.Eventhoughtilesatd \geq 2willproducesubstantiallylesstile - workthantheroottile,theywilluserecursivePoRtobe/spanematleast/span/emassecureasthecore.$

The purpose of this design is to enable the following argument for O(n) security: /spanul style="list-style-type:none"

/spanli

1. Attacking a single chain in a single simplex-tile is at least as difficult as attacking the entire tile and its neighbors (section /spana href="whitepaper.html#lateximage-whitepaper-63"4.10/span/a).

/span/li

/spanli

2. Attacking any tile is at least as difficult as attacking the core (via recursive PoR).

/span/li

/spanli

3. 99% of work is done in the core.

/span/li

/spanli

4. Therefore: the core is secure against an attacker with $q < 0.495$.

/span/li

/spanli

5. If $q < 0.5$ is O(n) secure, then $q < 0.495$ is also O(n) secure.

/span/li

/spanli

6. Therefore: attacking any single chain in any tile is as difficult as attacking the entire tiling, and thus the network is O(n) secure.

/span/li

/span/ul

<!--..............................-->
/spanspan class="paragraph" id="autosec-431"/spanspan class="sectionnumber"6.5.2.1&#x2003;/span/spanInter-tile Security Relationship/span/span

Since each tile has v-1 children, we will let u = v-1 for convenience.

We will limit each tile's maximum tile-work production, measured against the unit tile, using the following rules: /spanul style="list-style-type:none"

/spanli
1. The root tile produces at least $\mathcal{M}$units of tile-work ($\mathcal{M}\geq 1; \mathcal{M} stands for “multiplier''$).
/span/li
/spanli
2. Each non-root tile is limited to r$^{-d} units of tile-work (r > u \geq 2) where r is a network-wide constant.$
/span/li
/span/ul

The RAA sets the block reward so that each tile produces an appropriate amount of tile-work. The root tile has no upper limit on tile-work production. If a tile would have aggregate rewards greater than its upper limit, then the block rewards of constituent chains are proportionally capped and any excess is allocated to the parent tile. This process continues until all tiles are under their respective limits.

```
<!--...............................-->
```

/spanspan class="paragraph" id="autosec-432"/spanspan class="sectionnumber"6.5.2.2&#x2003;/span/spanTotal Work Across a Tiling/span/span

How much tile-work is produced over the complete tiling? Let's construct a sum of tile-work over the complete tiling.

The root tile produces $\mathcal{M}$units of tile-work. At depth d
$\geq 1, eachtileproducesr^{-d}unitsoftile-work, andthereareu^{d}suchtiles. Soatdepthdthereisatmost(ur^{-1})^{d}tile-workproduced. We'llassumethattilesproducethemaximumamountoftile-workpossible. Thesum(S)ofallworkoverthetilingisthus:$
$/spanspanclass="hidden"\backslash(\backslash seteqnumber\{0\}\{\}\{57\}\backslash)/span/span$
<!--

$$S = \mathcal{M} + \frac{u}{r} + \frac{u^2}{r^2} + \frac{u^3}{r^3} + \cdots \qquad \text{Sum \emph{all} chain-work}$$

$$S - (\mathcal{M} - 1) = 1 + \frac{u}{r} + \frac{u^2}{r^2} + \frac{u^3}{r^3} + \cdots$$

This is a geometric series, and since u$\overline{\scriptstyle r<1:}$

S - ($\mathcal{M}$-1) = 1 $\quad {\scriptstyle 1-\frac{u}{r}}$ Sum of geometric series

S - ($\mathcal{M}$-1) = r $\quad {\scriptstyle r-u\times\frac{r}{r}}$

S = $\mathcal{M}$-1 + r$\overline{\scriptstyle (58)}$

```
-->

\begin{align}
S &amp;= \Mroot + \frac {u}{r} + \frac {u^2}{r^2} + \frac {u^3}{r^3} + \cdots &amp;&amp; \text {Sum \emph {all} chain-work} \nonumber \\ S - (\Mroot -1) &amp;= 1 + \frac {u}{r} + \frac {u^2}{r^2} + \frac {u^3}{r^3} + \cdots \nonumber \\
\intertext {This is a geometric series, and since $\frac {u}{r} &lt; 1$:} S - (\Mroot -1) &amp;= \frac {1}{1 - \frac {u}{r}} &amp;&amp; \text {Sum of geometric series} \nonumber \\ S - (\Mroot -1) &amp;= \frac {r}{r - u} &amp;&amp; \times
\frac {r}r \nonumber \\ S &amp;= \Mroot -1 + \frac {r}{r - u} \label {eq:tree-tiling-total-work}
\end{align}
```

<!--...............................-->
/spanspan class="paragraph" id="autosec-433"/spanspan class="sectionnumber"6.5.2.3&#x2003;/span/spanDoublespend Requirements/span/span

A traditional PoW blockchain is secure against an attacker with less than 50% of the network hash rate. In terms of p and q, this threshold is $q < 0.5$.

While we maintained this threshold for simplexes, we will relax is very slightly for tiling. Instead of $q < 0.5$, we will pick a threshold that is very close, but slightly lower. This leaves some wiggle room for us to use PoW in other tiles (outside the core) without weakening the network in a significant way. Choosing $q < 0.495$ (half of 99%) is somewhat arbitrary – in that we don't have a reason to choose it over 0.496 or 0.494 – but it meets our other goals and gives us a nice round number (99%) for the proportion of work done in the core.

For the purpose of this paper, it is assumed that: $UT_{\aleph}$ being secure against an attacker with $q < 0.495$ counts as $O(n)$ secure. (That is: $q < 0.495$ is */spanemgood enough/span/em.*)

/spandiv role="note" class="footnotes"
/spanp
/spansup123/span/sup&nbsp; This also means that we’re only considering the PoR context of a single root token.
/span/p

/span/div

<!--................................-->
/spanh6 id="autosec-435"/spanspan class="sectionnumber"6.5.3&#x2003;/span/spanAnalysis/span/h6

How can we analyze the security of tree-tilings?

We will consider multiple cases, and in each case establish a */spanembound/span/em* on r in terms of u, q, and $\mathcal{M}$. The intersection of these bounds on r from each of these cases will yield the values of r for which */spanemtiling works/span/em* (i.e., it is secure). The value of q that we choose will determine how difficult the root tile is to attack, relative to the entire network — we will choose q $\leq 0.495 asanacceptableboundonq, soq = 0.495 isourthreshold.Thecasesthatweanalyzewillcoverallpossiblepositionsofatileinthetiling.$

The primary case for us to analyze is that the root tile is secure against an attacker with q proportion of the network-wide hash-rate. Additionally, we need to figure out when a tile (or branch) could be severed from the tiling.

```
<!--...............................-->
```

/spanspan class="paragraph" id="autosec-436"/spanspan class="sectionnumber"6.5.3.1&#x2003;/span/spanThe Core is (Almost) as Secure as the Network/span/span

Let's constrain the core to being secure against an attacker with up to q proportion of the network-wide hash-rate ($0 \leq q < 0.5$).

The amount of work that attacker is capable of is at most qS (via eq. /spanspan class="textup"(/spana href="whitepaper.html#lateximage-whitepaper-63"58/span/a)/span/span). And the amount of work securing the core is the sum of work contributed by the root tile and its immediate children: $\mathcal{M}$+ u·r^{-1}*.Thecoreisnotsecurewhentheattackerisresponsibleformorethanhalfoftheworksecuringthecore.Thus,thecoreissecurewhen* : */spanspanclass* = ”*hidden*”\(\seteqnumber{0}{}{58}\)*/span/span*
<!--

$$
\begin{aligned}
\frac{1}{2}(\mathcal{M}+\frac{u}{r}) &> qS && \text{Assume that the core is secure} \\
\frac{1}{2}(\mathcal{M}r+u) &> qr(\mathcal{M}-1+\frac{r}{r-u}) && \times r \text{ and substitute } S \text{ (eq. (58))} \\
(\mathcal{M}r+u)(r-u) &> 2qr((\mathcal{M}-1)(r-u)+r) && \times 2(r-u) && (59) \\
r &> \frac{u}{2\mathcal{M}}\left(\mathcal{M}-1+\sqrt{(\mathcal{M}-1)^2+\frac{4\mathcal{M}}{1-2q}}\right) && \text{Solve quadratic \& simplify}^{124} && (60)
\end{aligned}
$$

```
-->

\begin{align}
\frac 12 (\Mroot + \frac {u}r) &amp;&gt; qS &amp;&amp; \text {Assume that the core is secure} \nonumber \\ \frac 12 (\Mroot r + u) &amp;&gt; qr(\Mroot -1+\frac {r}{r-u}) &amp;&amp; \times r \text { and substitute } S \text { (\autoref
{eq:tree-tiling-total-work})} \nonumber \\ (\Mroot r + u)(r-u) &amp;&gt; 2qr((\Mroot -1)(r-u)+r) &amp;&amp; \times 2(r-u) \label {eq:tree-tiling-safe-r-nofrac} \\ r &amp;&gt; \frac {u}{2\Mroot }\left ( \Mroot - 1 + \sqrt {(\Mroot - 1)^2 +
\frac {4\Mroot }{1-2q}} \right ) &amp;&amp; \text {Solve quadratic \&amp; simplify\footnotemark } \label {eq:tree-tiling-safe-r}
\end{align}
```

This is our first bound on r. If u = 2, $\mathcal{M}$= 8, and q = 0.495, then r > 8. Therefore, the hash-rate dedicated to the root tile is ∼64 ×*thatofitsimmediatechildren.*

`<!--................................-->`
/spanspan class="subparagraph" id="autosec-437"/spanspan class="sectionnumber"6.5.3.1.1&#x2003;/span/spanLimiting the Number of Children of the Root Tile/span/span

Earlier, we limited the number of children of the root tile (in a tree-tiling) to v-1. This means we can fit twice as many chains in the root tile, and so the hash-rate of the root tile is spread over twice as many chains, and the confirmation rate is doubled (excluding confirmations from external reflections). It also means that hash-rate is less spread out over the tiling, and that the method used above and in **??**&nbsp;6.5.2.2 is relatively simple. Finally, it means that we can grow a simplex to /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"2/span/span/span/span of its maximal capacity without committing to any particular details of the tiling scheme (besides the proportion of reflection capacity left for child tiles).

`<!--...............................-->`
/spanspan class="paragraph" id="autosec-438"/spanspan class="sectionnumber"6.5.3.2&#x2003;/span/spanSevering Tiles/span/span

Is it possible to sever a tile from the tiling? Tiles outside the core have a very small proportion of the network's hash-rate. If all of that tile's miners deliberately censored PoRs with the parent tile, the tile would be severed from the rest of the tiling. If one was severed from the rest of the tiling, performing a double-spend might be trivial.

Assuming r > 2, r > u, and d
$\geq 2, atile'schain - workislessthanhalfthatofitsparent.Aswe'veseen, theamountofworkavailabletothetileviarecursivePoRismuchgreater.Thus, thePoRgraphofthattileiseasilydominatedbyanyblockthatcanestablishrecursivePoRswiththecore.$

Since we're outside the core, the local tile will produce approximately
r$^{-d} work(intermsoftheunittile), anditschildrenwillcontributeur^{-(d+1)}workatmost.Notethatur^{-(d+1)} < r^{-d}.Bycomparison, theworkcontributedbytheparenttileis\mathcal{M} + r^{-1} + r^{-2} + \cdots + r^{-(d-1)}.Given\mathcal{M} >$
$1, thevolumeofworkcontributedviarecursivePoRsisatleast/spanspanclass = "textrm\ textmd\ textup\ textulc"/spanspanclass = "numerator"\mathcal{M}r^{d}/span/span//spanspanclass = "denominator"2/span/span/span/span >$
$\mathcal{M}r^{d-1}timesgreaterthantheworkcontributedbythetileanditschildren.Therefore, onehonestminerinthattilingcanpreventseverancebyproducing, atmost, 1blockforevery\mathcal{M}r^{d-1}blocksproducedbytheattacker.$

If the attacker wishes to avoid this incredible disadvantage, they must attack the parent tile too. This continues recursively until the attacker reaches the core, at which point they must attack the entire network.

If an attacker is attempting to sever a child tile instead, the situation is similar, except that their disadvantage is at least a factor of $\mathcal{M}r^{d-2}comparedtoanhonestminer.$

Given that honest miners have an overwhelming advantage in these situations, we can skip examining both how this interacts with the DAA and RAA, and the opportunity cost to the attacker of attempting such an attack. Instead, we'll assume that such an attack is impractical, and leave the analysis for future research.

Regarding doublespends, we can conclude that, while it might be possible to disrupt a tile for a short while, all we have to do is wait for recursive PoRs via the core to thwart the attack.

<!--................................-->
/spanspan class="paragraph" id="autosec-439"/spanspan class="sectionnumber"6.5.3.3&#x2003;/span/spanAnalysis Conclusion/span/span

We have established bounds where the tiling is secure against severance attacks and doublespends: eq. /spanspan class="textup"(/spana href="whitepaper.html#lateximage-whitepaper-63"60/span/a)/span/span, u $\geq 2, r > u, and\mathcal{M} > 1. Practically, values of u = 2, \mathcal{M} = 8, r \geq 8 meet our goals.$

```
/spandiv role="note" class="footnotes"
/spanp
/spansup124/span/sup&nbsp; We can quickly verify that eq. /spanspan class="textup"(/spana href="whitepaper.html#lateximage-whitepaper-63"59/span/a)/span/span and eq. /spanspan class="textup"(/spana
href="whitepaper.html#lateximage-whitepaper-63"60/span/a)/span/span are identical for u,r > 1 /spana href="https://www.desmos.com/calculator/7pzmo7t8tc" target="_blank" by graphing them/span/a.
/span/p


/span/div
```

```
<!--................................-->
```

/spanh6 id="autosec-441"/spanspan class="sectionnumber"6.5.4&#x2003;/span/spanSecurity Propagation Speed / Finalization/span/h6

Typically, “finalization” refers to a breakpoint whereafter it is practically impossible to revert a transaction.

As we saw in section /spana href="whitepaper.html#lateximage-whitepaper-63"6.4.2/span/a, PoR paths form quickly between tiles. Particularly, this depends on the number of chains in each tile, $\mathrm{N}_1(v + 1)^{-1}, andtheblockperiod.$

How long does it take for a block in some tile to be secured by the root tile?

When a block in a tile is produced, miners in the parent tile will reflect it after some propagation delay,
$\phi. Theexpectedwaitingtimeisthepropagationdelayplustheexpectedtimebetweenblocks(ofanychain)intheparenttile. Afteranyblockintheparenttilehasreflectedthefirstblock, thisprocessrepeatsfortheparenttileanditsparent(i.e., thegrandparentoftheoriginaltile). Eventually, wereachtheroottile, andthereflectionsbegintopropagatebackdowntowardsthe$
$/spanspanclass = "hidden"\backslash(\backslash seteqnumber\{0\}\{\}\{60\}\backslash)/span/span$
<!--

$$\begin{aligned} & \Big(\frac{v+1}{N_1 B_f} + \phi\Big)(2d-2) + \Big(\frac{v+1}{2N_1 B_f} + \phi\Big) \\ &= \frac{(v+1)(4d-3)}{2N_1 B_f} + \phi(2d-1) \qquad (61) \end{aligned}$$

```
-->
```

```
\begin{align}
&amp; \;\;\;\; \Big ( \frac {v+1}{N_1 B_f} + \phi \Big ) (2d-2) + \Big ( \frac {v+1}{2 N_1 B_f} + \phi \Big ) \nonumber \\ &amp; = \frac {(v+1)(4d - 3)}{2 N_1 B_f} + \phi (2d - 1) \label {eq:tiling-finalization-delay}
\end{align}
```

Ignoring $\phi makesiteasytoseehowfastthisis : givenv = 3, d = 10, N_1 = 200chains, andB_f =$
$/spanspanclass = "textrm\ textmd\ textup\ textulc"/spanspanclass = "numerator"1/span/span//spanspanclass = "denominator"15/span/span/span/spanHz, wecanexpectarecursivePoRtobeavailableapproximately5.5secondsafterablockisproduced. d =$
$10impliesthatthetilinghasatleast10layers. Suchatilinghas \geq 1024/spanemtiles/span/em./spansup125/span/sup$

Alternatively, if we set $\phi = 0.5, thenwe'dexpectthesamerecursivePoRtobeavailablein{\sim}15seconds(whichhappenstobe1blockperiodinthiscase).$

/spandiv role="note" class="footnotes"
/spanp
/spansup125/span/sup&nbsp; The tiling needs at least 1 node at d=10, so we assume d=9 is full. In a full binary tree down to a depth of 9, there are $2^{10} - 1 nodes.2^{10} - 1 + 1 = 1024.$
$/span/p$

/span/div

```
<!--................................-->
/spanh5 id="autosec-443"/spanspan class="sectionnumber"6.6&#x2003;/span/spanScaling Complexity/span/h5
```

```
<!--...............................-->
```
/spanh6 id="autosec-444"/spanspan class="sectionnumber"6.6.1&#x2003;/span/spanCapacity/span/h6

The tiling is an (v-1)-ary tree; assuming it is balanced and has a maximum depth of d, the number of tiles, $\mathrm{N}_{\text{tiles}}, inthetilingis:$

/spanspan class="hidden" \(\seteqnumber{0}{}{61}\)/span/span <!--

(62)

```
-->
\begin{equation}

\end{equation}
```

Each tile in the tiling has $\mathrm{N}_1(v+1)^{-1}chains--except, of course, theroottilewhichhastwiceasmany. Thus, thetotalnumberofchainsinthetilingis/spanspanclass = "hidden"\backslash(\backslash seteqnumber\{0\}\{\}\{62\}\backslash)/span/span$
<!--

$$\begin{aligned} N_{\text{chains}} &= (N_{\text{tiles}}+1)\frac{N_1}{v+1} \\ &= \frac{N_1(v-1)^{d+1}-1}{(v-2)(v+1)} + \frac{N_1}{v+1} \end{aligned}$$

```
-->

\begin{align*}
N_{\text {chains}} &amp; = (N_{\text {tiles}} + 1) \frac {N_1}{v + 1} \\ &amp; = \frac {N_1 (v-1)^{d+1} - 1}{(v - 2)(v + 1)} + \frac {N_1}{v + 1}
\end{align*}
```
When v = 3, we have:

/spanspan class="hidden" \(\seteqnumber{0}{}{62}\)/span/span <!--

$$N_{\text{chains}} = 2^{d-1}N_1 \tag{63}$$

```
-->
\begin{equation}
N_{\text {chains}} = 2^{d-1} N_1
\end{equation}
```

Clearly, the number of chains depends on how many layers we create, which is determined by demand and systemic constraints.

Assuming the demand for capacity is O(n), and each chain provides O(c^{i})$capacity(wherei = 1withnodapp-chains, i = 2with1levelofnesting, etc), wecanarguethatO(2^{d-1}) = O(nc^{-i-1}).$

There is at least one physical constraint that will eventually limit us. Assuming all simplex-chains in the tiling use PoW, there will eventually come a layer where we run out of bits to use in the hash. If we had, say, 96 bits to work with (assuming a 256 bit hash), then we'd have ∼(96 - log$_2$($\mathcal{M}$))(log$_2$(r))$^{-1}layerstoworkwith.$

Given N$_1$ = 200, $\mathcal{M} = 8, r = 8, andv = 3, wecouldsupportupto{\sim}31layers, whichismorethan10^{11}simplex-chainsintotal. Granted, thereareotherconstraintsthatwewillbelimitedby.$

`<!--...............................-->`
/spanh6 id="autosec-445"/spanspan class="sectionnumber"6.6.2&#x2003;/span/spanRecursive PoRs/span/h6

Consider some block in some tile outside the core. Given v=3, there will be /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"N$_1$*/span/span*//spanspan class="denominator"2/span/span/span/span blocks produced in the root tile between this block and the next block. We only need to include one recursive PoR with the root tile to gain the weight of all those blocks — particularly, the recursive PoR goes via the most recent block we know of in the root tile. Also, the other chains in the tile will include recursive PoRs via many of those root tile blocks in the intervening period. In principle, then, we shouldn't need much additional data than these PoRs.

Each merkle (or verkle) branch in such a recursive PoR is O($\log c). There are O(d) = O(\log(nc^{-i-1})) many such branches. Thus the resulting complexity is /spanspanclass = "hidden"\(\seteqnumber{0}{}{63}\)/span/span$
<!--

$$\begin{aligned} O(\log(c)\log(nc^{-i-1})) &= O(\log(c)(\log(n)-(i+1)\log(c))) \\ &= O(\log(c)\log(n)) \end{aligned}$$

-->

```
\begin{align*}
O(\log (c) \log (n c^{-i-1})) &amp; = O(\log (c)(\log (n) - (i+1)\log (c))) \\ &amp; = O(\log (c) \log (n))
\end{align*}
```

`<!--...............................-->`
/spanh6 id="autosec-446"/spanspan class="sectionnumber"6.6.3&#x2003;/span/spanBandwidth/span/h6

The primary bandwidth constraint on a full node for some chain in some tile is the PoR graphs for each tile between the local tile and the root (inclusive). Since we've already established that following the PoR graph of a simplex is an O(c) bandwidth load, the overall bandwidth load is

/spanspan class="hidden" \(\seteqnumber{0}{}{63}\)/span/span `<!--`

$$O(c\log n - c(i+1)\log c) = O(c\log n)$$

```
-->
\begin{equation*}
O(c \log n - c(i+1)\log c) = O(c \log n)
\end{equation*}
```

A miner, though, needs to verify the availability of all those blocks – an
O(c^2)$bandwidthloadpertile.ThereforetheyhaveanassociatedbandwidthloadofO(c^2\log n).Assumingthatweareusingnetworkandchainparameterssuchthattheoverallbandwidthrequirementsofamaximalsimplexaretolerable(seesection\ /spanahref=$"whitepaper.html$\#lateximage-whitepaper-63$"5.7$/span/a),it'snotoutofthequestionthatmultipl$

If */spanemevery/span/em* block from */spanemevery/span/em* tile is downloaded, then we naturally have an O(n) load. However, in smaller tilings we actually */spanemsave/span/em* bandwidth compared to a maximal simplex of the same capacity. This is due to the simplex tiles themselves being smaller and reducing PoR graph overhead. So, even though the complexities of a tiling are naturally larger than a maximal simplex, there are configurations that increase overall capacity within the same constraints.

<!--...............................-->
/spanh4 id="autosec-447"/spanspan class="sectionnumber"7&#x2003;/span/spanAttacks/span/h4

\/spanemUltra Terminum/span/em — with appropriate configuration — is resistant to the following attacks; see the linked section for discussion:

/spanul style="list-style-type:none"

/spanli
- 51% Attack, see section /spana href="whitepaper.html#sec:proof-of-reflection"2/span/a, section /spana href="whitepaper.html#lateximage-whitepaper-63"4.8.3/span/a, and section /spana href="whitepaper.html#lateximage-whitepaper-63"4.10/span/a.

/span/li
/spanli
- Selfish mining, see section /spana href="whitepaper.html#lateximage-whitepaper-63"4.7/span/a.

/span/li
/spanli
- Reflection without publication, see section /spana href="whitepaper.html#lateximage-whitepaper-63"4.1/span/a.

/span/li
/spanli
- Empty block DoS and censorship, see section /spana href="whitepaper.html#lateximage-whitepaper-63"4.8/span/a.

/span/li
/spanli
- *\/spanemNothing at Stake/span/em* and *\/spanemlong range/span/em* attacks, see section /spana href="whitepaper.html#lateximage-whitepaper-63"2.6/span/a and section /spana href="whitepaper.html#lateximage-whitepaper-63"3.4/span/a.

/span/li
/spanli
- Intra-simplex cross-chain attacks, see section /spana href="whitepaper.html#lateximage-whitepaper-63"4.11/span/a.

/span/li
/span/ul

%

<!--................................-->
/spanh4 id="autosec-448"/spanspan class="sectionnumber"8&#x2003;/span/spanConclusions/span/h4

<!--................................-->
/spanh5 id="autosec-449"/spanspan class="sectionnumber"8.1&#x2003;/span/spanAddressing the */spanemBlockchain Trilemma/span/em*/span/h5

Our journey is coming to a close, and it's time we finally answer the question */spanemDoes UT solve the Trilemma?/span/em* First, let us consider UT$_1$.

The */spanemdecentralization/span/em* criterion states that the system must function when each participant has only O(c) resources. We codify this with the chain-parameter k, a generalization of block size, which we define to be O(c). Typically, we've set k = 3000 bytes/second since this is close to the real-world value of k for Bitcoin and pre-merge Ethereum, and is therefore, self-evidently, an acceptable value. We've also set k = 20000 in some tables so that we have an idea of what we could do if we pushed UT further. The /spana href="whitepaper.html#lateximage-whitepaper-63"Axiom of Availability/span/ameans that miners must download */spanemall/span/em* blocks — an
O(c^2)*load.However, minersdonotneedto/spanemvalidate/span/emallblocks, orstorethemforverylong; andtheseblocksaren′trequiredtosynchronizewiththePoRgraph.Therefore, ifwefailthedecentralizationcriterionitwillbebecauseofbandwidth* :
Δ*S.But, whatis*Δ*S?Waybackinsection /spanahref =* "whitepaper.html#sec:a-principle-of-scaling"1.1.2*/span/a, Isuggestedatestforwhether*UT$_1$*passesorfailsinthissortofsituation* :
isthisabottleneckoristhereexcesscapacity?Insection /spanahref = "whitepaper.html*#lateximage − whitepaper −* 63"5.10*/span/awesawthat, whenk* = 3000, *reasonablenetworkparametersmeant*Δ*S* ∈ [293, 2188]*kB/s.Thisequatesto*Δ*S* ∈
[759, 5184]*GB/month.Currently, DigitalOceanoffers/spansup*126*/span/supaVPSwith*5*TiB/month*(*outbound*)*for*48.00*USD.Byinspection, thisisO*(*c*), *so, atk* =
3000, *the*Δ*Srequirementsonminersisnotabottleneck./spanspanclass =* "*uline*"*Therefore, withappropriateparameters,* UT$_1$*passesthedecentralizationcriterion./span/span*

The */spanemsecurity/span/em* criterion states that the system must be secure against an attacker with O(n) resources. The standard way to check if a PoW system passes this criterion is whether it is resistant to a 51% attack. We saw in section /spana href="whitepaper.html#lateximage-whitepaper-63"4.10/span/a that UT$_1$ is indeed secure against a 51% attack. We also saw in section /spana href="whitepaper.html#lateximage-whitepaper-63"4.11/span/a that cross-chain transactions within the simplex were as secure as the network. /spanspan class="uline"Therefore UT$_1$ passes the security criterion./span/span

The */spanemscalability/span/em* criterion requires that the system can process more than O(c) transactions. Given k = 3000,
$\Sigma\mathrm{TPS}_1 \in [10^3, 8.4 \times 10^3]$*.Byinspection,* 1000*TPSismorethanO*(*c*)*./spanspanclass =* "*uline*"*Therefore*UT$_1$*passesthescalabilitycriterion./span/span*

/spanspan class="uline"Thus UT$_1$ satisfies all 3 trilemma criteria */spanemas put by Vitalik/span/em*./span/span

/spandiv role="note" class="footnotes"
/spanp
/spansup126/span/sup&nbsp; See /spana href="https://www.digitalocean.com/pricing/droplets" target="_blank" https://www.digitalocean.com/pricing/droplets/span/a (also via /spana
href="https://web.archive.org/web/20250126134024/https://www.digitalocean.com/pricing/droplets" target="_blank" web.archive.org/span/a however client-side js breaks the page shortly after loading). Note: inbound data is free.
/span/p

/span/div
/spandiv
class="tcolorbox"
style=" border: 1px solid #404040; background: #ECECEC; "

/spandiv
class="tcolorboxupper"
style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbAside/span/b**/span/span At k = 20000 B/s,
$\Delta S \in [10, 93] MB/s, or \Delta S \in [25, 241] TB/month. Thisisdefinitelybeyondmanyordinaryusers, butwouldworkforminersatsomeeconomyofscale. (241 TBcosts {\sim} 2235 USD/month/spansup127/span/supfromDigitalOcean.) Suchasimplexwouldhave N_1 \in$
$[10^3, 10^4] chains, and \Sigma \mathrm{TPS}_1 \in [4 \times 10^4, 3.8 \times 10^5] TPS./span/div$

/spandiv role="note" class="footnotes"
/spanp
/spansup127/span/sup&nbsp; Jan 2025; see /spana href="https://web.archive.org/web/20250126231118/https://docs.digitalocean.com/platform/billing/bandwidth/" target="_blank" https://docs.digitalocean.com/platform/billing/bandwidth//span/a
/span/p

/span/div /span/div

Second, let's consider UT$_2$ and UT$_3$.

/spanemOn the assumption/span/em that a secure O(c) PoS chain exists, then using that for dapp-chains provides an O(c) bump to scalability. The additional computational burden to a miner or full node is negligible, so UT$_2$ satisfies the decentralization criterion. The security of the simplex isn't affected, and the dapp-chain is more secure than the PoS chain would be in isolation, so UT$_2$ satisfies the security criterion. Since the dapp-chains are PoS, their effective header size will be greater than what is used for the calculations in section /spana href="whitepaper.html#lateximage-whitepaper-63"5.10/span/a. Assuming that the effective header size of dapp-chains is $10\times larger than in section\ /spanahref = $"whitepaper.html$\#lateximage - whitepaper - 63$"$5.10/span/a(and k = 3000, of course), we see that N_2 \in [5\times 10^3, 4\times 10^4] dapp - chains. We also see that \Sigma\mathrm{TPS}_2 \in [6\times 10^4, 5\times 10^5]. By inspection, 60,000 TPS is a lot./span span class = $"$uline$"$ Therefore, \mathrm{UT}_2 satisfies all 3 trilemma criteria/span em as put by Vitalik/span/em./span/span$

/spanemOn the assumption/span/em that a secure O(c^2)$PoS chain exists\ldots you know where this is going. If \mathrm{UT}_3 works, then we'retalking millions of TPS (\sim 10^6 to 10^7) at k = 3000B/s.$

Share security, and */spanemcapacity follows/span/em*.

`<!--................................-->`
/spanh5 id="autosec-452"/spanspan class="sectionnumber"8.2&#x2003;/span/spanAddressing the Stronger Trilemma/span/h5

In section /spana href="whitepaper.html#subsec:trilemma"1.1/span/a, I took issue with the scalability criterion and suggested a stronger version: O(n) transactions in O(1) time.

How can we reasonably estimate what an O(n) transaction load looks like? There are approximately 8 billion people on Earth. If everyone one of them made 1000 transactions per day, the network would require 8 $\times 10^{12} transactionsperday, whichisabout93million(\sim 10^8)TPS.Clearly, thisisgreaterthananyoftheTPSvalueswecalculatedabove.$

For $\mathrm{UT}_{1+\mathrm{HOT}}$ to reach $10^8 TPS, we'dneedalargek > 20000andmaybealongerblockperiod.$

$\mathrm{UT}_{2+\mathrm{HOT}}$, by comparison, is already around the
$\Sigma\mathrm{TPS}_2 \approx 5 \times 10^5 markwithourstandardchainparameters.IfB_f = /spanspanclass = "textrm textmd textup textulc"/spanspanclass = "numerator"1/span/span//spanspanclass = "denominator"60/span/span/span/span, then\Sigma\mathrm{TPS}_2 \approx$
$10^7; however, \Delta Swouldbearound5 \times larger.Thatisasubstantialincreaseinbandwidthrequirements, andpushingto10^8 TPSisnotreallypracticalwithinO(c)constraints.$

Alternatively, we can look to tiling for extra capacity in the first level of nesting. We found in section /spana href="whitepaper.html#lateximage-whitepaper-63"6.5/span/a that tree-tilings can be constructed to be O(n) secure, so all that remains to show is that $\mathrm{UT}_{\aleph}$ is O(n) scalable */spanemand/span/em* decentralized */spanemat the same time/span/em*. The big problem for arbitrarily large tilings is block availability. Smaller tilings with acceptable $\Delta Sarestilluseful, though.$

$\mathrm{UT}_{\aleph 1+\mathrm{HOT}}$ reaches $10^8 TPSaround15layers(d \leq 15)andwouldhaveabout10^7 chains.Ifblockavailabilityintilingsremainsanunsolvedproblem, thenthisisclearlynotpractical.$

On the other hand, $\mathrm{UT}_{\aleph 2+\mathrm{HOT}}$, with standard chain parameters, reaches $\Sigma\mathrm{TPS}_2 \approx 10^8 ataround8layers; a{\sim}100 \times increaseinthenumberoftotalchains(and\Delta S).Thisismuchclosertoreasonableparameters.$

Is it so close that, perhaps, an otherwise unacceptable technique could be used to ensure the availability of all simplex-tile blocks? An example of such a technique might be some kind of division of responsibility between miners so that they only need to ensure blocks from tiles above and below the given tile are available (in addition to blocks from that tile, of course). For all tiles other than the root tile, this would dramatically reduce $\Delta Stoabout2^{-d} \times whatitwouldotherwisebe, wheredisthedepthofthetile.$

So, */spanemif/span/em* the problem of block availability in a tiling can be and is solved, we are clearly O(n) scalable and decentralized with any tiling. However, since it is unsolved, $\mathrm{UT}_{\aleph 2}$ is */spanemnot currently/span/em* O(n) scalable */spanemand/span/em* decentralized, but could potentially reach that in the future.

Finally, what about UT_3 and $\mathrm{UT}_{\aleph 3}$?

A conservative estimate of UT_3's TPS with standard chain parameters is around $10^7.Giventhesensitivityofthisnumbertosmallchangesinchainparameters, wecouldeasilytweakthingsalittletoreach10^8 TPS.$

$\mathrm{UT}_{\aleph 3}$, instead of tweaking parameters, could increase capacity over UT_3 by
$4\times viatilingtojustbrushthesekindsofnumberswithonly3layers(thetilingshowninfig. /spanahref = "whitepaper.html\#lateximage - whitepaper - 63"35c/span/a).Inturn, \Delta Salsoincreasesby4 \times whatitwouldbeforamaximalsimplexwiththesamechainparameters.$

Arguably, if the UT_3 chain parameters are practical, then this is practical. Therefore, with carefully chosen parameters, $\mathrm{UT}_{\aleph 3}$ is secure, O(n) scalable, and decentralized */spanemat the same time/span/em*.

<!--...............................-->
/spanh5 id="autosec-453"/spanspan class="sectionnumber"8.3&#x2003;/span/spanTerminus Est/span/h5

What a ride, ey? Let's recap.

We started from a simple idea: that a blockchain confirming the history of another blockchain was worth */spanemsomething/span/em*. From there, we constructed */spanemProof of Reflection/span/em* — a new consensus primitive that facilitates the summation of chain-work from different chains. We then found ways to convert work between chains that is compatible with our goals and context, allowing otherwise incompatible consensus methods to cooperate. **??** shows how it is the missing idea that addresses the core conflict of the */spanemBlockchain Trilemma/span/em*.

Mutual PoR plus the conversion of work allowed us to construct */spanemthe Simplex/span/em* and UT$_1$ — a highly interconnected network of blockchains with O(c^2)$capacity.Byutilizingone-wayPoRincombinationwithexistingblockchaindesigns,weaddeddapp-chainstoconstruct\mathrm{UT}_2andbumptheoverallcapacitytoO(c^3).BasedontheobservationthatsomecurrentPoSnetworksclaimtobeO(c^2)scalable,weusedthesedesignsasdapp-chainsinsteadtoconstruct\mathrm{UT}_3andreachO(c^4)capacity.$

We then explored the practical side, and found that we needed the /spana href="whitepaper.html#lateximage-whitepaper-63"Axiom of Availability/span/ato ensure reflections between simplex-chains only included available simplex-blocks. As we looked into the specifics of how we would structure blocks and PoRs, we found several optimizations that significantly reduced both the computational and bandwidth requirements for network participants. We also discussed how to support stateless clients and fraud proofs – tools we would need later. This ultimately led us to construct the PoR graph and the /spana href="whitepaper.html#lateximage-whitepaper-63"Axiom of Maximal Reflection/span/a. We adapted NIPoPoWs into NIPoPoWRs, which work for the PoR graph instead of a standalone blockchain. We then integrated block-DAGs to prevent minority attackers from performing effective DoS or censorship attacks, and developed NIPoPPoWs which work with block-DAGs. We realized that we now had the conditions to */spanemrequire/span/em* miners to build on the latest block(s) without any appreciable downside, and enshrined this as the /spana href="whitepaper.html#lateximage-whitepaper-63"Axiom of Unified Ancestry/span/a. Throughout this, we found that these new constructions lacked multiple weaknesses of traditional chains, even though solving those problems was not a goal. Selfish Mining and PoS long range attacks are two such examples. Somewhat surprisingly, we also found that the confirmation rate of a simplex is */spaneminversely proportional to c/span/em*! Not only is this faster than any existing network, but the dynamics of block production cause */spanemminer resonance/span/em*, which acts to synchronize miners and reduces the variance in block production times in PoW simplexes. Using our newfound knowledge, we tested the */spanemConfirmation Equivalence Conjecture/span/em* and found it consistent with reality. Taking all of this, we devised a fast, succinct, secure, and reliable method for intra-simplex cross-chain transactions. As icing on the cake, we developed */spanemexpedited transactions/span/em* that not only allow for near-zero time-to-first-confirmation, but also allows a chain to dynamically expand its capacity by up to $\sim 5\times toheavilymitigatethenegativeeffectsofcongestion.Andwiththat,wecompletedthedesignfor\mathrm{UT}_1andtheSimplex.$

We analyzed this design more formally, and found that UT$_1$ was indeed an O(c^2)$construction.Wealsosawhoweveryrelevantaspectof\mathrm{UT}_1waswithinO(c)constraints,exceptfor\Delta S,whichwasO(c^2).However,whenwecheckedthe/spanemactualnumbers/span/em,wefound\Delta Swaswithinreasonablelimitsandiseasilyaccommodatedbytoday'sstandards.Atthispointwewerealreadyout-scalinganyexistingblockchaindesign,butthatdidn'tstopusconfirmingthescalabilityof\mathrm{UT}_2and\mathrm{UT}_3,andcalculatingsomegirthycapacityestimatesasaresult.$

After all this, we reconsidered how simplex-chains might be organized and discovered */spanemRecursive PoRs/span/em* and */spanemSimplex Tilings/span/em*, with a particular focus on tree-tilings. We developed a security model for tree-tilings that was secure against an attacker with up to 49.5% of the network hash rate. Were it not for the bandwidth requirements on miners imposed by the /spana href="whitepaper.html#lateximage-whitepaper-63"Axiom of Availability/span/a, this would be the ultimate scaling construction as it is practically unbounded — we could always add more tiles, growing the capacity exponentially with the depth of the tiling. We don't know if that availability problem is solvable, but even if it isn't, we saw how smaller tilings are still useful.

Penultimately, we evaluated UT$_1$ against the */spanemTrilemma/span/em* and found that it satisfied all three criteria as originally put. But that was not enough, so we examined each scaling configuration of UT against a stronger trilemma. Although boasting impressive capacity (see **??**, **??**), most configurations fell short of what we set as the standard for O(n) scalability: 100 million TPS. But! We did find one configuration that definitely worked: UT$_{\aleph 3}$.

With that, I think it's finally time to make */spanemthe/span/em* claim.

The */spanemBlockchain Trilemma/span/em* is solved. ■

/spanfigure id="autoid-55" class="table "

/spandiv class="figurecaption"Table 12: Table evaluating UT against trilemma criteria. TPS ranges are the minimum and maximum values for $3000 \leq k \leq 20000B/sacrossallUTvariants.Tilingsarenotlimitedind.$

/span/div

/spantable/spantr class="tbrule"
/spantd class="tdl"UT Config./span/td
/spantd class="tdl"Decentralized?/span/td
/spantd class="tdl"O(n) Secure?/span/td
/spantd colspan="2" class="tdl"O(n) Scalable?/span/td
/span/tr

/spantr class="hline"
/spantd class="tdl"UT$_1$/span/td
/spantd class="tdl"Yes/span/td
/spantd class="tdl"Yes/span/td
/spantd class="tdl"Unlikely/span/td
/spantd class="tdl"(Max. TPS: $\sim$ 1K - 400K)/span/td
/span/tr

/spantr
/spantd class="tdl"UT$_2$/span/td
/spantd class="tdl"Yes/span/td
/spantd class="tdl"Yes/span/td
/spantd class="tdl"Possibly/span/td
/spantd class="tdl"(Max. TPS: $\sim$ 60K - 2B)/span/td
/span/tr

/spantr
/spantd class="tdl"UT$_3$/span/td
/spantd class="tdl"Yes/span/td
/spantd class="tdl"Yes/span/td
/spantd class="tdl"Probably/span/td
/spantd class="tdl"(Max. TPS: $\sim$ 10M - 6T)/span/td
/span/tr

/spantr
/spantd class="tdl"UT$_{\aleph 1}$/span/td
/spantd class="tdl"Maybe/spansup128/span/sup/span/td
/spantd class="tdl"Yes/span/td
/spantd class="tdl"Probably/span/td
/spantd class="tdl"(Max. TPS: $\sim$ 50M - $10^{18})/span/td$
$/span/tr$

UT$_{\aleph 2}$/span/td
/spantd class="td"Maybe/span/td
/spantd class="td "Yes/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "Yes/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "(Max. TPS: $\sim$ 30B - $10^{21})/span/td$
$/spantdclass = ”tdat”/span/td$
$/spantdclass = ”tdbang”/span/td$
$/span/tr$

UT$_{\aleph 3}$/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "Yes/spansup129/span/sup/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "Yes/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "Yes/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "(Max. TPS: $\sim 10^{12} - 10^{25})/span/td$
$/spantdclass = ”tdat”/span/td$
$/spantdclass = ”tdbang”/span/td$
$/span/tr$

/span/tr/span/table /span/figure

/spanfigure id="autoid-56" class="table "

/spandiv class="figurecaption"Table 13: UT capacity vs. Earth with standard chain parameters and heavy dapp-chains headers. Parameters: k = 3000 B/s; B$_f$ $= /spanspanclass = ”textrm\ textmd\ textup\ textulc”/spanspanclass = ”numerator”1/span/span//spanspanclass = ”denominator”15/span/span/span/spanHz; B_h = 112B; D_f = /spanspanclass = ”textrm\ textmd\ textup\ textulc”/spanspanclass = ”numerator”1/span/span//spanspanclass = ”denominator”15/span/span/span/spanHz; D_h = 560B./spanemPercapita/span/emhereassumesapopulationof10billionpeople.Inallcases, theUTvariantis+$
$HO.Tilingshaved = 3./spanbr/$
$\Delta S : BWreq.forminers; \Delta s : syncBWreq.forfullnodes(PoRgraph + 1simplex - chain).$

/span/div

/spantable/spantr class="tbrule"
/spantd class="tdl"UT Config./span/td
/spantd class="tdr"Σ TPS*/span/td*
/spantdclass = ”tdr”Tx/day/capita/span/td
/spantdclass = ”tdr”$\Delta s(kB/s)$*/span/td*
/spantdclass = ”tdr”(TB/mo)/span/td
/spantdclass = ”tdr”$\Delta S(kB/s)$*/span/td*
/spantdclass = ”tdr”(TB/mo)/span/td
/span/tr

UT$_1$/span/td
/spantd class="tdr"4,219/span/td
/spantd class="tdr"0.036/span/td
/spantd class="tdr"10.8/span/td
/spantd class="tdr"0.028/span/td
/spantd class="tdr"1,097/span/td
/spantd class="td"2.884/span/td
/span/tr

/spantr
/spantd class="tdl"UT$_2$/span/td
/spantd class="tdr"339,007/span/td
/spantd class="tdr"2.929/span/td
/spantd class="tdr"10.8/span/td
/spantd class="tdr"0.028/span/td
/spantd class="tdr"1,097/span/td
/spantd class="tdr"2.884/span/td
/span/tr

/spantr
/spantd class="tdl"UT$_3$/span/td
/spantd class="tdr"27,241,610/span/td
/spantd class="tdr"235.368/span/td
/spantd class="tdr"10.8/span/td
/spantd class="tdr"0.028/span/td
/spantd class="tdr"1,097/span/td
/spantd class="tdr"2.884/span/td
/span/tr

/spantr
/spantd class="tdl"UT$_{\aleph 1}$ (d=3)/span/td
/spantd class="tdr"16,875/span/td
/spantd class="tdr"0.146/span/td
/spantd class="tdr"43.1/span/td
/spantd class="tdr"0.113/span/td
/spantd class="tdr"4,386/span/td
/spantd class="tdr"11.535/span/td
/span/tr

/spantr
/spantd class="tdl"UT$_{\aleph 2}$ (d=3)/span/td
/spantd class="tdr"1,356,027/span/td
/spantd class="tdr"11.716/span/td
/spantd class="tdr"43.1/span/td
/spantd class="tdr"0.113/span/td
/spantd class="tdr"4,386/span/td
/spantd class="tdr"11.535/span/td
/span/tr

/spantr
/spantd class="tdl"UT$_{\aleph 3}$ (d=3)/span/td
/spantd class="tdr"108,966,438/span/td
/spantd class="tdr"941.470/span/td
/spantd class="tdr"43.1/span/td
/spantd class="tdr"0.113/span/td
/spantd class="tdr"4,386/span/td
/spantd class="tdr"11.535/span/td
/span/tr

/spantr class="tbrule"
/spantd class="tdl"/span/td
/spantd class="tdr"/span/td
/spantd class="tdr"/span/td
/spantd class="tdr"/span/td
/spantd class="tdr"/span/td
/spantd class="tdr"/span/td
/spantd class="tdr"/span/td
/span/tr/span/table /span/figure

/spanfigure id="autoid-57" class="table "

/spandiv class="figurecaption"Table 14: UT capacity vs. Earth, as in **??** with k = 6000 B/s $(3.5\times Bitcoin's); D_h = 1024B$.

/span/div

```
/spantable/spantr class="tbrule"
/spantd class="tdl"UT Config./span/td
/spantd class="tdr"Σ TPS/span/td
/spantdclass = "tdr"Tx/day/capita/span/td
/spantdclass = "tdr"Δs(kB/s)/span/td
/spantdclass = "tdr"(TB/mo)/span/td
/spantdclass = "tdr"ΔS(kB/s)/span/td
/spantdclass = "tdr"(TB/mo)/span/td
/span/tr


UT$_1$/span/td
/spantd class="tdr"16,875/span/td
/spantd class="tdr"0.146/span/td
/spantd class="tdr"22.1/span/td
/spantd class="tdr"0.058/span/td
/spantd class="tdr"4,373/span/td
/spantd class="td"11.500/span/td
/span/tr


/spantr
/spantd class="tdl"UT$_2$/span/td
/spantd class="tdr"1,483,154/span/td
/spantd class="tdr"12.814/span/td
/spantd class="tdr"22.1/span/td
/spantd class="tdr"0.058/span/td
/spantd class="tdr"4,373/span/td
/spantd class="tdr"11.500/span/td
/span/tr


/spantr
/spantd class="tdl"UT$_3$/span/td
/spantd class="tdr"130,355,358/span/td
/spantd class="tdr"1,126.270/span/td
/spantd class="tdr"22.1/span/td
/spantd class="tdr"0.058/span/td
/spantd class="tdr"4,373/span/td
/spantd class="tdr"11.500/span/td
/span/tr


/spantr
/spantd class="tdl"UT$_{\aleph1}$ (d=3)/span/td
/spantd class="tdr"67,500/span/td
/spantd class="tdr"0.583/span/td
/spantd class="tdr"88.5/span/td
/spantd class="tdr"0.233/span/td
/spantd class="tdr"17,492/span/td
/spantd class="tdr"45.999/span/td
/span/tr


/spantr
/spantd class="tdl"UT$_{\aleph2}$ (d=3)/span/td
/spantd class="tdr"5,932,617/span/td
/spantd class="tdr"51.258/span/td
/spantd class="tdr"88.5/span/td
/spantd class="tdr"0.233/span/td
/spantd class="tdr"17,492/span/td
/spantd class="tdr"45.999/span/td
/span/tr


/spantr
/spantd class="tdl"UT$_{\aleph3}$ (d=3)/span/td
/spantd class="tdr"521,421,432/span/td
/spantd class="tdr"4,505.081/span/td
/spantd class="tdr"88.5/span/td
/spantd class="tdr"0.233/span/td
/spantd class="tdr"17,492/span/td
/spantd class="tdr"45.999/span/td
/span/tr


/spantr class="tbrule"
/spantd class="tdl"/span/td
/spantd class="tdr"/span/td
/spantd class="tdr"/span/td
/spantd class="tdr"/span/td
/spantd class="tdr"/span/td
/spantd class="tdr"/span/td
/spantd class="tdr"/span/td
/span/tr/span/table  /span/figure

/spanfigure id="autoid-58" class="figure "
/spana href="./includes/ut/diags/trilemma/conflict_resolution_sag.png" target="_blank" /spanimg
    src="./includes/ut/diags/trilemma/conflict_resolution_sag.png"
    class="inlineimage"
    alt="(image)"
/span/a


/spandiv class="figurecaption"Figure 38:  A solution to the core conflict of the /spaniTrilemma/span/i.

/span/div  /span/figure

%
```

/spandiv role="note" class="footnotes"
/spanp
/spansup128/span/sup&nbsp; If the block availability problem can be solved for tilings, then all tilings are O(n) scalable since their capacity is unbounded. If it cannot, tilings are still useful but do not provide O(n) scaling on their own.
/span/p

/spanp
/spansup129/span/sup&nbsp; $\text{UT}_{\aleph 3}$ is decentralized if the maximum tiling depth is kept low. As we found earlier, $\text{UT}_{\aleph 3}$ with d
$\leq 3 providedatolerable \Delta S, andleadtoanetwork-wideTPSofaround 10^8, eventhoughweassumeddapp-chainheaderstobe 10 \times worsethanthetheoreticalmaximum (dapp-chainheaderswouldbe \sim 1000 bytes)./spanemThisspecificcase/span/emiswhat “Yes'' refersto.$
/span/p

/span/div

<!--................................-->
/spanh4 id="autosec-458"/spanspan class="sectionnumber"9&#x2003;/span/spanCriticisms of UT/span/h4

This section documents the major reasons that UT might not work — or, equivalently, the most valuable contributions. I have also provided some commentary on the criticisms.

/spanspan class="uline"There are no known decisive criticisms of UT_1./span/span

`<!--................................-->`
/spanh5 id="autosec-459"/spanspan class="sectionnumber"9.1&#x2003;/span/spanOpen Problems/span/h5

<!--..............................-->
/spanh6 id="autosec-460"/spanspan class="sectionnumber"9.1.1&#x2003;/span/spanDapp-chains &amp; PoS/span/h6

Main content: section /spana href="whitepaper.html#lateximage-whitepaper-63"3.4/span/a.

/spanbCriticism:/span/b Dapp-chains are based on the assumption that a secure PoS method exists when adapted for one-way PoR. No specific PoS method is provided and it's not clear that PoS + PoR actually works without a clear design for such a system. The other claims about dapp-chains and UT_2 scaling and above is contingent on a suitable PoS method being found or created. PoS is insecure when used in isolation and it isn't clear that PoS can be secure, even with the help of PoR and a purely PoW simplex. Even if a suitable method of PoS is found, PoS + PoR is inefficient compared to PoW + PoR and some capacity estimates are therefore overly optimistic.

/spanbCommentary:/span/b This criticism is correct if PoS is never secure, although it is not the complete picture. There are some cases that work — PoA dapp-chains and non-financial dapps, for example. However, the goal is a universal method that works for any dapp-chain and allows for secure cross-chain transactions between the simplex and dapp-chains. And while we don't have an otherwise-working PoS + PoR implementation, we also have no reason to believe that conversion is impossible or that PoR is incompatible with PoS.

If PoS + PoR is secure in a PoW context, there is no other fundamental barrier to UT_2 or UT_3. Thus this criticism is /spanspan class="uline"possibly refuted/span/span */spanemif and only if/span/em* PoS + PoR to PoW is secure.

<!--................................-->
/spanh6 id="autosec-461"/spanspan class="sectionnumber"9.1.2&#x2003;/span/spanUT$_{\aleph 1} BlockAvailability\&amp;O(c^2 \log n)Bandwidth/span/h6$

Main content: section /spana href="whitepaper.html#lateximage-whitepaper-63"6.4.3/span/a and section /spana href="whitepaper.html#lateximage-whitepaper-63"6.6.3/span/a.

/spanbCriticism:/span/b Without a solution to block availability, UT$_\aleph$ doesn't work for d $\gtrsim 4. Itworkstoexpandcapacitybyupanorderofmagnitudebutnotbeyond. Eventhatmightnotworkbecausethesecuritymodelisuntestedandtherearenocomparable PoW - basedreal - worldexamplestoexamine. O(c^2 \log n) bandwidthforvalidatingnodesistoohigh. O(nc^{-i}) bandwidthforroot - tileminersistoohigh.$

/spanbCommentary:/span/b While UT$_\aleph$ is useful, it is not a complete solution to O(n) scaling. That it */spanemis/span/em* useful hints that there might be more and better PoR-based geometries to discover. If a satisfactory solution to block availability can be found then it will also solve the bandwidth problem and the bottleneck will be the */spanemgradient of work/span/em* required as distance from the core increases (d$_{/spanspanclass="textsc"Max/span/span} \in [10, 30]$).

`<!--................................-->`
/spanh5 id="autosec-462"/spanspan class="sectionnumber"9.2&#x2003;/span/spanMitigated Risks/span/h5

`<!--................................-->`
/spanh6 id="autosec-463"/spanspan class="sectionnumber"9.2.1&#x2003;/span/spanA Key Point for UT$_1$*IsFlawed/span/h*6

/spanbRisk:/span/b Several key components are required for UT$_1$ to work. There are /spanspan class="uline"no */spanemknown/span/em* decisive criticisms/span/span but maybe you can find one? PoR and Conversion of Work (section /spana href="whitepaper.html#sec:proof-of-reflection"2/span/a), Simplex Security (section /spana href="whitepaper.html#lateximage-whitepaper-63"4.10.1/span/a), and Intra-Simplex Cross-Chain Transactions (section /spana href="whitepaper.html#lateximage-whitepaper-63"4.11/span/a) are three such components.

<!--................................-->
/spanh6 id="autosec-464"/spanspan class="sectionnumber"9.2.2&#x2003;/span/spanUT$_1 O(c^2) Bandwidth/span/h6$

Main content: section /spana href="whitepaper.html#lateximage-whitepaper-63"4.1/span/a.

/spanbRisk:/span/b Whether
$\Delta S is tolerable depends on k (block size) and /spanspan class = "uline" is mitigated/span/span with a suitable k (see \textbf{??}, \textbf{??}). A k of 3000B/s has plenty of breathing room, and 20,000B/s is on the high end of what could possibly work. For comparison, Bitcoin has k \approx 1700B/s.$

`<!--................................-->`
/spanh4 id="autosec-465"Notation/span/h4

/spanfigure id="autoid-59" class="table "

```
/spantable/spantr
/spantd colspan="3" class="tdp"
/spandiv class="figurecaption"Table 15:&nbsp;Notation defined in this document. /span/div
/span/td
/span/tr

/spantr class="tbrule"
/spantd class="tdl"
```

/spandiv class="minipage" style="vertical-align:top ; justify-content:flex-start ; "
Term
/span/div
/span/td
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:top ; justify-content:flex-start ; "
Definition
/span/div
/span/td
/spantd class="tdl"

```
/spandiv class="minipage" style="vertical-align:top ; justify-content:flex-start ; "
Unit
/span/div
/span/td
/span/tr

/spantr class="hline"
/spantd class="tdl"/span/td
/spantd class="tdl"/span/td
/spantd class="tdl"/span/td
/span/tr

/spantr class="tbrule"
/spantd class="tdl"
```

/spandiv class="minipage" style="vertical-align:top ; justify-content:flex-start ; "
Term
/span/div
/span/td
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:top ; justify-content:flex-start ; "
Definition
/span/div
/span/td
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:top ; justify-content:flex-start ; "
Unit
/span/div
/span/td
/span/tr

/spantr class="hline"
/spantd class="tdl"

```
/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
c
/span/div
/span/td
/spantd class="tdl"
```

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
Abstract representation of per-node computational resources.
/span/div
/span/td
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
-
/span/div
/span/td
/span/tr

/spantr
/spantd class="tdl"

```
/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
n
/span/div
/span/td
/spantd class="tdl"
```

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
Abstract representation of network size.
/span/div
/span/td
/spantd class="tdl"

```
/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
-
/span/div
/span/td
/span/tr

/spantr
/spantd class="tdl"
```

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
p
/span/div
/span/td
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
Proportion of the network’s hash-rate controlled by honest nodes.
/span/div
/span/td
/spantd class="tdl"

```
/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
-
/span/div
/span/td
/span/tr

/spantr
/spantd class="tdl"
```

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
q
/span/div
/span/td
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
Proportion of the network's hash-rate controlled by the attacker.
/span/div
/span/td
/spantd class="tdl"

```
/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
-
/span/div
/span/td
/span/tr

/spantr
/spantd class="tdl"
```

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
L_d, R_d
/span/div
/span/td
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
The difficulty of chain L/R.
/span/div
/span/td
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
hashes/block
/span/div
/span/td
/span/tr

/spantr
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
L_r, R_r
/span/div
/span/td
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
The block reward of chain L/R.
/span/div
/span/td
/spantd class="tdl"

```
/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
coins/block
/span/div
/span/td
/span/tr

/spantr
/spantd class="tdl"
```

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
w
/span/div
/span/td
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
Some amount of */spanemwork/span/em*.
/span/div
/span/td
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
hashes
/span/div
/span/td
/span/tr

/spantr
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
$\mathrm{X}_{R\to L}$
/span/div
/span/td
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
Exchange rate between L-coins and R-coins.
/span/div
/span/td
/spantd class="tdl"

```
/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
L-coins/R-coin
/span/div
/span/td
/span/tr

/spantr
/spantd class="tdl"
```

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
C'
/span/div
/span/td
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
Confirmation rate.
/span/div
/span/td
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
Hz
/span/div
/span/td
/span/tr

/spantr
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
g
/span/div
/span/td
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
Hash digest size.
/span/div
/span/td
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
bytes
/span/div
/span/td
/span/tr

/spantr
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
k_i
/span/div
/span/td
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
A generalization of block size: the average per-chain raw throughput at the i$^{th}levelofnesting$.
/span/div
/span/td
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
bytes/second
/span/div
/span/td
/span/tr

/spantr
/spantd class="tdl"

```
/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
k
/span/div
/span/td
/spantd class="tdl"
```

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
Average per-chain raw throughput across nesting levels. k is used to simplify reasoning and equations, esp.&nbsp;when all k$_i$*areequal*.
/span/div
/span/td
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
bytes/second
/span/div
/span/td
/span/tr

/spantr
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
T_i
/span/div
/span/td
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
Network throughput at the i$^{th} levelofnesting$.
/span/div
/span/td
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
bytes/second
/span/div
/span/td
/span/tr

/spantr
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
N$_i$
/span/div
/span/td
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
Number of chains at the i$^{th}levelofnesting$.
/span/div
/span/td
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
chains
/span/div
/span/td
/span/tr

/spantr
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
N_{tiles}
/span/div
/span/td
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
Number of tiles in a simplex-tiling.
/span/div
/span/td
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
tiles
/span/div
/span/td
/span/tr

/spantr
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
B$_f$, L_f
/span/div
/span/td
/spantd class="tdl"

```
/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
Base-chain block frequency.
/span/div
/span/td
/spantd class="tdl"
```

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
Hz
/span/div
/span/td
/span/tr

/spantr
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
B_h
/span/div
/span/td
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
Base-chain header size.
/span/div
/span/td
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
bytes
/span/div
/span/td
/span/tr

/spantr
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
D_f
/span/div
/span/td
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
Dapp-chain block frequency.
/span/div
/span/td
/spantd class="tdl"

```
/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
Hz
/span/div
/span/td
/span/tr

/spantr
/spantd class="tdl"
```

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
D_h
/span/div
/span/td
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
Dapp-chain header size.
/span/div
/span/td
/spantd class="tdl"

```
/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
bytes
/span/div
/span/td
/span/tr

/spantr
/spantd class="tdl"
```

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
ϕ
/span/div
/span/td
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
Propagation delay across the network
/span/div
/span/td
/spantd class="tdl"

```
/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
seconds
/span/div
/span/td
/span/tr

/spantr
/spantd class="tdl"
```

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
Σ TPS_i
/span/div
/span/td
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
Network-wide transactions per second at the $\mathrm{i}^{\mathrm{th}} levelofnesting(givennoadditionallevels)$.
/span/div
/span/td
/spantd class="tdl"

```
/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
tx/s
/span/div
/span/td
/span/tr

/spantr
/spantd class="tdl"
```

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
Δs
/span/div
/span/td
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
Minimum network bandwidth for a full node to remain in sync with a single simplex-chain (whilst also validating PoRs).
/span/div
/span/td
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
bytes/second
/span/div
/span/td
/span/tr

/spantr
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
Δr
/span/div
/span/td
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
Minimum network bandwidth for a full node to fully reconstruct the PoR graph.
/span/div
/span/td
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
bytes/second
/span/div
/span/td
/span/tr

/spantr
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
ΔS
/span/div
/span/td
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
Minimum network bandwidth for a */spanemmining/span/em* (or fully validating) node to ensure block availability over all reflected simplex-chains.
/span/div
/span/td
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
bytes/second
/span/div
/span/td
/span/tr

/spantr
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
DAA$_N$
/span/div
/span/td
/spantd class="tdl"

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
The number of blocks over which the DAA operates.
/span/div
/span/td
/spantd class="tdl"

```
/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "
blocks
/span/div
/span/td
/span/tr

/spantr class="tbrule"
/spantd class="tdl"/span/td
/spantd class="tdl"/span/td
/spantd class="tdl"/span/td
/span/tr/span/table/span/figure
```

<!--................................-->
/spanh4 id="autosec-548"Nomenclature/span/h4

/spanfigure id="autoid-60" class="table "

/spandiv class="center"

/spandiv class="figurecaption"Table 16: Nomenclature defined in this document.

/span/div

/spantable/spantr class="tbrule"
/spantd class="tdl"Term/span/td
/spantd class="tdl"Definition/span/td
/spantd class="tdl"Reference/span/td
/span/tr

/spantr class="hline"
/spantd class="tdl"UT$_i$/span/td
/spantd class="tdl"The UT scaling configuration with i levels of nesting./span/td
/spantd class="tdl"section /spana href="whitepaper.html#lateximage-whitepaper-63"3/span/a/span/td
/span/tr

/spantr
/spantd class="tdl"UT$_1$/span/td
/spantd class="tdl"UT with base-level chains only — O(c^2)*scalability./span/td*
*/spantdclass = "tdl"section /spanahref = "*whitepaper.html#*lateximage − whitepaper −* 63"3.2*/span/a/span/td*
/span/tr

UT$_2$/span/td
/spantd class="tdl"UT with nested dapp-chains — O(c^3)*scalability./span/td*
*/spantdclass = "td"section /spanahref = "*whitepaper.html#*lateximage − whitepaper −* 63"3.4*/span/a/span/td*
/span/tr

UT$_3$/span/td
/spantd class="td"UT with nested dapp-dapp-chains — O(c^4)*scalability./span/td*
*/spantdclass = "td"section /spanahref = "*whitepaper.html#*lateximage − whitepaper −* 63"3.4*/span/a/span/td*
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/span/tr

UT$_{\aleph i}$/span/td
/spantd class="td "Tiling of UT$_i$ — O(n) scalability./span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "section /spana href="whitepaper.html#lateximage-whitepaper-63"6/span/a/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/span/tr

/spantr
/spantd class="tdl"+PoRs/span/td
/spantd class="tdl"Protocol variant: explicit proofs./span/td
/spantd class="tdl"section /spana href="whitepaper.html#lateximage-whitepaper-63"4.2/span/a/span/td
/span/tr

/spantr
/spantd class="tdl"+PoRTs/span/td
/spantd class="tdl"Protocol variant: explicit proofs + T./span/td
/spantd class="tdl"**??**/span/td
/span/tr

/spantr
/spantd class="tdl"+HOPoRs/span/td
/spantd class="tdl"Protocol variant: explicit proofs + header omission./span/td
/spantd class="tdl"section /spana href="whitepaper.html#lateximage-whitepaper-63"4.4/span/a/span/td
/span/tr

/spantr
/spantd class="tdl"+HOPoRTs/span/td
/spantd class="tdl"Protocol variant: explicit proofs + header omission + T./span/td
/spantd class="tdl"**??**/span/td
/span/tr

/spantr
/spantd class="tdl"+OP/span/td
/spantd class="tdl"Protocol variant: omitted proofs./span/td
/spantd class="tdl"section /spana href="whitepaper.html#lateximage-whitepaper-63"4.2/span/a/span/td
/span/tr

/spantr
/spantd class="tdl"+OPT/span/td
/spantd class="tdl"Protocol variant: omitted proofs + T./span/td
/spantd class="tdl"section /spana href="whitepaper.html#lateximage-whitepaper-63"4.4/span/a/span/td
/span/tr

/spantr
/spantd class="tdl"+HO/span/td
/spantd class="tdl"Protocol variant: omitted proofs + header omission./span/td
/spantd class="tdl"section /spana href="whitepaper.html#lateximage-whitepaper-63"4.4/span/a/span/td
/span/tr

/spantr
/spantd class="tdl"+HOT/span/td
/spantd class="tdl"Protocol variant: omitted proofs + header omission + T./span/td
/spantd class="tdl"section /spana href="whitepaper.html#lateximage-whitepaper-63"4.4/span/a/span/td
/span/tr

/spantr class="tbrule"
/spantd class="tdl"/span/td
/spantd class="tdl"/span/td
/spantd class="tdl"/span/td
/span/tr/span/table /span/div/span/figure

<!--.................................-->
/spanh4 id="autosec-551"List of Figures/span/h4

/spannav class="lof"/span/nav

`<!--................................-->`
/spanh4 id="autosec-552"List of Tables/span/h4

/spannav class="lot"/span/nav

<!--................................-->
/spanh4 id="autosec-553"Key References/span/h4

/spannav class="citation"/span/nav

<!--................................-->
/spanh4 id="autosec-554"Supplementary References/span/h4

/spannav class="suppcites"/span/nav

<!--................................-->
/spanh4 id="autosec-555"/spanspan class="sectionnumber"A&#x2003;/span/spanComparison: “The Big 4”/span/h4

The following comparisons (**??**, **??**, and **??**) are intended to be an */spanemapples to apples/span/em* comparison between UT variants and “The Big 4”: Bitcoin, Cardano, Eth2, and Polkadot. Cardano, as a cutting-edge network, is an exception here: the Cardano/IOHK teams have not been pursuing */spanemLayer 1/span/em* scalability solutions and are focusing instead on a */spanemLayer 2/span/em* solution: Hydra via EUTXOs. Ethereum is also an exception of sorts: these comparisons use the original */spanemEth2/span/em* sharding roadmap as an example of a sharded Layer 1, whereas the Ethereum network itself is now pursuing a rollup-centric scaling strategy via their beacon chain. I mention this particularly because the nature of an */spanemapples to apples/span/em* comparison casts Cardano and Ethereum in a light that some might consider to be misleading. However, these comparisons deliberately do not consider */spanemLayer 2/span/em* scalability solutions for one very simple reason: all networks can implement them in some fashion. It is not a fair (or accurate) comparison */spanemof blockchain architecture/span/em* if */spanemLayer 2/span/em* scaling solutions are considered for some networks and not for others.

In addition to those four, a network named */spanemOpt.Shard/span/em* (for */spanemOptimal Sharding/span/em*) is included in these comparisons. This is a theoretical network which uses the */spanembest parameters possible among UT configurations/span/em*. No real-world sharded network has come close to this level of performance, and it is incompatible with PoS.

/spandiv
    class="tcolorbox"
    style=" border: 1px solid #404040; background: #F2F2F2; "

/spandiv
    class="tcolorboxupper"
    style=" color: #000000; "
/spanspan role="note" class="marginpar"**/spanbTerm/span/b**/span/span **/spanb/spana href="whitepaper.html#??"Scaling Factor/span/a/span/b**: Also: “Scale
×″*.Foragivenk,itisthefactorbywhichTPSincreaseswithanadditionalnestinglevel.Ineffect,itallowsforcomparisonoftheefficacyofscalingschemeswhenkisfixed.Forsomedesigns,the/spanemScalingFactor/span/emcanchangebetweennestinglevels./span/div*
/span/div

/spanfigure id="autoid-61" class="table "

/spandiv class="center"

/spandiv class="figurecaption"Table 17: A comparison of quantitative scaling properties between UT and various networks given k = 3000 bytes/s. Transaction size is set to 250 bytes, D$_f = B_f, and D_h = B_h. Note : `\infty' heredoesnotliterallymeaninfinite; however, wecannotcalculateconcretevalueswithoutspecifyingparametersarbitrarilywhichareotherw$
/span/div

```
/spantable/spantr class="tbrule"
/spantd class="tdl"k, B_f, B_h/span/td
/spantdclass = ”tdl”Network/span/td
/spantdclass = ”tdr”E.B_h(B)/span/td
/spantdclass = ”tdr”E.D_h(B)/span/td
/spantdclass = ”tdr”Scale×/span/td
/spantdclass = ”tdr”N_2/span/td
/spantdclass = ”tdr”ΣTPS/span/td
/span/tr


3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"600/span/span/span/span, 80/span/td
/spantd class="tdl"Bitcoin/span/td
/spantd class="tdr"80/span/td
/spantd class="tdr"-/span/td
/spantd class="tdr"1/span/td
/spantd class="tdr"-/span/td
/spantd class="tdr"12/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"20/span/span/span/span, 1070/span/td
/spantd class="tdl"Cardano/span/td
/spantd class="tdr"1070/span/td
/spantd class="tdr"-/span/td
/spantd class="tdr"1/span/td
/spantd class="tdr"-/span/td
/spantd class="tdr"12/span/td
/span/tr


/spantr
/spantd class="tdl"3000, 1.58, 144/span/td
/spantd class="tdl"Solana/span/td
/spantd class="tdr"144/span/td
/spantd class="tdr"-/span/td
/spantd class="tdr"1/span/td
/spantd class="tdr"-/span/td
/spantd class="tdr"12/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 84/span/td
/spantd class="tdl"UT_1+PoRTs/span/td
/spantd class="tdr"99/span/td
/spantd class="tdr"-/span/td
/spantd class="tdr"1/span/td
/spantd class="tdr"-/span/td
/spantd class="tdr"1,363/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 84/span/td
/spantd class="tdl"UT_1+HOPoRTs/span/td
/spantd class="tdr"56/span/td
/spantd class="tdr"-/span/td
/spantd class="tdr"1/span/td
/spantd class="tdr"-/span/td
/spantd class="tdr"2,319/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 84/span/td
/spantd class="tdl"UT_1+HOT/span/td
/spantd class="tdr"16/span/td
/spantd class="tdr"-/span/td
/spantd class="tdr"1/span/td
/spantd class="tdr"-/span/td
/spantd class="tdr"8,437/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"6/span/span/span/span, 288/span/td
/spantd class="tdl"Polkadot/span/td
/spantd class="tdr"288/span/td
/spantd class="tdr"819/span/td
/spantd class="tdr"21/span/td
/spantd class="tdr"21/span/td
/spantd class="tdr"263/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"12/span/span/span/span, 200/span/td
/spantd class="tdl"Eth2/span/td
/spantd class="tdr"200/span/td
/spantd class="tdr"312/span/td
/spantd class="tdr"115/span/td
/spantd class="tdr"115/span/td
/spantd class="tdr"1,384/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 66/span/td
/spantd class="tdl"Opt.Shard/span/td
/spantd class="tdr"66/span/td
/spantd class="tdr"66/span/td
/spantd class="tdr"681/span/td
/spantd class="tdr"681/span/td
/spantd class="tdr"8,181/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 84/span/td
/spantd class="tdl"UT_2+PoRTs/span/td
/spantd class="tdr"99/span/td
/spantd class="tdr"66/span/td
/spantd class="tdr"681/span/td
/spantd class="tdr"77,479/span/td
/spantd class="tdr"929,750/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 84/span/td
/spantd class="tdl"UT_2+HOPoRTs/span/td
/spantd class="tdr"56/span/td
/spantd class="tdr"66/span/td
/spantd class="tdr"681/span/td
/spantd class="tdr"131,797/span/td
/spantd class="tdr"1,581,574/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 84/span/td
/spantd class="tdl"UT_2+HOT/span/td
/spantd class="tdr"16/span/td
/spantd class="tdr"66/span/td
/spantd class="tdr"681/span/td
/spantd class="tdr"479,403/span/td
/spantd class="tdr"5,752,840/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 84/span/td
/spantd class="tdl"UT_N2+HOT/span/td
/spantd class="tdr"16/span/td
/spantd class="tdr"66/span/td
/spantd class="tdr"681/span/td
/spantd class="tdr"∞/span/td
/spantdclass = ”tdr”∞/span/td
/span/tr


/span/td
/spantd class="tdr"/span/td
/span/tr/span/table /span/div/span/figure

/spanfigure id="autoid-62" class="table "
```

/spandiv class="center"

/spandiv class="figurecaption"Table 18: Similar to the previous table, but with $k = 20$ kB/s instead of 3 kB/s.
/span/div

```
/spantable/spantr class="tbrule"
/spantd class="tdl"k, $B_f$, $B_h$/span/td
/spantdclass = ”tdl”Network/span/td
/spantdclass = ”tdr”E.$B_h$(B)/span/td
/spantdclass = ”tdr”E.$D_h$(B)/span/td
/spantdclass = ”tdr”Scale×/span/td
/spantdclass = ”tdr”$N_2$/span/td
/spantdclass = ”tdr”ΣTPS/span/td
/span/tr

20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"600/span/span/span/span, 80/span/td
/spantd class="tdl"Bitcoin/span/td
/spantd class="tdr"80/span/td
/spantd class="tdr"-/span/td
/spantd class="tdr"1/span/td
/spantd class="tdr"-/span/td
/spantd class="tdr"80/span/td
/span/tr

/spantr
/spantd class="tdl"20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"20/span/span/span/span, 1070/span/td
/spantd class="tdl"Cardano/span/td
/spantd class="tdr"1070/span/td
/spantd class="tdr"-/span/td
/spantd class="tdr"1/span/td
/spantd class="tdr"-/span/td
/spantd class="tdr"80/span/td
/span/tr

/spantr
/spantd class="tdl"20000, 1.58, 144/span/td
/spantd class="tdl"Solana/span/td
/spantd class="tdr"144/span/td
/spantd class="tdr"-/span/td
/spantd class="tdr"1/span/td
/spantd class="tdr"-/span/td
/spantd class="tdr"80/span/td
/span/tr

/spantr
/spantd class="tdl"20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 84/span/td
/spantd class="tdl"$UT_{1+PoRTs}$/span/td
/spantd class="tdr"125/span/td
/spantd class="tdr"-/span/td
/spantd class="tdr"1/span/td
/spantd class="tdr"-/span/td
/spantd class="tdr"48,009/span/td
/span/tr

/spantr
/spantd class="tdl"20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 84/span/td
/spantd class="tdl"$UT_{1+HOPoRTs}$/span/td
/spantd class="tdr"77/span/td
/spantd class="tdr"-/span/td
/spantd class="tdr"1/span/td
/spantd class="tdr"-/span/td
/spantd class="tdr"77,078/span/td
/span/tr

/spantr
/spantd class="tdl"20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 84/span/td
/spantd class="tdl"$UT_{1+HOT}$/span/td
/spantd class="tdr"16/span/td
/spantd class="tdr"-/span/td
/spantd class="tdr"1/span/td
/spantd class="tdr"-/span/td
/spantd class="tdr"375,000/span/td
/span/tr

/spantr
/spantd class="tdl"20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"6/span/span/span/span, 288/span/td
/spantd class="tdl"Polkadot/span/td
/spantd class="tdr"288/span/td
/spantd class="tdr"819/span/td
/spantd class="tdr"146/span/td
/spantd class="tdr"146/span/td
/spantd class="tdr"11,721/span/td
/span/tr

/spantr
/spantd class="tdl"20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"12/span/span/span/span, 200/span/td
/spantd class="tdl"Eth2/span/td
/spantd class="tdr"200/span/td
/spantd class="tdr"312/span/td
/spantd class="tdr"769/span/td
/spantd class="tdr"769/span/td
/spantd class="tdr"61,538/span/td
/span/tr

/spantr
/spantd class="tdl"20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 66/span/td
/spantd class="tdl"Opt.Shard/span/td
/spantd class="tdr"66/span/td
/spantd class="tdr"66/span/td
/spantd class="tdr"4,545/span/td
/spantd class="tdr"4,545/span/td
/spantd class="tdr"363,636/span/td
/span/tr

/spantr
/spantd class="tdl"20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 84/span/td
/spantd class="tdl"$UT_{2+PoRTs}$/span/td
/spantd class="tdr"125/span/td
/spantd class="tdr"66/span/td
/spantd class="tdr"4,545/span/td
/spantd class="tdr"2,727,810/span/td
/spantd class="tdr"$2.18×10^8$/span/td
/span/tr

20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 84/span/td
/spantd class="td"$UT_{2+HOPoRTs}$/span/td
/spantd class="td "77/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "66/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "4,545/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "4,379,483/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "$3.50×10^8$/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/span/tr

20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 84/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "$UT_{2+HOT}$/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "16/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "66/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "4,545/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "21,306,818/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "$1.70×10^9$/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/span/tr

20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 84/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "$UT_{82+HOT}$/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "16/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "66/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "4,545/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "∞/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”∞/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/span/tr

/span/tr/span/table /span/div/span/figure

/spanfigure id="autoid-63" class="table "
```

/spandiv class="center"

/spandiv class="figurecaption"Table 19: A comparison of computational requirements (approximated by k) for 1 million TPS between UT and various other networks. UT$_2$'s equivalent TPS is also provided (given the same parameters of k, B$_f andB_h$).

/span/div

/spantable/spantr class="tbrule"
/spantd class="tdl"k, B$_f$, B_h/span/td
/spantdclass = ”tdl”Network/span/td
/spantdclass = ”tdr”N_2/span/td
/spantdclass = ”tdr”ΣTPS/span/td
/spantdclass = ”tdr”Equiv.UT$_{2+\mathrm{OPT}}\Sigma TPS$/span/td
/span/tr

$2.5{\times}10^8$, /spanspanclass = ”textrm textmd textup textulc”/spanspanclass = ”numerator”1/span/span//spanspanclass = ”denominator”600/span/span/span/span, 80/span/td
/spantdclass = ”tdl”Bitcoin/span/td
/spantdclass = ”tdr” − /span/td
/spantdclass = ”tdr”1, 000, 000/span/td
/spantdclass = ”tdr”1.37×10^{24}/span/td
/span/tr

$2.5{\times}10^8$, /spanspanclass = ”textrm textmd textup textulc”/spanspanclass = ”numerator”1/span/span//spanspanclass = ”denominator”20/span/span/span/span, 1070/span/td
/spantdclass = ”tdl”Cardano/span/td
/spantdclass = ”tdr” − /span/td
/spantdclass = ”tdr”1, 000, 000/span/td
/spantdclass = ”tdr”5.80×10^{18}/span/td
/span/tr

$2.5{\times}10^8$, 1.58, 144/span/td
/spantdclass = ”tdl”Solana/span/td
/spantdclass = ”tdr” − /span/td
/spantdclass = ”tdr”1, 000, 000/span/td
/spantdclass = ”tdr”4.99×10^{17}/span/td
/span/tr

93680, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 84/span/td
/spantd class="tdl"UT$_{1+\mathrm{PoRTs}}$/span/td
/spantd class="tdr"-/span/td
/spantd class="tdr"1,000,011/span/td
/spantd class="tdr"$4.25{\times}10^{10}$/span/td
/span/tr

74136, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 84/span/td
/spantd class="td"UT$_{1+\mathrm{HOPoRTs}}$/span/td
/spantd class="td "-/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "1,000,049/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "$2.10{\times}10^{10}$/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/span/tr

32660, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 84/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "UT$_{1+\mathrm{HOT}}$/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "-/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "999,999/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "$1.80{\times}10^{9}$/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/span/tr

184800, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"6/span/span/span/span, 288/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "Polkadot/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "1,353/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "1,000,763/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "$3.47{\times}10^{9}$/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/span/tr

244200, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"12/span/span/span/span, 200/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "Eth2/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "1,024/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "1,000,243/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "$5.03{\times}10^{10}$/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/span/tr

33166, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 66/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "Opt.Shard/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "7,537/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "1,000,000/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "$3.28{\times}10^{9}$/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/span/tr

3074, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 84/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "UT$_{2+\mathrm{PoRTs}}$/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "81,348/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "1,000,264/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "1,500,397/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/span/tr

/spantr

/spandiv class="center"
/spandiv
class="fminipage"
style="border: 0pt solid #000000 ; padding:0pt ; "

/spandiv class="figurecaption"Table 20: Table evaluating various other networks against trilemma criteria.

/span/div

/spantable/spantr class="tbrule"
/spantd class="tdl"Network/spanspan style="width:17pt; display:inline-block"<!---->/span/span Decentralized?/spanspan style="width:8.5pt; display:inline-block"<!---->/span/span O(n) Secure?/spanspan style="width:8.5pt; display:inline-block"<!---->/span/span
/spantd colspan="2" class="tdl"O(n) Scalable?/span/td
/spantd class="tdl"/span/td
/spantd class="tdl"/span/td
/spantd class="tdl"/span/td
/span/tr

/spantr class="hline"
/spantd class="tdl"Bitcoin Yes Yes No (Max. TPS: ∼ 5)/spansup/spanspan class="textrm"/spani*/span/i/span/span/span/sup/span/td
/spantd class="tdl"/span/td
/spantd class="tdl"/span/td
/spantd class="tdl"/span/td
/spantd class="tdl"/span/td
/span/tr

/spantr
/spantd class="tdl"Cardano Yes Maybe/spansup/spanspan class="textrm"/spaniPoS/span/i/span/span/span/sup   No (Max. TPS: ∼ 10 - 80)/spansup/spanspan class="textrm"/spani#/span/i/span/span/span/sup/span/td
/spantd class="tdl"/span/td
/spantd class="tdl"/span/td
/spantd class="tdl"/span/td
/spantd class="tdl"/span/td
/span/tr

/spantr
/spantd class="tdl"Solana No/spansup/spanspan class="textrm"/spani‡/span/i/span/span/span/sup   No (O(c))/spansup/spanspan class="textrm"/spani†/span/i/span/span/span/sup   Unlikely (Max. TPS: ∼ 50K)/spansup/spanspan
class="textrm"/spani‡/span/i/span/span/span/sup/span/td
/spantd class="tdl"/span/td
/spantd class="tdl"/span/td
/spantd class="tdl"/span/td
/spantd class="tdl"/span/td
/span/tr

/spantr
/spantd class="tdl"Polkadot Maybe Maybe/spansup/spanspan class="textrm"/spaniPoS/span/i/span/span/span/sup   Unlikely (Max. TPS: ∼ 200 - 12K)/spansup/spanspan class="textrm"/spani¶/span/i/span/span/span/sup/span/td
/spantd class="tdl"/span/td
/spantd class="tdl"/span/td
/spantd class="tdl"/span/td
/spantd class="tdl"/span/td
/span/tr

/spantr
/spantd class="tdl"Eth2 Maybe Maybe/spansup/spanspan class="textrm"/spaniPoS/span/i/span/span/span/sup   Unlikely (Max. TPS: ∼ 1K - 62K)/spansup/spanspan class="textrm"/spani‖/span/i/span/span/span/sup/span/td
/spantd class="tdl"/span/td
/spantd class="tdl"/span/td
/spantd class="tdl"/span/td
/spantd class="tdl"/span/td
/span/tr

/spantr
/spantd class="tdl"Opt.Shard Yes Yes Maybe (Max. TPS: ∼ 8K - 350K)/spansup/spanspan class="textrm"/spani§/span/i/span/span/span/sup/span/td
/spantd class="tdl"/span/td
/spantd class="tdl"/span/td
/spantd class="tdl"/span/td
/spantd class="tdl"/span/td
/span/tr

/spantr class="tbrule"
/spantd class="tdl"/span/td
/spantd class="tdl"/span/td
/spantd class="tdl"/span/td
/spantd class="tdl"/span/td
/spantd class="tdl"/span/td
/span/tr/span/table
/spandiv class="tnotes" /spansup/spanspan class="textrm"/spani*/span/i/span/span/span/sup &#x202f;Real-world performance of Bitcoin; k ≈ $1700B/s$; $\mathrm{Tx}_{\mathrm{avg}} \approx 375B$.

/spansup/spanspan class="textrm"/spani#,‖,¶/span/i/span/span/span/sup &#x202f;Prediction based on 3000 $\leq k \leq$ 20000.

/spansup/spanspan class="textrm"/spani§/span/i/span/span/span/sup &#x202f; Assuming 3000 $\leq k \leq 20000 andthatchild-chainsusePoW+PoRwiththesmallestpossibleheaders.$

/spansup/spanspan class="textrm"/spaniPoS/span/i/span/span/span/sup &#x202f; /spana href="https://web.archive.org/web/20220309063128/https://github.com/zack-bitcoin/amoveo-docs/blob/master/other_blockchains/proof_of_stake.md" target="_blank"
Unanswered criticisms of PoS/span/a (Zack Hess; /spana href="https://web.archive.org/web/20220309063254/https://github.com/zack-bitcoin/amoveo-docs/blob/master/other_blockchains/the_defence_of_pos.md" target="_blank" replies/span/a) mean that we cannot conclude that PoS is O(n) secure. Saying PoS is "Maybe" O(n) secure (especially when used in isolation) reflects an optimistic perspective — since there are unanswered criticisms, we should arguably conclude "No". (There are no such unanswered criticisms of PoW-based consensus.)

/spansup/spanspan class="textrm"/spani†/span/i/span/span/span/sup &#x202f; /spana href="https://web.archive.org/web/20241227202028/https://solana.com/solana-whitepaper.pdf" target="_blank" PoH/span/a is O(c) secure by design. Solana uses PoS on top of PoH, but this is applied after-the-fact and it's not clear that a well-placed and well-resourced attacker would require more than O(c) resources. Additionally, since a $\leq O(c) DoShasbroughtdownSolanabefore, thisisanupperlimitonSolana'ssecurity.$

/spansup/spanspan class="textrm"/spani‡/span/i/span/span/span/sup &#x202f; Even though Solana requires k ≈
$10^7 (\sim 12MB/s) for 50KTPS, thisisconsistentwiththeirpublished``strategy''for``scaling''.50Kischosenasthelimitbecause/spanahref="https://web.archive.org/web/20210914230947/https://solana.com/news/8-innovations-that-make-solana-the-first-web-scale-blockchain" target="\_blank" theofficialsiteclaims/spanema$
$thisalsoearnsSolanaadecisive``No''underthe``Decentralized?''column.)$

/span/div /span/div /span/div /span/figure

%

<!--................................-->
/spanh4 id="autosec-564"/spanspan class="sectionnumber"B&#x2003;/span/spanUT Variant Complexities/span/h4

%

`<!--................................-->`
/spanh5 id="autosec-565"/spanspan class="sectionnumber"B.1&#x2003;/span/span+PoRs: Explicit PoRs/span/h5

What is the throughput of the simplex if simplex-chains include explicit proofs of reflection (as merkle branches) and the headers of reflecting chains?

Let g be the length of the digest in bytes, i.e., the size of the hashes used in our merkle trees.

/spanspan class="hidden" \(\seteqnumber{0}{}{63}\)/span/span `<!--`

$$\begin{aligned} k_1 &= k_{1,tx} + k_{1,B} \\ &= k_{1,tx} + B_f \cdot N_1 \cdot (B_h + g \cdot \lceil \log_2 N_1 \rceil) \\ &\approx k_{1,tx} + B_f \cdot N_1 \cdot (B_h + g \cdot \log_2 N_1) \\ \implies T_1 &= N_1 \cdot (k_1 - B_f \cdot N_1 \cdot (B_h + g \cdot \log_2 N_1)) \\ \frac{dT_1}{dN_1} &= \frac{1}{\ln 2}(k_1 \cdot \ln 2 - B_f \cdot N_1 \cdot (g + B_h \cdot \ln 4) - 2 \cdot B_f \cdot g \cdot N_1 \cdot \ln N_1) \end{aligned}$$

/spana href="https://www.wolframalpha.com/input?i=y+\%3D+\%28k+log\%282\%29+-+f+x+\%28g+\%2B+h+log\%284\%29\%29+-+2+f+g+x+log\%28x\%29\%29\%2Flog\%282\%29" target="_blank" which has a zero at:/span/a

$$\begin{aligned} N_1 &= \frac{k_1 \cdot \ln 2}{2 \cdot B_f \cdot g \cdot W_0((2^{B_h g^{-1} - 1} \cdot \sqrt{e} \cdot k_1 \cdot \ln 2)(B_f \cdot g)^{-1})} \\ \implies k_{1,tx} &\neq k_{1,B} \end{aligned} \tag{64}$$

```
-->
\begin{equation}
\begin{split} \label {eq:simplex-N1-with-PoR} k_1 &amp; = k_{1,tx} + k_{1,B} \\ &amp; = k_{1,tx} + B_f \cdot N_1 \cdot (B_h + g \cdot \ceil {\log _2 N_1}) \\ &amp; \approx k_{1,tx} + B_f \cdot N_1 \cdot (B_h + g \cdot \log _2 N_1) \\
\implies T_1 &amp; = N_1 \cdot (k_{1} - B_f \cdot N_1 \cdot (B_h + g \cdot \log _2 N_1)) \\ \frac {dT_1}{dN_1} &amp; = \frac {1}{\ln 2}(k_1 \cdot \ln 2 - B_f \cdot N_1 \cdot (g + B_h \cdot \ln 4) - 2 \cdot B_f \cdot g \cdot N_1 \cdot \ln
N_1) \\ \mbox {\href {\wolframAlphaPorsRootUrl }{which has a zero at:}} \\ N_1 &amp; = \frac {k_1 \cdot \ln 2}{2 \cdot B_f \cdot g \cdot W_0((2^{B_h g^{-1} - 1} \cdot \sqrt e \cdot k_1 \cdot \ln 2)(B_f \cdot g)^{-1})} \\ \implies k_{1,tx}
&amp; \neq k_{1,B} \end {split}
\end{equation}
```

Note: W$_0$$(z) istheLambertWfunction, a.k.a.theproductlogarithm.$

?? gives an N$_1$$thatisoftheformN_1 = O(1) \cdot \frac{k_1}{W_0(O(1) \cdot k_1)}.$**??**$graphsf(x) = \frac{x}{W_0(x)}forvaluesofxthatwecareabout; f(x) = 0.0638xisincludedforcomparison.Regardingthisspecificcase, approximatingO(\frac{k}{W_0(k)}) = O(k)givesusO(N_1) = O(c)andO(T_1) = O(c^2).$

/spanfigure id="autoid-65" class="figure "

```
/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "


/spana href="./includes/ut/diags/graph__prodlog__sag.png" target="__blank" /spanimg
    src="./includes/ut/diags/graph__prodlog__sag.png"
    style="
    height:156pt ;
    "
    class="inlineimage"
    alt="(image)"
/span/a


/spandiv class="figurecaption"(a) Using linear axes

/span/div

/span/div

&#x2003;&#x2003;
```

/spandiv class="minipage" style="vertical-align:bottom ; justify-content:flex-start ; "

/spana href="./includes/ut/diags/graph__prodlog__loglog__sag.png" target="_blank" /spanimg
    src="./includes/ut/diags/graph__prodlog__loglog__sag.png"
    style="
    height:156pt ;
    "
    class="inlineimage"
    alt="(image)"
/span/a

/spandiv class="figurecaption"(b) Using log axes

/span/div

/span/div

/spandiv class="figurecaption"Figure 39: Graphs of f(x) = x $\frac{}{W_0(x) for x \in [10^2,10^8].}$

/span/div /span/figure

If we use verkle PoRs instead of merkle PoRs, then we'll have a different $k_{1,B}.Assumingthatvectorcommitmentsandproofsaregbytes, andvectorlocationsare1byte$:

/spanspan class="hidden" \(\seteqnumber{0}{}{64}\)/span/span `<!--`

(65)

```
-->
\begin{equation}

\end{equation}
```

Other than this change, the logic that is used in **??** still works, and the resulting complexities will be the same.

/spanfigure id="autoid-66" class="table "

/spandiv class="center"

/spandiv class="figurecaption"Table 21: $\text{UT}_{+\text{PoRs}}$ capacity given different parameters.

/span/div

```
/spantable/spantr class="tbrule"
/spantd class="tdl"k, B_f, B_h/span/td
/spantdclass = ”tdr”N_1/span/td
/spantdclass = ”tdr”Σ TPS_1/span/td
/spantdclass = ”tdr”N_2/span/td
/spantdclass = ”tdr”Σ TPS_2/span/td
/spantdclass = ”tdr”PoR(B)/span/td
/spantdclass = ”tdr”ℂ′(Hz)/span/td
/span/tr


3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"7.5/span/span/span/span, 112/span/td
/spantd class="tdr"78/span/td
/spantd class="tdr"465/span/td
/spantd class="tdr"7,793/span/td
/spantd class="tdr"93,516/span/td
/spantd class="tdr"33/span/td
/spantd class="tdr"10.4/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"7.5/span/span/span/span, 84/span/td
/spantd class="tdr"96/span/td
/spantd class="tdr"576/span/td
/spantd class="tdr"12,877/span/td
/spantd class="tdr"154,532/span/td
/spantd class="tdr"33/span/td
/spantd class="tdr"12.8/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 112/span/td
/spantd class="tdr"155/span/td
/spantd class="tdr"931/span/td
/spantd class="tdr"31,172/span/td
/spantd class="tdr"374,075/span/td
/spantd class="tdr"33/span/td
/spantd class="tdr"10.3/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 84/span/td
/spantd class="tdr"192/span/td
/spantd class="tdr"1,153/span/td
/spantd class="tdr"51,510/span/td
/spantd class="tdr"618,130/span/td
/spantd class="tdr"33/span/td
/spantd class="tdr"12.8/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"30/span/span/span/span, 112/span/td
/spantd class="tdr"276/span/td
/spantd class="tdr"1,813/span/td
/spantd class="tdr"121,457/span/td
/spantd class="tdr"1,457,487/span/td
/spantd class="tdr"35/span/td
/spantd class="tdr"9.2/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"30/span/span/span/span, 84/span/td
/spantd class="tdr"328/span/td
/spantd class="tdr"2,152/span/td
/spantd class="tdr"192,172/span/td
/spantd class="tdr"2,306,074/span/td
/spantd class="tdr"40/span/td
/spantd class="tdr"10.9/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"60/span/span/span/span, 112/span/td
/spantd class="tdr"511/span/td
/spantd class="tdr"3,320/span/td
/spantd class="tdr"444,647/span/td
/spantd class="tdr"5,335,770/span/td
/spantd class="tdr"49/span/td
/spantd class="tdr"8.5/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"60/span/span/span/span, 84/span/td
/spantd class="tdr"628/span/td
/spantd class="tdr"3,943/span/td
/spantd class="tdr"704,122/span/td
/spantd class="tdr"8,449,470/span/td
/spantd class="tdr"52/span/td
/spantd class="tdr"10.5/span/td
/span/tr


/spantr
/spantd class="tdl"20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"7.5/span/span/span/span, 112/span/td
/spantd class="tdr"445/span/td
/spantd class="tdr"18,797/span/td
/spantd class="tdr"314,698/span/td
/spantd class="tdr"25,175,874/span/td
/spantd class="tdr"47/span/td
/spantd class="tdr"59.3/span/td
/span/tr


/spantr
/spantd class="tdl"20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"7.5/span/span/span/span, 84/span/td
/spantd class="tdr"511/span/td
/spantd class="tdr"22,283/span/td
/spantd class="tdr"497,403/span/td
/spantd class="tdr"39,792,300/span/td
/spantd class="tdr"49/span/td
/spantd class="tdr"68.1/span/td
/span/tr


/spantr
/spantd class="tdl"20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 112/span/td
/spantd class="tdr"866/span/td
/spantd class="tdr"35,610/span/td
/spantd class="tdr"1,192,304/span/td
/spantd class="tdr"95,384,332/span/td
/spantd class="tdr"56/span/td
/spantd class="tdr"57.7/span/td
/span/tr


/spantr
/spantd class="tdl"20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 84/span/td
/spantd class="tdr"1,023/span/td
/spantd class="tdr"42,256/span/td
/spantd class="tdr"1,886,448/span/td
/spantd class="tdr"1.51×10^8/span/td
/spantdclass = ”tdr”57/span/td
/spantdclass = ”tdr”68.2/span/td
/span/tr


20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"30/span/span/span/span, 112/span/td
/spantd class="tdr"1,709/span/td
/spantd class="tdr"69,282/span/td
/spantd class="td"4,639,446/span/td
/spantd class="td "3.71×10^8/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”61/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”56.0/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/span/tr


20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"30/span/span/span/span, 84/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "2,027/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "82,206/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "7,339,839/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "5.87×10^8/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”61/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”67.6/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/span/tr


20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"60/span/span/span/span, 112/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "3,327/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "136,594/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "18,293,925/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "1.46×10^9/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”63/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”55.4/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/span/tr


20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"60/span/span/span/span, 84/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "3,839/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "161,784/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "28,890,120/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "2.31×10^9/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”63/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”63.0/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/span/tr


184800, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 112/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "7,167/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "2,874,121/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "96,231,756/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "7.11×10^10/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”64/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”477.8/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/span/tr


244200, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 112/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "9,471/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "5,011,856/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "1.68×10^8/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”1.64 × 10^11/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”65/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”631.4/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/span/tr


/span/tr/span/table /span/div/span/figure

%
```

<!--................................-->
/spanh6 id="autosec-570"/spanspan class="sectionnumber"B.1.1&#x2003;/span/span+PoRs with +T/span/h6

/spanfigure id="autoid-67" class="table "

/spandiv class="center"

/spandiv class="figurecaption"Table 22: $\mathrm{UT}_{+\mathrm{PoRTs}}$ capacity given different parameters.

/span/div

```
/spantable/spantr class="tbrule"
/spantd class="tdl"k, B_f, B_h/span/td
/spantdclass = ”tdr”N_1/span/td
/spantdclass = ”tdr”Σ TPS_1/span/td
/spantdclass = ”tdr”N_2/span/td
/spantdclass = ”tdr”Σ TPS_2/span/td
/spantdclass = ”tdr”PoR(B)/span/td
/spantdclass = ”tdr”ℂ′(Hz)/span/td
/span/tr


3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"7.5/span/span/span/span, 112/span/td
/spantd class="tdr"100/span/td
/spantd class="tdr"597/span/td
/spantd class="tdr"13,999/span/td
/spantd class="tdr"167,999/span/td
/spantd class="tdr"33/span/td
/spantd class="tdr"13.3/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"7.5/span/span/span/span, 84/span/td
/spantd class="tdr"114/span/td
/spantd class="tdr"681/span/td
/spantd class="tdr"19,369/span/td
/spantd class="tdr"232,435/span/td
/spantd class="tdr"33/span/td
/spantd class="tdr"15.2/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 112/span/td
/spantd class="tdr"199/span/td
/spantd class="tdr"1,194/span/td
/spantd class="tdr"56,001/span/td
/spantd class="tdr"672,013/span/td
/spantd class="tdr"33/span/td
/spantd class="tdr"13.3/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 84/span/td
/spantd class="tdr"227/span/td
/spantd class="tdr"1,363/span/td
/spantd class="tdr"77,479/span/td
/spantd class="tdr"929,750/span/td
/spantd class="tdr"33/span/td
/spantd class="tdr"15.1/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"30/span/span/span/span, 112/span/td
/spantd class="tdr"337/span/td
/spantd class="tdr"2,211/span/td
/spantd class="tdr"207,309/span/td
/spantd class="tdr"2,487,717/span/td
/spantd class="tdr"41/span/td
/spantd class="tdr"11.2/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"30/span/span/span/span, 84/span/td
/spantd class="tdr"373/span/td
/spantd class="tdr"2,445/span/td
/spantd class="tdr"277,935/span/td
/spantd class="tdr"3,335,230/span/td
/spantd class="tdr"43/span/td
/spantd class="tdr"12.4/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"60/span/span/span/span, 112/span/td
/spantd class="tdr"645/span/td
/spantd class="tdr"4,051/span/td
/spantd class="tdr"759,584/span/td
/spantd class="tdr"9,115,011/span/td
/spantd class="tdr"53/span/td
/spantd class="tdr"10.8/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"60/span/span/span/span, 84/span/td
/spantd class="tdr"714/span/td
/spantd class="tdr"4,480/span/td
/spantd class="tdr"1,018,358/span/td
/spantd class="tdr"12,220,304/span/td
/spantd class="tdr"54/span/td
/spantd class="tdr"11.9/span/td
/span/tr


/spantr
/spantd class="tdl"20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"7.5/span/span/span/span, 112/span/td
/spantd class="tdr"511/span/td
/spantd class="tdr"22,840/span/td
/spantd class="tdr"535,329/span/td
/spantd class="tdr"42,826,399/span/td
/spantd class="tdr"49/span/td
/spantd class="tdr"68.1/span/td
/span/tr


/spantr
/spantd class="tdl"20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"7.5/span/span/span/span, 84/span/td
/spantd class="tdr"600/span/td
/spantd class="tdr"25,338/span/td
/spantd class="tdr"719,836/span/td
/spantd class="tdr"57,586,909/span/td
/spantd class="tdr"52/span/td
/spantd class="tdr"80.0/span/td
/span/tr


/spantr
/spantd class="tdl"20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 112/span/td
/spantd class="tdr"1,023/span/td
/spantd class="tdr"43,372/span/td
/spantd class="tdr"2,033,097/span/td
/spantd class="tdr"1.63×10^8/span/td
/spantdclass = ”tdr”57/span/td
/spantdclass = ”tdr”68.2/span/td
/span/tr


20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 84/span/td
/spantd class="tdr"1,168/span/td
/spantd class="tdr"48,009/span/td
/spantd class="td"2,727,810/span/td
/spantd class="td "2.18×10^8/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”58/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”77.9/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/span/tr


20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"30/span/span/span/span, 112/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "2,047/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "84,433/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "7,915,621/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "6.33×10^8/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”61/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”68.2/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/span/tr


20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"30/span/span/span/span, 84/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "2,303/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "93,405/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "10,614,317/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "8.49×10^8/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”62/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”76.8/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/span/tr


20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"60/span/span/span/span, 112/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "3,839/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "165,714/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "31,071,522/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "2.49×10^9/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”63/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”63.0/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/span/tr


20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"60/span/span/span/span, 84/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "4,095/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "182,203/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "41,409,942/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "3.31×10^9/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”64/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”68.3/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/span/tr


184800, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 112/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "8,703/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "3,501,439/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "1.64×10^8/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”1.21 × 10^11/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”65/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”580.2/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/span/tr


244200, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 112/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "11,519/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "6,107,287/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "2.86×10^8/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”2.80 × 10^11/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”65/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”767.9/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/span/tr


/span/tr/span/table /span/div/span/figure

%
```

<!--................................-->
/spanh5 id="autosec-573"/spanspan class="sectionnumber"B.2&#x2003;/span/span+OP and +OPT/span/h5

The +OP variants exclude PoRs from simplex-chain blocks. This is reasonable if users running full nodes are willing to download and temporarily store all blocks from all simplex-chains (this allows each node to regenerate the PoRs). The PoRs are still processed as part of a chain's state transition (each reflecting header will have a corresponding PoR), and are thus provable. Additionally, since full nodes will need to recalculate these, a suitable P2P protocol will allow PoRs to be requested from full nodes on an ad-hoc basis.

The derivations of +OP's complexity was covered in section /spana href="whitepaper.html#lateximage-whitepaper-63"5/span/a.

/spanfigure id="autoid-68" class="table "

/spandiv class="center"

/spandiv class="figurecaption"Table 23: UT$_{+\mathrm{OP}}$ capacity given different parameters.

/span/div

/spantable/spantr class="tbrule"
/spantd class="tdl"k, B$_f, B_h/span/td$
$/spantdclass = ”tdr”N_1/span/td$
$/spantdclass = ”tdr”\Sigma$ TPS$_1/span/td$
$/spantdclass = ”tdr”N_2/span/td$
$/spantdclass = ”tdr”\Sigma$ TPS$_2/span/td$
$/spantdclass = ”tdr”PoR(B)/span/td$
$/spantdclass = ”tdr”\mathbb{C}'(Hz)/span/td$
$/span/tr$

3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"7.5/span/span/span/span, 112/span/td
/spantd class="tdr"100/span/td
/spantd class="tdr"602/span/td
/spantd class="tdr"10,089/span/td
/spantd class="tdr"121,073/span/td
/spantd class="tdr"33/span/td
/spantd class="tdr"13.4/span/td
/span/tr

/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"7.5/span/span/span/span, 84/span/td
/spantd class="tdr"133/span/td
/spantd class="tdr"803/span/td
/spantd class="tdr"17,936/span/td
/spantd class="tdr"215,242/span/td
/spantd class="tdr"33/span/td
/spantd class="tdr"17.9/span/td
/span/tr

/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 112/span/td
/spantd class="tdr"200/span/td
/spantd class="tdr"1,205/span/td
/spantd class="tdr"40,357/span/td
/spantd class="tdr"484,295/span/td
/spantd class="tdr"33/span/td
/spantd class="tdr"13.4/span/td
/span/tr

/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 84/span/td
/spantd class="tdr"267/span/td
/spantd class="tdr"1,607/span/td
/spantd class="tdr"71,747/span/td
/spantd class="tdr"860,969/span/td
/spantd class="tdr"34/span/td
/spantd class="tdr"17.9/span/td
/span/tr

/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"30/span/span/span/span, 112/span/td
/spantd class="tdr"401/span/td
/spantd class="tdr"2,410/span/td
/spantd class="tdr"161,431/span/td
/spantd class="tdr"1,937,181/span/td
/spantd class="tdr"45/span/td
/spantd class="tdr"13.4/span/td
/span/tr

/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"30/span/span/span/span, 84/span/td
/spantd class="tdr"535/span/td
/spantd class="tdr"3,214/span/td
/spantd class="tdr"286,989/span/td
/spantd class="tdr"3,443,877/span/td
/spantd class="tdr"50/span/td
/spantd class="tdr"17.9/span/td
/span/tr

/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"60/span/span/span/span, 112/span/td
/spantd class="tdr"803/span/td
/spantd class="tdr"4,821/span/td
/spantd class="tdr"645,727/span/td
/spantd class="tdr"7,748,724/span/td
/spantd class="tdr"55/span/td
/spantd class="tdr"13.4/span/td
/span/tr

/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"60/span/span/span/span, 84/span/td
/spantd class="tdr"1,071/span/td
/spantd class="tdr"6,428/span/td
/spantd class="tdr"1,147,959/span/td
/spantd class="tdr"13,775,510/span/td
/spantd class="tdr"58/span/td
/spantd class="tdr"17.9/span/td
/span/tr

/spantr
/spantd class="tdl"20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"7.5/span/span/span/span, 112/span/td
/spantd class="tdr"669/span/td
/spantd class="tdr"26,785/span/td
/spantd class="tdr"448,421/span/td
/spantd class="tdr"35,873,724/span/td
/spantd class="tdr"53/span/td
/spantd class="tdr"89.3/span/td
/span/tr

/spantr
/spantd class="tdl"20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"7.5/span/span/span/span, 84/span/td
/spantd class="tdr"892/span/td
/spantd class="tdr"35,714/span/td
/spantd class="tdr"797,193/span/td
/spantd class="tdr"63,775,510/span/td
/spantd class="tdr"56/span/td
/spantd class="tdr"119.0/span/td
/span/tr

/spantr
/spantd class="tdl"20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 112/span/td
/spantd class="tdr"1,339/span/td
/spantd class="tdr"53,571/span/td
/spantd class="tdr"1,793,686/span/td
/spantd class="tdr"1.43×10$^8/span/td$
$/spantdclass = ”tdr”59/span/td$
$/spantdclass = ”tdr”89.3/span/td$
$/span/tr$

20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 84/span/td
/spantd class="tdr"1,785/span/td
/spantd class="tdr"71,428/span/td
/spantd class="td"3,188,775/span/td
/spantd class="td "2.55×10$^8/span/td$
$/spantdclass = ”tdat”/span/td$
$/spantdclass = ”tdbang”/span/td$
$/spantdclass = ”td”61/span/td$
$/spantdclass = ”tdat”/span/td$
$/spantdclass = ”tdbang”/span/td$
$/spantdclass = ”td”119.0/span/td$
$/spantdclass = ”tdat”/span/td$
$/spantdclass = ”tdbang”/span/td$
$/span/tr$

20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"30/span/span/span/span, 112/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "2,678/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "107,142/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "7,174,744/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "5.74×10$^8/span/td$
$/spantdclass = ”tdat”/span/td$
$/spantdclass = ”tdbang”/span/td$
$/spantdclass = ”td”62/span/td$
$/spantdclass = ”tdat”/span/td$
$/spantdclass = ”tdbang”/span/td$
$/spantdclass = ”td”89.3/span/td$
$/spantdclass = ”tdat”/span/td$
$/spantdclass = ”tdbang”/span/td$
$/span/tr$

20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"30/span/span/span/span, 84/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "3,571/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "142,857/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "12,755,102/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "1.02×10$^9/span/td$
$/spantdclass = ”tdat”/span/td$
$/spantdclass = ”tdbang”/span/td$
$/spantdclass = ”td”63/span/td$
$/spantdclass = ”tdat”/span/td$
$/spantdclass = ”tdbang”/span/td$
$/spantdclass = ”td”119.0/span/td$
$/spantdclass = ”tdat”/span/td$
$/spantdclass = ”tdbang”/span/td$
$/span/tr$

20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"60/span/span/span/span, 112/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "5,357/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "214,285/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "28,698,979/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "2.30×10$^9/span/td$
$/spantdclass = ”tdat”/span/td$
$/spantdclass = ”tdbang”/span/td$
$/spantdclass = ”td”64/span/td$
$/spantdclass = ”tdat”/span/td$
$/spantdclass = ”tdbang”/span/td$
$/spantdclass = ”td”89.3/span/td$
$/spantdclass = ”tdat”/span/td$
$/spantdclass = ”tdbang”/span/td$
$/span/tr$

20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"60/span/span/span/span, 84/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "7,142/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "285,714/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "51,020,408/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "4.08×10$^9/span/td$
$/spantdclass = ”tdat”/span/td$
$/spantdclass = ”tdbang”/span/td$
$/spantdclass = ”td”64/span/td$
$/spantdclass = ”tdat”/span/td$
$/spantdclass = ”tdbang”/span/td$
$/spantdclass = ”td”119.0/span/td$
$/spantdclass = ”tdat”/span/td$
$/spantdclass = ”tdbang”/span/td$
$/span/tr$

184800, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 112/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "12,375/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "4,573,800/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "1.53×10$^8/span/td$
$/spantdclass = ”tdat”/span/td$
$/spantdclass = ”tdbang”/span/td$
$/spantdclass = ”td”1.13 \times 10^{11}/span/td$
$/spantdclass = ”tdat”/span/td$
$/spantdclass = ”tdbang”/span/td$
$/spantdclass = ”td”65/span/td$
$/spantdclass = ”tdat”/span/td$
$/spantdclass = ”tdbang”/span/td$
$/spantdclass = ”td”825.0/span/td$
$/spantdclass = ”tdat”/span/td$
$/spantdclass = ”tdbang”/span/td$
$/span/tr$

244200, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 112/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "16,352/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "7,986,648/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "2.67×10$^8/span/td$
$/spantdclass = ”tdat”/span/td$
$/spantdclass = ”tdbang”/span/td$
$/spantdclass = ”td”2.61 \times 10^{11}/span/td$
$/spantdclass = ”tdat”/span/td$
$/spantdclass = ”tdbang”/span/td$
$/spantdclass = ”td”65/span/td$
$/spantdclass = ”tdat”/span/td$
$/spantdclass = ”tdbang”/span/td$
$/spantdclass = ”td”1,090.2/span/td$
$/spantdclass = ”tdat”/span/td$
$/spantdclass = ”tdbang”/span/td$
$/span/tr$

/span/tr/span/table /span/div/span/figure

/spanfigure id="autoid-69" class="table "

/spandiv class="center"

/spandiv class="figurecaption"Table 24: $\mathrm{UT}_{+\mathrm{OPT}}$ capacity given different parameters.

/span/div

```
/spantable/spantr class="tbrule"
/spantd class="tdl"k, B_f, B_h/span/td
/spantdclass = "tdr"N_1/span/td
/spantdclass = "tdr"Σ TPS_1/span/td
/spantdclass = "tdr"N_2/span/td
/spantdclass = "tdr"Σ TPS_2/span/td
/spantdclass = "tdr"PoR(B)/span/td
/spantdclass = "tdr"ℂ′(Hz)/span/td
/span/tr


3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"7.5/span/span/span/span, 112/span/td
/spantd class="tdr"140/span/td
/spantd class="tdr"843/span/td
/spantd class="tdr"19,775/span/td
/spantd class="tdr"237,304/span/td
/spantd class="tdr"33/span/td
/spantd class="tdr"18.8/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"7.5/span/span/span/span, 84/span/td
/spantd class="tdr"170/span/td
/spantd class="tdr"1,022/span/td
/spantd class="tdr"29,054/span/td
/spantd class="tdr"348,657/span/td
/spantd class="tdr"33/span/td
/spantd class="tdr"22.7/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 112/span/td
/spantd class="tdr"281/span/td
/spantd class="tdr"1,687/span/td
/spantd class="tdr"79,101/span/td
/spantd class="tdr"949,218/span/td
/spantd class="tdr"36/span/td
/spantd class="tdr"18.8/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 84/span/td
/spantd class="tdr"340/span/td
/spantd class="tdr"2,045/span/td
/spantd class="tdr"116,219/span/td
/spantd class="tdr"1,394,628/span/td
/spantd class="tdr"41/span/td
/spantd class="tdr"22.7/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"30/span/span/span/span, 112/span/td
/spantd class="tdr"562/span/td
/spantd class="tdr"3,375/span/td
/spantd class="tdr"316,406/span/td
/spantd class="tdr"3,796,875/span/td
/spantd class="tdr"51/span/td
/spantd class="tdr"18.8/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"30/span/span/span/span, 84/span/td
/spantd class="tdr"681/span/td
/spantd class="tdr"4,090/span/td
/spantd class="tdr"464,876/span/td
/spantd class="tdr"5,578,512/span/td
/spantd class="tdr"53/span/td
/spantd class="tdr"22.7/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"60/span/span/span/span, 112/span/td
/spantd class="tdr"1,125/span/td
/spantd class="tdr"6,750/span/td
/spantd class="tdr"1,265,625/span/td
/spantd class="tdr"15,187,500/span/td
/spantd class="tdr"58/span/td
/spantd class="tdr"18.8/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"60/span/span/span/span, 84/span/td
/spantd class="tdr"1,363/span/td
/spantd class="tdr"8,181/span/td
/spantd class="tdr"1,859,504/span/td
/spantd class="tdr"22,314,049/span/td
/spantd class="tdr"59/span/td
/spantd class="tdr"22.7/span/td
/span/tr


/spantr
/spantd class="tdl"20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"7.5/span/span/span/span, 112/span/td
/spantd class="tdr"937/span/td
/spantd class="tdr"37,500/span/td
/spantd class="tdr"878,906/span/td
/spantd class="tdr"70,312,500/span/td
/spantd class="tdr"57/span/td
/spantd class="tdr"125.0/span/td
/span/tr


/spantr
/spantd class="tdl"20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"7.5/span/span/span/span, 84/span/td
/spantd class="tdr"1,136/span/td
/spantd class="tdr"45,454/span/td
/spantd class="tdr"1,291,322/span/td
/spantd class="tdr"1.03×10^8/span/td
/spantdclass = "tdr"58/span/td
/spantdclass = "tdr"151.5/span/td
/span/tr


20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 112/span/td
/spantd class="tdr"1,875/span/td
/spantd class="tdr"75,000/span/td
/spantd class="td"3,515,625/span/td
/spantd class="td "2.81×10^8/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"61/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"125.0/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/span/tr


20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 84/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "2,272/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "90,909/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "5,165,289/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "4.13×10^8/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"62/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"151.5/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/span/tr


20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"30/span/span/span/span, 112/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "3,750/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "150,000/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "14,062,500/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "1.13×10^9/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"63/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"125.0/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/span/tr


20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"30/span/span/span/span, 84/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "4,545/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "181,818/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "20,661,157/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "1.65×10^9/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"64/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"151.5/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/span/tr


20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"60/span/span/span/span, 112/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "7,500/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "300,000/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "56,250,000/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "4.50×10^9/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"65/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"125.0/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/span/tr


20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"60/span/span/span/span, 84/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "9,090/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "363,636/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "82,644,628/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "6.61×10^9/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"65/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"151.5/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/span/tr


184800, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 112/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "17,325/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "6,403,320/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "3.00×10^8/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"2.22 × 10^11/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"65/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"1, 155.0/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/span/tr


244200, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 112/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "22,893/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "11,181,307/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "5.24×10^8/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"5.12 × 10^11/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"65/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"1, 526.3/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/span/tr


/span/tr/span/table /span/div/span/figure

%
```

`<!--...............................-->`
/spanh5 id="autosec-578"/spanspan class="sectionnumber"B.3&#x2003;/span/span+HO and +HOT/span/h5

The +HO variants replace the headers of reflecting chains with the respective header's hash. This is reasonable since the headers of each simplex-chain (that would otherwise be recorded in simplex-chain blocks) are */spanemcommon/span/em* among all simplex-chains. If a user is running nodes for multiple simplex-chains, they should only need to download each header once — including raw headers in each block is redundant.

Thus, +HO has k$_{1,B}of$:

/spanspan class="hidden" \(\seteqnumber{0}{}{65}\)/span/span `<!--`

(66)

`-->`
`\begin{equation}`

`\end{equation}`

This is equivalent to +OP with very small headers — 32 bytes instead of 80+ bytes.

/spanfigure id="autoid-70" class="table "

/spandiv class="center"

/spandiv class="figurecaption"Table 25: UT$_{+\mathrm{HO}}$ capacity given different parameters.

/span/div

```
/spantable/spantr class="tbrule"
/spantd class="tdl"k, B_f, B_h/span/td
/spantdclass = "tdr"N_1/span/td
/spantdclass = "tdr"Σ TPS_1/span/td
/spantdclass = "tdr"N_2/span/td
/spantdclass = "tdr"Σ TPS_2/span/td
/spantdclass = "tdr"PoR(B)/span/td
/spantdclass = "tdr"ℂ'(Hz)/span/td
/span/tr


3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"7.5/span/span/span/span, 112/span/td
/spantd class="tdr"351/span/td
/spantd class="tdr"2,109/span/td
/spantd class="tdr"35,313/span/td
/spantd class="tdr"423,758/span/td
/spantd class="tdr"42/span/td
/spantd class="tdr"46.9/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"7.5/span/span/span/span, 84/span/td
/spantd class="tdr"351/span/td
/spantd class="tdr"2,109/span/td
/spantd class="tdr"47,084/span/td
/spantd class="tdr"565,011/span/td
/spantd class="tdr"42/span/td
/spantd class="tdr"46.9/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 112/span/td
/spantd class="tdr"703/span/td
/spantd class="tdr"4,218/span/td
/spantd class="tdr"141,252/span/td
/spantd class="tdr"1,695,033/span/td
/spantd class="tdr"54/span/td
/spantd class="tdr"46.9/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 84/span/td
/spantd class="tdr"703/span/td
/spantd class="tdr"4,218/span/td
/spantd class="tdr"188,337/span/td
/spantd class="tdr"2,260,044/span/td
/spantd class="tdr"54/span/td
/spantd class="tdr"46.9/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"30/span/span/span/span, 112/span/td
/spantd class="tdr"1,406/span/td
/spantd class="tdr"8,437/span/td
/spantd class="tdr"565,011/span/td
/spantd class="tdr"6,780,133/span/td
/spantd class="tdr"60/span/td
/spantd class="tdr"46.9/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"30/span/span/span/span, 84/span/td
/spantd class="tdr"1,406/span/td
/spantd class="tdr"8,437/span/td
/spantd class="tdr"753,348/span/td
/spantd class="tdr"9,040,178/span/td
/spantd class="tdr"60/span/td
/spantd class="tdr"46.9/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"60/span/span/span/span, 112/span/td
/spantd class="tdr"2,812/span/td
/spantd class="tdr"16,875/span/td
/spantd class="tdr"2,260,044/span/td
/spantd class="tdr"27,120,535/span/td
/spantd class="tdr"63/span/td
/spantd class="tdr"46.9/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"60/span/span/span/span, 84/span/td
/spantd class="tdr"2,812/span/td
/spantd class="tdr"16,875/span/td
/spantd class="tdr"3,013,392/span/td
/spantd class="tdr"36,160,714/span/td
/spantd class="tdr"63/span/td
/spantd class="tdr"46.9/span/td
/span/tr


/spantr
/spantd class="tdl"20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"7.5/span/span/span/span, 112/span/td
/spantd class="tdr"2,343/span/td
/spantd class="tdr"93,750/span/td
/spantd class="tdr"1,569,475/span/td
/spantd class="tdr"1.26×10^8/span/td
/spantdclass = "tdr"62/span/td
/spantdclass = "tdr"312.5/span/td
/span/tr


20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"7.5/span/span/span/span, 84/span/td
/spantd class="tdr"2,343/span/td
/spantd class="tdr"93,750/span/td
/spantd class="td"2,092,633/span/td
/spantd class="td "1.67×10^8/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"62/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"312.5/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/span/tr


20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 112/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "4,687/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "187,500/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "6,277,901/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "5.02×10^8/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"64/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"312.5/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/span/tr


20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 84/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "4,687/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "187,500/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "8,370,535/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "6.70×10^8/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"64/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"312.5/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/span/tr


20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"30/span/span/span/span, 112/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "9,375/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "375,000/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "25,111,607/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "2.01×10^9/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"65/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"312.5/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/span/tr


20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"30/span/span/span/span, 84/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "9,375/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "375,000/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "33,482,142/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "2.68×10^9/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"65/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"312.5/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/span/tr


20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"60/span/span/span/span, 112/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "18,750/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "750,000/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "1.00×10^8/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"8.04 × 10^9/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"65/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"312.5/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/span/tr


20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"60/span/span/span/span, 84/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "18,750/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "750,000/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "1.34×10^8/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"1.07 × 10^10/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"65/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"312.5/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/span/tr


184800, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 112/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "43,312/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "16,008,300/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "5.36×10^8/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"3.96 × 10^11/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"65/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"2, 887.5/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/span/tr


244200, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 112/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "57,234/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "27,953,268/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "9.36×10^8/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"9.14 × 10^11/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"65/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"3, 815.6/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/span/tr


/span/tr/span/table /span/div/span/figure

/spanfigure id="autoid-71" class="table "
```

/spandiv class="center"

/spandiv class="figurecaption"Table 26: UT$_{+\mathrm{HOT}}$ capacity given different parameters.

/span/div

```
/spantable/spantr class="tbrule"
/spantd class="tdl"k, B_f, B_h/span/td
/spantdclass = "tdr"N_1/span/td
/spantdclass = "tdr"Σ TPS_1/span/td
/spantdclass = "tdr"N_2/span/td
/spantdclass = "tdr"Σ TPS_2/span/td
/spantdclass = "tdr"PoR(B)/span/td
/spantdclass = "tdr"ℂ'(Hz)/span/td
/span/tr


3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"7.5/span/span/span/span, 112/span/td
/spantd class="tdr"703/span/td
/spantd class="tdr"4,218/span/td
/spantd class="tdr"98,876/span/td
/spantd class="tdr"1,186,523/span/td
/spantd class="tdr"54/span/td
/spantd class="tdr"93.8/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"7.5/span/span/span/span, 84/span/td
/spantd class="tdr"703/span/td
/spantd class="tdr"4,218/span/td
/spantd class="tdr"119,850/span/td
/spantd class="tdr"1,438,210/span/td
/spantd class="tdr"54/span/td
/spantd class="tdr"93.8/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 112/span/td
/spantd class="tdr"1,406/span/td
/spantd class="tdr"8,437/span/td
/spantd class="tdr"395,507/span/td
/spantd class="tdr"4,746,093/span/td
/spantd class="tdr"60/span/td
/spantd class="tdr"93.8/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 84/span/td
/spantd class="tdr"1,406/span/td
/spantd class="tdr"8,437/span/td
/spantd class="tdr"479,403/span/td
/spantd class="tdr"5,752,840/span/td
/spantd class="tdr"60/span/td
/spantd class="tdr"93.8/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"30/span/span/span/span, 112/span/td
/spantd class="tdr"2,812/span/td
/spantd class="tdr"16,875/span/td
/spantd class="tdr"1,582,031/span/td
/spantd class="tdr"18,984,375/span/td
/spantd class="tdr"63/span/td
/spantd class="tdr"93.8/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"30/span/span/span/span, 84/span/td
/spantd class="tdr"2,812/span/td
/spantd class="tdr"16,875/span/td
/spantd class="tdr"1,917,613/span/td
/spantd class="tdr"23,011,363/span/td
/spantd class="tdr"63/span/td
/spantd class="tdr"93.8/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"60/span/span/span/span, 112/span/td
/spantd class="tdr"5,625/span/td
/spantd class="tdr"33,750/span/td
/spantd class="tdr"6,328,125/span/td
/spantd class="tdr"75,937,500/span/td
/spantd class="tdr"64/span/td
/spantd class="tdr"93.8/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"60/span/span/span/span, 84/span/td
/spantd class="tdr"5,625/span/td
/spantd class="tdr"33,750/span/td
/spantd class="tdr"7,670,454/span/td
/spantd class="tdr"92,045,454/span/td
/spantd class="tdr"64/span/td
/spantd class="tdr"93.8/span/td
/span/tr


/spantr
/spantd class="tdl"20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"7.5/span/span/span/span, 112/span/td
/spantd class="tdr"4,687/span/td
/spantd class="tdr"187,500/span/td
/spantd class="tdr"4,394,531/span/td
/spantd class="tdr"3.52×10^8/span/td
/spantdclass = "tdr"64/span/td
/spantdclass = "tdr"625.0/span/td
/span/tr


20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"7.5/span/span/span/span, 84/span/td
/spantd class="tdr"4,687/span/td
/spantd class="tdr"187,500/span/td
/spantd class="td"5,326,704/span/td
/spantd class="td "4.26×10^8/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"64/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"625.0/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/span/tr


20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 112/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "9,375/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "375,000/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "17,578,125/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "1.41×10^9/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"65/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"625.0/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/span/tr


20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 84/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "9,375/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "375,000/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "21,306,818/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "1.70×10^9/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"65/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"625.0/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/span/tr


20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"30/span/span/span/span, 112/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "18,750/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "750,000/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "70,312,500/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "5.63×10^9/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"65/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"625.0/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/span/tr


20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"30/span/span/span/span, 84/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "18,750/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "750,000/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "85,227,272/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "6.82×10^9/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"65/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"625.0/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/span/tr


20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"60/span/span/span/span, 112/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "37,500/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "1,500,000/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "2.81×10^8/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"2.25 × 10^10/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"65/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"625.0/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/span/tr


20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"60/span/span/span/span, 84/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "37,500/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "1,500,000/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "3.41×10^8/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"2.73 × 10^10/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"65/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"625.0/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/span/tr


184800, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 112/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "86,625/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "32,016,600/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "1.50×10^9/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"1.11 × 10^12/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"74/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"5, 775.0/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/span/tr


244200, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 112/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "114,468/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "55,906,537/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "2.62×10^9/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"2.56 × 10^12/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"80/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"7, 631.3/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/span/tr


/span/tr/span/table /span/div/span/figure

%
```

```
<!--...............................-->
```

/spanh5 id="autosec-583"/spanspan class="sectionnumber"B.4&#x2003;/span/span+HOPoRs and +HOPoRTs/span/h5

+HOPoRs is the combination of +HO and +PoRs — headers are omitted but PoRs are still explicitly recorded.

Thus, +HOPoRs has $k_{1,B} of : /spanspanclass = "hidden"\backslash(\backslash seteqnumber\{0\}\{\}\{66\}\backslash)/span/span$
<!--

$$\begin{aligned} \text{Merkle PoRs:} \quad & k_{1,B} = B_f \cdot N_1 \cdot g \cdot (1 + \log_2 N_1) \\ \text{Verkle PoRs:} \quad & k_{1,B} = B_f \cdot N_1 \cdot (g + (g + 1) \cdot \max(1, \log_{256} N_1)) \end{aligned} \tag{67}$$

```
-->

\begin{align}
\text {Merkle PoRs:} &amp; &amp; k_{1,B} &amp;= B_f \cdot N_1 \cdot g \cdot (1 + \log _2 N_1) &amp; \nonumber \\ \text {Verkle PoRs:} &amp; &amp; k_{1,B} &amp;= B_f \cdot N_1 \cdot (g + (g + 1) \cdot \max (1, \log _{256} N_1)) &amp;
\end{align}
```

/spanfigure id="autoid-72" class="table "

/spandiv class="center"

/spandiv class="figurecaption"Table 27: $\mathrm{UT}_{+\mathrm{HOPoRs}}$ capacity given different parameters.

/span/div

```
/spantable/spantr class="tbrule"
/spantd class="tdl"k, B_f, B_h/span/td
/spantdclass = ”tdr”N_1/span/td
/spantdclass = ”tdr”Σ TPS_1/span/td
/spantdclass = ”tdr”N_2/span/td
/spantdclass = ”tdr”Σ TPS_2/span/td
/spantdclass = ”tdr”PoR(B)/span/td
/spantdclass = ”tdr”ℂ′(Hz)/span/td
/span/tr


3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"7.5/span/span/span/span, 112/span/td
/spantd class="tdr"173/span/td
/spantd class="tdr"1,038/span/td
/spantd class="tdr"17,384/span/td
/spantd class="tdr"208,619/span/td
/spantd class="tdr"33/span/td
/spantd class="tdr"23.1/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"7.5/span/span/span/span, 84/span/td
/spantd class="tdr"173/span/td
/spantd class="tdr"1,038/span/td
/spantd class="tdr"23,179/span/td
/spantd class="tdr"278,159/span/td
/spantd class="tdr"33/span/td
/spantd class="tdr"23.1/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 112/span/td
/spantd class="tdr"273/span/td
/spantd class="tdr"1,940/span/td
/spantd class="tdr"64,986/span/td
/spantd class="tdr"779,834/span/td
/spantd class="tdr"35/span/td
/spantd class="tdr"18.2/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 84/span/td
/spantd class="tdr"273/span/td
/spantd class="tdr"1,940/span/td
/spantd class="tdr"86,648/span/td
/spantd class="tdr"1,039,779/span/td
/spantd class="tdr"35/span/td
/spantd class="tdr"18.2/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"30/span/span/span/span, 112/span/td
/spantd class="tdr"502/span/td
/spantd class="tdr"3,294/span/td
/spantd class="tdr"220,606/span/td
/spantd class="tdr"2,647,279/span/td
/spantd class="tdr"49/span/td
/spantd class="tdr"16.7/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"30/span/span/span/span, 84/span/td
/spantd class="tdr"502/span/td
/spantd class="tdr"3,294/span/td
/spantd class="tdr"294,142/span/td
/spantd class="tdr"3,529,705/span/td
/spantd class="tdr"49/span/td
/spantd class="tdr"16.7/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"60/span/span/span/span, 112/span/td
/spantd class="tdr"961/span/td
/spantd class="tdr"6,033/span/td
/spantd class="tdr"808,020/span/td
/spantd class="tdr"9,696,249/span/td
/spantd class="tdr"57/span/td
/spantd class="tdr"16.0/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"60/span/span/span/span, 84/span/td
/spantd class="tdr"961/span/td
/spantd class="tdr"6,033/span/td
/spantd class="tdr"1,077,361/span/td
/spantd class="tdr"12,928,333/span/td
/spantd class="tdr"57/span/td
/spantd class="tdr"16.0/span/td
/span/tr


/spantr
/spantd class="tdl"20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"7.5/span/span/span/span, 112/span/td
/spantd class="tdr"767/span/td
/spantd class="tdr"34,040/span/td
/spantd class="tdr"569,881/span/td
/spantd class="tdr"45,590,480/span/td
/spantd class="tdr"55/span/td
/spantd class="tdr"102.3/span/td
/span/tr


/spantr
/spantd class="tdl"20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"7.5/span/span/span/span, 84/span/td
/spantd class="tdr"767/span/td
/spantd class="tdr"34,040/span/td
/spantd class="tdr"759,841/span/td
/spantd class="tdr"60,787,306/span/td
/spantd class="tdr"55/span/td
/spantd class="tdr"102.3/span/td
/span/tr


/spantr
/spantd class="tdl"20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 112/span/td
/spantd class="tdr"1,535/span/td
/spantd class="tdr"64,614/span/td
/spantd class="tdr"2,163,431/span/td
/spantd class="tdr"1.73×10^8/span/td
/spantdclass = ”tdr”60/span/td
/spantdclass = ”tdr”102.3/span/td
/span/tr


20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 84/span/td
/spantd class="tdr"1,535/span/td
/spantd class="tdr"64,614/span/td
/spantd class="td"2,884,575/span/td
/spantd class="td "2.31×10^8/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”60/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”102.3/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/span/tr


20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"30/span/span/span/span, 112/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "2,815/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "124,703/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "8,350,697/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "6.68×10^8/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”63/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”93.8/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/span/tr


20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"30/span/span/span/span, 84/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "2,815/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "124,703/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "11,134,263/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "8.91×10^8/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”63/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”93.8/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/span/tr


20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"60/span/span/span/span, 112/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "5,375/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "244,038/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "32,683,743/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "2.61×10^9/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”64/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”89.6/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/span/tr


20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"60/span/span/span/span, 84/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "5,375/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "244,038/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "43,578,324/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "3.49×10^9/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”64/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”89.6/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/span/tr


184800, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 112/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "12,287/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "5,159,789/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "1.73×10^8/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”1.28 × 10^11/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”65/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”819.1/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/span/tr


244200, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 112/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "16,127/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "8,983,624/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "3.01×10^8/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”2.94 × 10^11/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”65/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/spantdclass = ”td”1, 075.1/span/td
/spantdclass = ”tdat”/span/td
/spantdclass = ”tdbang”/span/td
/span/tr


/span/tr/span/table /span/div/span/figure

/spanfigure id="autoid-73" class="table "
```

/spandiv class="center"

/spandiv class="figurecaption"Table 28: $\mathrm{UT}_{+\mathrm{HOPoRTs}}$ capacity given different parameters.

/span/div

```
/spantable/spantr class="tbrule"
/spantd class="tdl"k, B_f, B_h/span/td
/spantdclass = "tdr"N_1/span/td
/spantdclass = "tdr"Σ TPS_1/span/td
/spantdclass = "tdr"N_2/span/td
/spantdclass = "tdr"Σ TPS_2/span/td
/spantdclass = "tdr"PoR(B)/span/td
/spantdclass = "tdr"ℂ'(Hz)/span/td
/span/tr


3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"7.5/span/span/span/span, 112/span/td
/spantd class="tdr"230/span/td
/spantd class="tdr"1,377/span/td
/spantd class="tdr"32,286/span/td
/spantd class="tdr"387,434/span/td
/spantd class="tdr"33/span/td
/spantd class="tdr"30.7/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"7.5/span/span/span/span, 84/span/td
/spantd class="tdr"230/span/td
/spantd class="tdr"1,377/span/td
/spantd class="tdr"39,134/span/td
/spantd class="tdr"469,618/span/td
/spantd class="tdr"33/span/td
/spantd class="tdr"30.7/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 112/span/td
/spantd class="tdr"326/span/td
/spantd class="tdr"2,319/span/td
/spantd class="tdr"108,733/span/td
/spantd class="tdr"1,304,798/span/td
/spantd class="tdr"40/span/td
/spantd class="tdr"21.7/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 84/span/td
/spantd class="tdr"326/span/td
/spantd class="tdr"2,319/span/td
/spantd class="tdr"131,797/span/td
/spantd class="tdr"1,581,574/span/td
/spantd class="tdr"40/span/td
/spantd class="tdr"21.7/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"30/span/span/span/span, 112/span/td
/spantd class="tdr"511/span/td
/spantd class="tdr"3,850/span/td
/spantd class="tdr"360,976/span/td
/spantd class="tdr"4,331,721/span/td
/spantd class="tdr"49/span/td
/spantd class="tdr"17.0/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"30/span/span/span/span, 84/span/td
/spantd class="tdr"511/span/td
/spantd class="tdr"3,850/span/td
/spantd class="tdr"437,547/span/td
/spantd class="tdr"5,250,571/span/td
/spantd class="tdr"49/span/td
/spantd class="tdr"17.0/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"60/span/span/span/span, 112/span/td
/spantd class="tdr"1,023/span/td
/spantd class="tdr"7,124/span/td
/spantd class="tdr"1,335,820/span/td
/spantd class="tdr"16,029,847/span/td
/spantd class="tdr"57/span/td
/spantd class="tdr"17.1/span/td
/span/tr


/spantr
/spantd class="tdl"3000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"60/span/span/span/span, 84/span/td
/spantd class="tdr"1,023/span/td
/spantd class="tdr"7,124/span/td
/spantd class="tdr"1,619,176/span/td
/spantd class="tdr"19,430,117/span/td
/spantd class="tdr"57/span/td
/spantd class="tdr"17.1/span/td
/span/tr


/spantr
/spantd class="tdl"20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"7.5/span/span/span/span, 112/span/td
/spantd class="tdr"966/span/td
/spantd class="tdr"40,771/span/td
/spantd class="tdr"955,579/span/td
/spantd class="tdr"76,446,341/span/td
/spantd class="tdr"57/span/td
/spantd class="tdr"128.8/span/td
/span/tr


/spantr
/spantd class="tdl"20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"7.5/span/span/span/span, 84/span/td
/spantd class="tdr"966/span/td
/spantd class="tdr"40,771/span/td
/spantd class="tdr"1,158,277/span/td
/spantd class="tdr"92,662,232/span/td
/spantd class="tdr"57/span/td
/spantd class="tdr"128.8/span/td
/span/tr


/spantr
/spantd class="tdl"20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 112/span/td
/spantd class="tdr"1,791/span/td
/spantd class="tdr"77,078/span/td
/spantd class="tdr"3,613,073/span/td
/spantd class="tdr"2.89×10^8/span/td
/spantdclass = "tdr"61/span/td
/spantdclass = "tdr"119.4/span/td
/span/tr


20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 84/span/td
/spantd class="tdr"1,791/span/td
/spantd class="tdr"77,078/span/td
/spantd class="td"4,379,483/span/td
/spantd class="td "3.50×10^8/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"61/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"119.4/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/span/tr


20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"30/span/span/span/span, 112/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "3,327/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "148,711/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "13,941,710/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "1.12×10^9/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"63/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"110.9/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/span/tr


20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"30/span/span/span/span, 84/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "3,327/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "148,711/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "16,899,042/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "1.35×10^9/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"63/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"110.9/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/span/tr


20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"60/span/span/span/span, 112/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "6,143/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "288,296/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "54,055,558/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "4.32×10^9/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"64/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"102.4/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/span/tr


20000, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"60/span/span/span/span, 84/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "6,143/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "288,296/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "65,521,889/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "5.24×10^9/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"64/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"102.4/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/span/tr


184800, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 112/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "14,335/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "6,128,357/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "2.87×10^8/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"2.12 × 10^11/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"65/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"955.7/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/span/tr


244200, /spanspan class="textrm textmd textup textulc"/spanspan class="numerator"1/span/span//spanspan class="denominator"15/span/span/span/span, 112/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "18,687/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "10,647,792/span/td
/spantd class="tdat "/span/td
/spantd class="tdbang "/span/td
/spantd class="td "4.99×10^8/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"4.88 × 10^11/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"65/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/spantdclass = "td"1, 245.8/span/td
/spantdclass = "tdat"/span/td
/spantdclass = "tdbang"/span/td
/span/tr


/span/tr/span/table /span/div/span/figure

.7ex]0.5pt/spanspan style="width:3.4445pt; display:inline-block"<!----->/span/spanBegin DRAFT/spanspan style="width:3.4445pt; display:inline-block"<!----->/span/span.7ex]0.5pt

%
```

```
<!--. . . . . . . . . . . . . . . . . . . . . . . . . . . . . . .-->
/spanh4 id="autosec-588"/spanspan class="sectionnumber"C&#x2003;/span/spanPoS and Bribe attacks/span/h4



/spandiv
      class="tcolorbox"
      style=" border: 1px solid #404040; background: #FCD1E9; "

/spandiv
      class="tcolorboxtitle"
      style=" border-top: 1px solid #404040; border-bottom: 1px solid #404040; background: #404040; color: #FFFFFF; "
TODO [14]/span/div
/spandiv
      class="tcolorboxupper"
      style=" color: #000000; "
  This section on PoS is mostly notes, see source for commented stuff  /span/div /span/div

.7ex]0.5pt/spanspan style="width:3.4445pt; display:inline-block"<!---->/span/spanEnd DRAFT/spanspan style="width:3.4445pt; display:inline-block"<!---->/span/span.7ex]0.5pt
```

```
<!--................................-->
/spanh4 id="autosec-589"/spanspan class="sectionnumber"D&#x2003;/span/spanCEC Experiment Extra/span/h4
```

<!--...............................-->
/spanh5 id="autosec-590"/spanspan class="sectionnumber"D.1&#x2003;/span/spanResults with q = 0.48/span/h5

?? shows results of the simulation for q=0.48. Unlike earlier figures, this figure uses c = 20 as the basis for comparison instead of c = 5 to accommodate the extended duration of attacks performed by an attacker close to 50% of the global hash-rate. These results do converge as predicted by the CEC, but not as closely as the results in section /spana href="whitepaper.html#lateximage-whitepaper-63"4.10.4/span/a.

/spanfigure id="autoid-74" class="figure "

/spandiv class="center"
/spana href="simulation/__results__extra__q=0.48__daa=500__nofrac" target="__blank" /spanimg
src="simulation/__results__extra__q=0.48__daa=500__nofrac"
class="inlineimage"
alt="(image)"
/span/a

/spandiv class="figurecaption"Figure 40: Main results for q=0.48 for the experiment documented in section /spana href="whitepaper.html#lateximage-whitepaper-63"4.10.4/span/a

/span/div /span/div/span/figure

Blockchain security breaks down at q $\geq 0.5, so, as q \rightarrow 0.5 weshouldexpectthat/spanempreviouslyinsignificant/span/emimplementationdetailsbecomemoreandmoresignificant. Anypreviouslyinsignificantadvantagesforthehonestnetworkortheattacker (atlowervaluesofq) canbecomesubstantialasqapproachesthatthreshold. Itmightbepossibletofurtherrefinethesimulation (viaadditionaliterationsande$

So we should expect the simulation to lose accuracy as q $\rightarrow 0.5. Giventhis, itseemsreasonabletoconcludethat \textbf{??} isconsistentwiththeCEC'spredictions, albeitwiththecaveatofloweraccuracy.$

`<!--...............................-->`
/spanh5 id="autosec-592"/spanspan class="sectionnumber"D.2&#x2003;/span/spanSimulator Iteration/span/h5

The purpose of this section is to avoid excluding all but the best results, and to document the steps taken that increased the accuracy of the simulation.

Other than the major changes discussed in **??** and on, many other configurations were also tested to establish that certain implementation details were */spanemnot/span/em* significant. These include: /spanul style="list-style-type:none"

/spanli
- Hashing algorithm: `/spankbdxxh3/span/kbd` was compared against `/spankbdblake3/span/kbd` and `/spankbdsha256/span/kbd`. No significant differences were observed.

/span/li
/spanli
- B_f^{-1} :
$Nosignificantdifferencewasfoundforvaluesof50, 75, and100.Thefinalvalueof75was, inpart, chosentoensuretherewassufficientexcesscapacityinB_f{}^{-1}(sothatitdidnotaffectresults).Lowervalues(e.g., 10)seemedtoworkokayduringearlytesting, butalsoresultinmoreblocksbeinggeneratedsimultaneously.$

/span/li
/spanli
- H: this value is important because
q$\cdot Hshouldresultinawholenumber.IfHistoosmall, orq \cdot Hisnotawholenumber, lossofprecisionoccurs.H = 50issufficienttotestvaluesofqthataremultiplesof0.02.H = 75worksforvaluesofqthataremultiplesof0.04, andwaschosenoverH = 50toensureexcesscapacity.$

/span/li
/spanli
- Iteration between H and B_f^{-1} $: thedifficultyisthenumberofhashestofindablock, which, beforetheattackstarts, is{\sim}H \cdot B_f{}^{-1}.Thus, H = B_f{}^{-1} = 75isabout/spanemhalf/span/emthecomputationcostofusingH = B_f{}^{-1} =$
$100.Smallervaluesforbothcouldwork, butrisksinvalidatingresultsifeitheristoosmall.$

/span/li
/span/ul

```
<!--................................-->
/spanh6 id="autosec-593"/spanspan class="sectionnumber"D.2.1&#x2003;/span/spanInitial Results/span/h6


/spanfigure id="autoid-75" class="figure "
```

/spandiv class="center"
/spana href="./includes/ut/simulation/lp__early__results.svg" target="__blank" /spanimg
    src="./includes/ut/simulation/lp__early__results.svg"
    class="inlineimage"
    alt="(image)"
/span/a

/spandiv class="figurecaption"Figure 41: Early results showed that, although $\mathrm{P}'(q, c = Cx, N_1 = 1) \rightarrow 0asexpected(traditionalchains), P'(q, c = C, N_1 = x)didnotapproach0asN_1increased(simplexes).Thiswaspriortothesubstantiveerrorcorrectionsshowninsubsequentfigures.$

/span/div /span/div/span/figure

<!--................................-->
/spanh6 id="autosec-595"/spanspan class="sectionnumber"D.2.2&#x2003;/span/spanAccounting for Draft Reflecting Work/span/h6

/spanfigure id="autoid-76" class="figure "

```
/spandiv class="center"
/spana href="./includes/ut/simulation/lp_results_after_draft_refl_work.svg" target="_blank" /spanimg
    src="./includes/ut/simulation/lp_results_after_draft_refl_work.svg"
    class="inlineimage"
    alt="(image)"
/span/a
```

/spandiv class="figurecaption"Figure 42: Early results (legend: ‘no DRW’) did not show P$' \rightarrow 0asN_1increased.Thiswasfixedafteraccountingfordraftreflectedwork(‘w/DRW')inthefunctionthatdetermineswhetheranattackhassucceeded.Thesignificanceofthisisexplainedinsection /spanahref =$ "whitepaper.html$\#lateximage-whitepaper-63$"4

/span/div /span/div/span/figure

```
<!--................................-->
/spanh6 id="autosec-597"/spanspan class="sectionnumber"D.2.3&#x2003;/span/spanAttacker’s Hash-Rate Distribution: Uniform vs Random/span/h6

/spanfigure id="autoid-77" class="figure "
```

```
/spandiv class="center"
/spana href="./includes/ut/simulation/__interim__randhr.png" target="__blank" /spanimg
       src="./includes/ut/simulation/__interim__randhr.png"
       class="inlineimage"
       alt="(image)"
/span/a
```

/spandiv class="figurecaption"Figure 43: These results indicate that simplexes are *spanemmore/span/em* secure when an attacker has the same hash-rate over all simplex-chains. Randomizing hash-rates (of both the honest network and the attacker) over all simplex-chains, while globally maintaining p + q = 1, improves the accuracy of the simulation with respect to the CEC's predictions.

/span/div /span/div/span/figure

When hash-rates are */spanemuniformly/span/em* distributed: the miners of each chain do H hashes per tick, and the attacker does q$\cdot Hofthosehashespertick$.

When hash-rates are */spanemrandomly/span/em* distributed: for all chains but the primary chain, H $\cdot (N_1 - 1) hashespertickaredistributedrandomlyoverthosechains, withtheattackerresponsibleforq \cdot H \cdot (N_1 - 1) ofthosehashespertick. Somechainswillhaveahash - rategreaterthanHhashespertick, andsomechainsfewer. It's possibleforanattackertohave/spanemany/span/emproportionofaparticularchain's hash - rate, butp + q = 1 ismaintainedglobally. Boththeattacker's andhonestminers' hash - ratesareuniformlydistributed, andthusthehash - rateofeachchainhasatrapezoidaldistribution. PMFsofthehash - rateofeachchain, thehonestminers, andtheattackercanbeobtainedvia/spankbdmakeprint - randhr - pmfs/span/kbd.$

<!--................................-->
/spanh6 id="autosec-599"/spanspan class="sectionnumber"D.2.4&#x2003;/span/spanThe Attacker's “Bonus Block”/span/h6

/spanfigure id="autoid-78" class="figure "

/spandiv class="center"
/spana href="./includes/ut/simulation/_interim_bonusblock.png" target="_blank" /spanimg
src="./includes/ut/simulation/_interim_bonusblock.png"
class="inlineimage"
alt="(image)"
/span/a

/spandiv class="figurecaption"Figure 44: In general, an attacker has the opportunity to prepare for a doublespend attack. Particularly, they can wait to mine their first private block before making the public transaction (which is the target of the doublespend). When the attacker waits to mine this first private block before starting the attack, it's called (in this paper) the “bonus block”. Implementing this substantially increased the accuracy of the simulation with respect to the CEC's predictions for smaller values of c and/or N_1.

/span/div /span/div/span/figure

`<!--................................-->`
/spanh6 id="autosec-601"/spanspan class="sectionnumber"D.2.5&#x2003;/span/spanEffects of $\mathrm{DAA}_N/span/h$6

/spanfigure id="autoid-79" class="figure "

/spandiv class="center"
/spana href="simulation/__interim__daa" target="__blank" /spanimg
    src="simulation/__interim__daa"
    class="inlineimage"
    alt="(image)"
/span/a

/spandiv class="figurecaption"Figure 45: This was the last iteration prior to generating the final results (**??**&nbsp;4.10.4.5). Early results indicated that lower values of $\mathrm{DAA}_N weremoresecurefortraditionalchains, andthat\mathrm{DAA}_N \approx 500wassufficientforresultstocloselymatchtheanalyticalsolution.\mathrm{DAA}_N isnotassignificantforthesecurityofsimplexesa$
$20areincludedtobetterdemonstratetheeffectofincreasing\mathrm{DAA}_N$.

/span/div /span/div/span/figure

.7ex]0.5pt/spanspan style="width:3.4445pt; display:inline-block"<!---->/span/spanBegin DRAFT/spanspan style="width:3.4445pt; display:inline-block"<!---->/span/span.7ex]0.5pt

.7ex]0.5pt/spanspan style="width:3.4445pt; display:inline-block"<!---->/span/spanEnd DRAFT/spanspan style="width:3.4445pt; display:inline-block"<!---->/span/span.7ex]0.5pt

.7ex]0.5pt/spanspan style="width:3.4445pt; display:inline-block"<!---->/span/spanBegin DRAFT/spanspan style="width:3.4445pt; display:inline-block"<!---->/span/span.7ex]0.5pt

.7ex]0.5pt/spanspan style="width:3.4445pt; display:inline-block"<!---->/span/spanEnd DRAFT/spanspan style="width:3.4445pt; display:inline-block"<!---->/span/span.7ex]0.5pt

/span/section /span/main /span/div

/span/body
/span/html